MINISTRY OF EDUCATION AND TRAINING

FPT UNIVERSITY

Capstone Project Document

Free Driver - Platform connecting drivers and cars owners

Group Members	Mã Ngọc Long Bảo - SE171157 (Leader) Phạm Đức Thịnh - SE171147 Nguyễn Huỳnh Anh Tài - SE171213 Trần Hoàng Trung Anh - SE151299
Supervisor	Tôn Thất Hoàng Minh
Ext Supervisor	
Capstone Project code	SP25SE136

- HCM, 04/2025 -

Table of Contents

Acknowledgement....3
Definition and Acronyms....3
I. Project Introduction....4
1. Overview....4
1.1 Project Information....4
1.2 Project Team....4
2. Product Background....4
3. Existing Systems....4
3.1 Mioto....4
3.2 BonBonCar....5
4. Business Opportunity....6
5. Software Product Vision....6
6. Project Scope & Limitations....6
6.1 Major Features....6
6.2 Limitations & Exclusions....8
II. Project Management Plan....10
1. Overview....10
1.1 Scope & Estimation....10
1.2 Project Objectives....11
1.3 Project Risks....11
2. Management Approach....13
2.1 Project Process....13
2.2 Quality Management....13
2.3 Training Plan....14
3. Project Deliverables....14
4. Responsibility Assignments....14
5. Project Communications....15
6. Configuration Management....15
6.1 Document Management....15
6.2 Source Code Management....15
6.3 Tools & Infrastructures....16
III. Software Requirement Specification....16
1. Product Overview....16
2. User Requirements....18
2.1 Actors....18
2.2 Use Cases....18
3. Functional Requirements....23
3.1 System Functional Overview....23
3.2 View Statistics....39

3.3 Manage Amenities....42
3.4 Manage Manufacturers....46
3.5 Manage Models (Dòng xe)....50
3.6 Manage Fuel Types (Loại Nhiên Liệu)....54
3.7 Manage Transmission Types (Loại Truyền Động)....58
3.8 Manage GPS Devices (Thiết Bị GPS)....62
3.9 View Transactions (Giao dịch)....63
3.10 Process Withdrawal Requests (Lệnh rút tiền)....64
3.11 View Owners (Chủ xe)....66
3.12 View Drivers (Người Thuê)....67
3.13 Manage Consultants (Tư vấn viên)....68
3.14 Manage Technicians (Kĩ thuật viên)....70
3.15 View Cars (Xe)....72
3.16 View Inspection Schedules....73
3.17 Create Inspection Schedule (Tạo lịch kiểm tra)....74
3.18 View Pending Cars (Xe chờ xử lý)....75
3.19 View User Reports (Báo cáo người dùng)....76
3.20 View Report Details (Chi tiết báo cáo)....77
3.21 Create Compensation (Tạo bồi thường)....78
3.22 Conditional Compensation Creation Trigger....79
3.23 Reject Report (Từ chối báo cáo)....80
3.24 Approve Report with Compensation Proof (Phê duyệt báo cáo với bằng chứng bồi thường)....81
3.25 Technician View Inspection Schedule Todo Task....81
3.26 Technician Approve Inspection Schedule Todo Task (Mark as In-Progress)....82
3.27 Technician View Inspection Schedule Todo Task Detail....83
3.28 Technician Reject Inspection Schedule....83
3.29 Technician View Contract (Update Technician Info and Navigate)....84
3.30 Technician View Contract....84
3.31 Technician Approve Contract....85
3.32 Technician Import Images Then Approve....85
3.33.1 Send OTP....85
3.33.2 Create Account....88
3.33.3 Get current user....90
3.33.4 Upload image avatar....90
3.33.5 Update user profile....91
3.33.6 Update password account....92
3.33.7 Login....94
3.34.1 Create License....95
3.34.2 Upload Image License....96
3.35.1 List Cars....97
3.35.2 Get car by id....98
3.35.3 Get unavailability date....99
3.35.4 Delete Car....101

3.35.5 View Contact....102
3.35.6 Disable Car....103
3.35.7 Enable Car....104
3.35.9 List Fuel....105
3.35.10 List Amenity....106
3.35.11 List Transmission....107
3.35.12 Create Car....108
3.35.13 Update Car....109
3.35.14 Upload Image Car....110
3.35.15 Create unavailability date....111
3.35.16 View Contract Car....112
3.35.17 Contract sign car....113
3.35.18 Tracking location car....114
3.36.1 Contract sign booking....114
3.36.2 List Booking....115
3.36.3 Get Booking by id....117
3.36.4 View Contract Booking....118
3.36.5 Approve Booking....119
3.36.6 Reject Booking....119
3.36.7 Feedback Booking....120
a3.36.8 Create Booking Report....122
3.36.9 Get list schedule....123
3.36.10 Get schedule by id....124
3.36.11 Get list booking reports....125
3.36.12 Get booking report by id....126
3.36.13 Get list car reports....126
3.36.14 Get car report by id....128
3.37.1 Get list transaction....129
3.37.2 Get list bank account....129
3.37.3 Get bank account by id....130
3.37.4 Update bank acount....132
3.37.5 Create Withdraw....134
3.38.1 View detail gps device....135
3.38.2 Unassign device....136
3.38.3 Get List car for staff....137
3.38.4 Assign device to car....138
4. Non-Functional Requirements....139
4.1 External Interfaces....139
4.2 Quality Attributes....139
5. Requirement Appendix....140
5.1 Business Rules....140
5.2 Common Requirements....141
5.3 Application Messages List....141
IV. Software Design Description....144

1. System Design....144
1.1 System Architecture....144
1.2 Package Diagram....145
2. Database Design....147
3. Detailed Design....151
3.1 Authentication and Authorization....151
3.3 Car Management....156
3.4 Booking Management....162
V. Software Testing Documentation....171
1. Scope of Testing....171
2. Test Strategy....171
2.1 Testing Types....171
2.2 Test Levels....171
2.3 Supporting Tools....171
3. Test Plan....171
3.1 Human Resources....171
3.2 Test Environment....172
3.3 Test Milestones....172
4. Test Cases....172
- Other test reports: test-report.html, PCDACO-performance-report-20.pdf....172
- Detail Unit test: Report5_Unit Test.xlsx....172
- Detail User Acceptance test: Report5_Test Report.xlsx....172
5. Test Reports....172
5.1. Unit test....172
5.2 User acceptance test....173
VI. Release Package & User Guides....173
1. Deliverable Package....173
2. Installation Guides....174
2.1 System Requirements....174
2.1.2 Mobile Application....174
2.2 Installation Instruction....174
3. User Manual....182
3.1 Overview....182
3.2 USER REGISTER....184
3.4 DRIVER CREATE BOOKING....195
3.5 USER CREATE REPORT....200
3.6 USER CREATE WITHDRAWAL....204

Acknowledgement

We want to express our heart-felt thanks to everyone who may have helped us and guided us in the completion of our Capstone Project, FreeDriver – A Platform that Connects Drivers and Car Owners.

First of all, we want to acknowledge our supervisor, Mr. Tôn Thất Hoàng Minh, for all his time, guidance, feedback, and encouragement through every aspect of this project. Mr. Minh was the expert who directed our overall path and quality of our team's work.

We would like to thank FPT University for providing the project environment and resources to do our project, and having a structured Capstone Project model that provided an opportunity to put our knowledge to use in a real-life context.

We would like to acknowledge our families and friends, for providing their moral support to us, especially during the stressful parts of the project.

Finally, we would like to acknowledge all of our team members, Mã Ngọc Long Bảo, Phạm Đức Thịnh, Nguyễn Huỳnh Anh Tài, and Trần Hoàng Trung Anh, for working together, being committed, and having common determination to complete the project. We truly feel that the project would not have been a success without combined efforts and support of each member of our team.

Definition and Acronyms

Acronym	Definition
PWM	Psychology website
AWS	Amazon Web Services
BA	Business Analysis
BR	Business Rule
ERD	Entity Relationship Diagram
GUI	Graphical User Interface
PM	Project Manager
SDD	Software Design Description
SPMP	Software Project Management Plan
SRS	Software Requirement Specification
UAT	User Acceptance Test
UC	Use Case
API	Application Program Interface

I. Project Introduction

1. Overview

1.1 Project Information

- Project name: FreeDriver - Platform connecting drivers and cars owners
- Project code: SP25SE136
- Group name: GSP25SE94
- Software type: Web App and Mobile App

1.2 Project Team

Full Name	Role	Email	Mobile
Ton That Hoang Minh	Lecturer	minhtth5@fpt.edu.vn	0936668995
Ma Ngoc Long Bao	Leader	baomnlse171157@fpt.edu.vn	0764418461
Pham Duc Thinh	Member	thinhpdse171147@fpt.edu.vn	0938396953
Nguyen Huynh Anh Tai	Member	tainhase171213@fpt.edu.vn	0797909177
Tran Hoang Trung Anh	Member	anhthtse151299@fpt.edu.vn	0961790276

2. Product Background

Within Vietnam's rapidly changing urban landscape, two major disconnects between licensed drivers who have no vehicle and vehicle owners who do not fully utilize their vehicles. Such a situation entails many issues:

- There are so many qualified drivers out there who can't afford to buy a car of their own.
- Car owners face high maintenance costs for vehicles that sit idle.
- There are fewer options for flexible vehicle sharing on the market.
- Waste of vehicle capacity and no extra earnings in the process.

The idea of FreeDriver came by observing all the above challenges and finding an opportunity to develop a solution that would help both parties while contributing toward resource efficiency for the transport sector.

3. Existing Systems

3.1 Mioto

- **Link**: https://mioto.vn/
- **Description**: Mioto is a car rental platform in Vietnam that connects car owners with renters, offering both short-term and long-term rental options. The platform focuses on providing a convenient and secure way to rent cars through their mobile application.
- **System Actors**:
  - **Car Owners**: Individuals who list their vehicles for rent
  - **Renters**: Users who want to rent cars for personal or business use
  - **Platform Administrators**: Staff managing the platform operations
- **Features**:
  - **Car Listing Management**: Owners can list their vehicles with detailed specifications and pricing
  - **Booking System**: Users can book cars for specific dates and times

        - **Payment Integration**: Secure payment processing through various methods
        - **Insurance Coverage**: Basic insurance included in rental fees
        - **User Verification**: KYC (Know Your Customer) process for both owners and renters
        - **Rating System**: Feedback and rating mechanism for both parties
        - **GPS Tracking**: Vehicle tracking during rental periods
    - **Advantages and Disadvantages**:
        - **Advantages**:
            - Established brand in Vietnam with a good reputation
            - Comprehensive insurance coverage for peace of mind
            - User-friendly mobile application
            - Wide range of vehicle options
            - Flexible rental periods (hourly, daily, monthly)
        - **Disadvantages**:
            - Limited to specific regions in Vietnam
            - Higher rental rates compared to traditional rental services
            - Strict verification process may deter some users
            - Limited availability during peak seasons

3.2 BonBonCar

- **Link**: https://www.bonboncar.vn/
- **Description**: BonBonCar is a peer-to-peer car rental platform in Vietnam that focuses on connecting car owners with renters, offering a more community-driven approach to car sharing.
- **System Actors**:
    - **Car Owners**: Individuals who share their vehicles
    - **Renters**: Users looking to rent cars
    - **Community Moderators**: Platform staff managing user interactions
- **Features**:
    - **Community-Based Platform**: Focus on building trust between users
    - **Flexible Pricing**: Owners can set their own rental rates
    - **Instant Booking**: Quick booking process for available vehicles
    - **Document Verification**: Digital verification of driving licenses and vehicle documents
    - **Review System**: Detailed feedback mechanism for both parties
    - **Location-Based Search**: Find vehicles based on proximity
- **Advantages and Disadvantages**:
    - **Advantage**s:
        - Lower rental rates compared to traditional rental services
        - More personalized rental experience
        - Strong community focus
        - Flexible booking options
        - Direct communication between parties
    - **Disadvantages**:
        - Quality of vehicles may vary significantly
        - Less structured support system
        - Potential inconsistency in service quality
        - Limited insurance coverage options

4. Business Opportunity

FreeDriver is taking advantage of this opportunity in Vietnam for the following reasons:

- Growing middle class with increase in car owners
- Growing need for flexible transportation
- Poorly served need by existing platforms so far
- Increase in additional income streams for car owners
- Optimization in the use of vehicles, especially in the urban parts

Market research indicates the following:

- Above 50% of private cars are not being utilized during working hours
- Increasing amount of licensed drivers looking for job flexibility
- Widening cultural acceptance of a sharing economy-based platform

5. Software Product Vision

FreeDriver will be the all-encompassing web-based platform that changes how car utilization is taking place in Vietnam by matching car owners with licensed drivers. It creates a sustainable ecosystem where:

- Car owners monetize their idle vehicle
- Licensed drivers have access to vehicles without a major capital investment
- Passengers have increased transportation options
- The community benefits via optimized vehicle usage, reduced traffic

6. Project Scope & Limitations

6.1 Major Features

a. Car Owners (Mobile Application)

FE-01: Register and login into the application with secure authentication

FE-02: Manage personal profile and account settings

FE-03: List cars with detailed specifications (model, transmission type, fuel type, amenities)

FE-04: Set car availability periods

FE-05: Upload and manage car images and documents

FE-06: Set car location and pickup details

FE-07: View and manage incoming booking requests

FE-08: Track car usage and status during rental periods

FE-09: View earnings and transaction history

FE-10: Request withdrawals and manage bank account information

FE-11: Provide feedback and ratings for drivers

FE-12: Report issues or damages during rental periods

FE-13: View car statistics and rental history

FE-14: Manage car amenities and specifications

FE-15: Set collateral requirements for car rentals

b. Drivers (Mobile Application)

FE-16: Register and login with driver's license verification

FE-17: Browse available cars with advanced filtering options

FE-18: Search cars by location, price, and specifications

FE-19: View detailed car information and images

FE-20: Make booking requests with specific time periods

FE-21: Track ongoing trips with GPS integration

FE-22: View booking history and status

FE-23: Process payments through integrated payment gateways

FE-24: Provide feedback and ratings for car owners

FE-25: Report issues or concerns during rental periods

FE-26: View personal statistics and earnings

FE-27: Manage personal profile and documents

FE-28: Receive notifications for booking updates

FE-29: View and manage active bookings

c. Admin (Web Application)

FE-30: Manage user accounts (car owners and drivers)

FE-31: Review and approve driver's licenses

FE-32: Monitor platform transactions and financial reports

FE-33: Manage car listings

FE-34: View system analytics and statistics

FE-35: Monitor GPS devices

FE-36: Review and process withdrawal requests

FE-37: Manage car amenities, manufacturer and models

FE-38: Monitor cars, which are needed to be reviewed

FE-39: Assign inspection schedules for technicians

FE-40: Monitor inspection schedules of technicians in platform

FE-41: Handle user reports and disputes

FE-42: Assign compensations for users

FE-43: Update stages of inspection schedules

FE-44: Approve cars contract

FE-45: Approve inspection schedules with proofs

d. System Features

FE-46: Secure payment processing and transaction management

FE-47: Real-time GPS tracking and location services

FE-48: Automated booking confirmation and cancellation

FE-49: Document verification and storage

FE-50: Encrypted data storage for sensitive information

FE-51: Email notifications and communication system

FE-52: Rating and feedback management

FE-53: Report handling and resolution system

FE-54: Analytics and reporting tools

FE-55: Multi-language support

6.2 Limitations & Exclusions

- **Limitations:**

LI-01: The platform operates exclusively within Vietnam's geographical boundaries

LI-02: Car rentals are limited to licensed drivers with valid Vietnamese driver's licenses

LI-03: GPS tracking requires continuous internet connectivity

LI-04: Payment processing may be subject to banking system limitations

LI-05: Car availability depends on owner's manual updates

LI-06: Platform performance may be affected by network conditions

LI-07: Some features may require specific mobile device capabilities

LI-08: Real-time tracking accuracy depends on GPS signal quality

LI-09: Payment processing times may vary based on banking systems

LI-10: Platform availability may be affected by maintenance windows

- **Exclusions**:

EX-01: The system does not provide insurance coverage (users must arrange their own insurance)

EX-02: The platform does not handle vehicle maintenance or repairs

EX-03: No support for international driver's licenses

EX-04: No automated vehicle inspection system

EX-05: No built-in navigation system

EX-06: No support for commercial vehicle rentals

EX-07: No integration with external ride-hailing services

EX-08: No automated fuel cost calculation

EX-09: No support for long-term leasing (beyond 30 days)

EX-10: No built-in emergency roadside assistance

II. Project Management Plan

1. Overview

1.1 Scope & Estimation

#	WBS Item	Complexity	Est. Effort (man-days)
1	**Requirement analysis**		**12**
1.1	Define background and context of product	Complex	7
1.2	Analyze users pain point	Medium	3
1.3	Business opportunity analysis	Medium	2
2	**Technology Training**		**25**
2.1	Learn NextJS	Medium	7
2.2	Learn Expo	Medium	7
2.3	Learn AES encryption	Medium	1
2.4	Learn Testing	Medium	5
2.5	Learn real-time GPS Tracking	Medium	5
3	**Workspace setup**		**6**
3.1	Setup backend	Medium	4
3.2	Setup frontend	Medium	1
3.3	Setup mobile	Medium	1
4	**Documentation**		**32**
4.1	Project introduction	Simple	2
4.2	Project management plan	Medium	3
4.3	Software requirement specification	Medium	7
4.4	Software design document	Medium	5
4.5	Test report	Medium	5
4.6	User guide	Simple	3
4.7	Final document	Complex	7
5	**Design**		**55**
5.1	Design UI for web app	Complex	20
5.2	Design UI for mobile app	Complex	30
5.3	Design database	Complex	5

6	**Implementation**		**89**
6.1	Authentication	Medium	5
6.2	Car Management	Complex	20
6.3	Booking Management	Complex	20
6.4	Contract Management	Medium	10
6.5	User Management	Medium	10
6.6	Gps Device Management	Medium	5
6.7	Payment gateway	Medium	6
6.8	Email notification	Medium	3
6.9	Feedback management	Medium	5
6.10	Wallet Management	Medium	5
7	**Testing**		**50**
7.1	Unit Test	Complex	30
7.2	Acceptant Test	Complex	20
8	**Closing**		**15**
8.1	Maintenance	Medium	10
8.2	Prepare slide for capstone	Medium	5
		Total Estimated Effort (man-days)	***284***

1.2 Project Objectives

#	Testing Stage	Test Coverage	No. of Defects	% of Defect	Notes
1	Reviewing	100%	N/A	N/A	Reviewing requirements and initial test case
2	Unit Test	90%	0	0%	
3	Acceptance Test	90%	0	0%	Core functionalities tested Focus on critical paths and user workflows

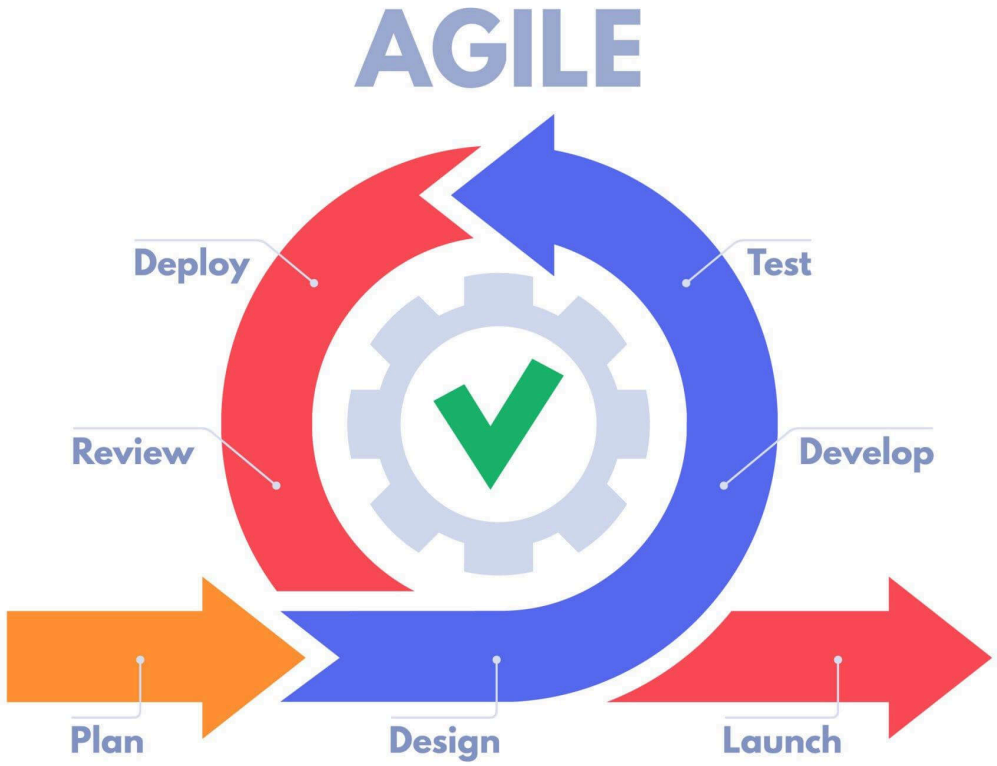
1.3 Project Risks

#	Risk Description	Impact	Possibility	Response Plans
1	Standard insurance policies may not cover car	High financial liability for	Medium	Partner with insurance companies to create specialized policies. Clearly

	rentals or commercial use	accidents; legal issues.		define insurance coverage in user agreements.
2	Risk of damage to cars during rentals or potential theft.	Financial losses for car owners; loss of assets.	Medium	
3	The third-party payment method(PayOS)'s server crashes	High	Low	Integrate another bank to avoid high peak usage
4	Data Security Risk	High	Medium	Implement advanced security measures such as data encryption
5	User Interaction Risk	Medium	Low	Improve user interface and provide detailed user guides to enhance the user experience
6	Gps device lost internet connection	Medium	Low	
7	User un-attach Gps out of car	High	Low	

2. Management Approach

2.1 Project Process



Reference: devops.vinahost.vn

2.2 Quality Management

2.2.1 Defect Prevention

- Code Standards: Establish coding guidelines to minimize common issues and enhance readability and maintainability.

2.2.2. Code Review

- Code Reviews: Implement peer code reviews to ensure adherence to coding standards and early identification of potential issues.
- Design Reviews: Conduct design reviews to verify that the architecture and design align with the project's requirements and objectives

2.2.3. Unit Test

- Develop unit tests to validate the functionality of individual components in isolation.
- Automate unit testing to ensure consistent execution after every update, reducing the risk of introducing new defects.

2.2.4. Continuous Improvement

- Regularly review and update quality management processes based on
- feedback and results from testing and review sessions.
- Use metrics such as defect rates, testing coverage, and user feedback to identify areas for improvement.

2.3 Training Plan

Training Area	Participants	When, Duration	Waiver Criteria
NextJS	ThinhPDSE171147	2w	Mandatory
Expo	AnhthtSE151299	3w	Mandatory
Testing	BaoMNLSE171157 TaiNHASE171213	1w	Mandatory
Tracking GPS	BaoMNLSE171157 AnhthtSE151299	1w	Mandatory

3. Project Deliverables

#	Deliverable	Due Date	Notes
1	Report 1	03/02/2025	Week 3
2	Report 2	10/02/2025	Week 4
3	Report 3	08/03/2025	Week 7
4	Report 4	15/03/2025	Week 8
5	Admin Web App	1/4/2025	Week 11
6	Driver App	1/4/2025	Week 11
7	Owner App	1/4/2025	Week 11
8	Tracking App	1/4/2025	Week 11
9	Report 5	17/04/2025	Week 13
10	Report 6	20/04/2025	Week 14
11	Report 7	25/04/2025	Week 14

4. Responsibility Assignments

D~Do; R~Review; S~Support; I~Informed; <blank>- Omitted

Responsibility	ThinhPDSE171147	BaoMNLSE171157	TaiNHASE171213	AnhTHTSE151299
Project Planning & Tracking	D	D	D	D
Prepare Project Introduction Document	S	S	D	R

Prepare SRS Document (Overview Part)	R	D	S	S
Prepare SRS Document (User Requirements)	D	R	D	D
Design System Architecture	D	S	S	R
Develop the Backend (API)	D	D	D	R
Develop the Web App (Frontend)	D	R	I	S
Develop the Mobile App (Frontend)	S	S	S	D
Perform Unit Testing	I	D	D	I
Prepare User Manual	D	D	D	D

5. Project Communications

Communication Item	Who/ Target	Purpose	When, Frequency	Type, Tool, Method(s)
Discord	Members	Solving Problems	Daily	Online
Google Meet	Members + Mentor	Report	Weekly	Online
Zalo	Members + Mentor	Communicate	Daily	Online

6. Configuration Management

6.1 Document Management

- Drive Structure: Project folder with subfolders (e.g., Requirements, Design).
- Naming: [DocType] - [Title] - [YYYYMMDD] - [v#].
- Versioning: Use Google Docs' "Version history." Name key versions.
- Changes: "Suggesting" mode + comments for feedback.
- Access: Share with specific people (view, comment, edit).
- Owners: Assign document owners for key documents.
- Archive: Move old versions to an "Archive" folder.

6.2 Source Code Management

GitHub is used for version control and GitHub Issues for tracking tasks.

- Branching Strategy:
  - main: Stable production code
  - develop: Integrates tested features
  - feat/*, fix/*, build/*: Specific changes
- Workflow:
  - Create issues → Work on a branch → Submit PR → Code review → Merge
  - Use semantic commit messages (e.g., feat: add search feature)
  - Squash & merge for clean history
- Versioning & Releases:
  - Use branch for tracking versions
- Security & Backup:
  - Manage sensitive data with .gitignore

- Control access using GitHub permissions

This ensures efficient code management, tracking, and collaboration.

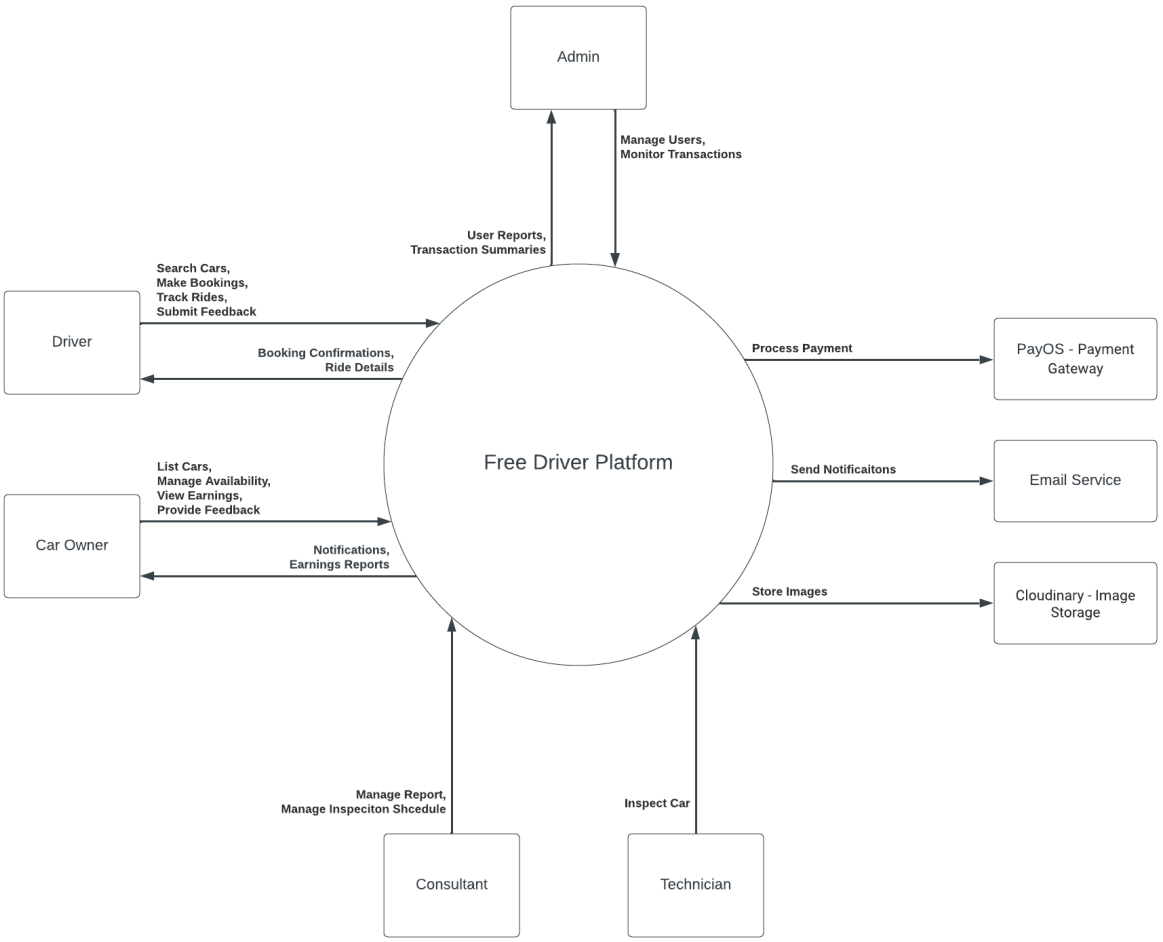
6.3 Tools & Infrastructures

Category	Tools / Infrastructure
Technology	NextJS 15 (Front-End), Expo (App), ASP.Net 8 (Back-end)
Database	PostgreSQL
IDEs/Editors	Visual Studio Code, NeoVim
Diagramming	Lucid chart, DrawIO, Eraser, Excalidraw
Documentation	Google Docs/Sheets, Canva
Version Control	Github(Source Codes), Docker Hub (Docker Images)
Deployment server	Contabo
Project management	Trello

III. Software Requirement Specification

1. Product Overview

FreeDriver is a new platform in Vietnam matching licensed drivers and car owners. The scheme allows car owners to make money by renting unused cars to drivers, who then use the car as taxis to earn money. The web and mobile applications will assist users in listing cars, scheduling rides, payments, and tracking rides. It wraps the website with payment handling, notification service, and photo storage capabilities. Along with providing technical and consulting assistance, it will manage vehicle inspection and maintenance. FreeDriver creates a win-win situation addressing mobility challenges while giving economic advantages to both drivers and car owners.


Admin
Manage Users,
Monitor Transactions
User Reports,
Transaction Summaries
Search Cars,
Make Bookings,
Track Rides,
Submit Feedback
Driver
Booking Confirmations,
Ride Details
Process Payment
PayOS - Payment Gateway
Free Driver Platform
List Cars,
Manage Availability,
View Earnings,
Provide Feedback
Car Owner
Notifications,
Earnings Reports
Send Notificaitons
Email Service
Store Images
Cloudinary - Image Storage
Manage Report,
Manage Inspeciton Shcedule
Consultant
Inspect Car
Technician

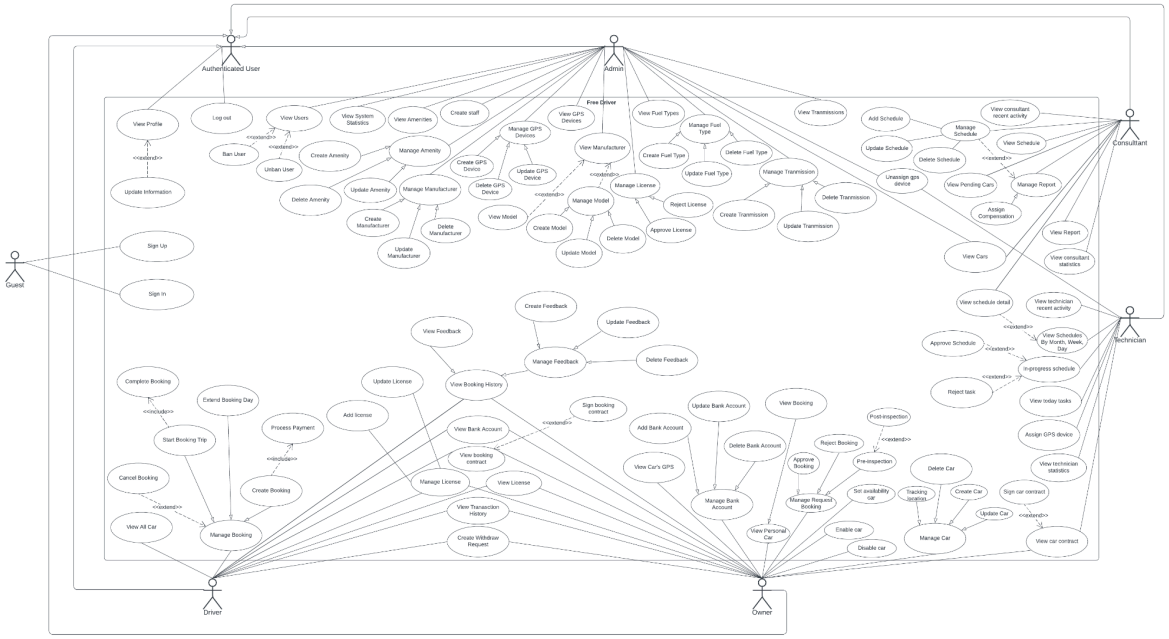
2. User Requirements

2.1 Actors

#	Actor	Description
1	Administrator	Who manages and observes systems data, approves users' requests.
2	Consultant	Contact with Owner and Drivers to create an inspection schedule date
3	Technician	Check Owners' cars and Drivers' cars in order to ensure that cars meet platform's car requirements
4	Owner	Let drivers rent their cars
5	Driver	Rent cars from the Owners

2.2 Use Cases

2.2.1 Diagram(s)



2.2.2 Descriptions

ID	Use Case	Actors	Use Case Description
1	Sign In	Admin, Consultant, Technician, Owner, Driver	Allows users to access their respective account by providing valid credentials. The Technician can also log in to the car tracking app.
2	Logout	Admin, Consultant, Technician, Owner, Driver	Ends the user's session, logging them out of the system.

3	Sign Up	Guest	Allows users to create new account to access to the system with role Owner or Driver, including sending an OTP for verification and creating account with license information and image upload.
4	Update Profile	Admin, Consultant, Technician, Owner, Driver	Enables users to update personal or account-related information like name, license, and upload a profile picture.
5	View System Statistic	Admin	View statistics of the system including numbers and usage history.
6	View Users	Admin	View list of users in all states
7	View Cars	Admin, Consultant	View list of cars in all states
8	View Amenities	Admin	View amenities in the platform
9	Create Amenity	Admin	Create a new amenity for platform
10	Update Amenity	Admin	Update an existing amenity
11	Delete Amenity	Admin	Delete an existing amenity
12	View Manufacturer	Admin	View list of manufacturers in the platform
13	Create Manufacturer	Admin	Create a new manufacturer for the platform
14	Update Manufacturer	Admin	Update manufacturer for the platform
15	Delete Manufacturer	Admin	Delete manufacturer for the platform
16	View Models	Admin	View list of models of selected manufacturer.
17	Create Model	Admin	Create a new model of selected manufacturer
18	Update Model	Admin	Update an existing model
19	Delete Model	Admin	Delete an existing model
20	View Fuel Types	Admin	View list of fuel types currently in the platform
21	Create Fuel Type	Admin	Create a new fuel type for the platform
22	Update Fuel Type	Admin	Update an existing fuel type
23	Delete Fuel Type	Admin	Delete an existing fuel type
24	View Transmission Types	Admin	View list of transmission types in platform
25	Create Transmission Type	Admin	Create a new transmission type for platform
26	Update Transmission Type	Admin	Update an existing transmission type
27	Delete Transmission Type	Admin	Delete an existing transmission type
28	View GPS Devices	Admin	View list of gps devices in all states in the platform
29	Unattach GPS Device	Admin, Technician	Unattach GPS Device
30	View License Pending Approval	Admin	View license submission that are currently pending

31	Approve License Pending Approval	Admin	Approve license submission
32	Reject License Pending Approval	Admin	Reject license submission
33	VIew Owners	Admin	View list of users that are currently cars owners.
34	Ban Owners	Admin	Ban an cars owner
35	Unban Owners	Admin	Unban an cars owner
36	View Drivers	Admin	View list of users that are currently customers
37	Ban Drivers	Admin	Ban an customer
38	Unban Drivers	Admin	Unban an customer
39	View Consultants	Admin	View list of users that are currently consultants
40	Create Consultants	Admin	Create a new consultant for the platform
41	View Technicians	Admin	View list of users that are currently technicians
42	Create Technicians	Admin	Create a new technician for the platform
43	View Transactions	Admin	View all transactions in all states for the platform
44	View Withdrawal Request	Admin	View all withdrawal requests
45	Process Withdrawal Request	Admin	Process additional withdrawal request
46	Update Withdrawal Request	Admin	Update the withdrawal request status with notes and proofs.
47	View Technician Todo Tasks	Technician	View in-date technician todo tasks
48	Mark Technician Todo Tasks As In-progress	Technician	Mark a selected task as in-progress.
49	View Schedule By Month, Week, Day	Technician	View all schedule assigned to this technician, these schedules can be filtered by month, week, day.
50	View Technician Recent Activity	Technician	View all activities of this technician with handling schedules
51	View Schedule Detail	Technician, Consultant	View detail of selected inspection schedule
52	Reject Schedule	Technician	Reject the existing schedule
53	Approve Schedule	Technician	Approve the schedule with images, which is proof.
54	View Car Contract	Technician, Owner	View a contract of car
55	View Technician Statistics	Technician	View statistics of a technician
56	View Consultant Statistics	Consultant	View statistics of a consultant
57	View Consultant Recent Activity	Consultant	View all activities of this consultant:
58	View Overall Inspection	Consultant	View inspection schedules in calendar form of

	Schedules		all technicians in the platform
59	Create Inspection Schedule	Consultant	Create a new inspection schedule for the technician
60	Delete Inspection Schedule	Consultant	Delete an existing inspection schedule
61	Update Inspection Schedule	Consultant	Change technician if needed
62	View User Reports	Consultant	View users reports
63	View Report Details	Consultant	View details of reports
64	Create Compensation Request	Consultant	Create a compensation request for the driver who cause the incident
65	Reject Report	Consultant	Reject the report
66	Approve Report	Consultant	Approve the report with notes
67	List Cars (Owner)	Owner	Displays owner's registered cars, allowing them to view, delete, enable, and disable cars. This includes listing available models, fuel types, amenities, and transmission types for each car.
68	Manage Car Details (Owner)	Owner	Allows owner to get car by ID to view renter's contact information, unavailability dates, create/update car details, upload car images, create car reports with image uploads, and view car contracts, enable car for driver booking or disable car if owner doesn’t want car to be rented.
69	Manage Car Unavailability (Owner)	Owner	Enables owner to create unavailability date to block out dates when the car is not available for rent.
70	Sign Car Contracts	Owner, Technician	Allows owner or technician to digitally sign car registration contracts.
71	Track Car Location (Owner)	Owner	Enables owner to track the location of their cars if they are equipped with GPS.
72	Manage Bookings (Owner)	Owner	Displays list of car bookings and allows owner to get booking by ID, approve/reject bookings, provide feedback, view booking details for compensation forms, upload pre/post compensation images.
73	Upload Pre-inspection images	Owner	After approving booking, the owner needs to provide car images to ensure the car is in the best condition.
74	Upload Post-inspection images	Owner	After the driver completes booking, the owner will upload car images to check the car condition.

75	View Booking Contract	Owner, Driver	Owner and Driver view a booking contract.
76	Sign Booking Contract	Owner, Driver	Owner and Driver sign a booking contract.
77	Manage Inspection Schedules (Owner)	Owner	Allows owner to view a list of scheduled vehicle inspections and detailed information for each schedule.
78	Manage Profile (Owner)	Owner	Allows owner to view and update their profile information, upload a profile picture, update their account password, and manage linked bank accounts for payments, including creating withdrawal requests and viewing transaction lists.
79	Manage Booking Reports (Owner)	Owner	Allows owner to view a list of booking reports, view booking report details.
80	Manage Car Reports (Owner)	Owner	Allows owner to view a list of car reports, view car report details.
81	List Cars (Driver)	Driver	Displays available cars for rent, including viewing car details and unavailability dates.
82	Manage Bookings (Driver)	Driver	Allows driver to create new bookings, get unavailability dates, view booking details, check payment status, process payments, cancel bookings, start/complete trips, provide feedback, extent booking day.
83	Manage Profile (Driver)	Driver	Allows driver to view and update their profile information, upload a profile picture, and update their account password.
84	Manage Booking Reports (Driver)	Driver	Allows driver to create booking reports, view a list of their booking reports, and view booking report details.
85	Manage Transactions (Driver)	Driver	Allows driver to view a list of their payment transactions.
86	View GPS Device Details (Technician)	Technician	Displays detailed information about a specific GPS tracking device.
87	Manage GPS Device Assignment (Technician)	Technician	Allows the technician to unassign a GPS device from a car and assign a GPS device to a car.
88	View Cars (Technician)	Technician	Retrieves and displays a list of cars with 'pending' and 'maintain' status for attaching gps device for car.

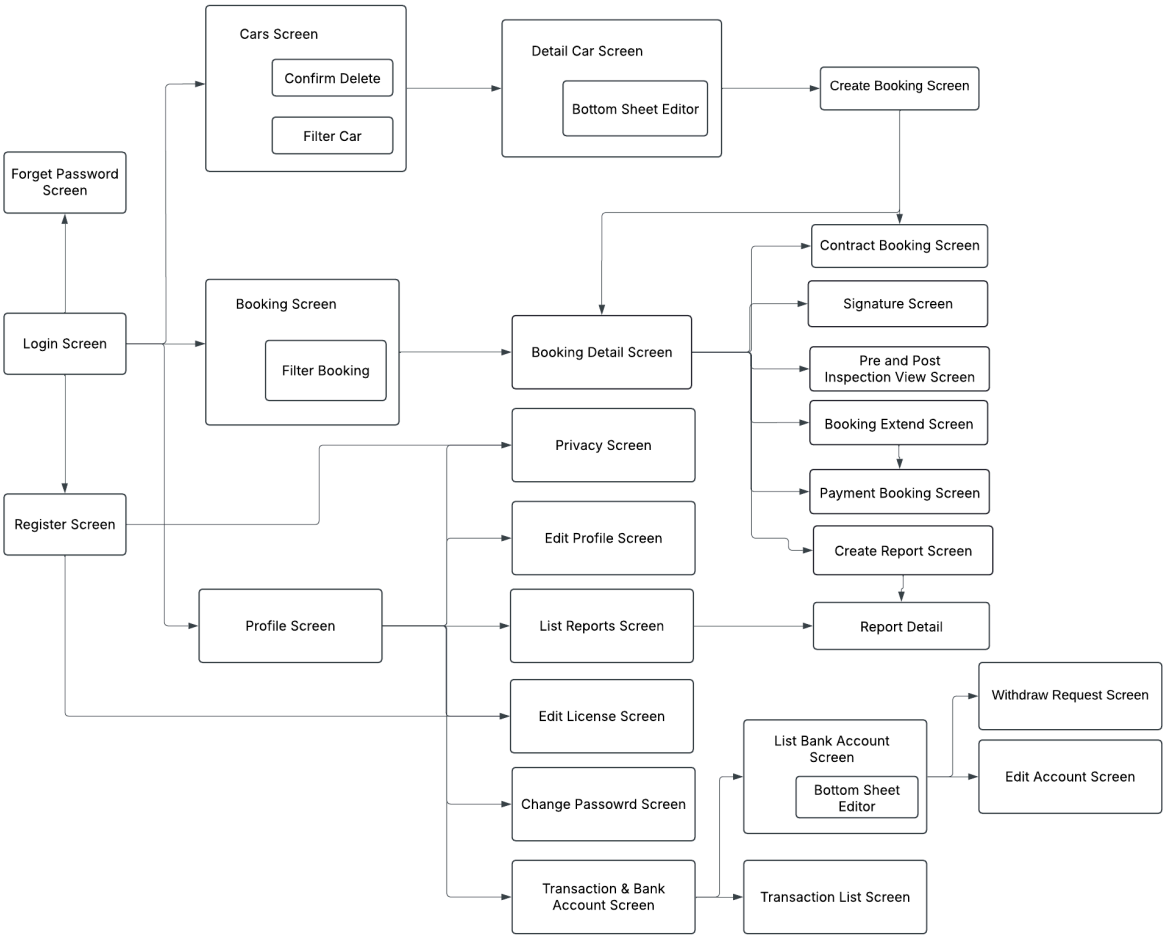
3. Functional Requirements

3.1 System Functional Overview

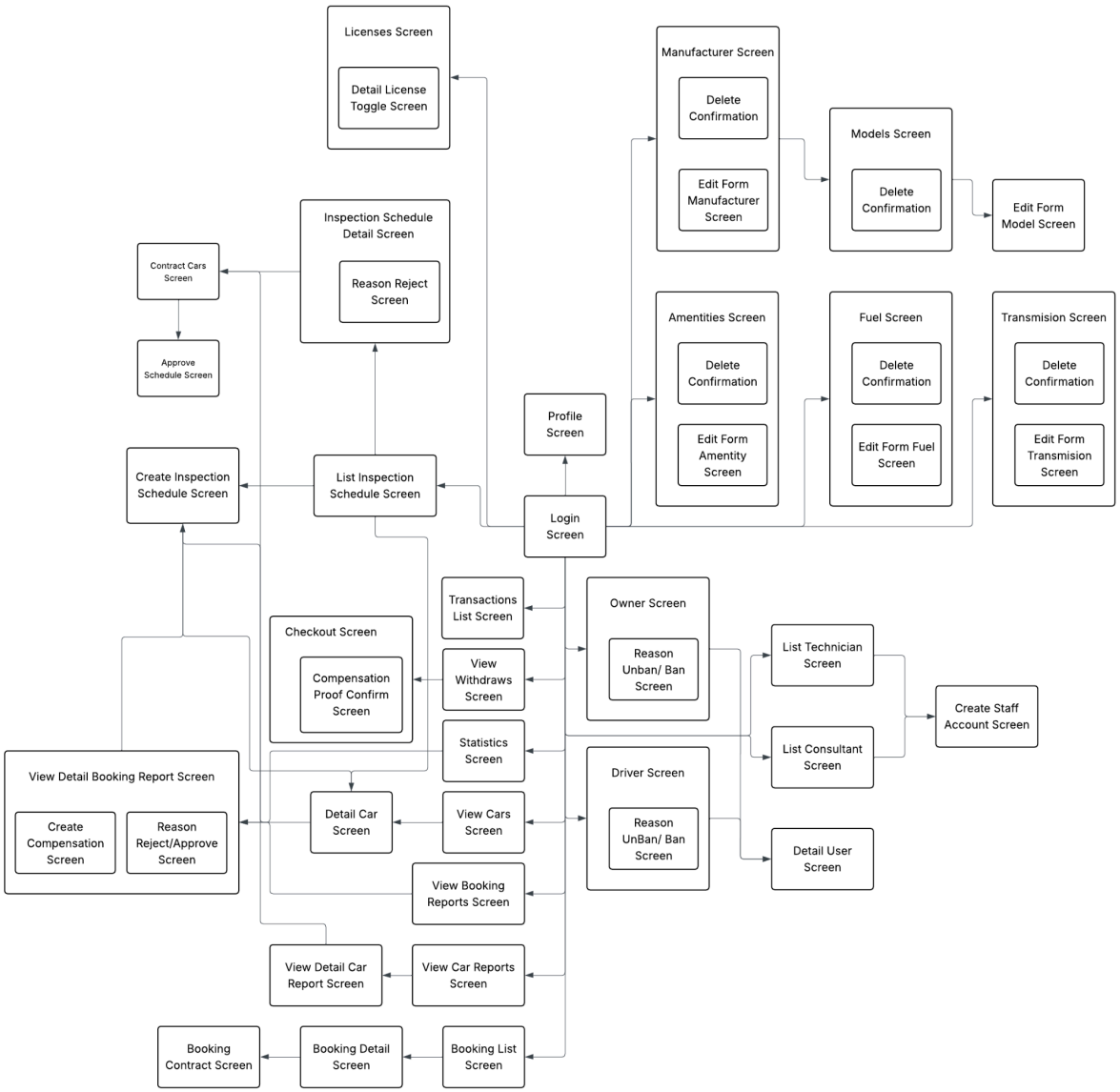
3.1.1 Screens Flow

3.1.1.1 Owner Screen App

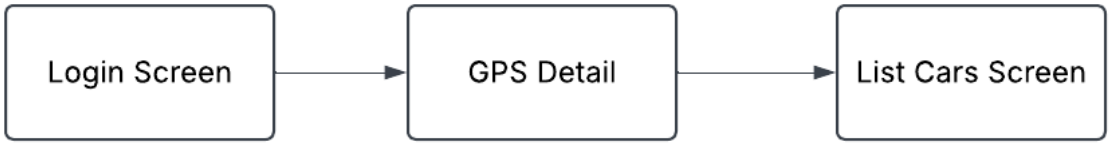
3.1.1.2 Driver Screen App



3.1.1.3 Admin Screen Web App



3.1.1.4 Tracking location App



3.1.2 Screen Descriptions

3.1.2.1 System Web

#	Feature	Screen	Description
1	View inspection schedules	Inspection Schedule Page	See inspection schedules of all technicians

2	Delete inspection schedule	Inspection Schedule Page	Delete a specific inspection schedule
3	Create inspection schedule	Create Inspection Schedule Page	Create a new inspection schedule
4	View in-date inspection schedules	Todo Page	See personal inspection schedules of today
5	Approve Inspection Schedule	Todo Page	Approve the inspection schedule, which means the inspection schedule is considered success
6	Reject Inspection Schedule	Todo Page	Reject the inspection schedule, which means the inspection schedule is considered failed
7	View Inspection Schedule Details	Inspection Schedule Detail Page	Displays detailed information about a specific inspection schedule.
8	View contract	Contract Page	Displays details of rental contracts within the system.
9	Approve contract	Contract Page	Approves a rental contract.
10	Approve inspection schedule	Approve Inspection Page	Approves a specific inspection schedule.
11	View Report Page	Report Page	Displays a list of all reports (e.g., car reports, booking reports).
12	View Report Details Page	Report Details Page	Shows detailed information about a specific report.
13	Create Compensation Request	Report Details Page	Initiates a request for compensation based on a report.
14	Reject Report	Report Details Page	Declines or rejects a submitted report.
15	Approve Report	Report Details Page	Approves a submitted report.
16	View Transactions	Transaction Page	View all financial transactions processed within the system.
17	View Withdrawal Requests	Withdrawal Request Page	View all withdrawal requests submitted by owners.
18	Process Withdrawal Request	Checkout Page	Processes the withdrawal requests for payment.
19	Update Withdrawal Request	Checkout Page	Updates the status or details of a withdrawal request processing.
20	View bookings	Booking Page	View all rental bookings in the system.
21	View booking details	Booking Detail Page	View all details of a specific booking.
22	View Cars	Car Page	See all cars registered on the platform.

23	View Car Details	Car Details Page	See detailed information about a specific car.
24	View Amenities	Amenity Page	See all available amenities for cars.
25	Create Amenity	Amenity Page	Create a new amenity option.
26	Update Amenity	Amenity Page	Update an existing amenity option.
27	Delete Amenity	Amenity Page	Delete an existing amenity option.
28	View Fuel Types	Fuel Type Page	See all available fuel types.
29	Create Fuel Type	Fuel Type Page	Create a new fuel type option.
30	Update Fuel Type	Fuel Type Page	Update an existing fuel type option.
31	Delete Fuel Type	Fuel Type Page	Delete an existing fuel type option.
32	View Manufacturers	Manufacturer Page	See all car manufacturers in the system.
33	Create Manufacturer	Manufacturer Page	Create a new car manufacturer entry.
34	Update Manufacturer	Manufacturer Page	Update an existing car manufacturer entry.
35	Delete Manufacturer	Manufacturer Page	Delete an existing car manufacturer entry.
36	View Models	Model Page	See all car models in the system.
37	Create Model	Model Page	Create a new car model entry.
38	Update Model	Model Page	Update an existing car model entry.
39	Delete Model	Model Page	Delete an existing car model entry.
40	View Transmissions	Transmission Page	See all available transmission types.
41	Create Transmission	Transmission Page	Create a new transmission type option.
42	Update Transmission	Transmission Page	Update an existing transmission type option.
43	Delete Transmission	Transmission Page	Delete an existing transmission type option.
44	View Owners	Owner Page	See all registered owners on the platform.
45	View Drivers	Driver Page	See all registered drivers on the platform.
46	View Technicians	Technician Page	See all registered technicians on the platform.
47	View Consultants	Consultant Page	See all registered consultants on the platform.

3.1.2.2 Owner App

#	Feature	Screen	Description
1	Send OTP	Register Screen	Sends OTP for user registration verification.
2	Create Account	Register Screen	Creates a new owner account.
3	Create License	Edit License Screen	Inputs owner's driver's license information.

4	Upload Image License	Edit License Screen	Uploads image of owner's driver's license.
5	List Cars	Home Screen	Displays owner's registered cars.
6	List Bookings	Home Screen	Shows current/upcoming bookings for owner's cars.
7	List Schedules	Home Screen	Displays scheduled maintenance/inspection appointments.
8	List Reports	Home Screen	Provides access to various owner reports.
9	Get current use	Home Screen	Displays logged-in owner's profile information.
10	List Cars	Cars Screen	Shows detailed list of owner's registered cars.
11	Delete Car	Cars Screen	Removes a registered car from the list.
12	Get car by id	Car Detail Screen	Displays detailed information for a specific car.
13	View Contact	Car Detail Screen	Views renter's contact info for a car's booking.
14	Get unavailability date	Car Detail Screen	Display date car is unavailable for booking.
15	Disable Car	Car Detail Screen	Temporarily disables a car from being booked.
16	Enable Car	Car Detail Screen	Re-enables a disabled car for booking.
17	List Model	Edit Car Screen	Lists car models for adding/editing cars.
18	List Fuel	Edit Car Screen	Lists fuel types for adding/editing cars.
19	List Amenity	Edit Car Screen	Lists amenities for adding/editing cars.
20	List Transmission	Edit Car Screen	Lists transmission types for adding/editing cars.
21	Create Car	Edit Car Screen	Adds a new car to the registered vehicle list.
22	Update Car	Edit Car Screen	Modifies details of an existing car listing.
23	Upload Image Ca	Edit Car Screen	Upload photos of the car.
24	Create Car Report	Create Car Report Screen	Creates a report on car condition after rental.
25	Upload Image Car Report	Create Car Report Screen	Uploads images for a car report.
26	Create unavailability date	Unavailability Schedule Screen	Blocks out dated cars are not available for rent.
27	View Contract Car	Contract Car Screen	Displays car registration contract.
28	Contract sign ca	Signature Screen	Digitally signs a car registration contract.
29	Contract sign booking	Signature Screen	Digitally signs a booking-related contract.
30	Tracking location	Map Screen	Displays the car's current location (if GPS

	car		equipped).
31	List Booking	Booking Screen	Displays list of all car bookings.
32	Get Booking by id	Booking Detail Screen	Shows detailed information for a specific booking.
33	Approve Booking	Booking Detail Screen	Accepts a pending booking request.
34	Reject Booking	Booking Detail Screen	Declines a pending booking request.
35	Feedback Booking	Booking Detail Screen	Provides feedback or notes about a booking.
36	Get Booking by id	Pre and Post Compensation Screen	Retrieves booking details for compensation forms.
37	Upload image field pre-compensation	Pre-compensation Form Screen	Uploads images of car condition before rental.
38	Upload image field post-compensation	Post-compensation Form Screen	Uploads images of car condition after rental.
39	View Contract Booking	Contract Booking Screen	Displays the contract for a specific booking.
40	Create Booking Report	Create Booking Report Screen	Creates a report related to a specific booking.
41	Get list schedule	View Inspection Schedule Screen	Displays list of scheduled vehicle inspections.
42	Get schedule by id	View Detail Inspection Schedule Screen	Shows detailed information for a specific inspection schedule.
43	Get current use	Profile Screen	Displays owner's profile information.
44	Upload image avatar	Profile Screen	Uploads a profile picture for the owner.
45	Get list booking reports	List Booking Reports Screen	Displays list of booking reports created by owner.
46	Get booking report by id	Booking Reports Detail Screen	Shows detailed information for a specific booking report.
47	Get list car reports	List car Reports Screen	Displays list of car reports created by owner.
48	Get car report by id	Car Reports Detail Screen	Shows detailed information for a specific car report.
49	Update user profile	Edit Profile Screen	Modifies owner's profile information.
50	Update password account	Change password Screen	Changes the owner's account password.
51	Get list bank account	List Bank Account Screen	Displays owner's linked bank accounts for payments.
52	Get bank account by id	Edit Bank Account Screen	Shows detailed information for a specific bank account.
53	Update bank account	Edit Bank Account Screen	Modifies details of a linked bank account.

54	Create Withdraw	Withdraw Request Screen	Requests withdrawal of owner's earnings.
55	Get list transaction	Transaction List Screen	Displays list of owner's financial transactions.

3.1.2.3 Driver App

#	Feature	Screen	Description
1	Send OTP	Register Screen	Sends OTP for driver registration verification.
2	Create Account	Register Screen	Creates a new driver account.
3	Create License	Edit License Screen	Inputs driver's license information for verification.
4	Upload Image License	Edit License Screen	Uploads image of driver's license for verification.
5	List Cars	Cars Screen	Displays available cars for rent.
6	Get car by id	Car Detail Screen	Shows detailed information for a specific car.
7	Get unavailability date	Car Detail Screen	Displays dates a specific car is unavailable.
8	Contract sign booking	Signature Screen	Digitally signs the booking rental contract.
9	List Booking	Booking Screen	Displays a list of the driver's bookings.
10	Get unavailability date	Create Booking Screen	Check car availability before creating a booking.
11	Create Booking	Create Booking Screen	Initiates a new rental booking request.
12	Get Booking by id	Booking Detail Screen	Shows detailed information for a specific booking.
13	Check Payment	Payment Booking Screen	Verifies payment status for a booking.
14	Payment Booking	Booking Detail Screen	Processes payment for a booking.
15	Cancel Booking	Booking Detail Screen	Allows the driver to cancel a booking.
16	Start-trip Booking	Booking Detail Screen	Marks the beginning of the rental period.
17	Complete Booking	Booking Detail Screen	Marks the end of the rental period.
18	Feedback Booking	Booking Detail Screen	Provides feedback on a completed booking.
19	Get Booking by id	Pre and Post Compensation Screen	Retrieves booking details for pre/post forms.
20	View Contract Booking	Contract Booking Screen	Displays the rental contract for a specific booking.
21	Create Booking Report	Create Booking Report Screen	Creates a report related to a specific booking issue.
22	Get current user	Profile Screen	Displays the driver's profile information.
23	Upload image	Profile Screen	Uploads a profile picture for the driver.

	avatar		
24	Get list booking reports	List Booking Reports Screen	Displays list of booking reports created by driver.
25	Get booking report by id	Booking Reports Detail Screen	Shows detailed information for a specific booking report.
26	Update user profile	Edit Profile Screen	Modifies driver's profile information.
27	Update password account	Change password Screen	Change the driver's account password.
28	Get list transaction	Transaction List Screen	Displays list of the driver's payment transactions.
29	Booking Extend	Booking Extend Screen	Extend booking schedule

3.1.2.3 Tracking location app

#	Feature	Screen	Description
1	Login	Login Screen	Allows the technician to log in to the tracking system.
2	View detail gps device	Home Screen	Displays detailed information about a specific GPS tracking device.
3	Unassign device	Home Screen	Removes the assignment of a GPS device from a car.
4	Get List car for staff	List Cars Screen	Retrieves and displays a list of cars with 'pending' and 'maintain' status for staff management.
5	Assign device to car	List Cars Screen	Assigns a specific GPS tracking device to a designated car.

3.1.3 Screen Authorization

Screen	Admin	Consultant	Technician	Driver	Owner
<<Inspection Schedule Page>>		x			
View Inspection Schedules		x			
Delete Inspection Schedules		x			
Update Inspection Schedules		x			
<<Create Inspection Schedule Page>>					
Create Inspection Schedule		x			
<<List Cars Page>>	x	x		x	x
View list cars	x	x		x	x

<<Car Detail Page>>	x	x		x	x
View car details	x	x		x	x
Edit car				x	
Create compensation request		x			x
Reject Report		x			
Approve Report		x			
<<Todo Page>>			x		
View Technician Todo Task			x		
Mark Inspection Schedule as In-Progress			x		
Reject Inspection Schedule			x		
<<Contract Page>>		x	x		
View Contract		x	x		
Approve Contract			x	x	x
<<Approve Inspection Schedule Page>>			x		
Approve Inspection Schedule			x		
<<Booking Page>>	x			x	x
View all bookings	x			x	x
<<Booking Detail Page>>	x			x	x
View booking details	x			x	x
<<Transaction Page>>	x			x	x
View all transactions	x			x	x
<<Withdrawal Requests Page>>	x				
View all withdrawal requests	x				
<<Checkout Page>>	x				
View checkout details	x				
Update withdrawal requests	x				

<<Car Page>>	x				
View all cars	x			x	x
<<Car Detail Page>>	x				
View car details	x			x	x
<<Amenity Page>>	x				
View amenities	x				
Create amenity	x				
Update amenity	x				
Delete amenity	x				
<<Fuel Type Page>>	x				
View fuel types	x				
Create fuel types	x				
Update fuel types	x				
Delete fuel types	x				
<<Manufacturer Page>>	x				
View manufacturers	x				
Create manufacturers	x				
Update manufacturers	x				
Delete manufacturers	x				
<<Model Page>>	x				
View models	x				
Create model type	x				
Update model	x				
Delete model	x				
<<Transmission Page>>	x				
View transmissions	x				
Create transmission type	x				

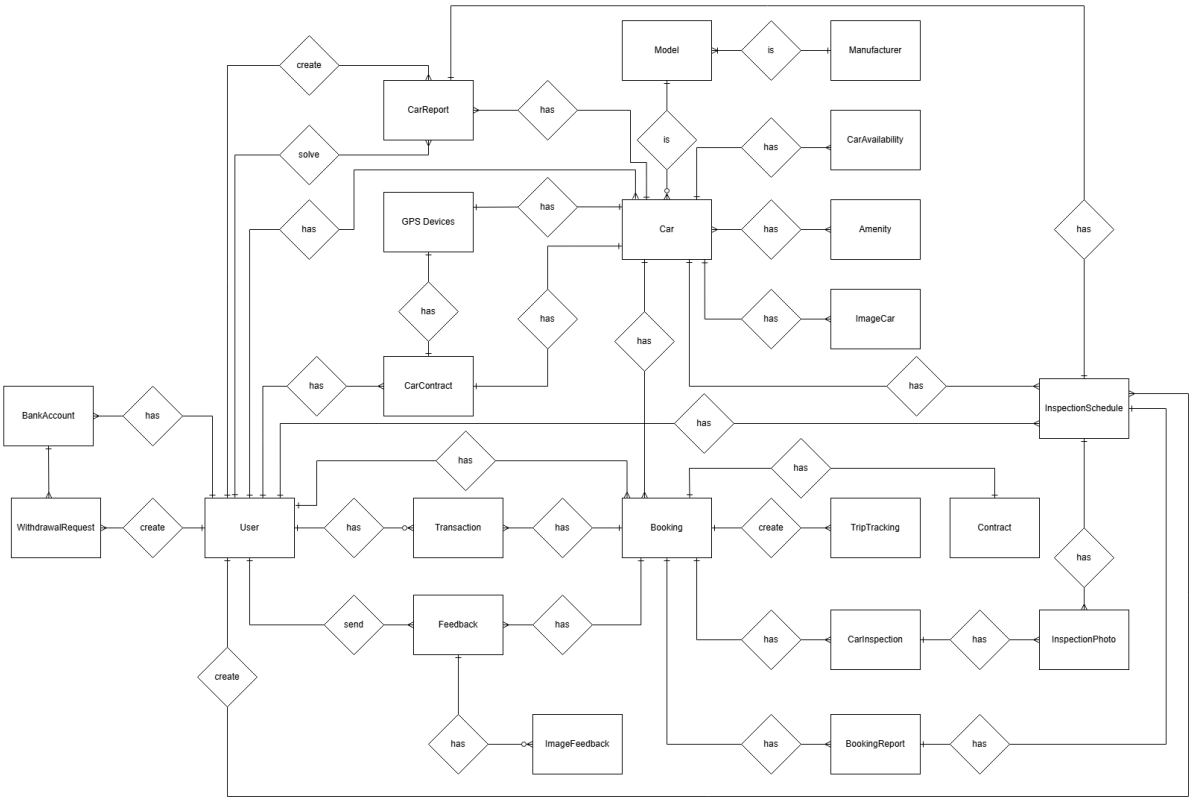
Update transmission	x				
Delete transmission	x				
View Owners	x	x			
View Drivers	x	x			
View Technicians	x	x	x		
View Consultants	x	x			

3.1.4 Non-Screen Functions

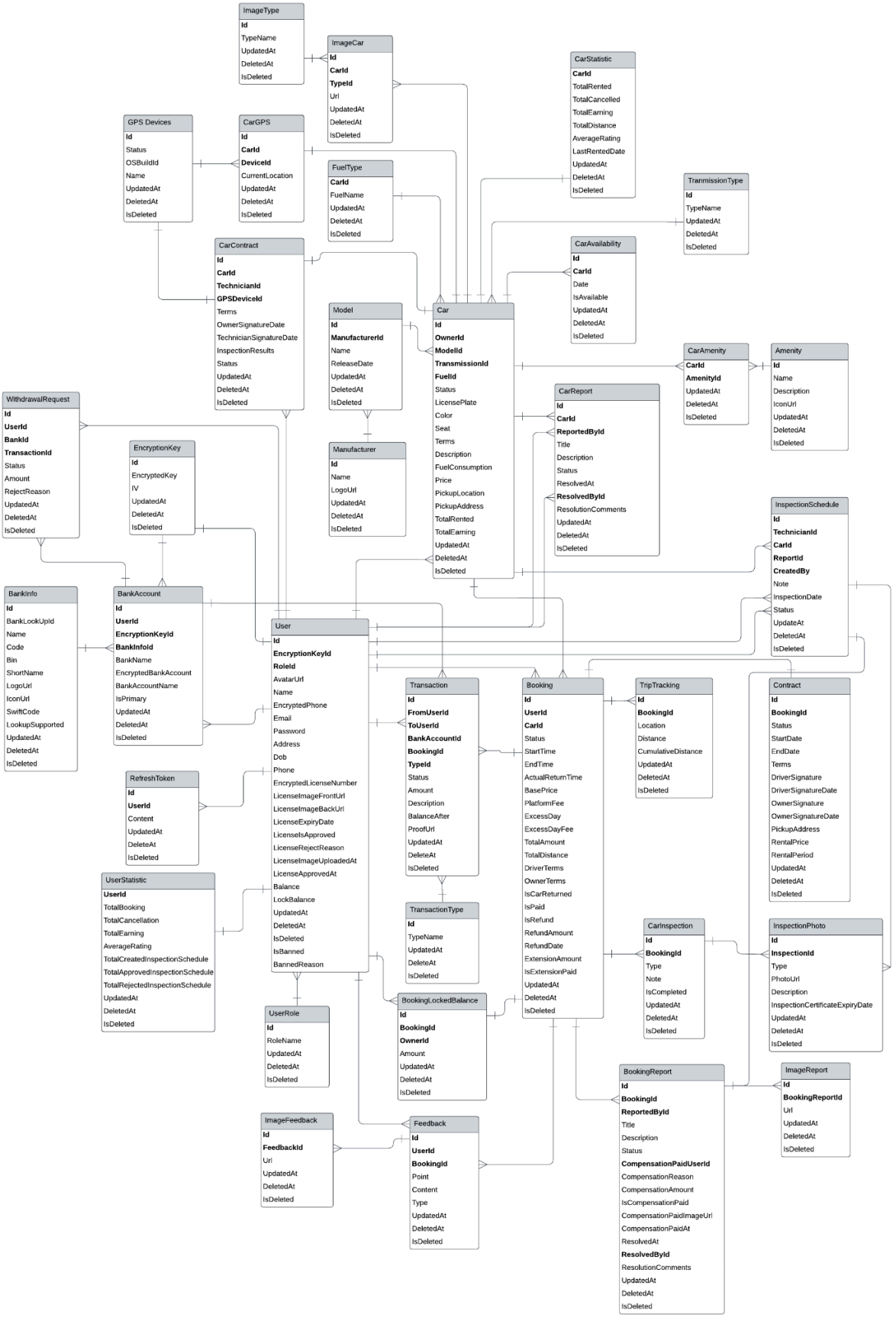
#	Feature	System Function	Description
1	Booking Reminder	Send Booking Reminder	Background job to send email to owner for approval
2	Booking Expired	Expired Booking	Background job to expired booking if it is not approved or overdue
3	Update Statistics	Update Statistics	Background job to update car and user statistics
4	Unlock Balance	Unlock Owner Balance	Background job to unlock owner balance when it's close to start time of booking, or when booking is completed or cancelled so owner can make withdraw
5	Booking Cancel	Update Booking Status Cancel	Background job to update all bookings affected by overdue booking to cancelled

3.1.5 Entity Relationship Diagram

3.1.5.1 Conceptual Entity Relational Diagram



3.1.5.2 Logical Entity Relational Diagram



Entities Description

#	Entity	Description
1	Amenities	Store amenities of a car. - Primary Key: Id - Foreign Key:
2	BankAccount	Store bank accounts of a user, which is used for send request of withdrawal. - Primary Key: Id - Foreign Key: EncryptionKeyId, BankInfoId,UserId
3	BankInfo	Store bank info of all banks, which is used for withdrawal requests. - Primary Key: Id - Foreign Key:
4	Booking	Store all bookings that are created by drivers. - Primary Key: Id - Foreign Key: UserId, CarId
5	BookingLockedBalance	Store current payment amount belong to a booking, to prevent owner withdraw current balance - Primary Key: Id - Foreign Key: BookingId, OwnerId
6	BookingReport	Store reports of a booking, including fee, accident, conflict,... - Primary Key: Id, - Foreign Key: BookingId, ReportedById, ResolvedById
7	Car	Store cars of platform. - Primary Key: Id, - Foreign Key: EncryptionKeyId, RoleId
8	CarAmenity	Store amenities for each car - Primary Key: Id - Foreign Key: Carid, AmenityId
9	CarAvailability	Store available time of a car - Primary Key: Id - Foreign Key: CarId
10	CarContract	Store a contract of a car, which is created by system when the car is created in platform, which is just tagged as Pending. - Primary Key: Id, - Foreign Key: CarId, TechnicianId, GPSDeviceId
11	CarGPS	Store GPS Device's location, which is attached in the car already. - Primary Key: Id, - Foreign Key: Car Id, DeviceId
12	CarInspection	Store car reviews before and after each booking, which ensures cars' overall condition to save proofs for conflicts. - Primary Key: Id - Foreign Key: BookingId
13	CarReport	Store all reports for owner's cars. Owner create report when they need assistance. - Primary Key: Id - Foreign Key: CarId, ReportedById, ResolvedById
14	CarStatistic	Store cars' statistics for overall review. - Primary Key: Id

		- Foreign Key: CarId
15	Contract	Store bookings' contracts between drivers and owners - Primary Key: Id - Foreign Key: BookingId
16	EncryptionKey	Store encryption information of entities for encrypting datas. - Primary Key: Id
17	Feedback	Store feedbacks of booking when finish a booking, which is describe personal thoughts about the opposite - Primary Key: Id - Foreign Key: UserId, BookingId
18	FuelType	Store types of fuel that car used. - Primary Key: Id
19	GPSDevice	Store information on all devices that are used for collecting GPS Coordination and send that information back to platform's server. - Primary Key: Id
20	ImageCar	Store images of a car, including the car itself and its paperworks - Primary Key: Id - Foreign Key: CarId, TypeId
21	ImageFeedback	Store image of a feedbacks. - Primary Key: Id - Foreign Key: FeedbackId,
22	ImageReport	Store images of a user report. - Primary Key: Id - Foreign Key: ReportId
23	ImageType	Store types of an image, which is used for specifying the purpose of an image usage. - Primary Key: Id - Foreign Key: TypeId
24	InspectionPhoto	Store images of a car, which is taken by the drivers itself for evidence. - Primary Key: Id - Foreign Key: InspectionId
25	InspectionSchedule	Store dates of inspections, which is used for technicians and consultants to know dates and time for inspecting purposes. - Primary Key: Id - Foreign Key: TechnicianId, CarId, ReportId
26	Manufacturer	Store manufacturers of a car, to know which brand produces the car. - Primary Key: Id
27	Model	Store models of manufacturers, to know which model the car is . - Primary Key: Id - Foreign Key: ManufacturerId
28	RefreshToken	Store refresh token of the users, which is used for revalidate the JWT Token. - Primary Key: Id - Foreign Key: UserId

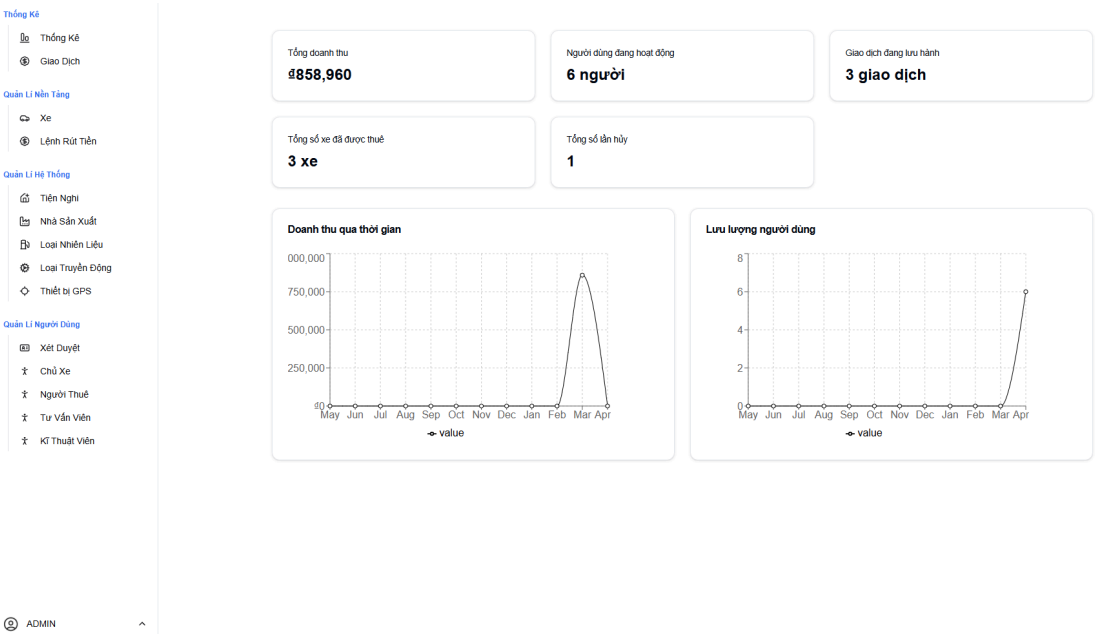
29	Transaction	Store transactions of system, which is used for control the money flow of the system. - Primary Key: Id - Foreign Key: FromUserId, ToUserId, BankAccountId, TypeId
30	TransactionType	Store types of transaction, to distinguish transactions. - Primary Key: Id
31	TransmissionType	Store type informations of transmission type - Primary Key: Id
32	TripTracking	Store all the locations that are received from the GPS Devices for logging the history of car locations. - Primary Key: Id - Foreign Key: BookingId
33	User	Store users of the platform. - Primary Key: Id - Foreign Key: EncryptionKeyId, RoleId
34	UserRole	Store role informations of users. - Primary Key: Id
35	UserStatistic	Store statistics of activities of users in the platform. - Primary Key: Id - Foreign Key: UserId
36	WithdrawalRequest	Store withdrawal requests of owners who want to retrieve profits from the platform. - Primary Key: Id - Foreign Key: UserId, StatusId, BankId, TransactionId

3.2 View Statistics

3.2.1 View System Statistics

- **Function Trigger:** When a user logs in.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Watch overall system statistics.
  - **Interface:** Charts, Graphs, Tables
- **Data Processing:**
  - **Data Collect:** Data is collected from the primary database.
  - **Data Computation:** Data is filtered through multiple layers.
  - **Data Binding:** Data is displayed on the screen.

- **Screen Layout:**

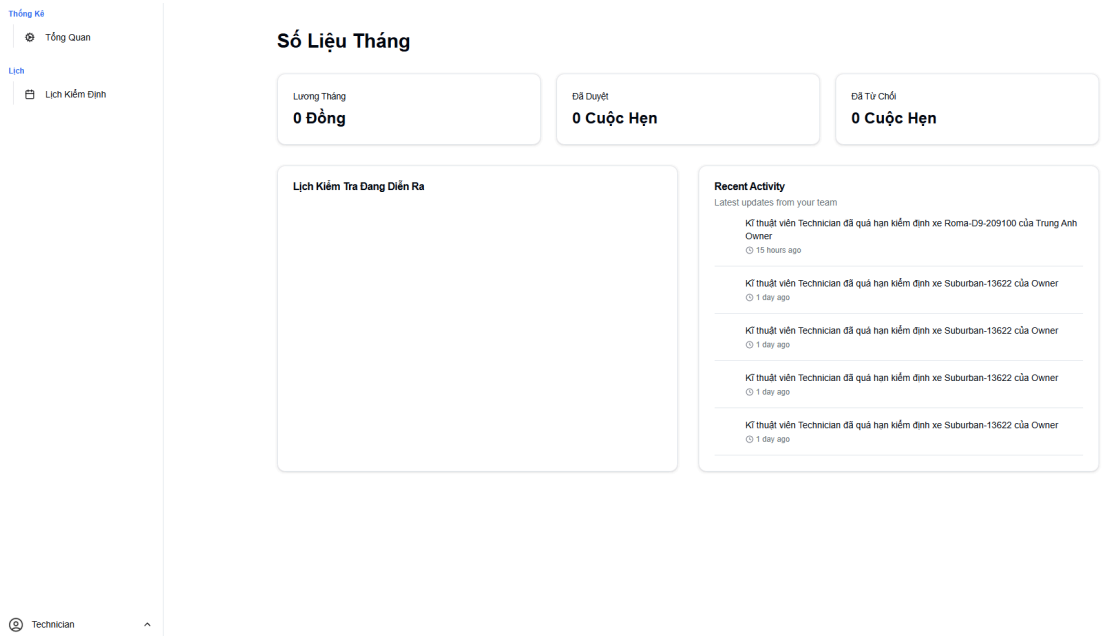


- **Function Details:** When logged in, if the user is an admin, the system displays overall statistics such as income, active users, transactions, etc.

3.2.2 View Personal Technician Statistics

- **Function Trigger:** When a technician logs in.
- **Function Description:**
  - **Actors:** Technician
  - **Purpose:** Watch overall technician KPIs.
  - **Interface:** Card
- **Data Processing:**
  - **Data Collect:** Data is collected from the primary database.
  - **Data Computation:** Data is filtered through multiple layers.
  - **Data Binding:** Data is displayed on the screen.

- **Screen Layout:**

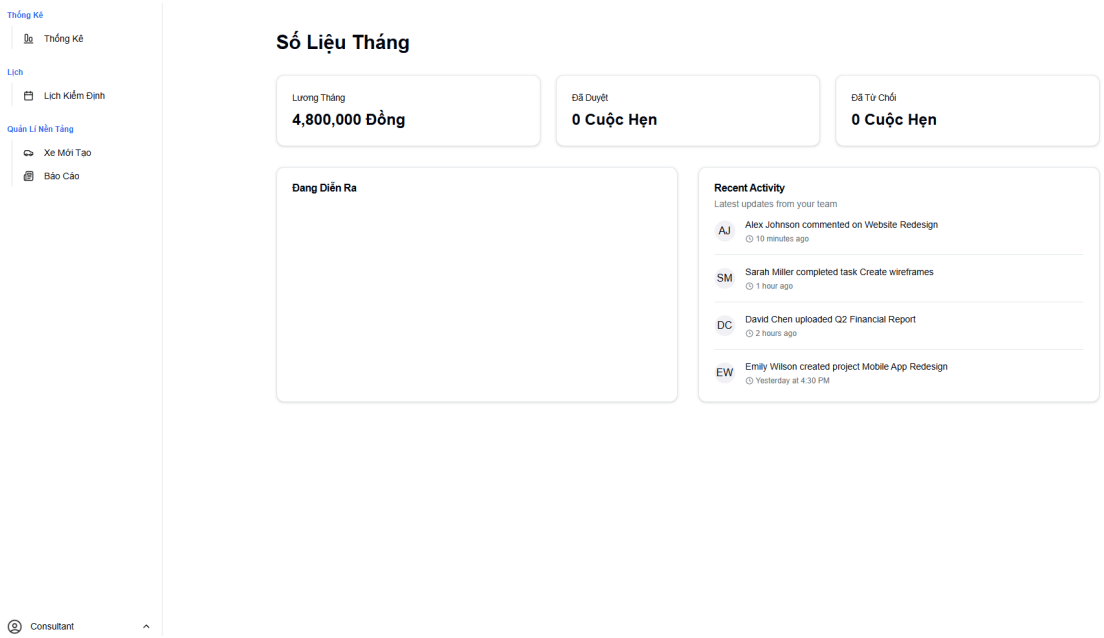


- **Function Details:** When logged in, a technician sees their monthly KPIs including salary, completed inspections, ongoing inspections, and recent activities.

3.2.3 View Personal Consultant Statistics

- **Function Trigger:** When a consultant logs in.
- **Function Description:**
    + **Actors:** Consultant
    + **Purpose:** Watch overall consultant KPIs.
    + **Interface:** Card
- **Data Processing:**
    + **Data Collect:** Data is collected from the primary database.
    + **Data Computation:** Data is filtered through multiple layers.
    + **Data Binding:** Data is displayed on the screen.

- **Screen Layout:**



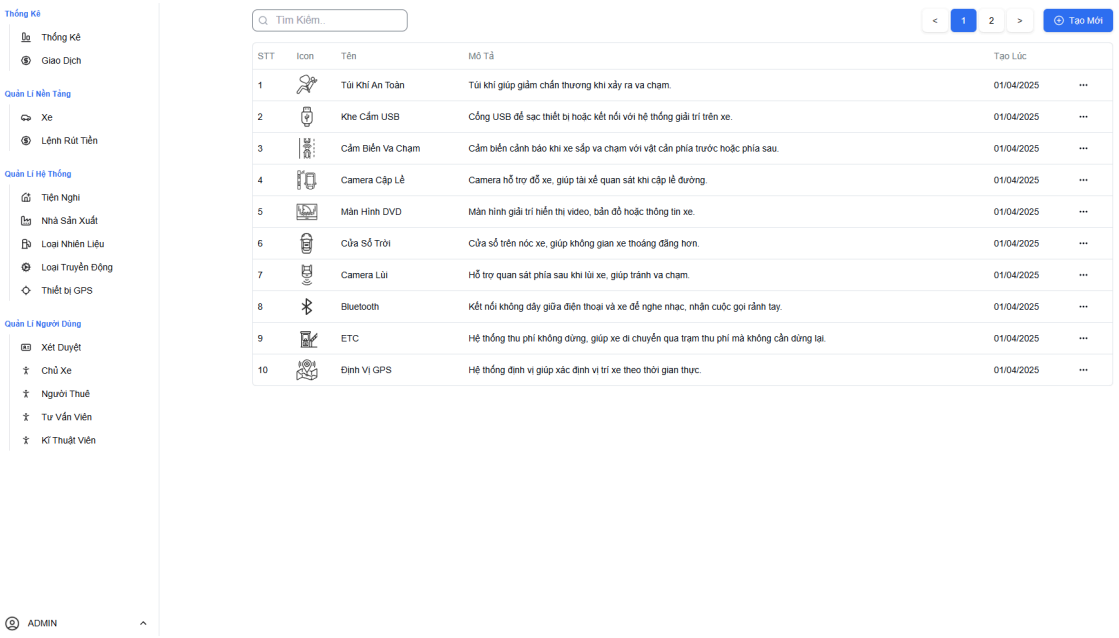
- **Function Details:** When logged in, a consultant sees their monthly KPIs including salary, completed inspections, ongoing inspections, and recent activities.

3.3 Manage Amenities

3.3.1 View Amenities

- **Function Trigger:** When clicking the “Tiện Nghi” button.
- **Function Description:**
    + **Actors:** Admin
    + **Purpose:** Observe available amenities.
    + **Interface:** Table
- **Data Processing:**
    + **Data Collect:** Data is collected from the primary database.
    + **Data Computation:** Data is filtered through multiple layers.
    + **Data Binding:** Data is displayed on the screen.

- **Screen Layout:**

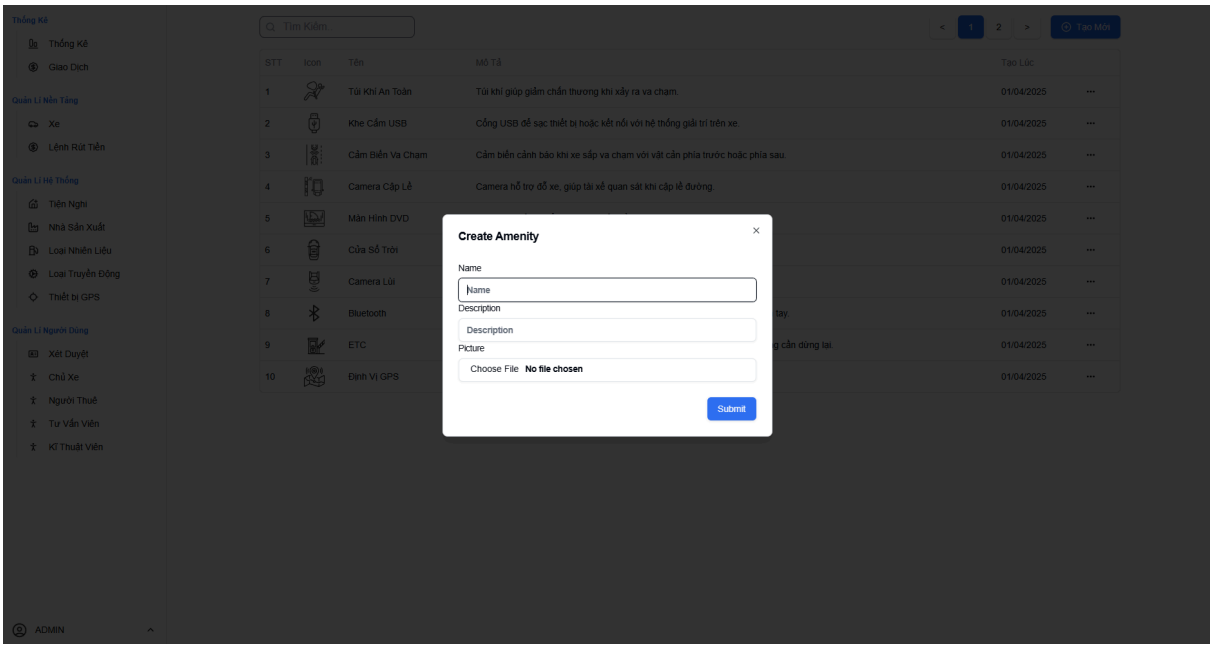


- **Function Details:** When the “Tiện Nghi” button is clicked, the admin can view all amenities with details such as icon image, name, description, and created date.

3.3.2 Create Amenities

- **Function Trigger:** When clicking the “Tạo” button.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Create a new amenity for the platform.
  - **Interface:** Form interface (modal or dedicated page)
- **Data Processing:**
  - **Receive Data Input:** The system receives the admin’s input.
  - **Data Computation:** Input is validated and processed.
  - **Data Storage:** Data is stored in the primary storage.

- **Screen Layout:**

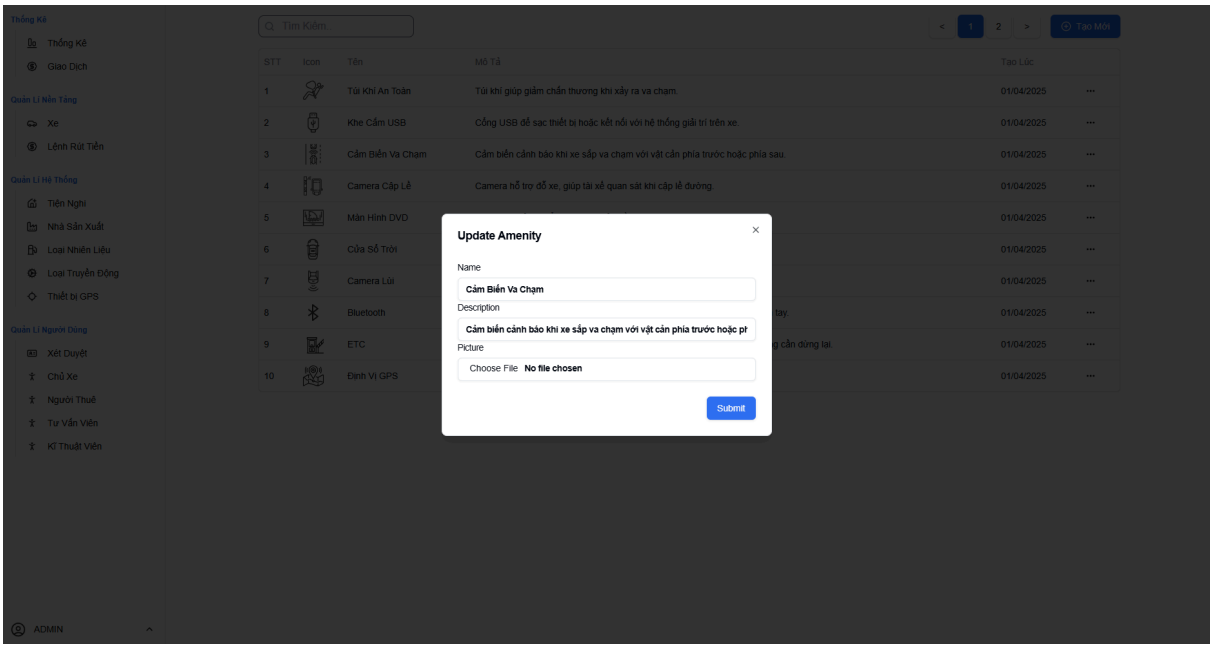


- **Function Details:** On clicking “Tạo”, the admin enters all required information to add a new amenity.

3.3.3 Update Amenity

- **Function Trigger:** When clicking the three-dot button, then selecting “Cập nhật”.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Update an existing amenity.
  - **Interface:** Modal
- **Data Processing:**
  - **Receive Data Input:** The system receives the updated input.
  - **Data Computation:** Input is validated.
  - **Data Storage:** Updated information is stored in the primary storage.

- **Screen Layout:**

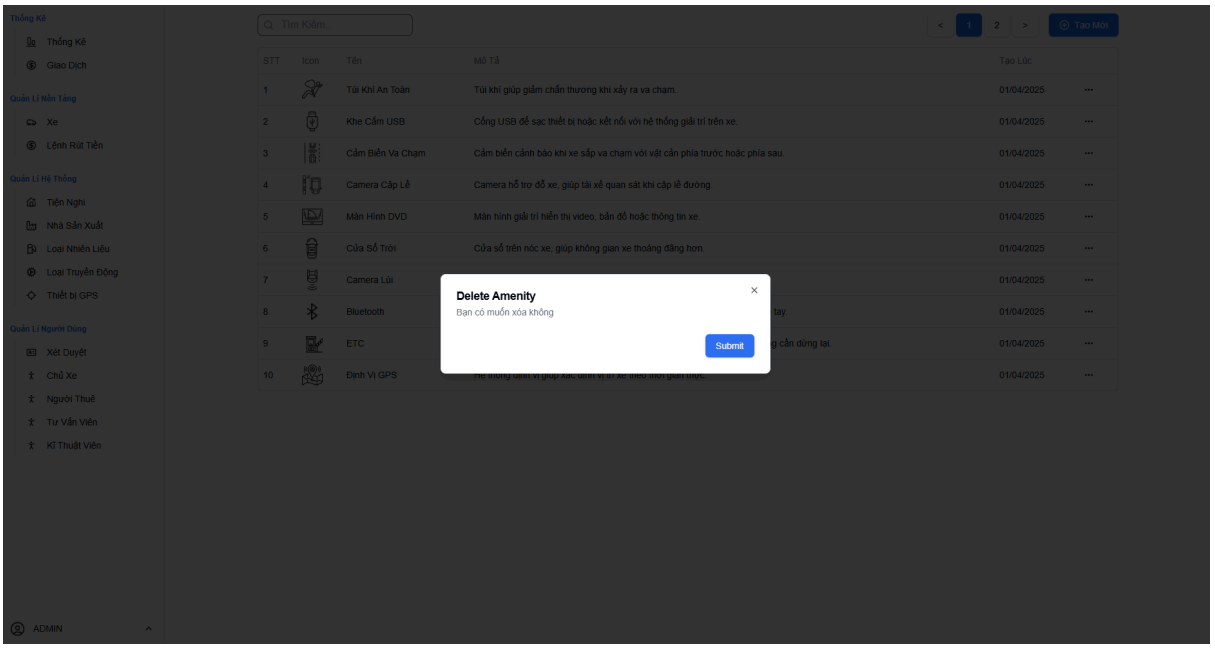


- **Function Details:** When “Cập nhật” is selected, the admin modifies the amenity details. All fields are required.

3.3.4 Delete Amenity

- **Function Trigger:** When clicking the three-dot button, then selecting “Xóa”.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Delete an existing amenity.
  - **Interface:** Modal
- **Data Processing:**
  - **Data Computation:** The system validates the existence of the amenity.
  - **Data Storage:** The deletion is recorded in the primary storage.

- **Screen Layout:**



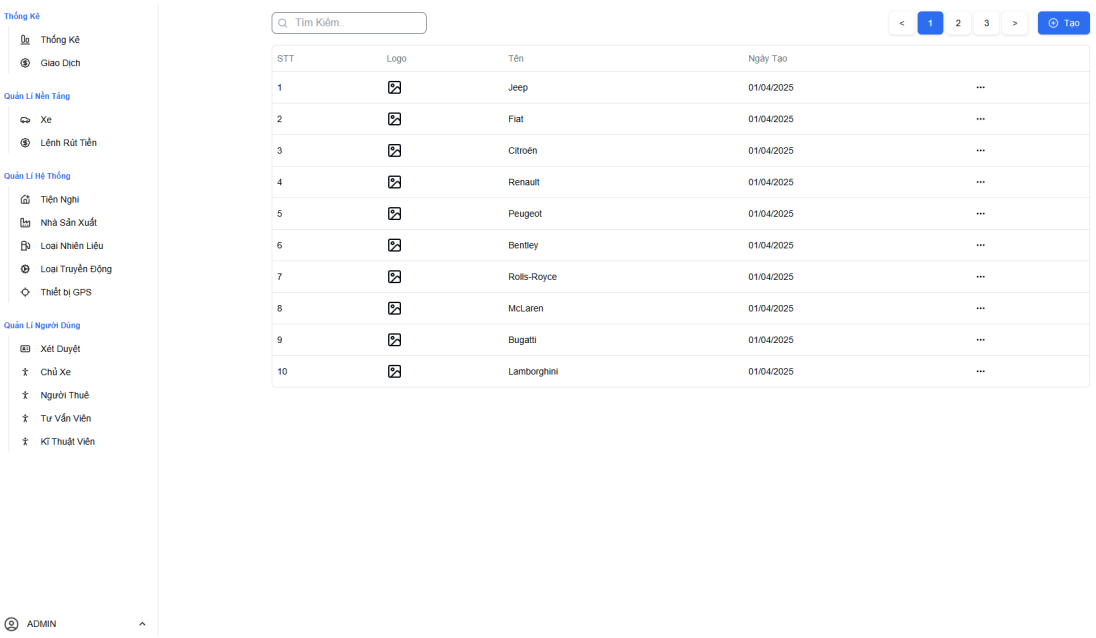
- **Function Details:** On clicking “Xóa”, the admin confirms the deletion by clicking the “Submit” button.

3.4 Manage Manufacturers

3.4.1 View Manufacturers

- **Function Trigger:** When clicking the “Nhà sản xuất” button.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Observe available manufacturers.
  - **Interface:** Table
- **Data Processing:**
  - **Data Collect:** Data is collected from the primary database.
  - **Data Computation:** Data is filtered through multiple layers.
  - **Data Binding:** Data is displayed on the screen.

- **Screen Layout:**

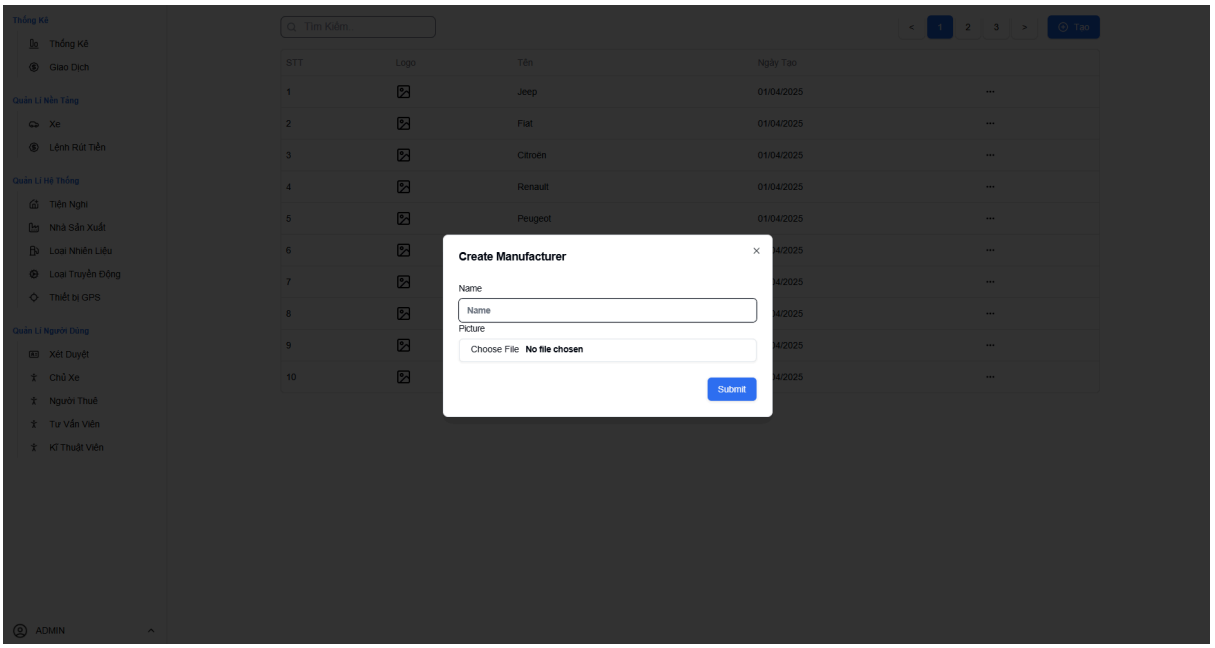


- **Function Details:** On clicking “Nhà sản xuất”, the admin can view all manufacturers with details like icon, name, and creation date.

3.4.2 Create Manufacturers

- **Function Trigger:** When clicking the “Tạo” button.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Create a new manufacturer.
  - **Interface:** Modal
- **Data Processing:**
  - **Receive Data Input:** The system receives the admin’s input.
  - **Data Computation:** Input is validated.
  - **Data Storage:** New manufacturer data is stored.

- **Screen Layout:**

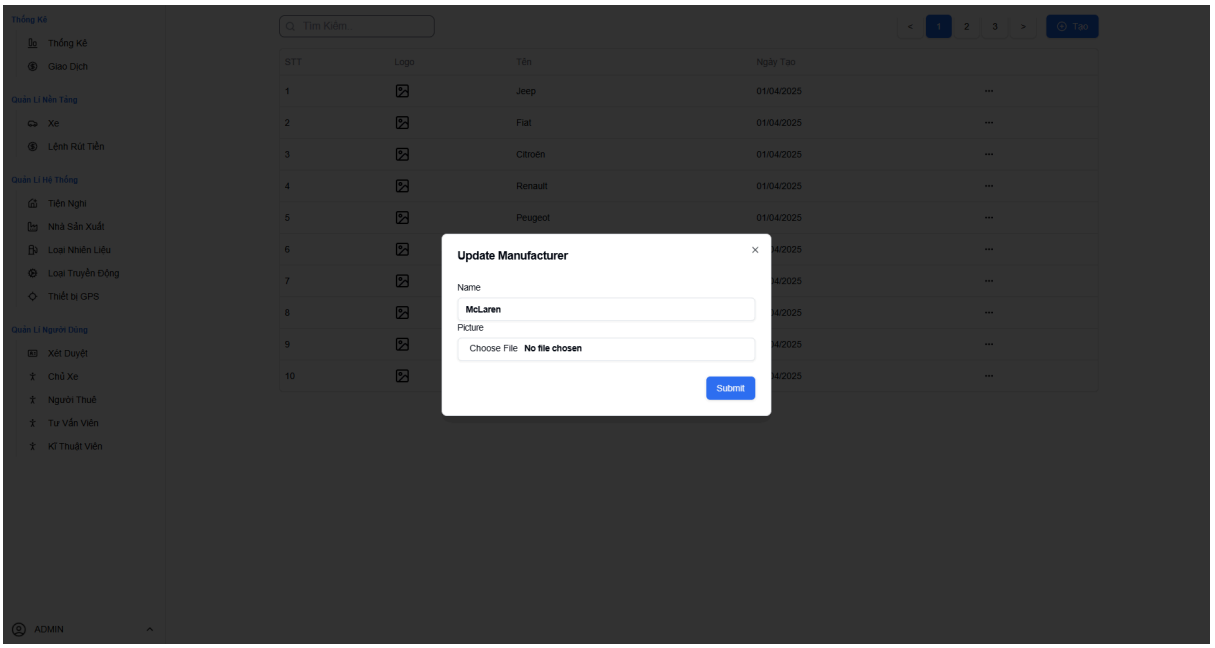


- **Function Details:** On clicking “Tạo”, the admin enters all required information to create a new manufacturer.

3.4.3 Update Manufacturer

- **Function Trigger:** When clicking the three-dot button, then selecting “Cập nhật”.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Update an existing manufacturer.
  - **Interface:** Modal
- **Data Processing:**
  - **Receive Data Input:** The system receives updated data.
  - **Data Computation:** Input is validated.
  - **Data Storage:** Updated data is stored in the primary storage.

- **Screen Layout:**

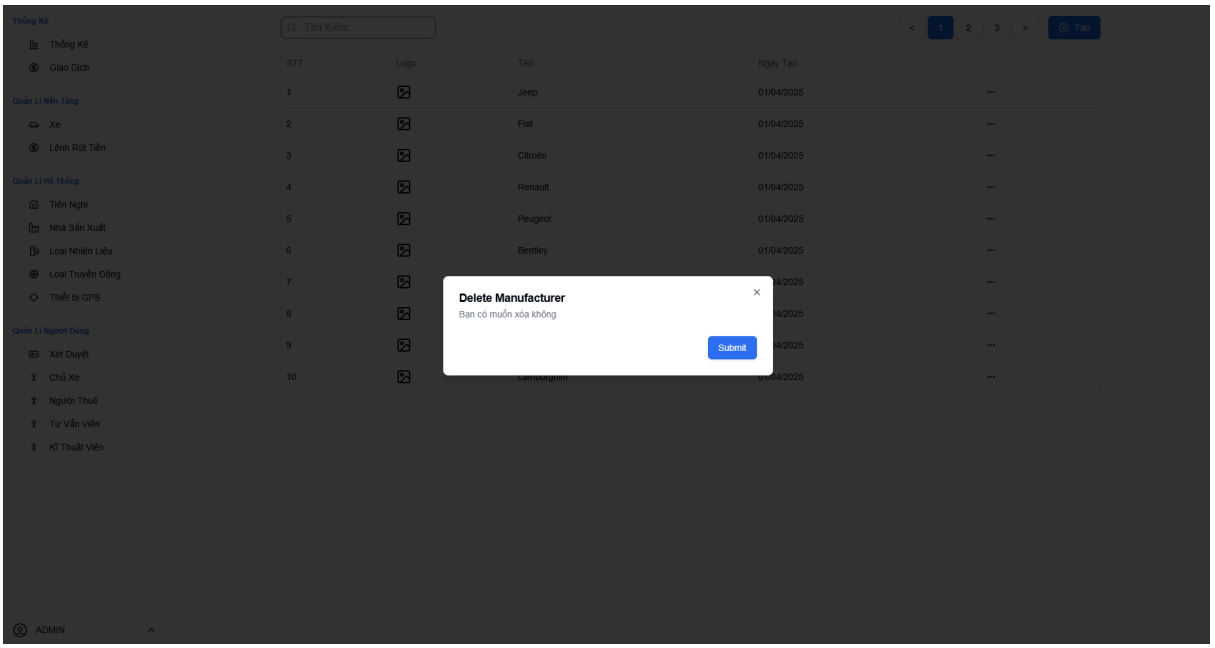


- **Function Details:** On selecting “Cập nhật”, the admin updates manufacturer details. All required fields must be provided.

3.4.4 Delete Manufacturer

- **Function Trigger:** When clicking the three-dot button, then selecting “Xóa”.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Delete an existing manufacturer.
  - **Interface:** Modal
- **Data Processing:**
  - **Data Computation:** The system verifies the existence of the manufacturer.
  - **Data Storage:** Deletion is recorded in the primary storage.

- **Screen Layout:**



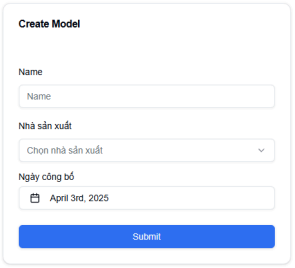
- **Function Details:** On clicking “Xóa”, the admin confirms the deletion by clicking the “Submit” button.

3.5 Manage Models (Dòng xe)

3.5.1 View Models (Dòng xe)

- **Function Trigger:** When clicking the “Dòng xe” button from the manufacturer page.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Observe all available models.
  - **Interface:** Table
- **Data Processing:**
  - **Data Collect:** Data is collected from the primary database.
  - **Data Computation:** Data is filtered and validated.
  - **Data Binding:** Data is rendered on the screen.

- **Screen Layout:**

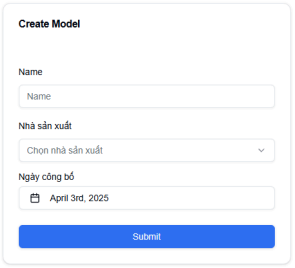


- **Function Details:** On clicking “Dòng xe”, the admin can view model details such as icon, name, and creation date.

3.5.2 Create Models (Dòng xe)

- **Function Trigger:** When clicking the “Tạo” button.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Create a new model.
  - **Interface:** Modal
- **Data Processing:**
  - **Receive Data Input:** The system receives admin’s input.
  - **Data Computation:** Input is validated.
  - **Data Storage:** New model data is stored.

- **Screen Layout:**

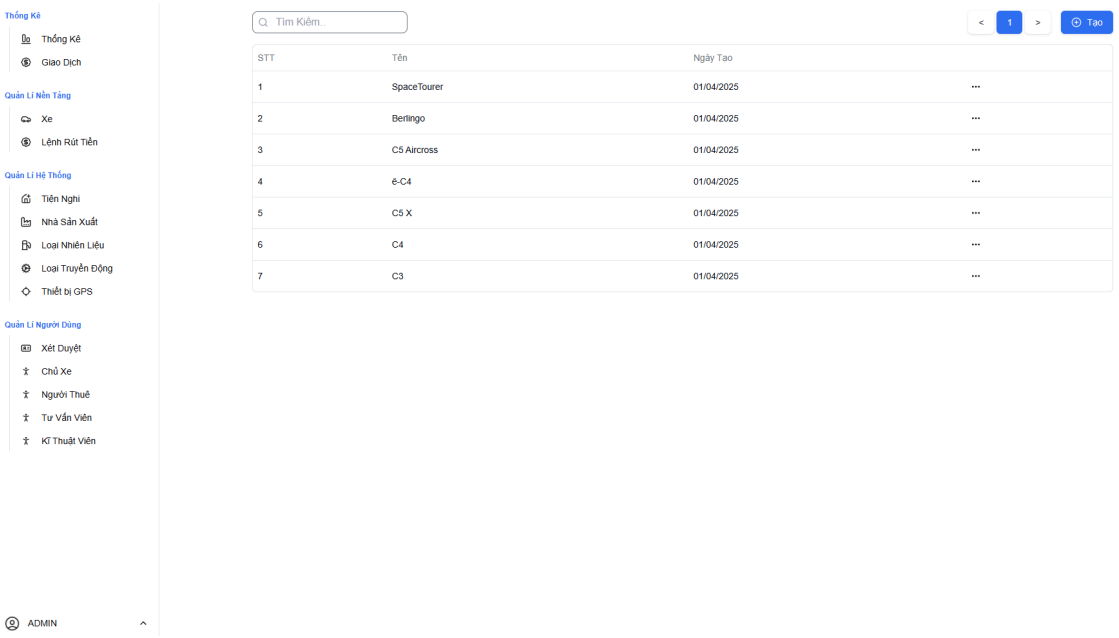


- **Function Details:** On clicking “Tạo”, the admin provides all required details to create a new model.

3.5.3 Update Model (Dòng xe)

- **Function Trigger:** When clicking the three-dot button, then selecting “Cập nhật”.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Update an existing model.
  - **Interface:** Modal
- **Data Processing:**
  - **Receive Data Input:** The system receives updated model data.
  - **Data Computation:** Modifications are validated.
  - **Data Storage:** Updated data is stored.

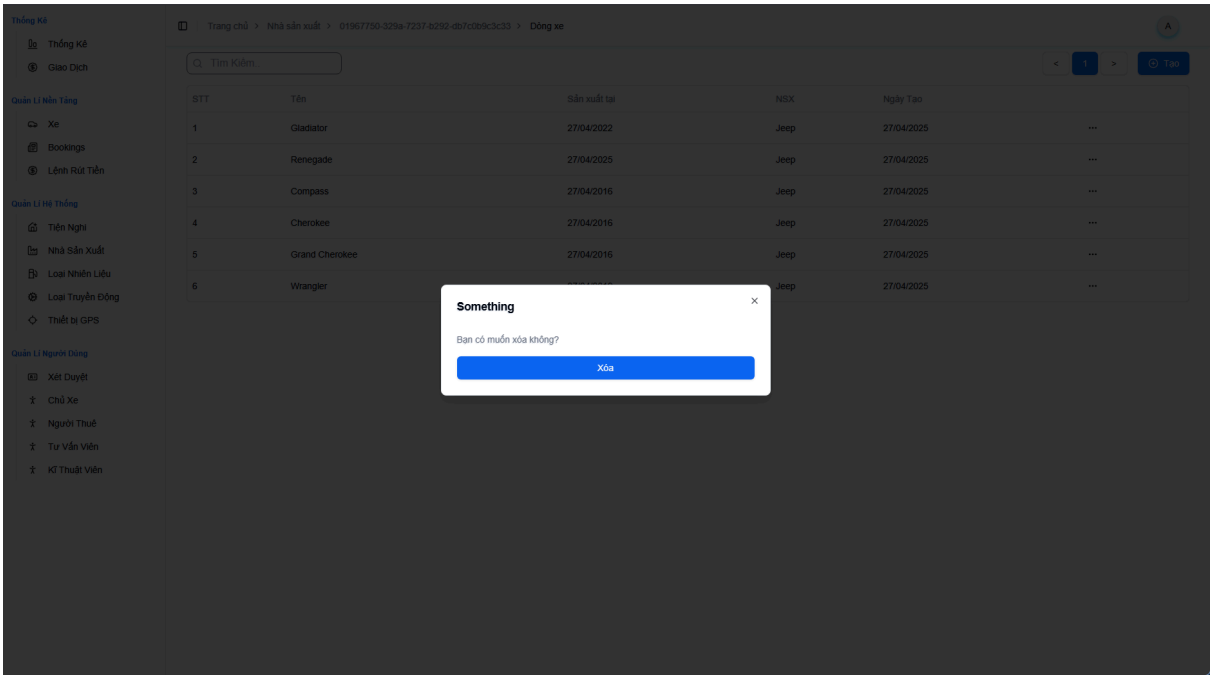
- **Screen Layout:**



- **Function Details:** On selecting “Cập nhật”, the admin updates the model’s details. All fields are mandatory.

3.5.4 Delete Model (Dòng xe)

- **Function Trigger:** When clicking the three-dot button, then selecting “Xóa”.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Delete an existing model.
  - **Interface:** Modal
- **Data Processing:**
  - **Data Computation:** The system verifies deletion eligibility.
  - **Data Storage:** The deletion action is recorded.
- **Screen Layout:**



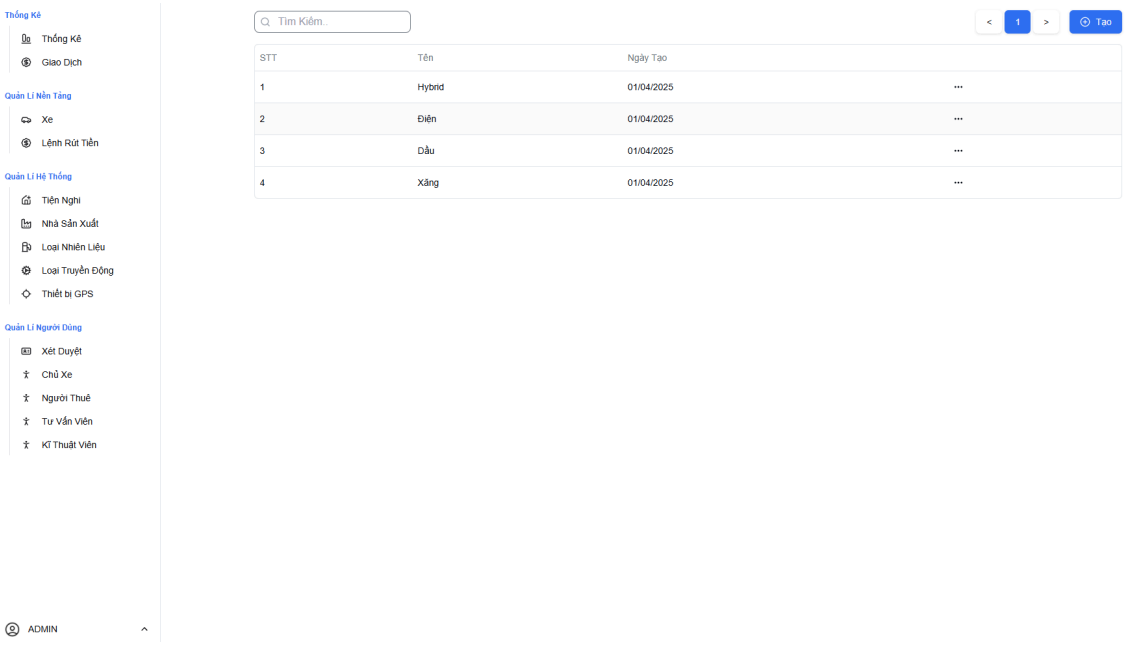
- **Function Details:** On clicking “Xóa”, the admin confirms deletion by clicking the “Submit” button.

3.6 Manage Fuel Types (Loại Nhiên Liệu)

3.6.1 View Fuel Types (Loại Nhiên Liệu)

- **Function Trigger:** When clicking the “Loại Nhiên Liệu” button.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Observe all available fuel types.
  - **Interface:** Table
- **Data Processing:**
  - **Data Collect:** Data is collected from the primary database.
  - **Data Computation:** Data is filtered and validated.
  - **Data Binding:** Data is rendered on the screen.

- **Screen Layout:**

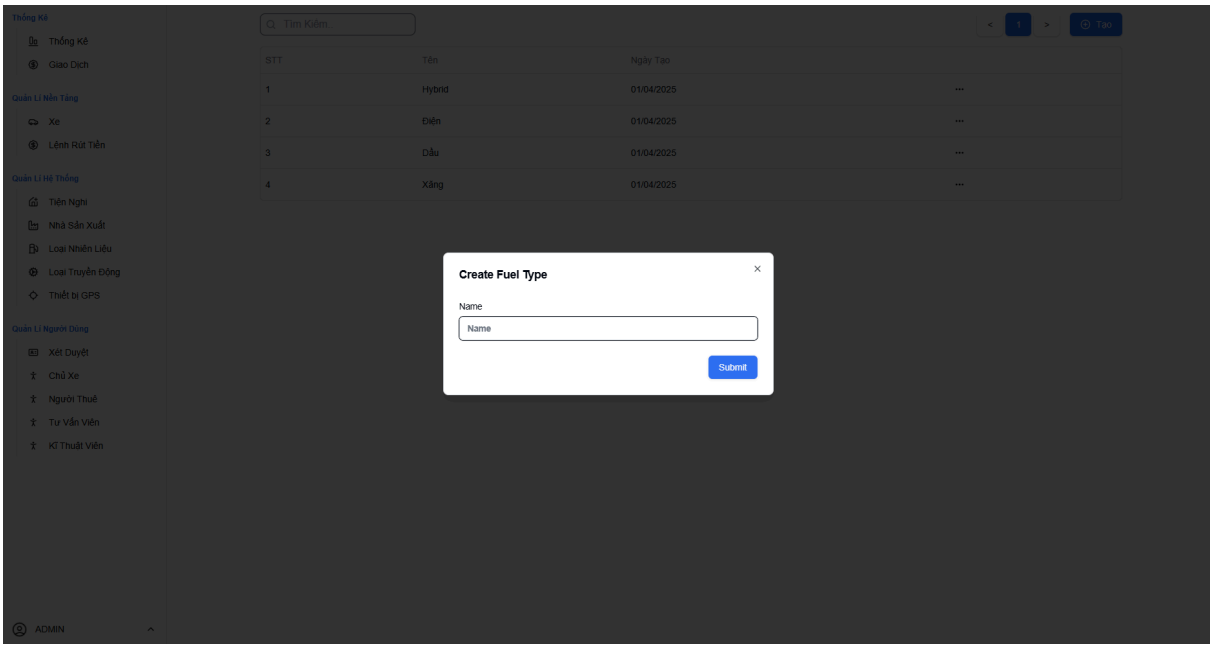


- **Function Details:** On clicking “Loại Nhiên Liệu”, the admin can view fuel types including details such as icon, name, and creation date.

3.6.2 Create Fuel Types (Loại Nhiên Liệu)

- **Function Trigger:** When clicking the “Tạo” button.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Create a new fuel type.
  - **Interface:** Modal
- **Data Processing:**
  - **Receive Data Input:** The system receives admin’s input.
  - **Data Computation:** Input is validated.
  - **Data Storage:** New fuel type data is stored.

- **Screen Layout:**

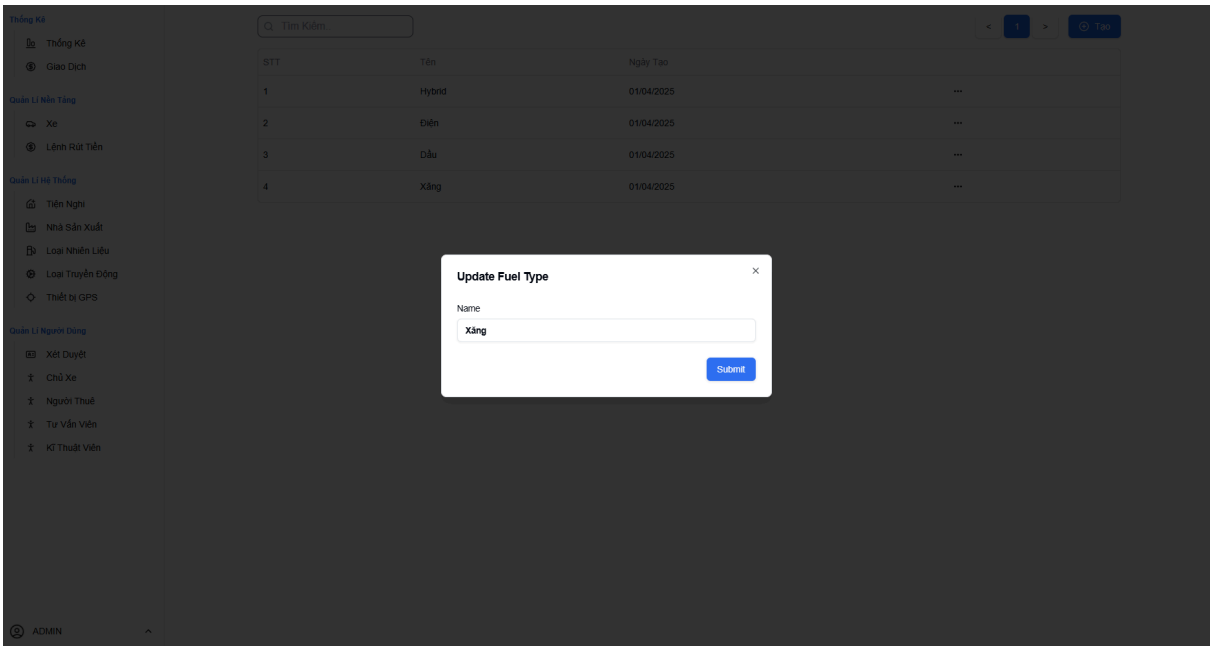


- **Function Details:** On clicking “Tạo”, the admin enters all required information to add a new fuel type.

3.6.3 Update Fuel Type (Loại Nhiên Liệu)

- **Function Trigger:** When clicking the three-dot button, then selecting “Cập nhật”.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Update an existing fuel type.
  - **Interface:** Modal
- **Data Processing:**
  - **Receive Data Input:** The system receives updated information.
  - **Data Computation:** Changes are validated.
  - **Data Storage:** Updated data is stored.

- **Screen Layout:**

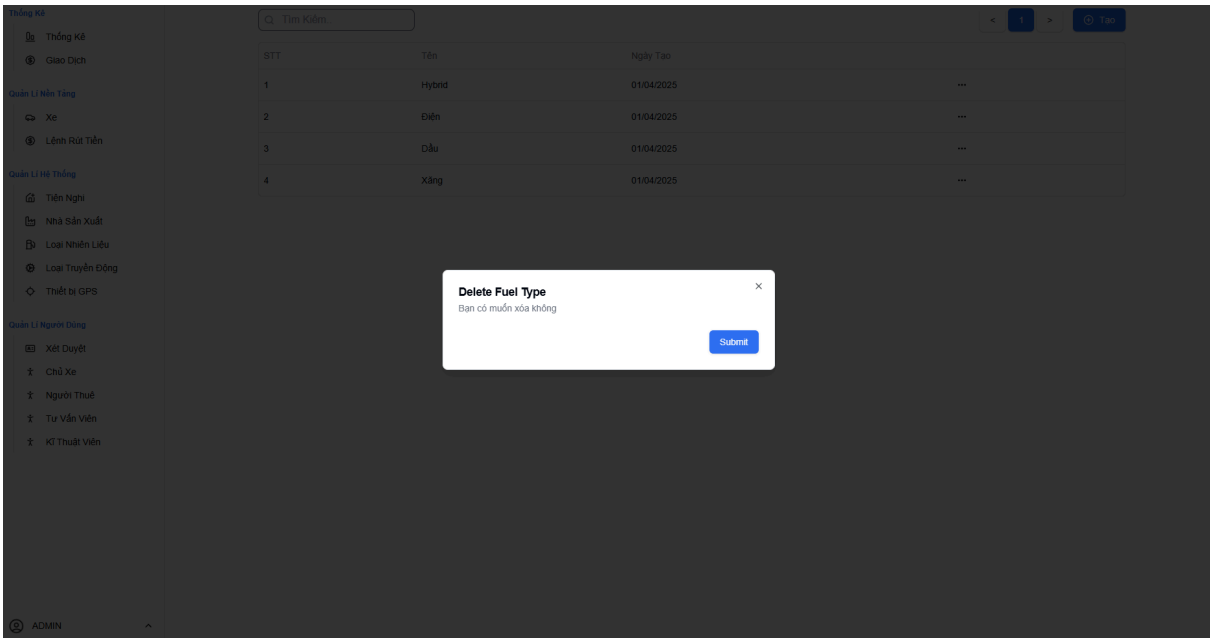


- **Function Details:** On selecting “Cập nhật”, the admin must provide all required fields to update the fuel type.

3.6.4 Delete Fuel Type (Loại Nhiên Liệu)

- **Function Trigger:** When clicking the three-dot button, then selecting “Xóa”.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Delete an existing fuel type.
  - **Interface:** Modal
- **Data Processing:**
  - **Data Computation:** The system validates the existence of the fuel type.
  - **Data Storage:** Deletion is recorded in the primary storage.

- **Screen Layout:**



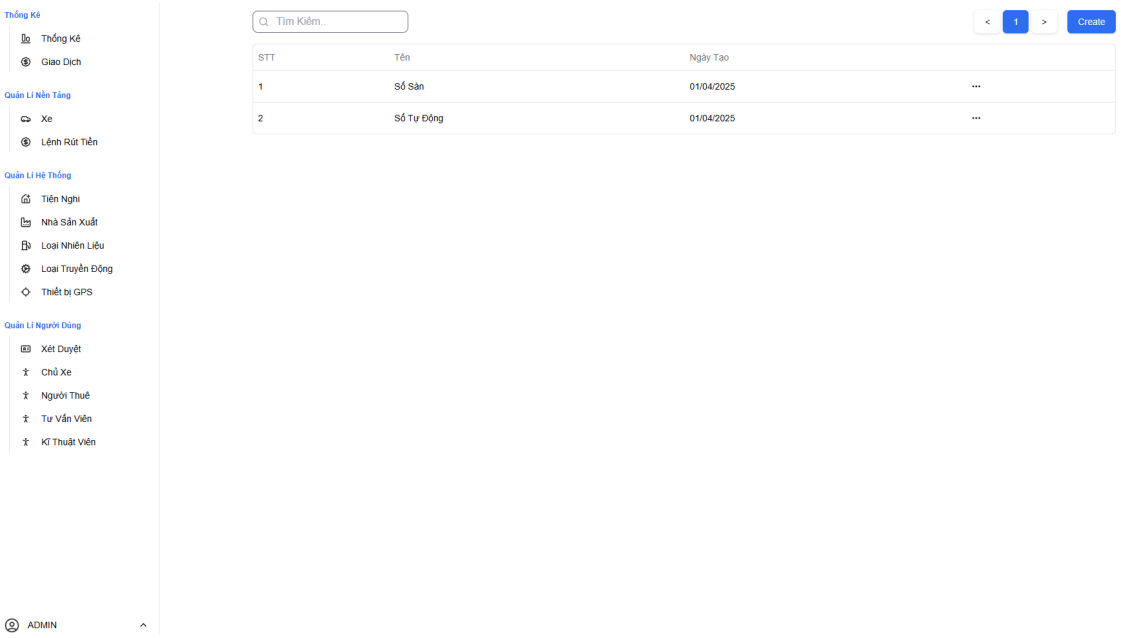
- **Function Details:** On clicking “Xóa”, the admin confirms the deletion by clicking the “Submit” button.

3.7 Manage Transmission Types (Loại Truyền Động)

3.7.1 View Transmission Types (Loại Truyền Động)

- **Function Trigger:** When clicking the “Loại Truyền Động” button.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Observe all available transmission types.
  - **Interface:** Table
- **Data Processing:**
  - **Data Collect:** Data is collected from the primary database.
  - **Data Computation:** Data is filtered and prepared.
  - **Data Binding:** Processed data is rendered on the screen.

- **Screen Layout:**

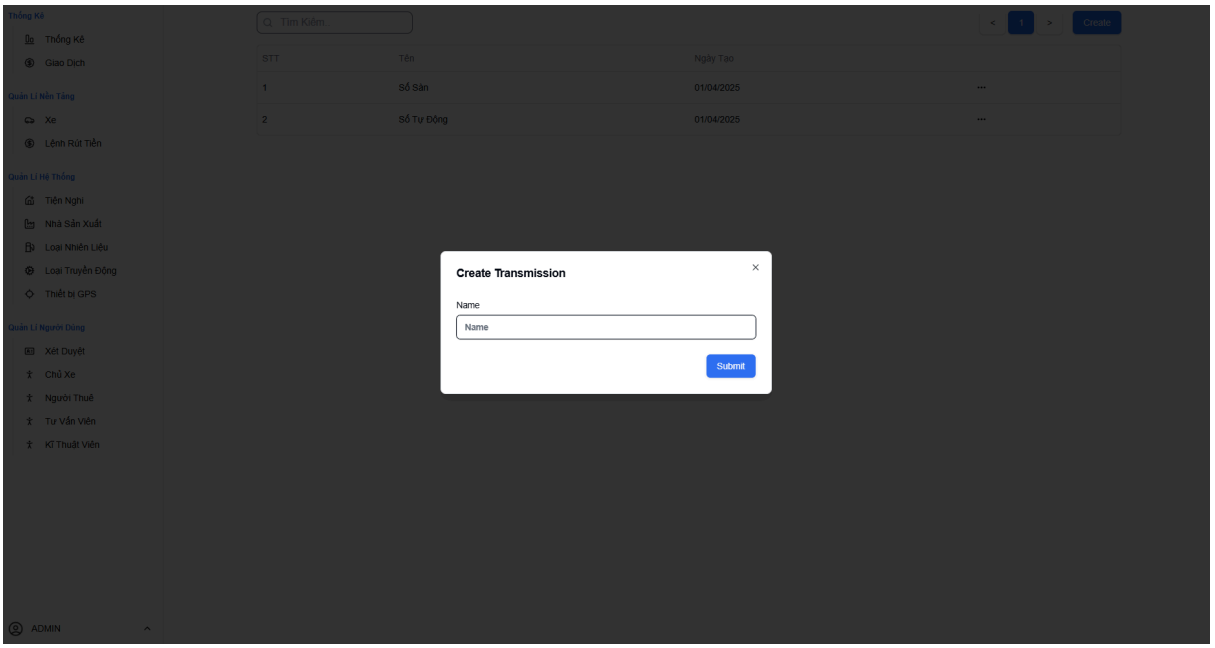


- **Function Details:** On clicking “Loại Truyền Động”, the admin can view transmission type details such as icon, name, and creation date.

3.7.2 Create Transmission Types (Loại Truyền Động)

- **Function Trigger:** When clicking the “Tạo” button.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Create a new transmission type.
  - **Interface:** Modal
- **Data Processing:**
  - **Receive Data Input:** The system receives admin’s input.
  - **Data Computation:** Input is validated.
  - **Data Storage:** New transmission type data is stored.

- **Screen Layout:**

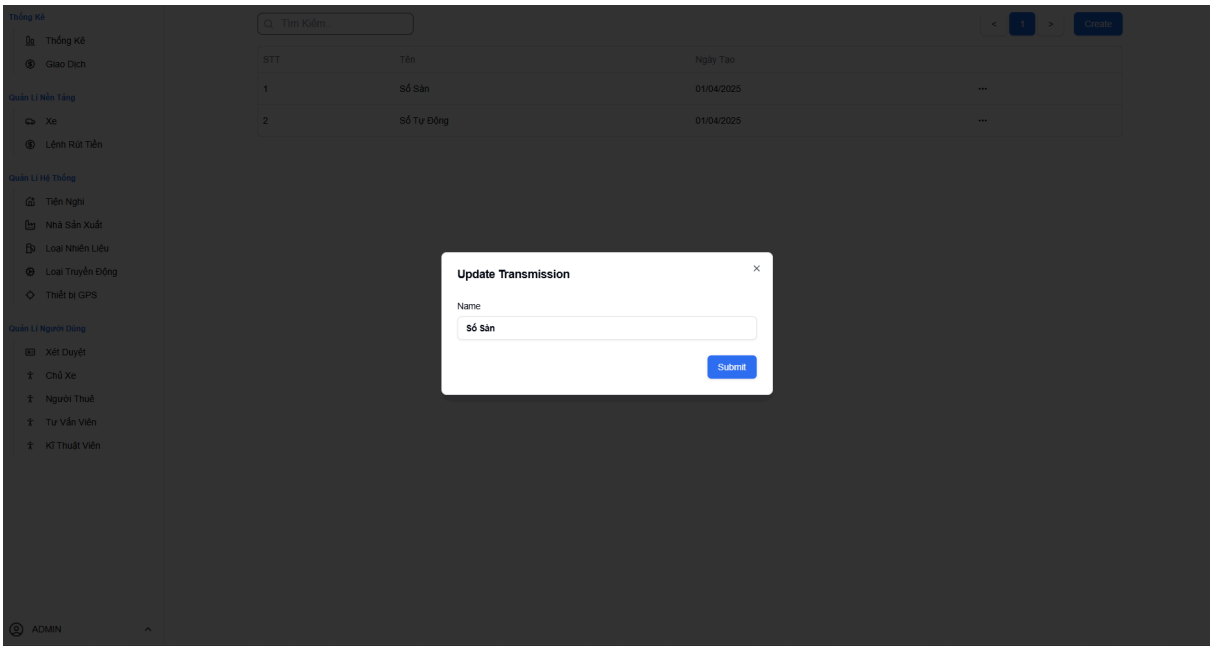


- **Function Details:** On clicking “Tạo”, the admin enters all required information to add a new transmission type.

3.7.3 Update Transmission Type (Loại Truyền Động)

- **Function Trigger:** When clicking the three-dot button, then selecting “Cập nhật”.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Update an existing transmission type.
  - **Interface:** Modal
- **Data Processing:**
  - **Receive Data Input:** The system receives updated details.
  - **Data Computation:** Changes are validated.
  - **Data Storage:** Updated data is stored in the primary storage.

- **Screen Layout:**

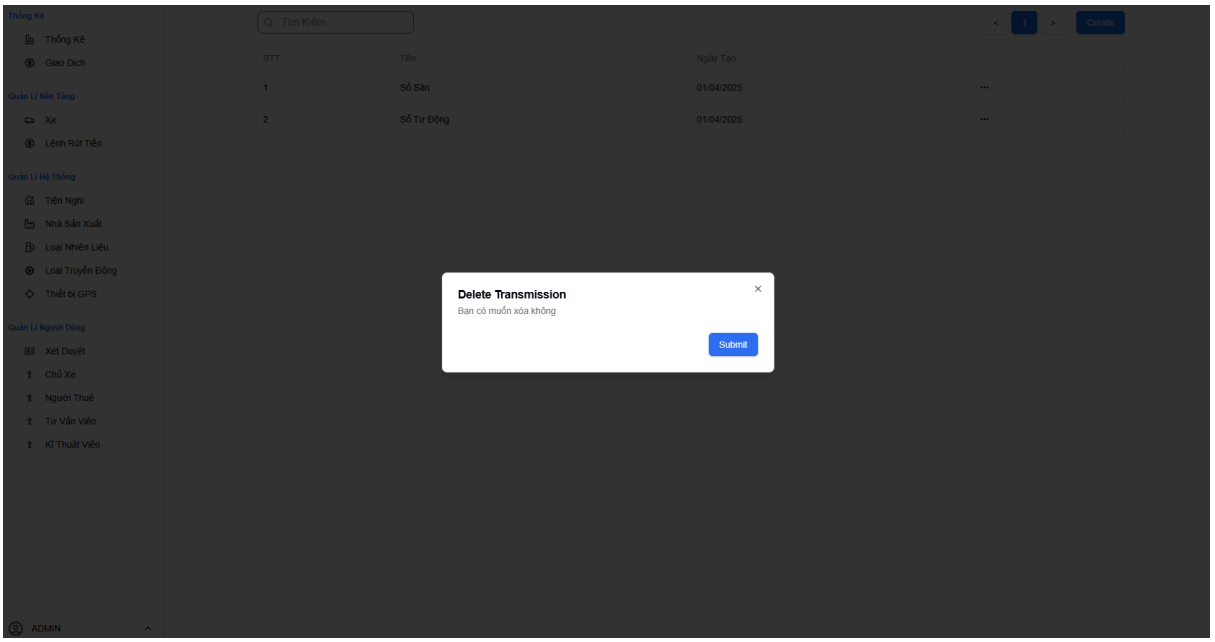


- **Function Details:** On selecting “Cập nhật”, the admin updates the transmission type details. All fields are required.

3.7.4 Delete Transmission Type (Loại Truyền Động)

- **Function Trigger:** When clicking the three-dot button, then selecting “Xóa”.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Delete an existing transmission type.
  - **Interface:** Modal
- **Data Processing:**
  - **Data Computation:** The system verifies deletion eligibility.
  - **Data Storage:** Deletion is recorded.

- **Screen Layout:**



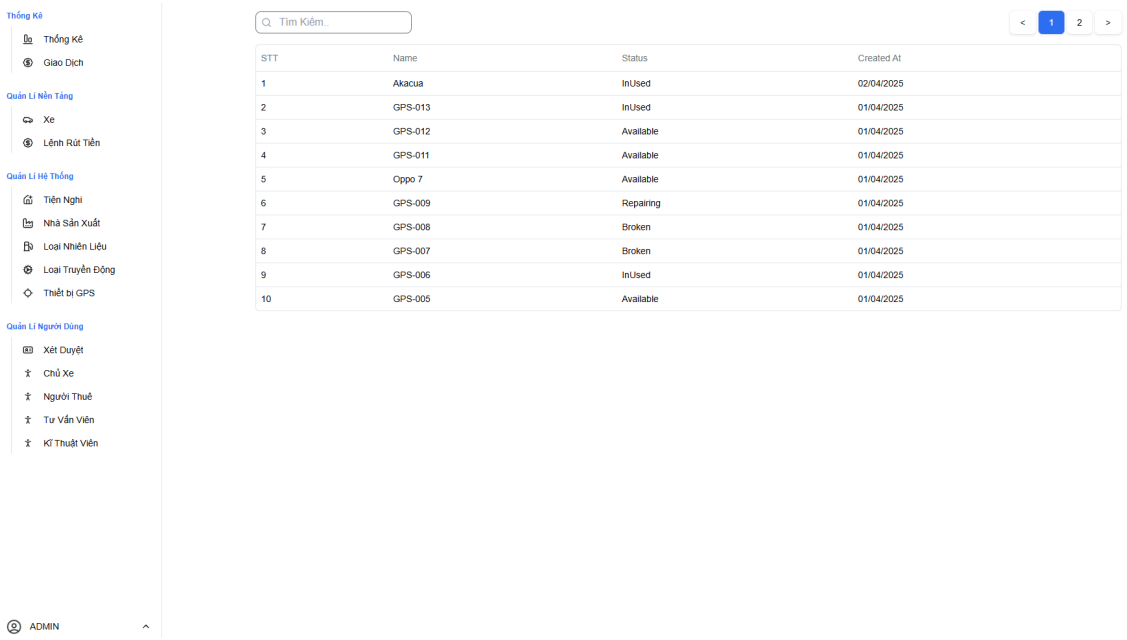
- **Function Details:** On clicking “Xóa”, the admin confirms deletion by clicking the “Submit” button.

3.8 Manage GPS Devices (Thiết Bị GPS)

3.8.1 View GPS Devices (Thiết Bị GPS)

- **Function Trigger:** When clicking the “Thiết Bị GPS” button.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Observe all configured GPS devices.
  - **Interface:** Table
- **Data Processing:**
  - **Data Collect:** Data is collected from the primary database.
  - **Data Computation:** Data is filtered and verified.
  - **Data Binding:** Data is rendered on the screen.

- **Screen Layout:**

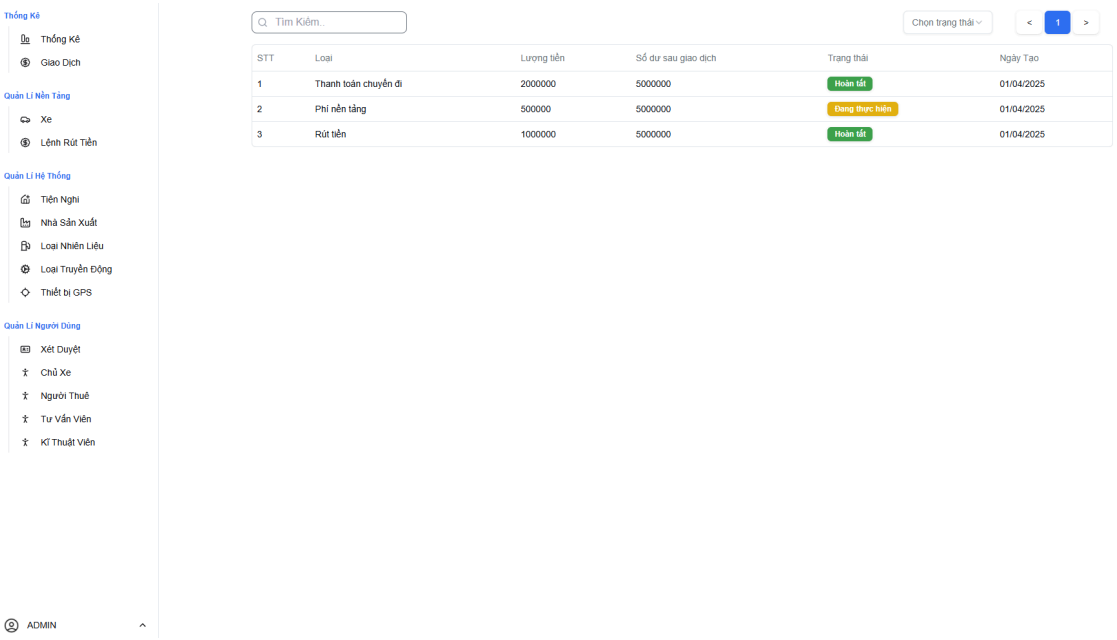


- **Function Details:** On clicking “Thiết Bị GPS”, the admin can view details such as device icon, name, and creation date.

3.9 View Transactions (Giao dịch)

- **Function Trigger:** When clicking the “Giao dịch” button.
- **Function Description:**
    + **Actors:** Admin
    + **Purpose:** Observe all recorded transactions.
    + **Interface:** Table
- **Data Processing:**
    + **Data Collect:** Data is collected from the primary database.
    + **Data Computation:** Data is filtered and validated.
    + **Data Binding:** Data is rendered on the screen.

- **Screen Layout:**



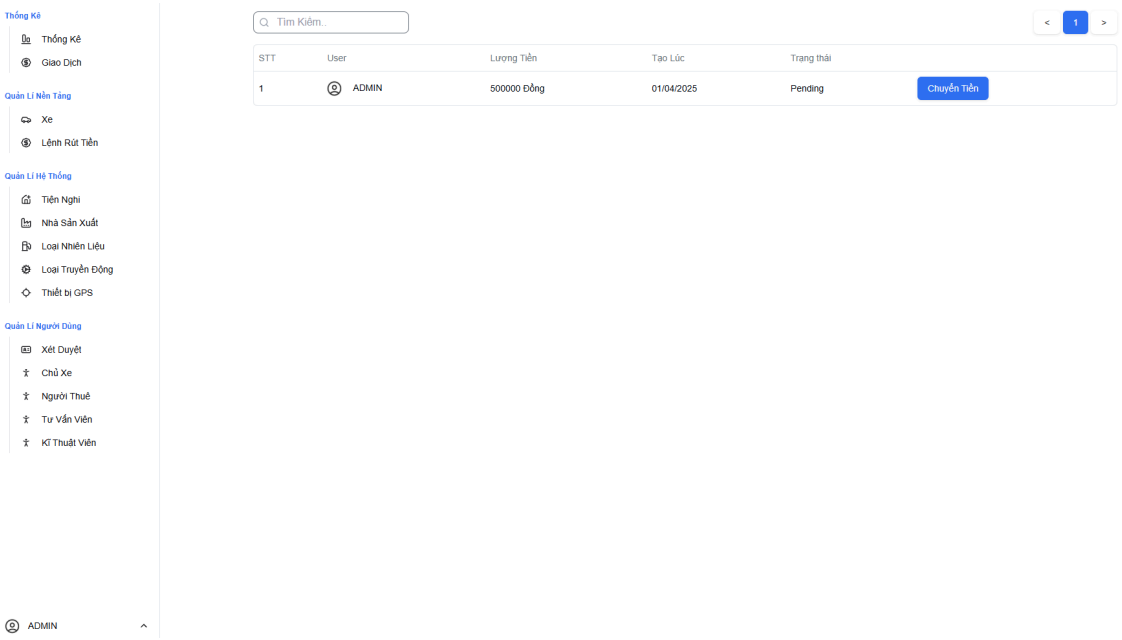
- **Function Details:** On clicking “Giao dịch”, the admin can view transaction details such as ID, amount, date, and status.

3.10 Process Withdrawal Requests (Lệnh rút tiền)

3.10.1 View Withdrawal Request

- **Function Trigger:** When clicking the “Lệnh rút tiền” button.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Observe all withdrawal requests.
  - **Interface:** Table
- **Data Processing:**
  - **Data Collect:** Data is collected from the primary database.
  - **Data Computation:** Data is filtered and validated.
  - **Data Binding:** Data is rendered on the screen.

- **Screen Layout:**

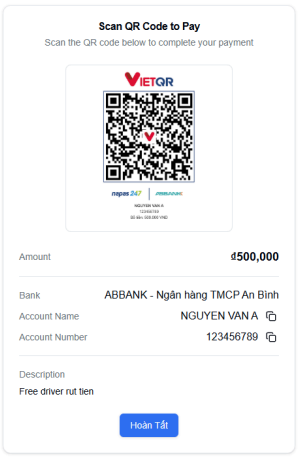


- **Function Details:** On clicking “Lệnh rút tiền”, the admin can view details such as request ID, amount, requester info, and request date.

3.10.2 Process Withdraw Request

- **Function Trigger:** When clicking the “Chuyển tiền” button.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Process withdrawal requests.
  - **Interface:** Card
- **Data Processing:**
  - **Data Collect:** Data is collected from the primary database.
  - **Data Computation:** Data is filtered and validated.
  - **Data Binding:** Processed data is displayed.

- **Screen Layout:**



- **Function Details:** On clicking “Chuyển tiền”, the admin can view transaction details including QR Card and recipient information.

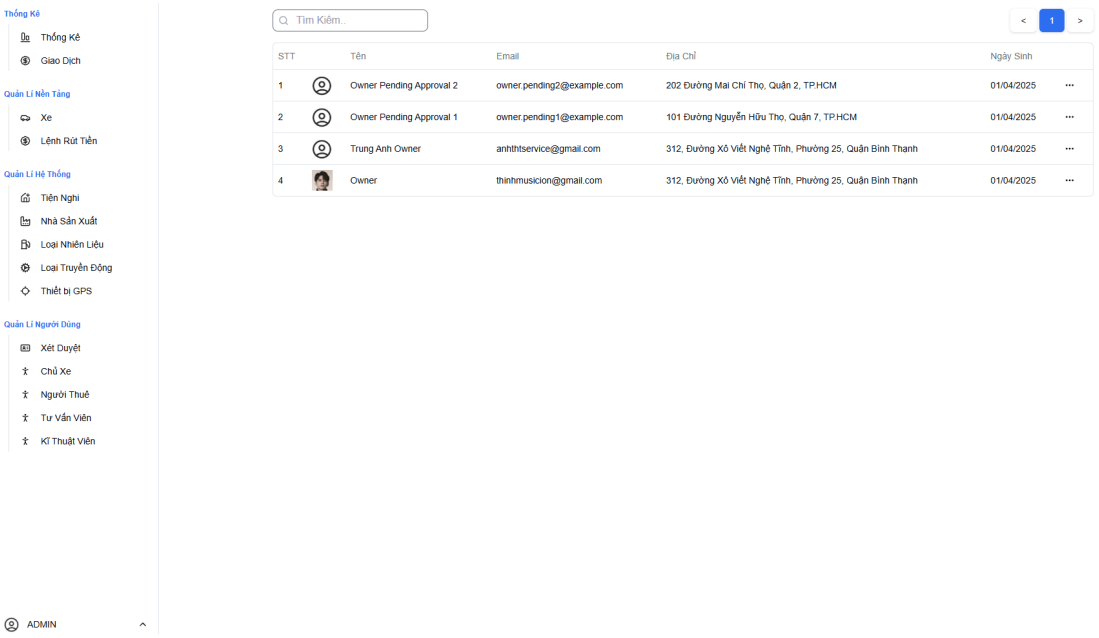
3.10.3 Update Withdraw Request

- **Function Trigger:** When clicking the “Hoàn tất” button.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Confirm completion of a withdrawal request.
  - **Interface:** Modal
- **Data Processing:**
  - **Receive Data Input:** The system receives the admin’s input.
  - **Data Computation:** Input is validated.
  - **Data Storage:** The updated withdrawal request is saved and marked as completed.
- **Screen Layout:**
- **Function Details:** On clicking “Hoàn tất”, the admin fills in the checkout form with proofs to confirm completion.

3.11 View Owners (Chủ xe)

- **Function Trigger:** When clicking the “Chủ xe” button.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Observe all vehicle owners registered on the platform.
  - **Interface:** Table
- **Data Processing:**
  - **Data Collect:** Data is collected from the primary database.
  - **Data Computation:** Data is filtered and prepared.
  - **Data Binding:** Processed data is rendered on the screen.

- **Screen Layout:**

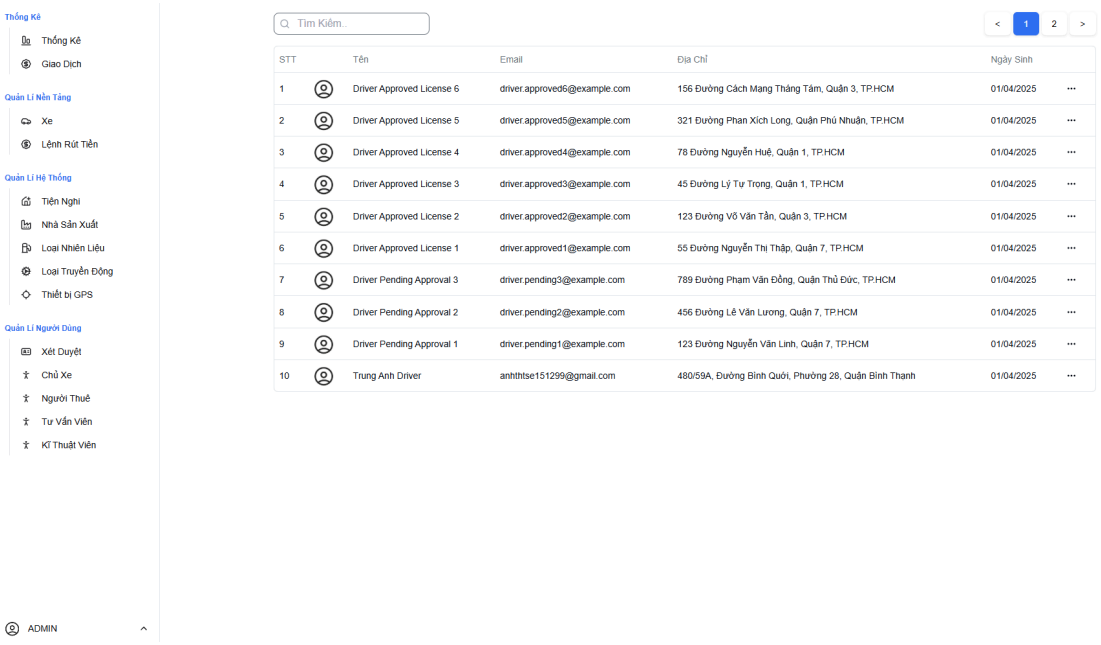


- **Function Details:** On clicking “Chủ xe”, the admin can view details such as owner name, contact information, and registration date.

3.12 View Drivers (Người Thuê)

- **Function Trigger:** When clicking the “Người Thuê” button.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Observe all drivers registered on the platform.
  - **Interface:** Table
- **Data Processing:**
  - **Data Collect:** Data is collected from the primary database.
  - **Data Computation:** Data is filtered and validated.
  - **Data Binding:** Data is rendered on the screen.

- **Screen Layout:**



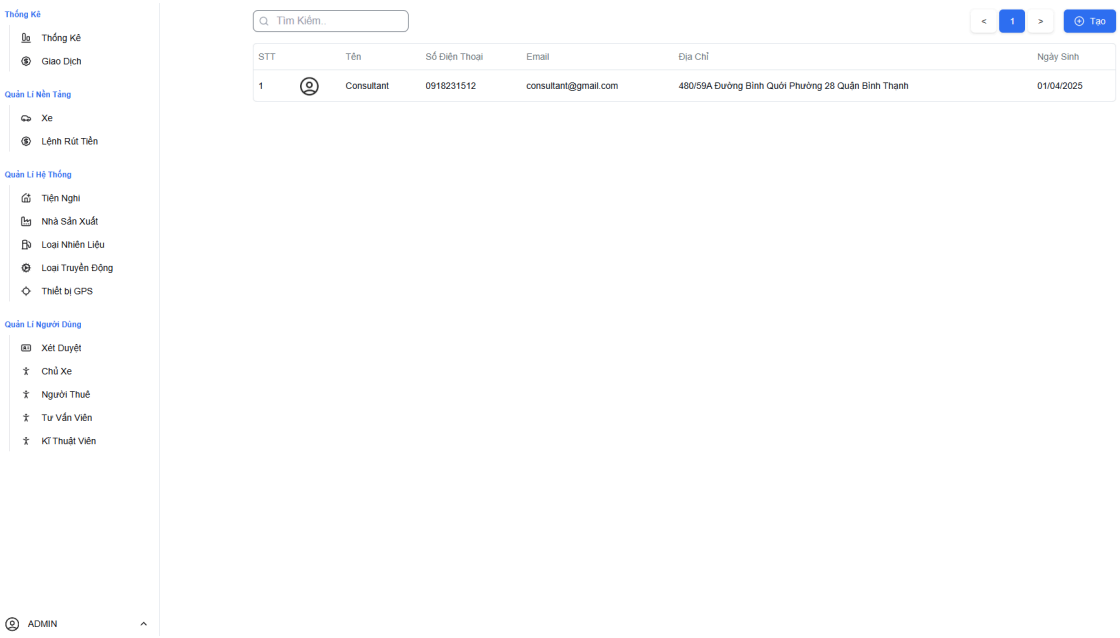
- **Function Details:** On clicking “Người Thuê”, the admin can view details such as driver name, license information, and registration date.

3.13 Manage Consultants (Tư vấn viên)

3.13.1 View Consultants

- **Function Trigger:** When clicking the “Tư vấn viên” button.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Observe all active consultants on the platform.
  - **Interface:** Table
- **Data Processing:**
  - **Data Collect:** Data is collected from the primary database.
  - **Data Computation:** Data is filtered and validated.
  - **Data Binding:** Data is rendered on the screen.

- **Screen Layout:**

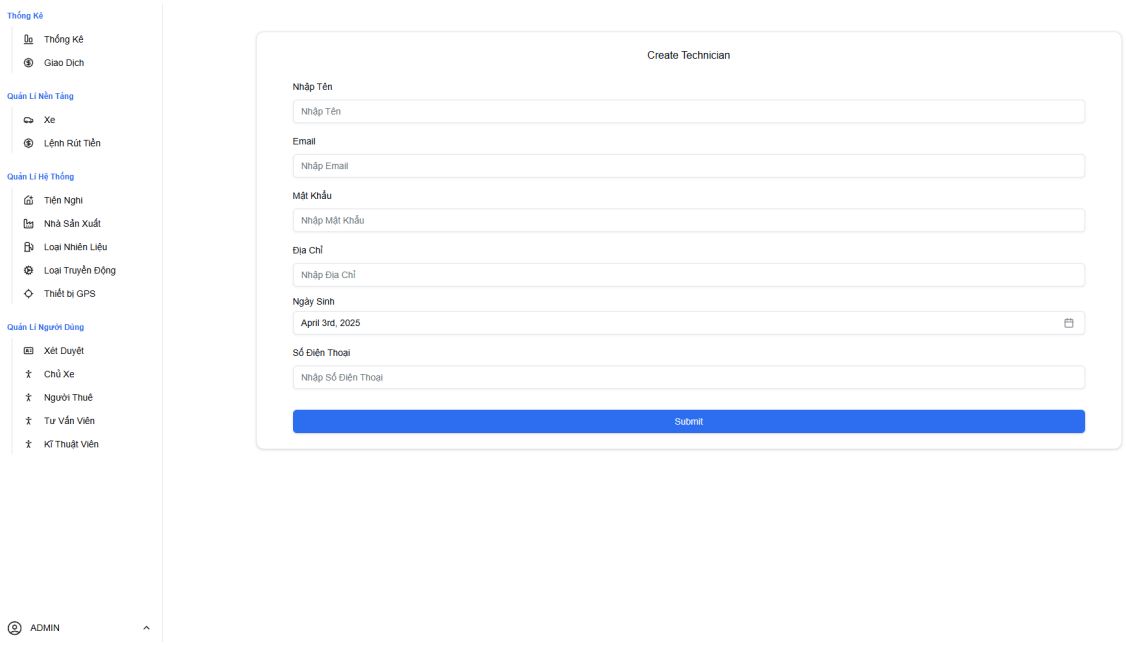


- **Function Details:** On clicking “Tư vấn viên”, the admin can view consultant details such as name, area of expertise, and registration date.

3.13.2 Create Consultants

- **Function Trigger:** When clicking the “Tạo” button.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Create a new consultant.
  - **Interface:** Modal
- **Data Processing:**
  - **Receive Data Input:** The system receives the admin’s input.
  - **Data Computation:** Input is validated.
  - **Data Storage:** New consultant data is stored.

- **Screen Layout:**



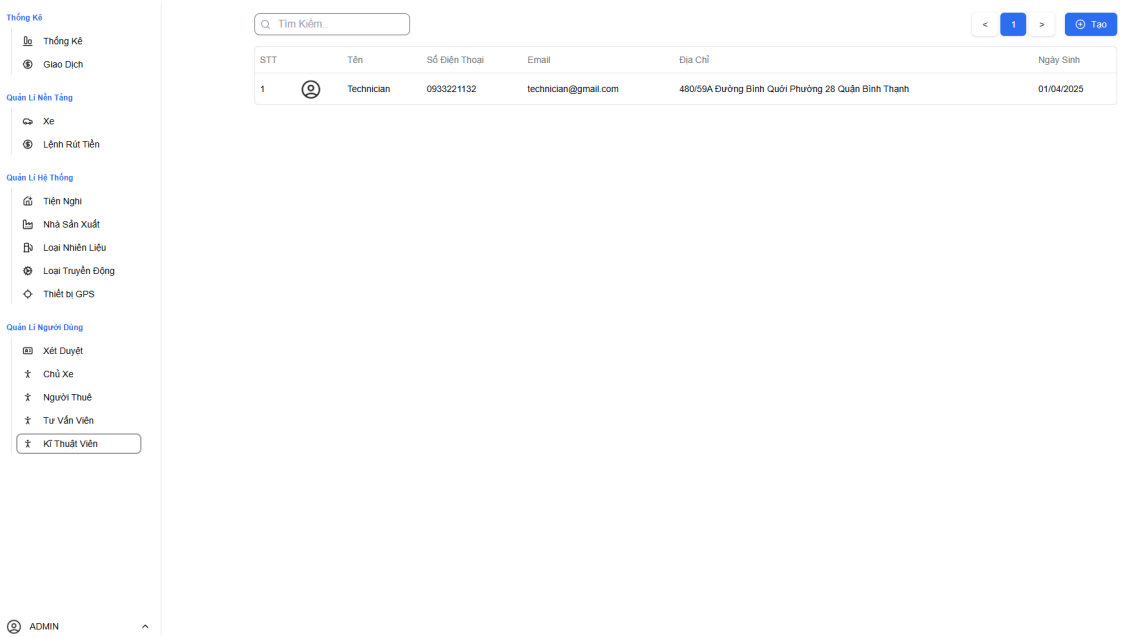
- **Function Details:** On clicking “Tạo”, the admin enters all required details to create a new consultant.

3.14 Manage Technicians (Kĩ thuật viên)

3.14.1 View Technicians

- **Function Trigger:** When clicking the “Kĩ thuật viên” button.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Observe all technicians available on the platform.
  - **Interface:** Table
- **Data Processing:**
  - **Data Collect:** Data is collected from the primary database.
  - **Data Computation:** Data is filtered and validated.
  - **Data Binding:** Data is rendered on the screen.

- **Screen Layout:**

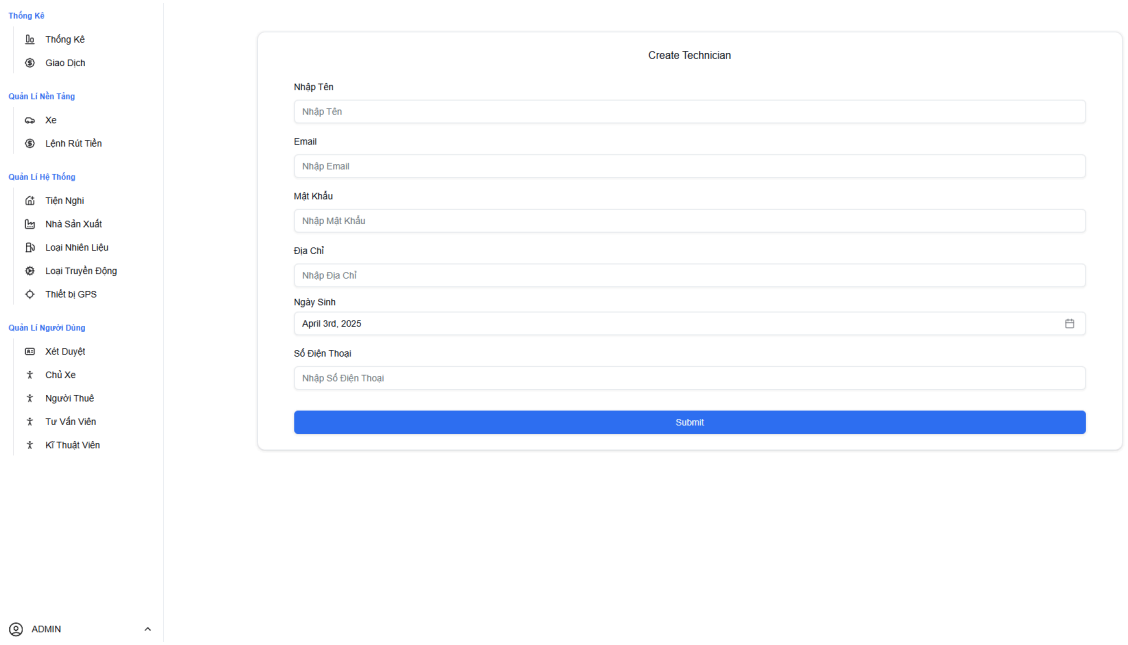


- **Function Details:** On clicking “Kĩ thuật viên”, the admin can view technician details such as name, expertise, and registration date.

3.14.2 Create Technicians

- **Function Trigger:** When clicking the “Tạo” button.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Create a new technician.
  - **Interface:** Modal
- **Data Processing:**
  - **Receive Data Input:** The system receives the admin’s input.
  - **Data Computation:** Input is validated.
  - **Data Storage:** New technician data is stored.

- **Screen Layout:**

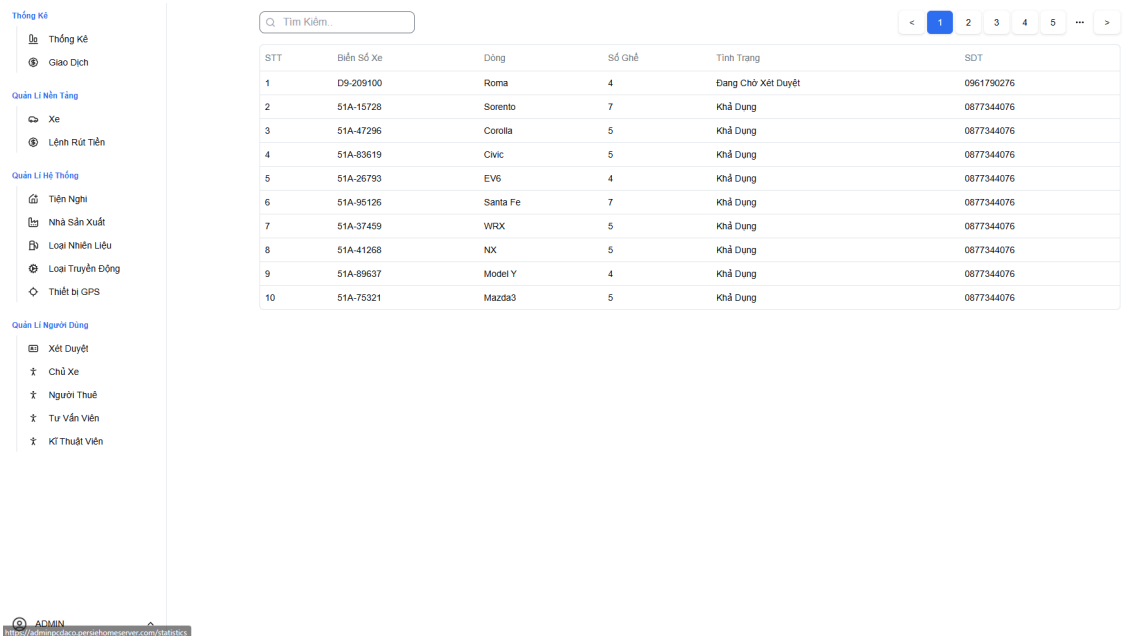


- **Function Details:** On clicking “Tạo”, the admin enters all required information for the new technician.

3.15 View Cars (Xe)

- **Function Trigger:** When clicking the “Xe” button.
- **Function Description:**
  - **Actors:** Admin
  - **Purpose:** Observe all cars on the platform.
  - **Interface:** Table
- **Data Processing:**
  - **Data Collect:** Data is collected from the primary database.
  - **Data Computation:** Data is filtered and validated.
  - **Data Binding:** Data is rendered on the screen.

- **Screen Layout:**

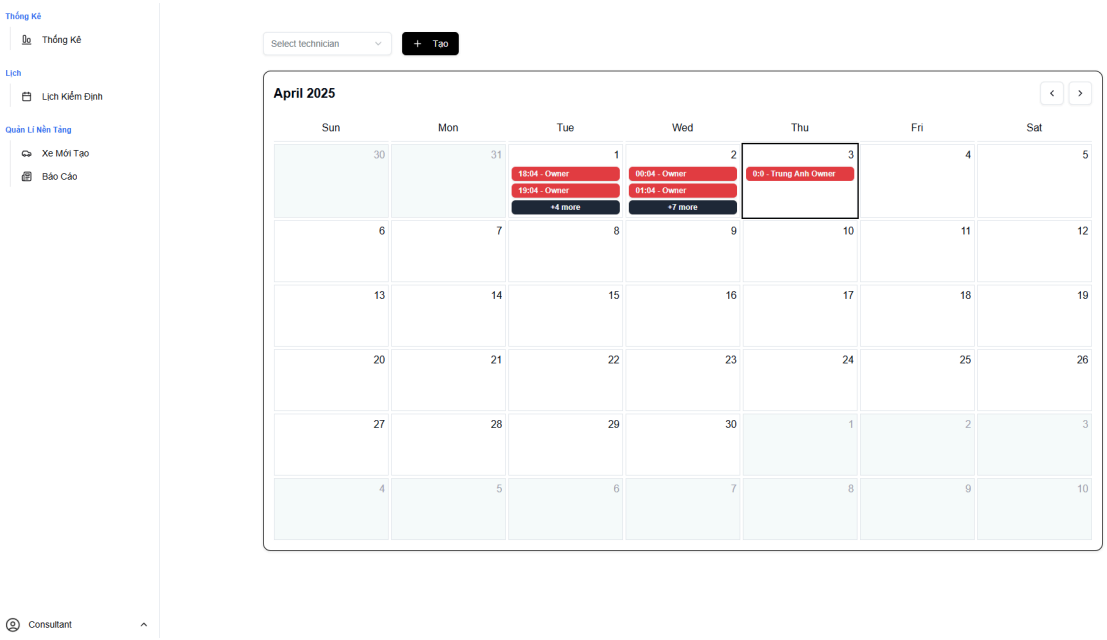


- **Function Details:** On clicking “Xe”, the admin can view car details such as name, specifications, and status.

3.16 View Inspection Schedules

- **Function Trigger:** When clicking the “Lịch kiểm định” tab/button.
- **Function Description:**
  - **Actors:** Consultant
  - **Purpose:** Review technician details for scheduling inspections.
  - **Interface:** Calendar (Inspection Schedule Calendar)
- **Data Processing:**
  - **Data Collect:** Technician data is retrieved from the inspection schedule-linked database.
  - **Data Computation:** Data is filtered and sorted by availability and scheduled dates.
  - **Data Binding:** Processed data is bound to the table.

- **Screen Layout:**

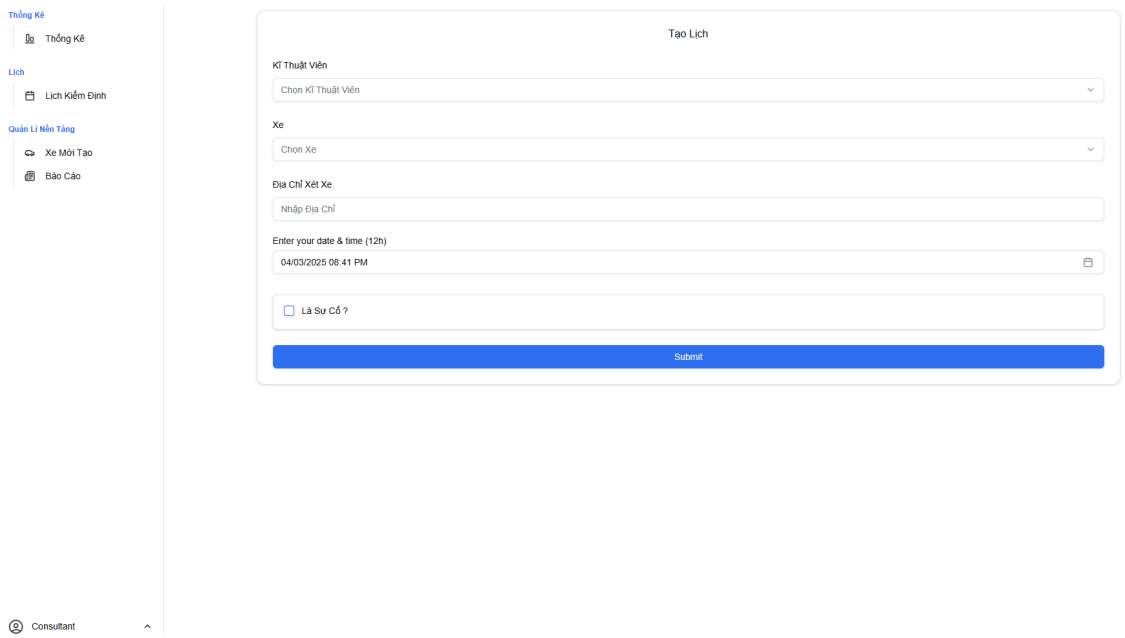


- **Function Details:** The consultant sees a table of technician details with options to sort or filter based on availability.

3.17 Create Inspection Schedule (Tạo lịch kiểm tra)

- **Function Trigger:** When the Consultant clicks the “Tạo lịch” button within the Inspection Schedule module.
- **Function Description:**
  - **Actors:** Consultant
  - **Purpose:** Assign a technician for a vehicle inspection on a specified date and address.
  - **Interface:** Form interface (dropdowns for technician selection, date picker, address field)
- **Data Processing:**
  - **Data Collect:** User input (technician, inspection date, address) is captured.
  - **Data Computation:** The system validates the input (e.g., date format, technician availability, valid address).
  - **Data Binding:** Upon validation, the new schedule record is saved and displayed.

- **Screen Layout:**

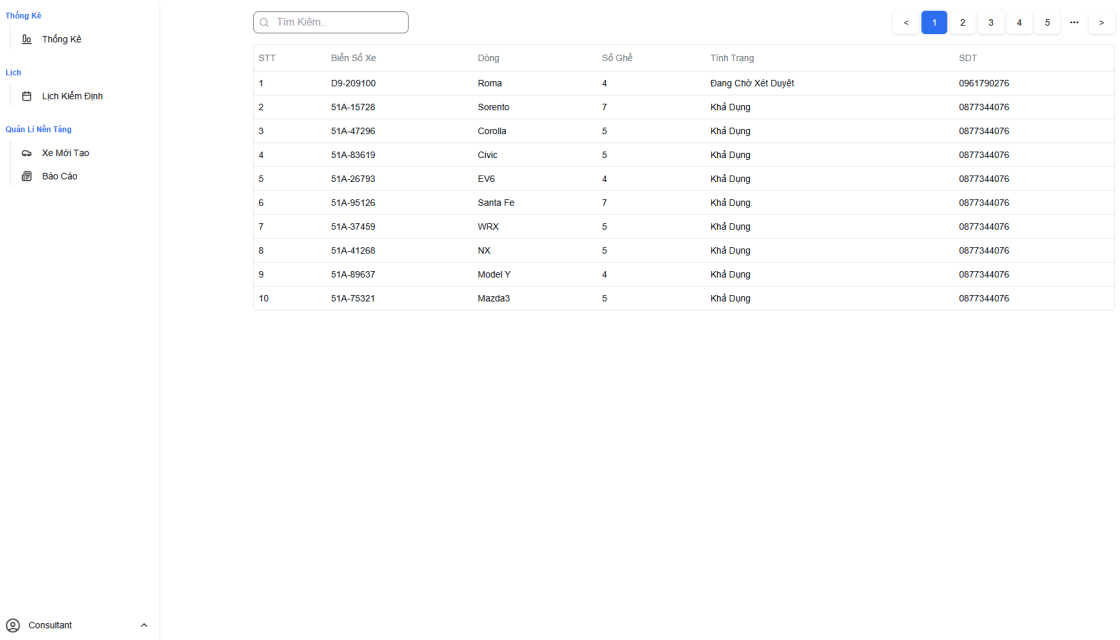


- **Function Details:** When "Tạo lịch" is clicked and the form data is valid, a new inspection schedule is added along with appropriate notifications.

3.18 View Pending Cars (Xe chờ xử lý)

- **Function Trigger:** When the Consultant selects the "Pending Cars" tab/filter.
- **Function Description:**
    + **Actors:** Consultant
    + **Purpose:** Monitor car records that are newly created and pending approval.
    + **Interface:** Table (filtered by pending status)
- **Data Processing:**
    + **Data Collect:** Car records with a "pending" status are fetched from the database.
    + **Data Computation:** The system verifies these records as freshly created and unprocessed.
    + **Data Binding:** Pending records are displayed in the table.

- **Screen Layout:**

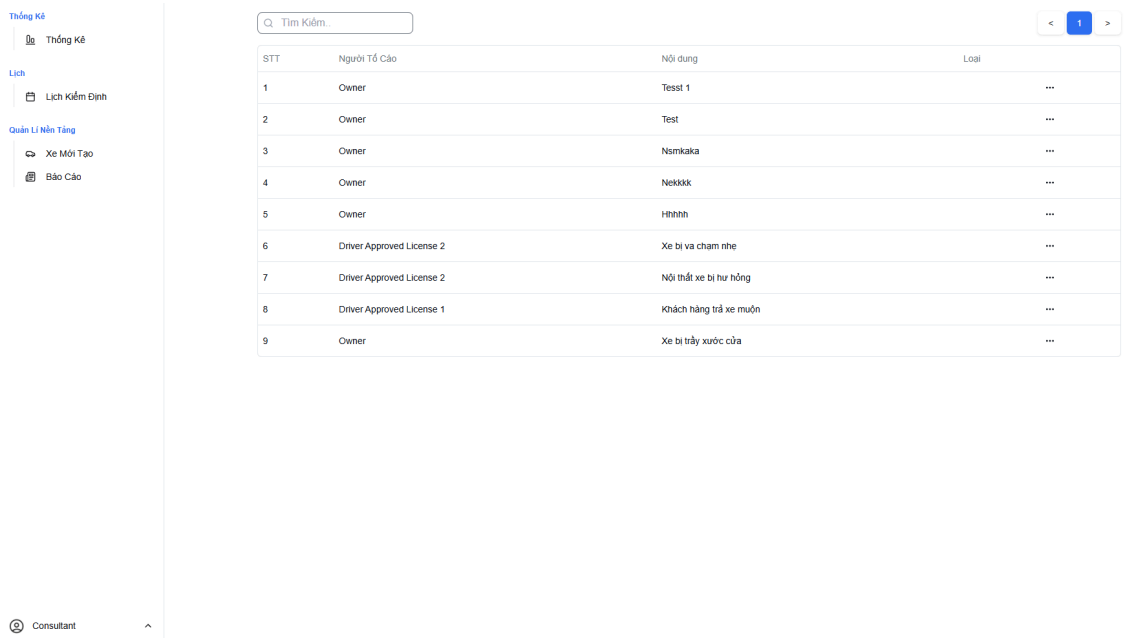


- **Function Details:** Only pending cars are visible, enabling the consultant to take further action if required.

3.19 View User Reports (Báo cáo người dùng)

- **Function Trigger:** When the Consultant clicks the “User Reports” button.
- **Function Description:**
  - **Actors:** Consultant
  - **Purpose:** Review reports submitted by users on platform issues.
  - **Interface:** Table (listing key report attributes)
- **Data Processing:**
  - **Data Collect:** Report data is retrieved from the user reports database.
  - **Data Computation:** Data is sorted and filtered (by date, status, severity, etc.).
  - **Data Binding:** The data is dynamically rendered in the table.

- **Screen Layout:**

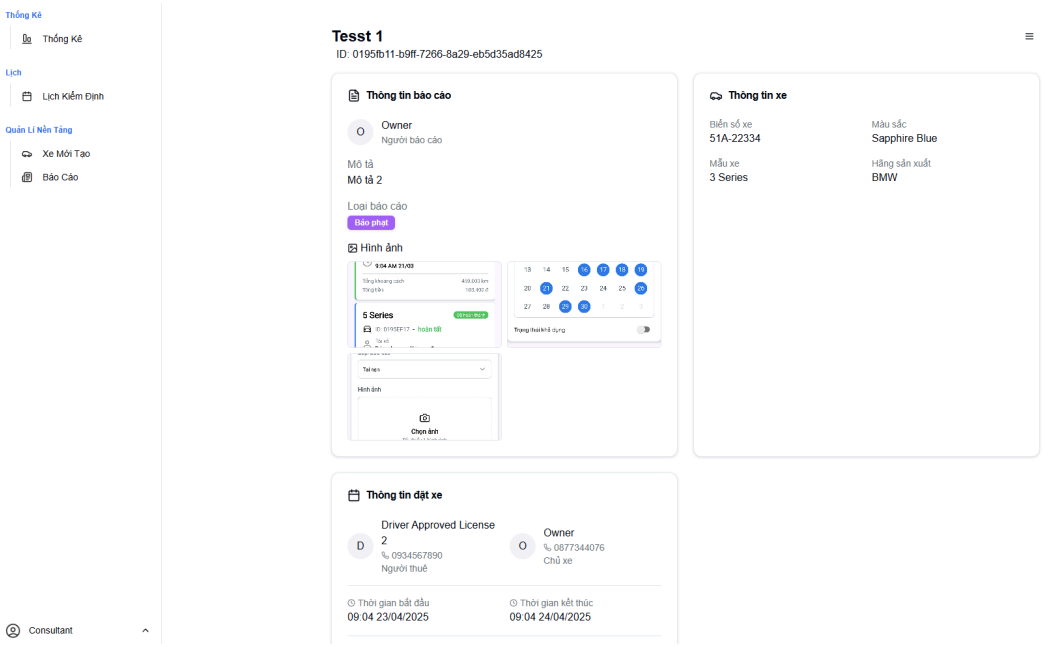


- **Function Details:** The consultant can sort, filter, and click on individual reports for further investigation.

3.20 View Report Details (Chi tiết báo cáo)

- **Function Trigger:** When the Consultant selects a report (e.g., via “View Details”).
- **Function Description:**
  - **Actors:** Consultant
  - **Purpose:** Display comprehensive report details including attached proofs.
  - **Interface:** Detailed view page or modal dialog
- **Data Processing:**
  - **Data Collect:** Full report details are retrieved using the report ID.
  - **Data Computation:** Information is aggregated and formatted for display.
  - **Data Binding:** Detailed report data is bound to the view.

- **Screen Layout:**

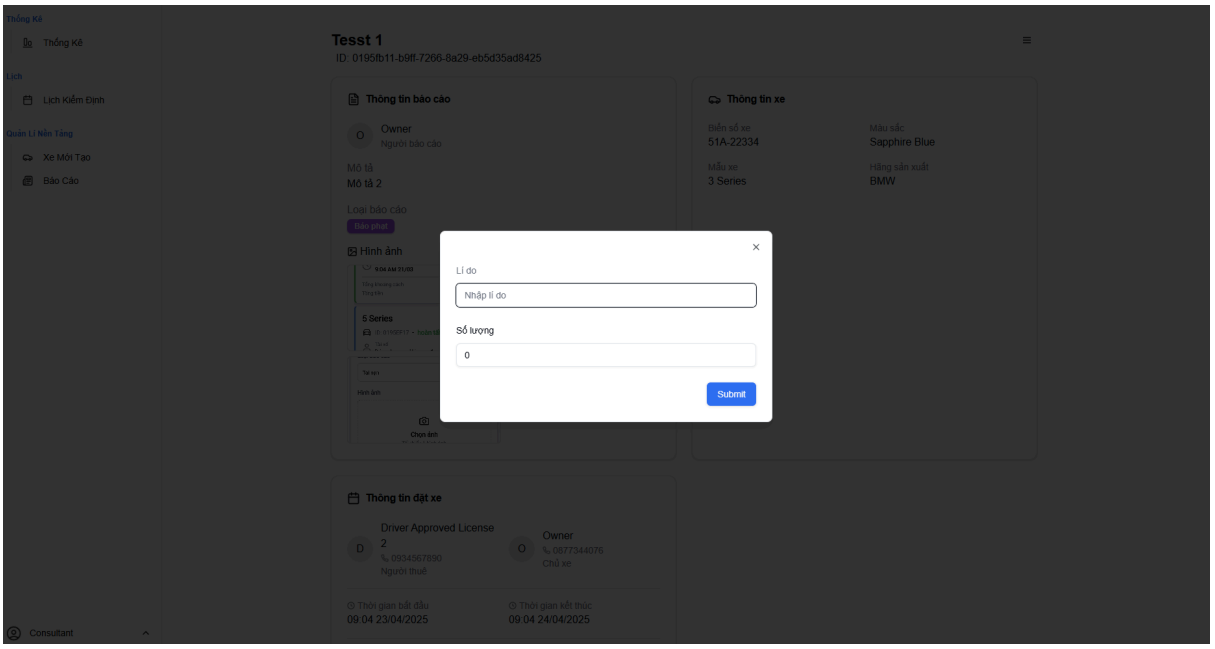


- **Function Details:** The consultant reviews the complete report and then decides whether to approve or reject it.

3.21 Create Compensation (Tạo bồi thường)

- **Function Trigger:** When the Consultant selects the “Create Compensation” option (typically from a report or driver details view).
- **Function Description:**
  - **Actors:** Consultant
  - **Purpose:** Record and manage a new compensation for a driver responsible for fees (accident, traffic, etc.).
  - **Interface:** Compensation creation form
- **Data Processing:**
  - **Data Collect:** Inputs such as selected driver, fee details, and incident description are captured.
  - **Data Computation:** Inputs are validated and the total compensation may be computed.
  - **Data Binding:** The compensation record is saved and linked to the report/driver.

- **Screen Layout:**

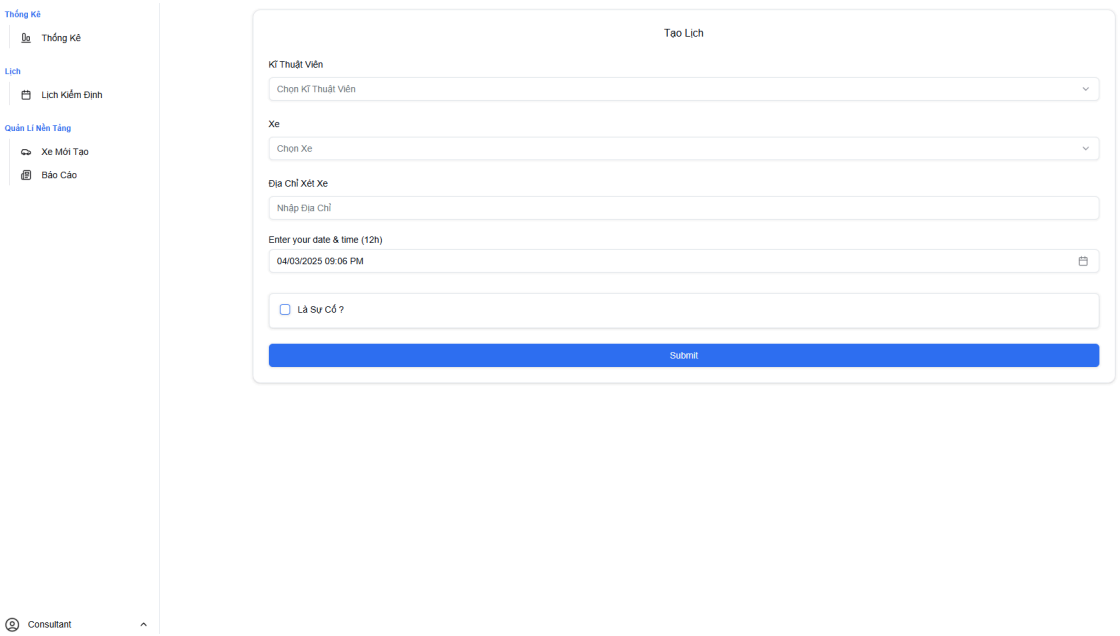


- **Function Details:** After form submission, a confirmation message is displayed and the new record is immediately available for reference.

3.22 Conditional Compensation Creation Trigger

- **Function Trigger:** When the Consultant clicks the “Tạo lịch” button within the report processing interface and no compensation record exists for that report.
- **Function Description:**
    + **Actors:** Consultant
    + **Purpose:** Prevent duplicate compensation records.
    + **Interface:** “Tạo lịch” button that is conditionally displayed.
- **Data Processing:**
    + **Data Collect:** The system checks for an existing compensation record linked to the report.
    + **Data Computation:** If none exists, the button is active; otherwise, it is disabled or hidden.
    + **Data Binding:** The UI dynamically reflects whether a compensation has been created.

- **Screen Layout:**

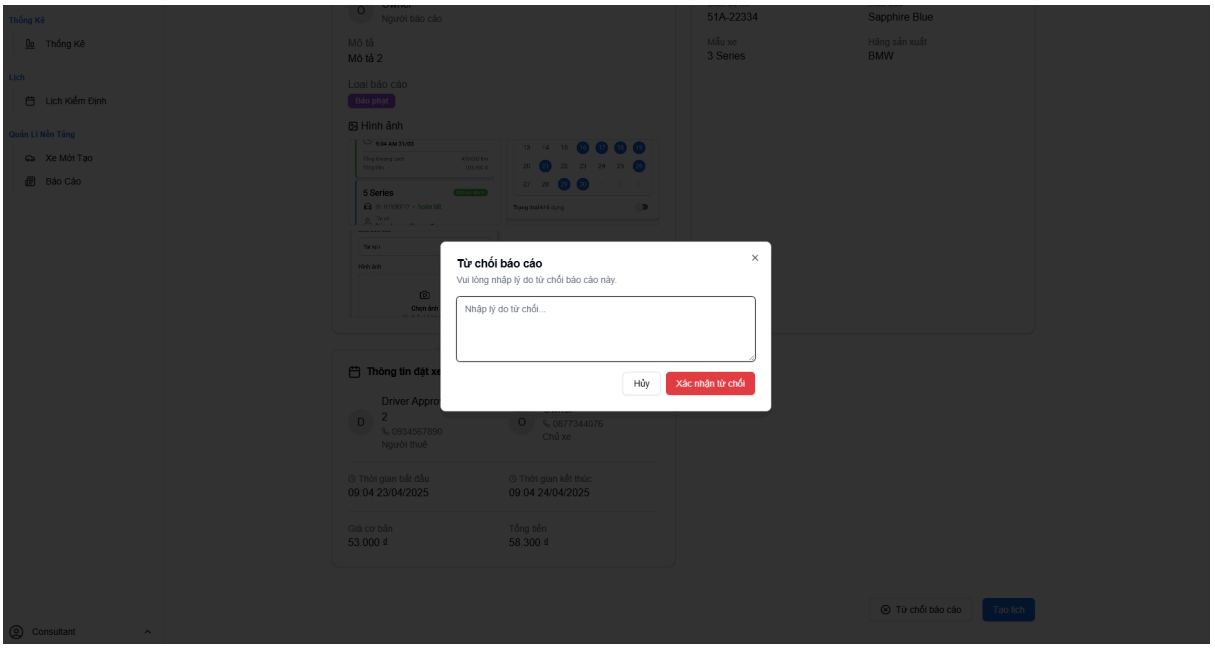


- **Function Details:** If no record exists and the “Tạo lịch” button is clicked, the system transitions into the compensation creation process as detailed in Section 3.21.

3.23 Reject Report (Từ chối báo cáo)

- **Function Trigger:** When the Consultant clicks the “Reject” button in the report details view.
- **Function Description:**
  - **Actors:** Consultant
  - **Purpose:** Mark a report as rejected and optionally add a rejection comment.
  - **Interface:** Report management interface with actionable buttons
- **Data Processing:**
  - **Data Collect:** The system collects the report ID and any rejection comments.
  - **Data Computation:** The report status is updated to “Rejected” and logged.
  - **Data Binding:** The updated status is immediately reflected in both the report list and detailed view.

- **Screen Layout:**



- **Function Details:** After rejection, the report is locked from further modifications and the rejection history can be viewed.

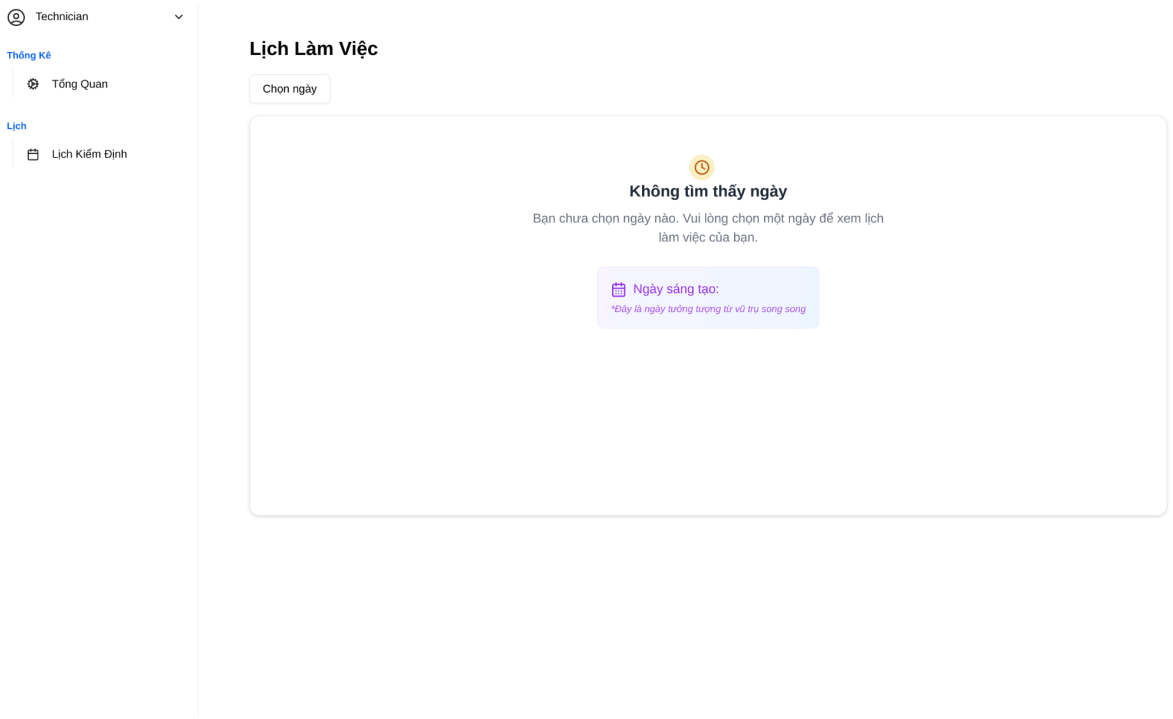
3.24 Approve Report with Compensation Proof (Phê duyệt báo cáo với bằng chứng bồi thường)

- **Function Trigger:** When the Consultant clicks the “Approve” button in the report details view and attaches the required compensation proof images.
- **Function Description:**
  - **Actors:** Consultant
  - **Purpose:** Formally approve the report with visual evidence supporting compensation.
  - **Interface:** Approval interface with file upload components
- **Data Processing:**
  - **Data Collect:** Both the approval action and the uploaded image files are collected.
  - **Data Computation:** The system validates the image formats and resolutions, then associates the images with the report.
  - **Data Binding:** The report status is updated to “Approved” and the images are bound to the detailed view.
- **Screen Layout:**
- **Function Details:** Upon successful approval, the report status changes to “Approved” and any subsequent automated processes (e.g., notifications) are triggered.

3.25 Technician View Inspection Schedule Todo Task

- **Function Trigger:** When the Technician navigates to the "Inspection Tasks" section or clicks the "Todo" icon.
- **Function Description:** Allows the Technician to view all assigned inspection schedule tasks in a to-do list format.

    - **Actors:** Technician
    - **Purpose:** To provide a clear overview of pending inspection tasks.
    - **Interface:** To-do list or table view within the inspection schedule module.
- **Data Processing:**
    - **Data Collect:** The system retrieves tasks assigned to the technician from the inspection task database.
    - **Data Computation:** Tasks are filtered by status (e.g., "Pending") and sorted by scheduled date or priority.
    - **Data Binding:** The retrieved tasks are dynamically bound to the UI elements.
- **Screen Layout:**



A dashboard page displaying a list or table of tasks with columns such as Task ID, Car ID, Scheduled Date, and Status.

- **Function Details:**
  The Technician is presented with all pending inspection tasks and can click on individual tasks to view further details.

3.26 Technician Approve Inspection Schedule Todo Task (Mark as In-Progress)

- **Function Trigger:** When the Technician clicks the “Approve” or “Start” button on a pending inspection task.
- **Function Description:** Enables the Technician to approve an inspection task, thereby updating its status to "In-Progress".
    - **Actors:** Technician
    - **Purpose:** To confirm the initiation of the inspection process and update the task status accordingly.
    - **Interface:** Approval interface with a confirmation prompt.
- **Data Processing:**
    - **Data Collect:** The system collects the task identifier along with the Technician’s approval action.

    - **Data Computation:** The task status is updated to "In-Progress" in the database.
    - **Data Binding:** The updated status is immediately reflected on the Technician's dashboard.
- **Screen Layout:**

- **Function Details:**
  Once approved, the task is removed or re-categorized from the "Pending" list, triggering subsequent workflow processes.

3.27 Technician View Inspection Schedule Todo Task Detail

- **Function Trigger:** When the Technician clicks on an in-progress or upcoming inspection task from the statistics page.
- **Function Description:** Provides a detailed view of the selected inspection task, including car details, scheduled time, and the inspection checklist.
    - **Actors:** Technician
    - **Purpose:** To review comprehensive details necessary for performing the inspection task.
    - **Interface:** Detailed view page or modal dialog.
- **Data Processing:**
    - **Data Collect:** The system retrieves complete task details using the task's unique identifier.
    - **Data Computation:** Data is aggregated and formatted for an in-depth display.
    - **Data Binding:** The detailed task data is bound to the view layout.
- **Screen Layout:**
  A modal or dedicated detail page featuring sections for task summary, car details, inspection checklist, and additional instructions.
- **Function Details:**
  The Technician can verify all critical information before proceeding with the inspection.

3.28 Technician Reject Inspection Schedule

- **Function Trigger:** When the Technician clicks the "Reject" button on an assigned inspection task.
- **Function Description:** Allows the Technician to reject an assigned inspection task, optionally providing a rejection reason.
    - **Actors:** Technician
    - **Purpose:** To indicate inability or refusal to perform the scheduled inspection.
    - **Interface:** Rejection interface with a comment box.
- **Data Processing:**
    - **Data Collect:** The system collects the task identifier along with the optional rejection reason.
    - **Data Computation:** The task status is updated to "Rejected" in the database and the reason is logged.
    - **Data Binding:** The updated status is immediately reflected in the task list.
- **Screen Layout:**
  A pop-up or modal dialog prompting the Technician to enter a rejection comment.

- **Function Details:**
  The rejection action updates the task status and can trigger notifications for task redistribution.

3.29 Technician View Contract (Update Technician Info and Navigate)

- **Function Trigger:** When the Technician selects the "View Contract" option from the inspection task or contract module.
- **Function Description:** Displays the contract details while updating the Technician's information in the car contract record, then navigates to the contract page.
  - **Actors:** Technician
  - **Purpose:** To ensure the contract record reflects the current Technician's info before reviewing its content.
  - **Interface:** Contract update and view interface.
- **Data Processing:**
  - **Data Collect:** The system retrieves the Technician's profile information and the relevant car contract data.
  - **Data Computation:** The Technician's details are merged into the contract record.
  - **Data Binding:** The updated contract data is dynamically bound to the contract view page.
- **Screen Layout:**
  A dedicated contract page that displays updated contract details including Technician information.
- **Function Details:**
  Upon selection, the system updates the contract record and seamlessly navigates the Technician to the detailed contract view.

3.30 Technician View Contract

- **Function Trigger:** When the Technician navigates to the contract section or selects a contract from a list.
- **Function Description:** Provides a read-only view of the detailed contract information.
  - **Actors:** Technician
  - **Purpose:** To review contract terms, conditions, and related clauses without making modifications.
  - **Interface:** Contract display interface.
- **Data Processing:**
  - **Data Collect:** Contract details are fetched from the contract database.
  - **Data Computation:** The contract data is organized and formatted for clear display.
  - **Data Binding:** The retrieved information is bound to the UI elements on the contract page.
- **Screen Layout:**
  A comprehensive page featuring sections for contract terms, obligations, and signatories.
- **Function Details:**
  The Technician can fully review the contract in a non-editable format to ensure proper understanding of all terms.

3.31 Technician Approve Contract

- **Function Trigger:** When the Technician clicks the “Approve” button on the contract view page.
- **Function Description:** Enables the Technician to formally approve the contract, confirming their acceptance of its terms.
    + **Actors:** Technician
    + **Purpose:** To validate and record the Technician’s agreement with the contract.
    + **Interface:** Contract approval interface with a confirmation prompt.
- **Data Processing:**
    + **Data Collect:** The system collects the contract identifier along with the Technician’s approval action.
    + **Data Computation:** The contract status is updated to "Approved" in the database.
    + **Data Binding:** The updated status is immediately reflected on the contract view page.
- **Screen Layout:**
  An updated contract page clearly indicating the approved status, possibly with a confirmation message.
- **Function Details:**
  The approval action triggers notifications to other stakeholders and may initiate subsequent contractual workflows.

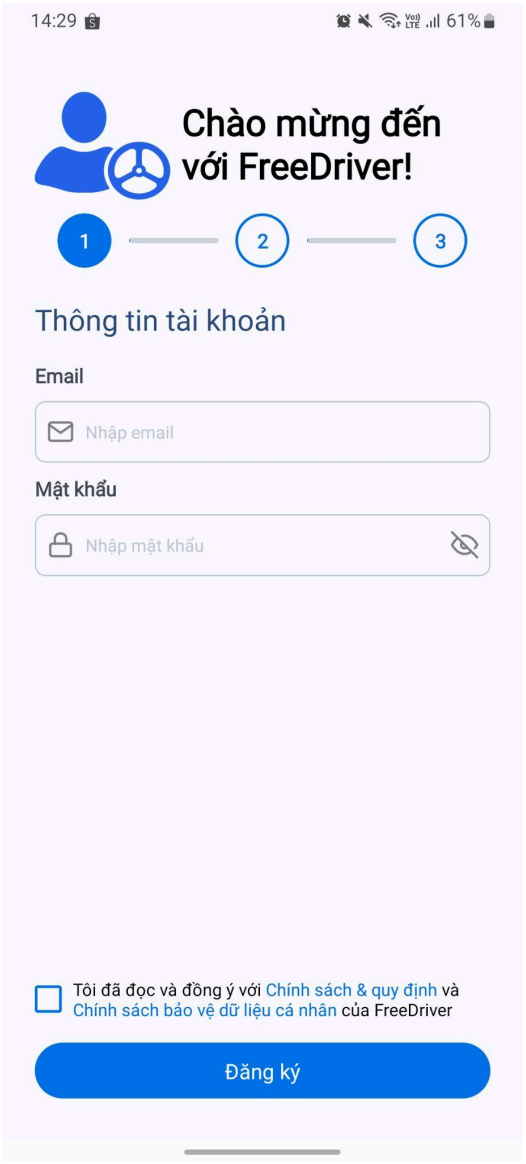
3.32 Technician Import Images Then Approve

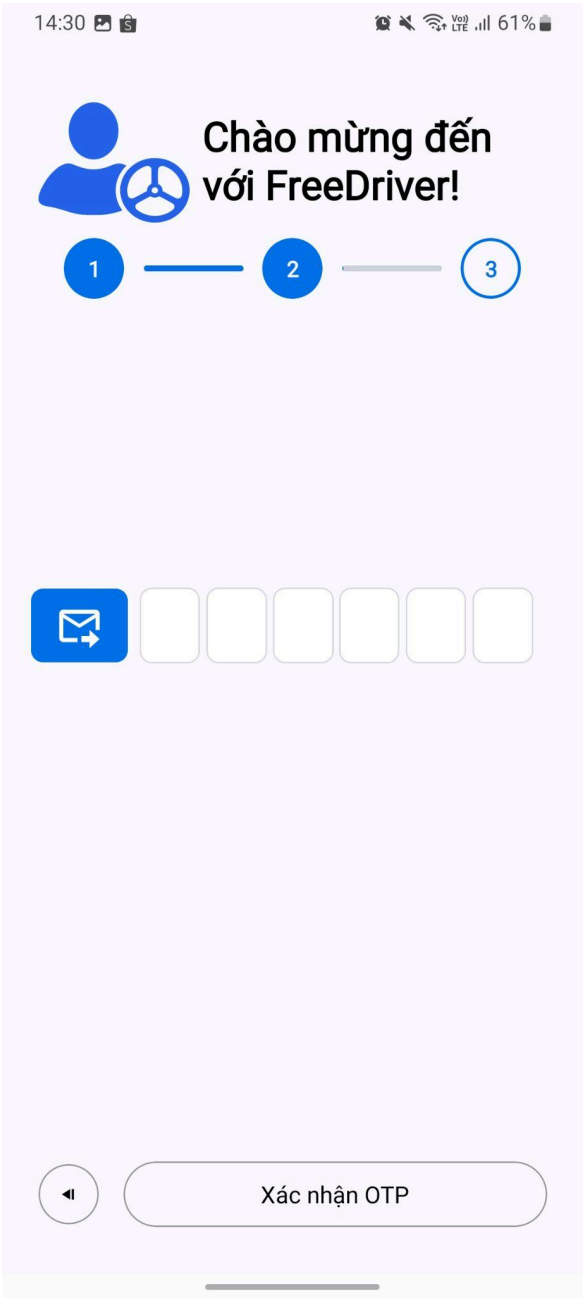
- **Function Trigger:** When the Technician uploads image proofs and clicks the “Approve” button.
    + **Function Description:** Allows the Technician to attach images as evidence and approve the corresponding task or contract.
    + **Actors:** Technician
    + **Purpose:** To validate task completion or contract conditions with supporting visual evidence.
    + **Interface:** Image upload interface paired with an approval mechanism.
- **Data Processing:**
    + **Data Collect:** The system collects the uploaded image files along with the Technician’s approval confirmation.
    + **Data Computation:** The images are validated (format, resolution, etc.) and associated with the relevant record.
    + **Data Binding:** The images and the updated approval status are bound to the record and reflected in the UI.
- **Screen Layout:**
  A form with an image upload section, preview pane, and an “Approve” button.
- **Function Details:**
  Upon successful image upload and approval, the record is updated, and notifications are sent to trigger any associated processes.

3.33.1 Send OTP

- **Function Trigger:** When the user (Driver/Owner) clicks the "Send OTP" button on the Register Screen.

- **Function Description:**
    - **Actors:** Driver, Owner
    - **Purpose:** To initiate the OTP (One-Time Password) sending process for user registration verification.
    - **Interface:** Register Screen
- **Data Processing:**
    - **Data Collect:** Collects the user's phone number entered on the Register Screen.
    - **Data Computation:** Generates a random OTP, associates it with the user's phone number, and potentially hashes it for security.
    - **Data Binding:** Displays a confirmation message to the user indicating that the OTP has been sent.
- **Screen Layout:**


14:29
61%
Chào mừng đến với FreeDriver!
1
2
3
Thông tin tài khoản
Email
Nhập email
Mật khẩu
Nhập mật khẩu
Tôi đã đọc và đồng ý với Chính sách & quy định và Chính sách bảo vệ dữ liệu cá nhân của FreeDriver
Đăng ký



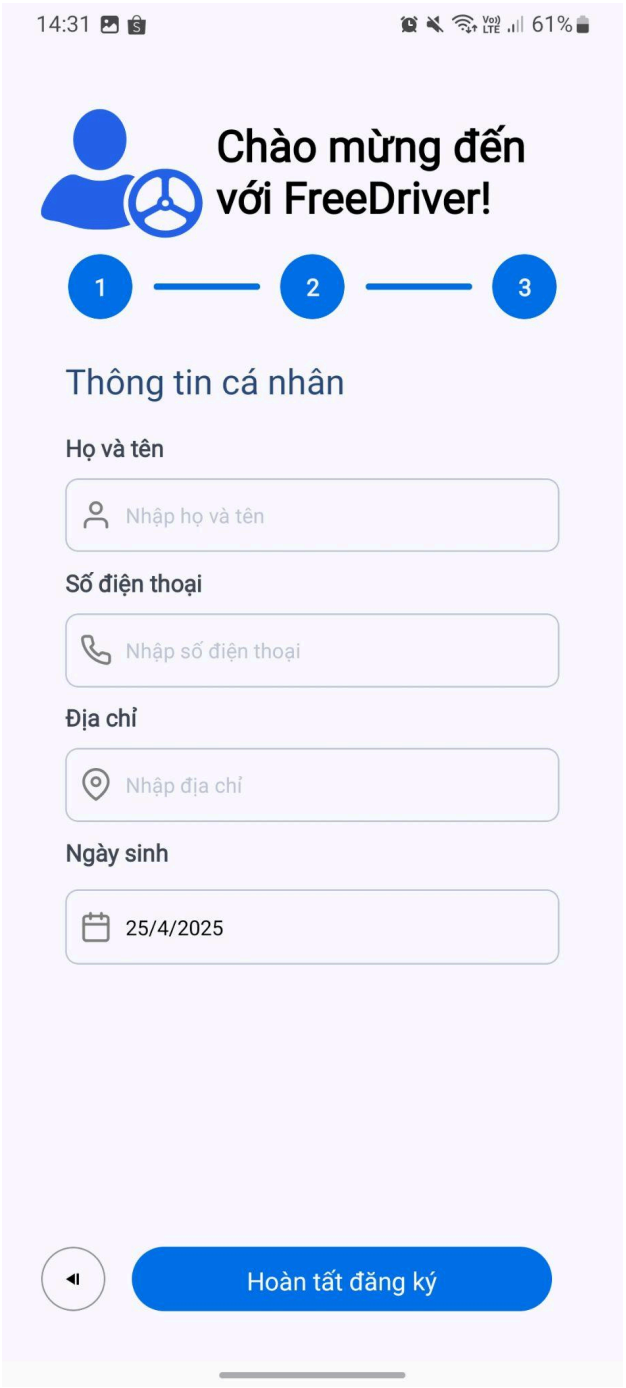
- **Function Details:** On clicking "Send OTP," the system sends an OTP to the user's provided phone number. The user needs this OTP to complete the registration process.

3.33.2 Create Account

- **Function Trigger:** When the user (Driver/Owner) clicks the "Create Account" button on the Register Screen.
- **Function Description:**
    - **Actors:** Driver, Owner
    - **Purpose:** To create a new user account in the system.
    - **Interface:** Register Screen
- **Data Processing:**

- **Receive Data Input:** Receives user's information (phone number, password, potentially name, email, etc.) from the Register Screen. Also receives the OTP entered by the user.
- **Data Computation:** Validates the input data (e.g., password strength, phone number format, matching OTP).
- **Data Storage:** Stores the new user account information (including hashed password) in the database.

- **Screen Layout:**



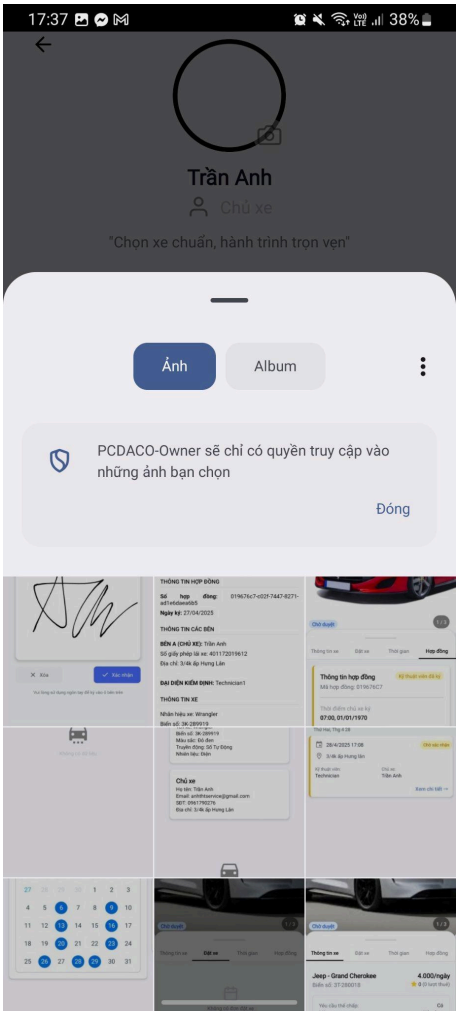
- **Function Details:** On clicking "Create Account," the system validates the entered information and the OTP. If valid, a new user account is created and stored. The user is then typically logged in.

3.33.3 Get current user

- **Function Trigger:** When the user (Driver/Owner) navigates to the "Profile Screen" or "Home Screen".
- **Function Description:**
    - **Actors:** Driver, Owner
    - **Purpose:** To display the user's profile information.
    - **Interface:** Profile Screen, Home Screen
- **Data Processing:**
    - **Data Collect:** Retrieves the logged-in user's profile information from the database.
    - **Data Computation:** No significant computation is needed.
    - **Data Binding:** Displays the user's profile information on the Profile Screen or Home Screen.
- **Screen Layout:** or sidebar on the Home Screen.
- **Function Details:** Shows user profile information. For owners, this may appear as a small display in the header or sidebar on the Home Screen.

3.33.4 Upload image avatar

- **Function Trigger:** When the user (Driver/Owner) selects an image file and confirms the upload on the "Profile Screen".
- **Function Description:**
    - **Actors:** Driver, Owner
    - **Purpose:** To upload a profile picture for the user's account.
    - **Interface:** Profile Screen
- **Data Processing:**
    - **Receive Data Input:** Receives the image file uploaded by the user.
    - **Data Computation:** May perform image validation (e.g., file size, file type).
    - **Data Storage:** Stores the image file in a storage service and saves the image's URL or path in the database, associated with the user's account.
- **Screen Layout**

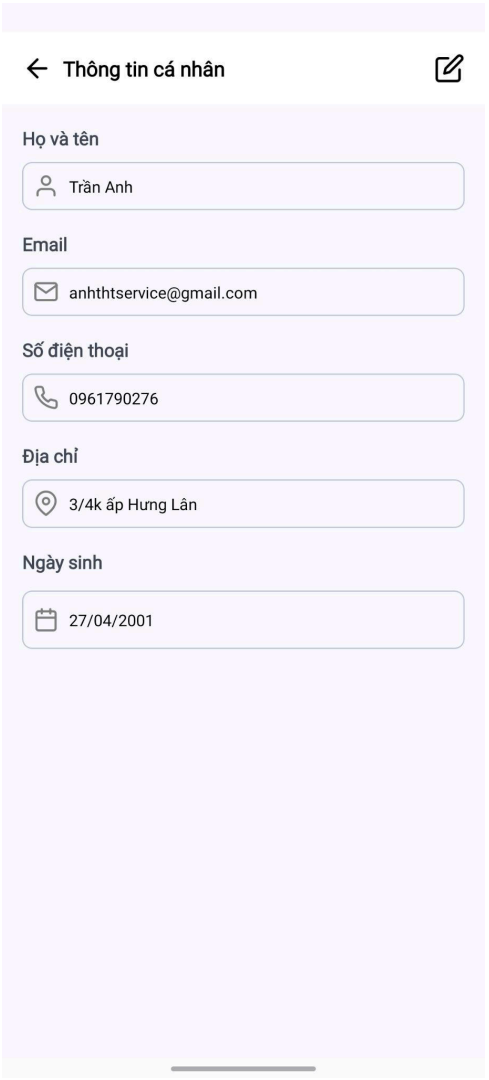


:

- **Function Details:** Uploads user profile picture.

3.33.5 Update user profile

- **Function Trigger:** When the user (Driver/Owner) clicks the "Save" or "Update" button on the "Edit Profile Screen".
- **Function Description:**
    - **Actors:** Driver, Owner
    - **Purpose:** To modify the user's profile information.
    - **Interface:** Edit Profile Screen
- **Data Processing:**
    - **Receive Data Input:** Receives the updated profile information from the Edit Profile Screen.
    - **Data Computation:** Validates the input data (e.g., email format, phone number format).
    - **Data Storage:** Updates the user's profile information in the database.
- **Screen Layout**



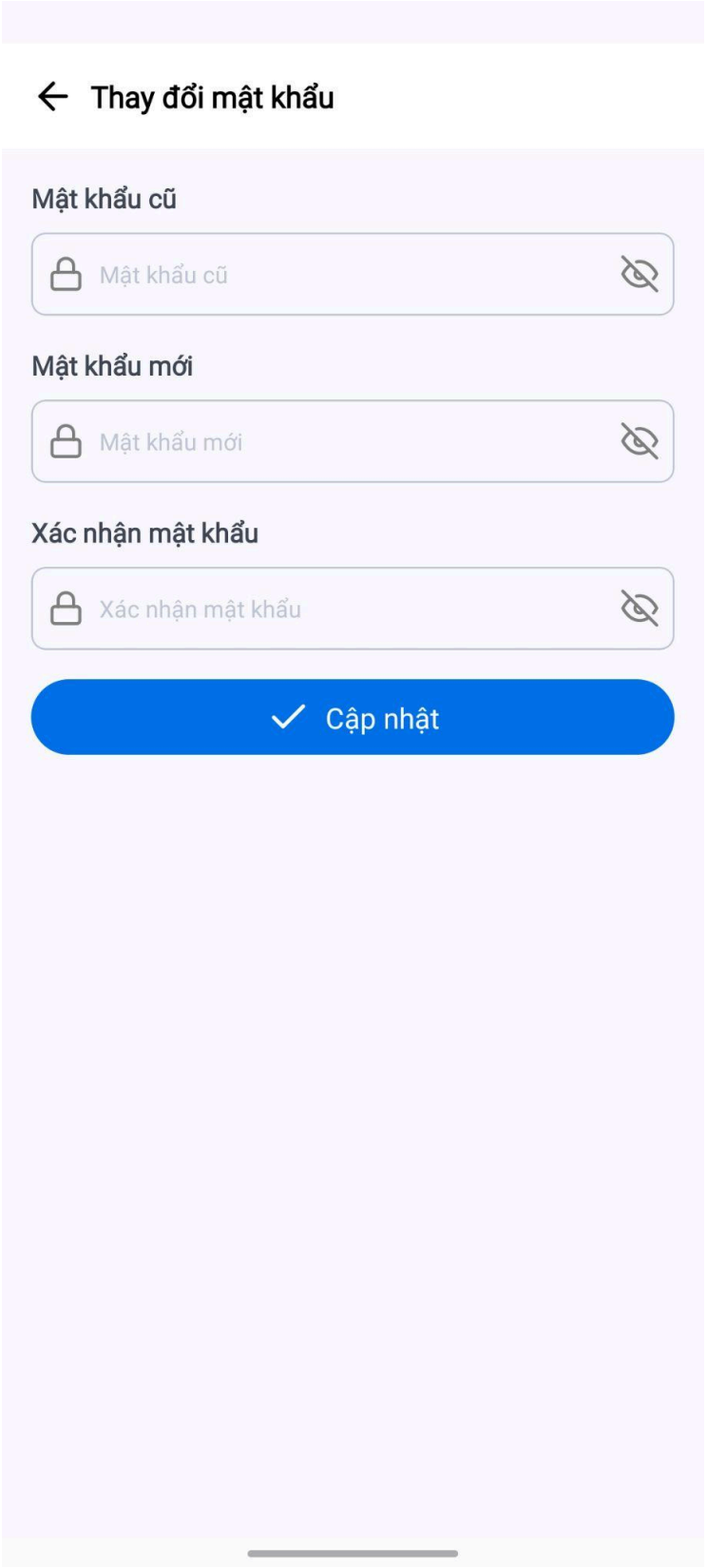
:

- **Function Details:** Updates user profile information.

3.33.6 Update password account

- **Function Trigger:** When the user (Driver/Owner) clicks the "Change Password" or "Save" button on the "Change password Screen".
- **Function Description:**
    - **Actors:** Driver, Owner
    - **Purpose:** To change the user's account password.
    - **Interface:** Change password Screen
- **Data Processing:**
    - **Receive Data Input:** Receives the old password, new password, and confirmation password from the Change password Screen.
    - **Data Computation:** Validates that the old password is correct and that the new password and confirmation password match. Enforces password complexity requirements.

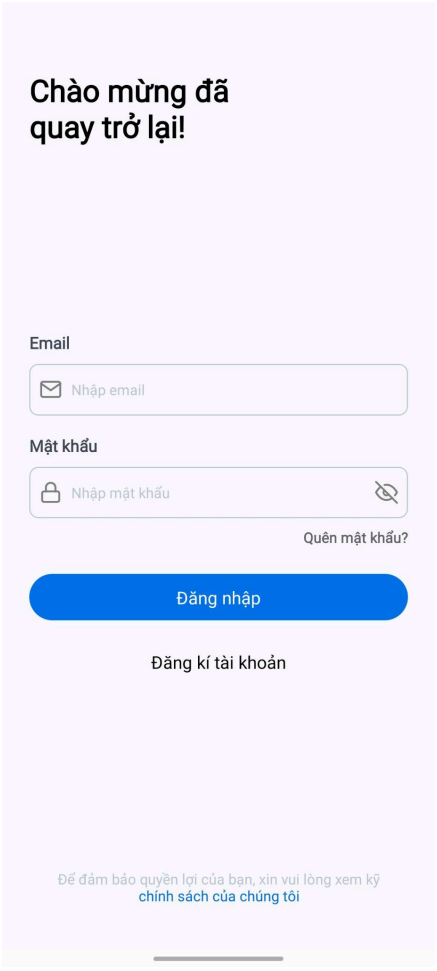
    - **Data Storage:** Updates the user's password in the database (after hashing it securely).
- **Screen Layout:**



- **Function Details:** Changes user password.

3.33.7 Login

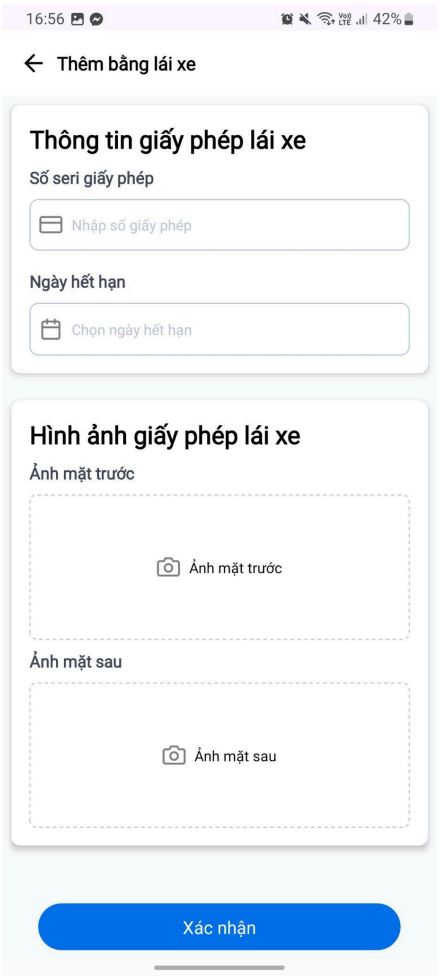
- **Function Trigger:** When the technician enters their credentials and clicks the "Login" button on the Login Screen.
- **Function Description:**
    - **Actors:** Technician
    - **Purpose:** To allow the technician to log in to the tracking system.
    - **Interface:** Login Screen
- **Data Processing:**
    - **Receive Data Input:** Receives the technician's username/email and password from the Login Screen.
    - **Data Computation:** Authenticates the technician's credentials against the stored user data (e.g., compares the entered password to a stored hashed password).
    - **Data Binding:** On successful authentication, redirects the technician to the Home Screen (or another designated screen). On failed authentication, displays an error message.
- **Screen Layout:**



- **Function Details:** Technician gets access.

3.34.1 Create License

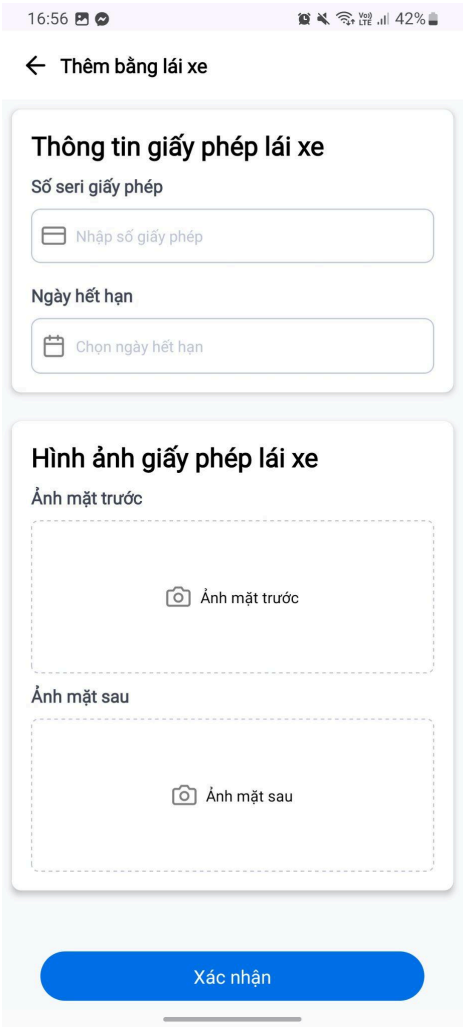
- **Function Trigger:** When the user (Driver/Owner) clicks the "Save" or "Submit" button on the Edit License Screen.
- **Function Description:**
    - **Actors:** Driver, Owner
    - **Purpose:** To input and store the user's driver's license information.
    - **Interface:** Edit License Screen
- **Data Processing:**
    - **Receive Data Input:** Receives user's license information (license number, expiry date, issuing state, etc.) from the Edit License Screen.
    - **Data Computation:** Validates the input data (e.g., date format, license number format).
    - **Data Storage:** Stores the user's license information in the database, associated with the user's account.
- **Screen Layout:**



- **Function Details:** On clicking "Save," the system validates the entered license information. If valid, the information is stored in the database.

3.34.2 Upload Image License

- **Function Trigger:** When the user (Driver/Owner) selects an image file and confirms the upload on the Edit License Screen.
- **Function Description:**
    - **Actors:** Driver, Owner
    - **Purpose:** To upload an image of the user's driver's license for verification purposes.
    - **Interface:** Edit License Screen
- **Data Processing:**
    - **Receive Data Input:** Receives the image file uploaded by the user.
    - **Data Computation:** May perform image validation (e.g., file size, file type).
    - **Data Storage:** Stores the image file in a storage service and saves the image's URL or path in the database, associated with the user's account.
- **Screen Layout:**

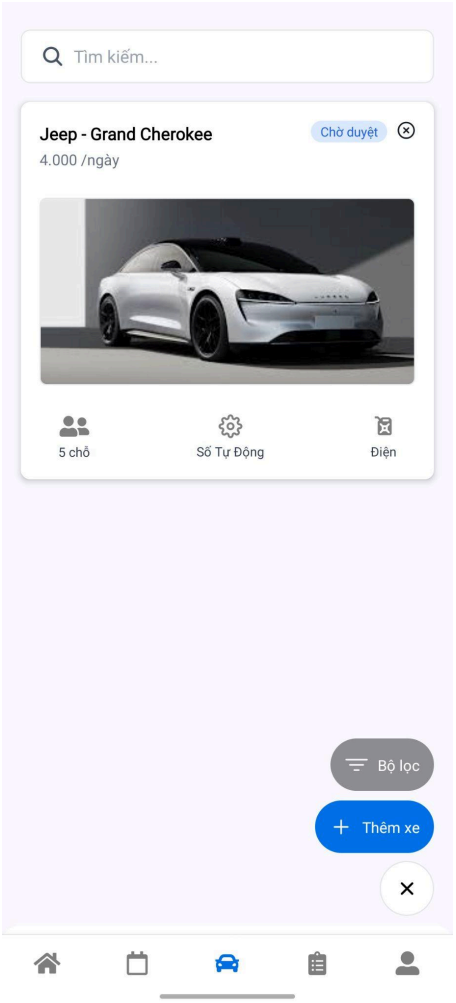


.

- **Function Details:** On confirming the upload, the system stores the image and associates it with the user's profile.

3.35.1 List Cars

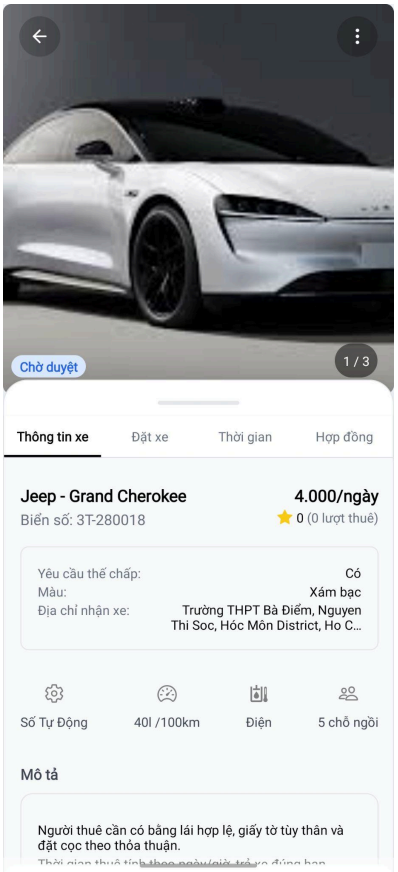
- **Function Trigger:** When the user (Driver/Owner) navigates to the "Cars" Screen or "Home Screen".
- **Function Description:**
    - **Actors:** Driver, Owner
    - **Purpose:** To display a list of cars (available for rent for drivers, registered cars for owners).
    - **Interface:** Cars Screen, Home Screen
- **Data Processing:**
    - **Data Collect:**
        - For Drivers: Retrieves a list of available cars from the database. May apply filters (e.g., location, car type, availability dates).
        - For Owners: Retrieves a list of cars registered by the logged-in owner from the database.
    - **Data Computation:** Sorts and filters the car list based on defined criteria or user preferences.
    - **Data Binding:** Displays the list of cars on the Cars Screen or Home Screen, including relevant information (e.g., image, model, price/availability).
- **Screen Layout:**



- **Function Details:** On navigating to the Cars screen or Home screen, the system displays a list of cars based on the user's role. For owners, the Home Screen shows a summary view while the Cars Screen provides more detailed functionality.

3.35.2 Get car by id

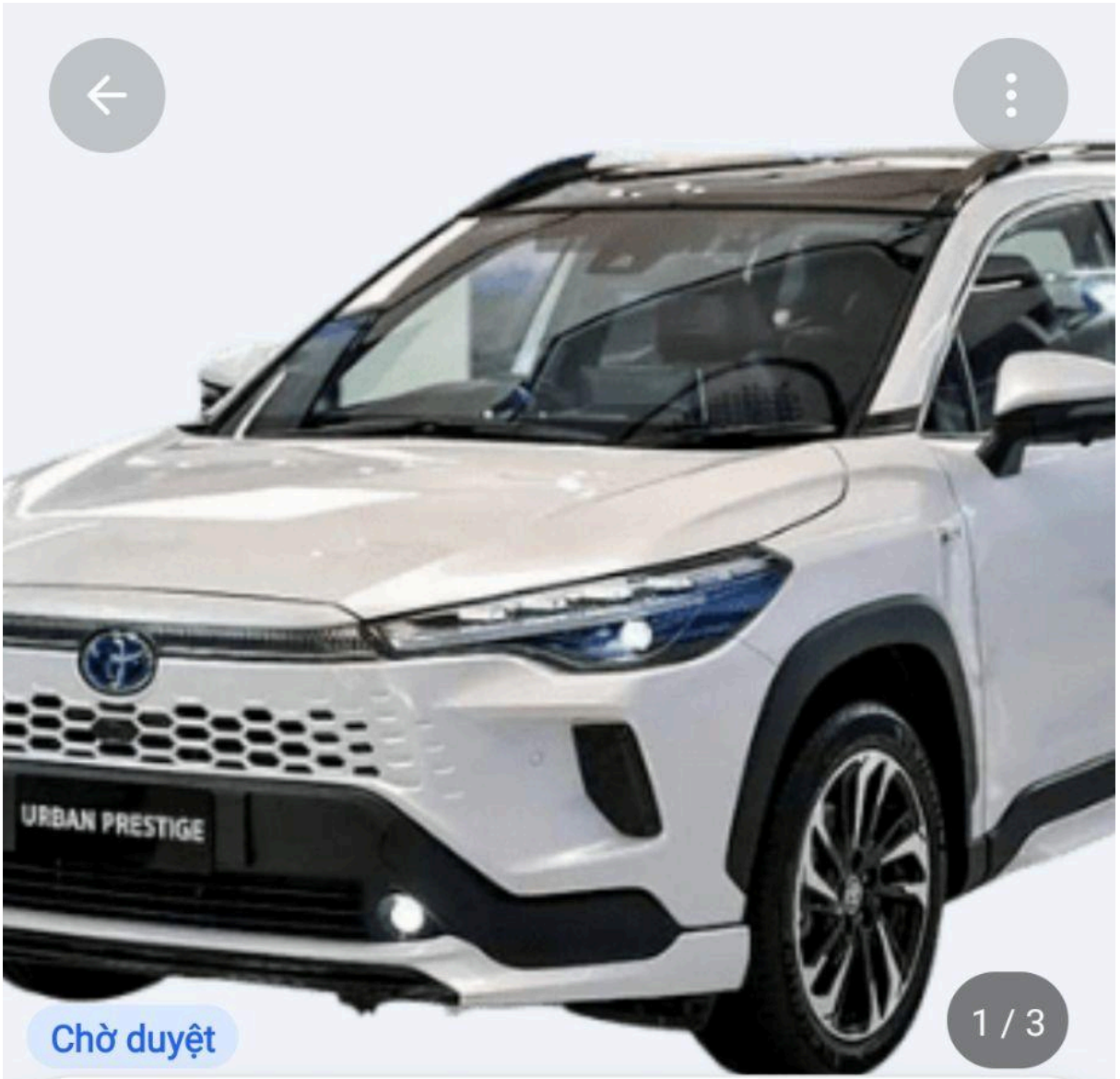
- **Function Trigger:** When the user (Driver/Owner) clicks on a specific car from the "Cars" Screen or from a search result.
- **Function Description:**
    - **Actors:** Driver, Owner
    - **Purpose:** To view detailed information about a specific car.
    - **Interface:** Car Detail Screen
- **Data Processing:**
    - **Data Collect:** Collects the car's unique ID. Retrieves the car's details from the database using the ID.
    - **Data Computation:** No significant computation is needed.
    - **Data Binding:** Displays all relevant information about the car on the Car Detail Screen (e.g., images, description, features, rental price/availability).
- **Screen Layout:**



- **Function Details:** On clicking on a specific car, the system displays detailed information about the car.

3.35.3 Get unavailability date

- **Function Trigger:** When the user (Driver/Owner) views the "Car Detail Screen" or during the booking process.
- **Function Description:**
    - **Actors:** Driver, Owner
    - **Purpose:** To view/check the dates when the car is not available for booking.
    - **Interface:** Car Detail Screen, Create Booking Screen
- **Data Processing:**
    - **Data Collect:** Collects the car's unique ID. Retrieves a list of booked dates (or unavailable dates) associated with the car from the database.
    - **Data Computation:** No significant computation is needed.
    - **Data Binding:** Displays the unavailable dates on the Car Detail Screen, possibly using a calendar view.
- **Screen Layout:**



Chờ duyệt

1 / 3

Thông tin xe | Đặt xe | **Thời gian** | Hợp đồng

Thời gian không cho thuê xe

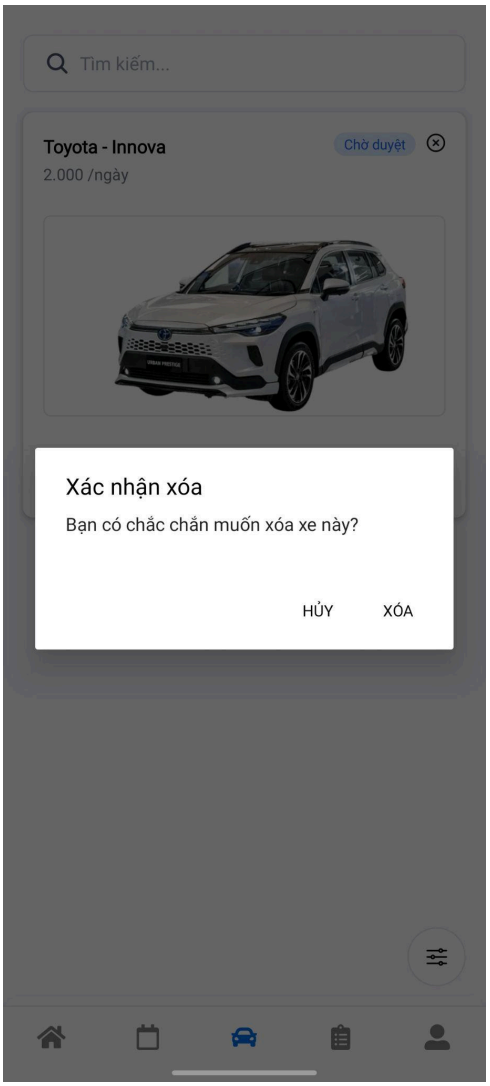
Tháng 5 2025

CN	T2	T3	T4	T5	T6	T7
27	28	29	30	1	2	3

- **Function Details:** Shows which dates the car is not available for rental.

3.35.4 Delete Car

- **Function Trigger:** When the owner clicks the "Delete" button next to a car on the "Cars Screen".
- **Function Description:**
    - **Actors:** Owner
    - **Purpose:** To remove a registered car from the list.
    - **Interface:** Cars Screen
- **Data Processing:**
    - **Data Collect:** Collects the car's unique ID.
    - **Data Computation:** Verifies that the car can be deleted (e.g., no active bookings). May display a confirmation dialog.
    - **Data Storage:** Deletes the car from the database.
- **Screen Layout:**



- **Function Details:** Deletes the car and all info of the car.

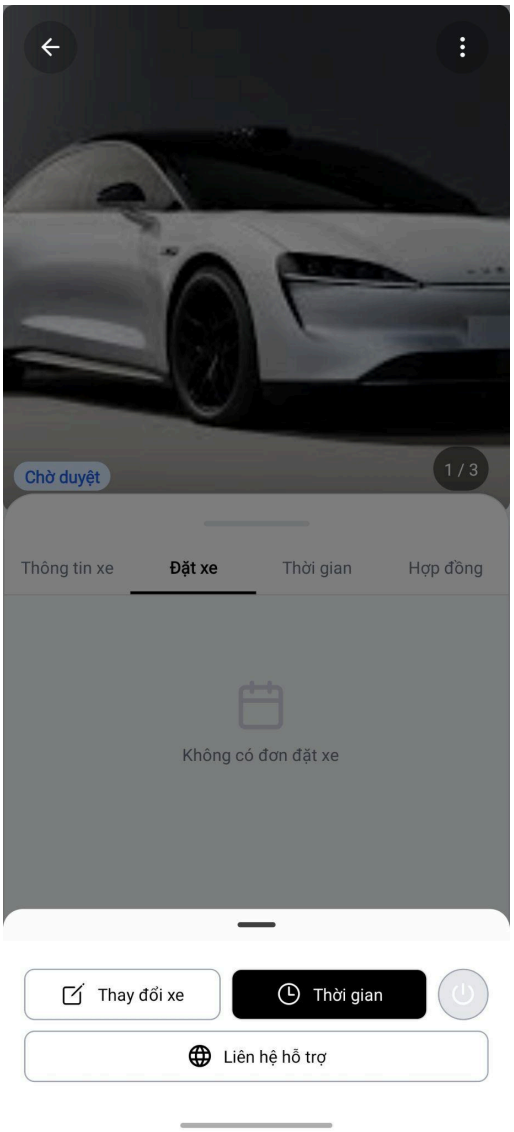
3.35.5 View Contact

- **Function Trigger:** When the owner clicks the "View Renter Contact" button on the "Car Detail Screen" (presumably only when a car is currently booked).
- **Function Description:**
    - **Actors:** Owner
    - **Purpose:** To view the renter's contact information for a specific car's booking.
    - **Interface:** Car Detail Screen
- **Data Processing:**
    - **Data Collect:** Collects the car's ID. Identifies the current booking associated with that car. Retrieves the renter's contact information from the database, linked to that booking.
    - **Data Computation:** No significant computation is needed.
    - **Data Binding:** Displays the renter's contact information (e.g., name, phone number) on the Car Detail Screen.
- **Screen Layout:**

- **Function Details:** See the renter contact.

3.35.6 Disable Car

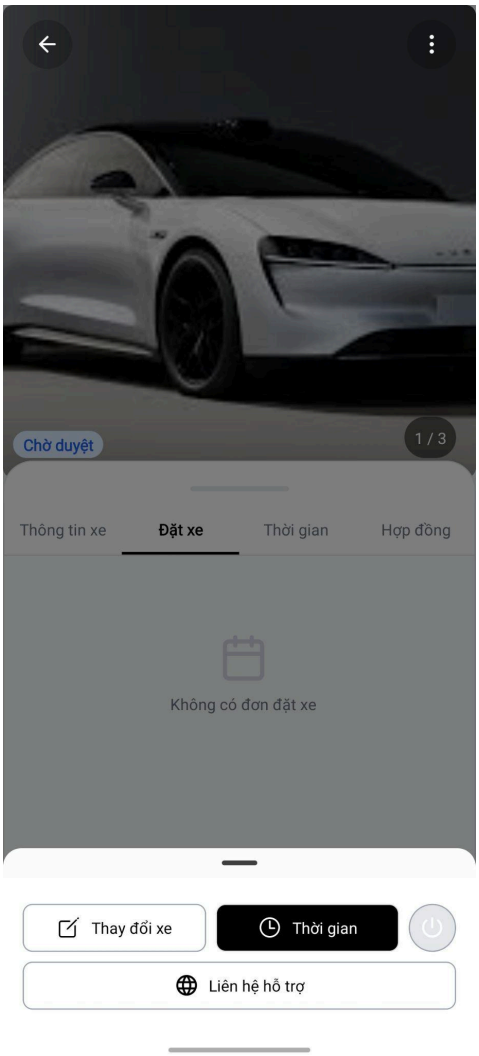
- **Function Trigger:** When the owner clicks the "Disable Car" button on the "Car Detail Screen".
- **Function Description:**
    - **Actors:** Owner
    - **Purpose:** To temporarily prevent a car from being booked.
    - **Interface:** Car Detail Screen
- **Data Processing:**
    - **Data Collect:** Collects the car's unique ID.
    - **Data Computation:** No significant computation is needed.
    - **Data Storage:** Updates the car's status in the database to "Disabled". This status should prevent new bookings from being created.
- **Screen Layout:**



- **Function Details:** Prevent the car from any booking.

3.35.7 Enable Car

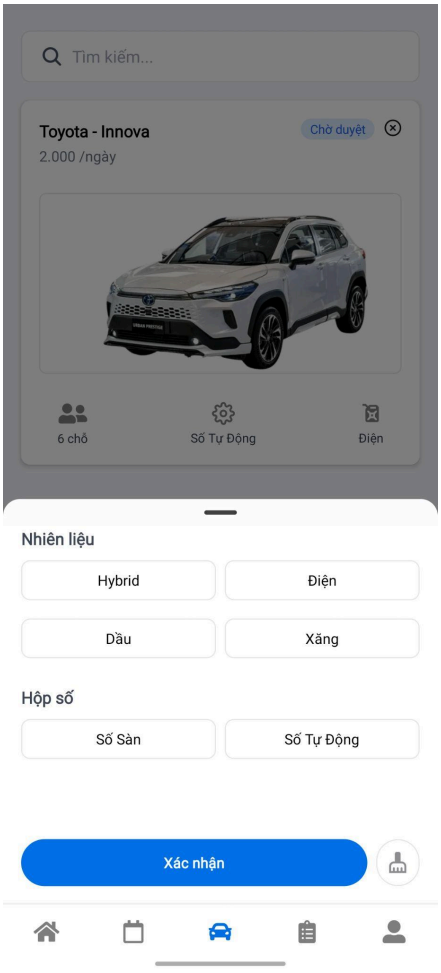
- **Function Trigger:** When the owner clicks the "Enable Car" button on the "Car Detail Screen" (only displayed when the car is currently disabled).
- **Function Description:**
    - **Actors:** Owner
    - **Purpose:** To re-enable a disabled car, allowing it to be booked again.
    - **Interface:** Car Detail Screen
- **Data Processing:**
    - **Data Collect:** Collects the car's unique ID.
    - **Data Computation:** No significant computation is needed.
    - **Data Storage:** Updates the car's status in the database to "Enabled".
- **Screen Layout:**



- **Function Details:** Allow the car can be book again.

3.35.9 List Fuel

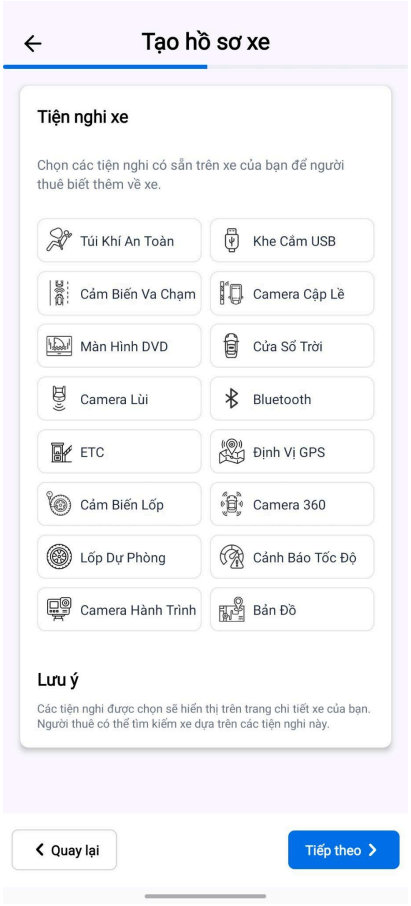
- **Function Trigger:** When the owner navigates to the "Edit Car Screen" (either to create a new car or edit an existing one). Specifically, when the owner interacts with the "Fuel Type" field.
- **Function Description:**
    - **Actors:** Owner
    - **Purpose:** To provide a list of available fuel types for selection when adding or editing a car.
    - **Interface:** Edit Car Screen
- **Data Processing:**
    - **Data Collect:** Retrieves a list of fuel types from the database (e.g., Gasoline, Diesel, Electric, Hybrid).
    - **Data Computation:** No significant computation is needed.
    - **Data Binding:** Displays the list of fuel types in a dropdown menu or similar UI element on the Edit Car Screen.
- **Screen Layout:**



- **Function Details:** Display list of fuel.

3.35.10 List Amenity

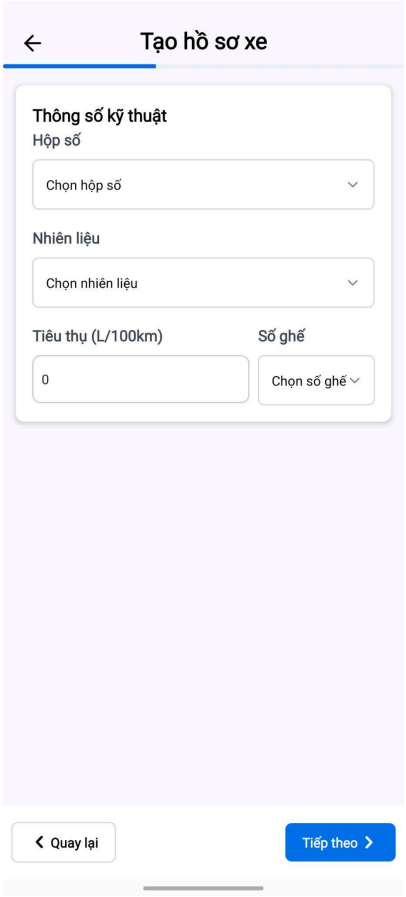
- **Function Trigger:** When the owner navigates to the "Edit Car Screen" (either to create a new car or edit an existing one). Specifically, when the owner interacts with the "Amenities" section.
- **Function Description:**
    - **Actors:** Owner
    - **Purpose:** To provide a list of available car amenities for selection when adding or editing a car.
    - **Interface:** Edit Car Screen
- **Data Processing:**
    - **Data Collect:** Retrieves a list of car amenities from the database (e.g., Air Conditioning, GPS, Bluetooth, Sunroof).
    - **Data Computation:** No significant computation is needed.
    - **Data Binding:** Displays the list of car amenities using checkboxes, a multi-select dropdown, or a similar UI element on the Edit Car Screen.
- **Screen Layout:**



- **Function Details:** Displays all amenities.

3.35.11 List Transmission

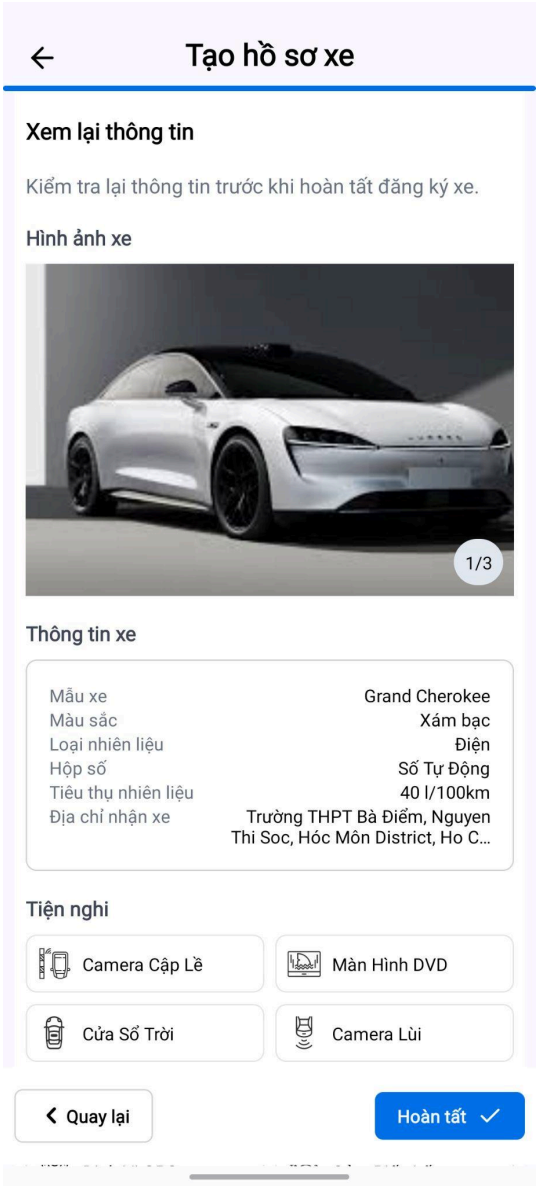
- **Function Trigger:** When the owner navigates to the "Edit Car Screen" (either to create a new car or edit an existing one). Specifically, when the owner interacts with the "Transmission Type" field.
- **Function Description:**
    - **Actors:** Owner
    - **Purpose:** To provide a list of available transmission types for selection when adding or editing a car.
    - **Interface:** Edit Car Screen
- **Data Processing:**
    - **Data Collect:** Retrieves a list of transmission types from the database (e.g., Automatic, Manual).
    - **Data Computation:** No significant computation is needed.
    - **Data Binding:** Displays the list of transmission types in a dropdown menu or similar UI element on the Edit Car Screen.
- **Screen Layout:**



- **Function Details:** List transmission type.

3.35.12 Create Car

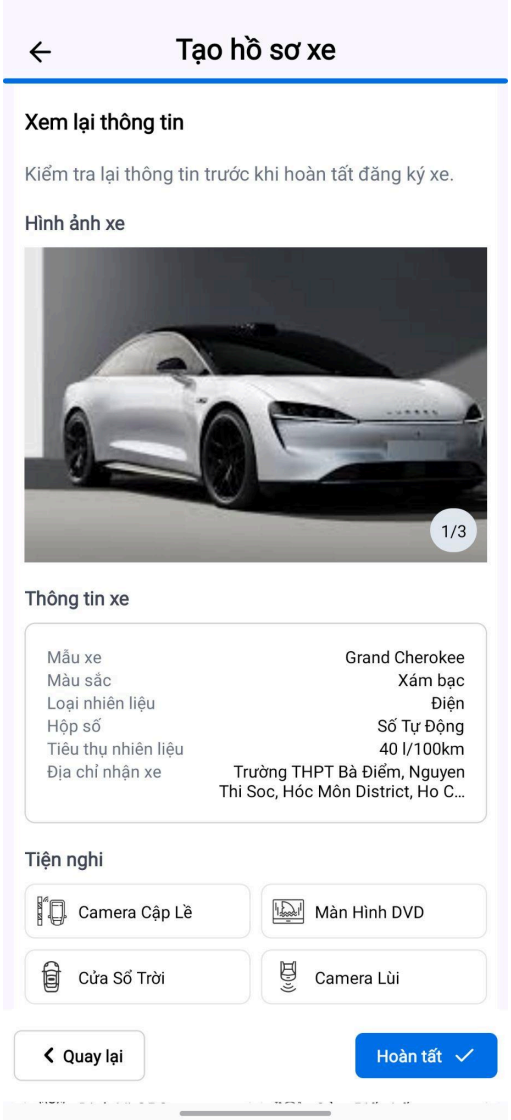
- **Function Trigger:** When the owner clicks the "Save" or "Create Car" button on the "Edit Car Screen" after filling in the car details.
- **Function Description:**
    - **Actors:** Owner
    - **Purpose:** To add a new car to the registered vehicle list.
    - **Interface:** Edit Car Screen
- **Data Processing:**
    - **Receive Data Input:** Receives car details from the Edit Car Screen (e.g., model, year, fuel type, amenities, license plate, rental price).
    - **Data Computation:** Validates the input data.
    - **Data Storage:** Stores the new car information in the database, associating it with the owner's account.
- **Screen Layout:**



- **Function Details:** New car and details.

3.35.13 Update Car

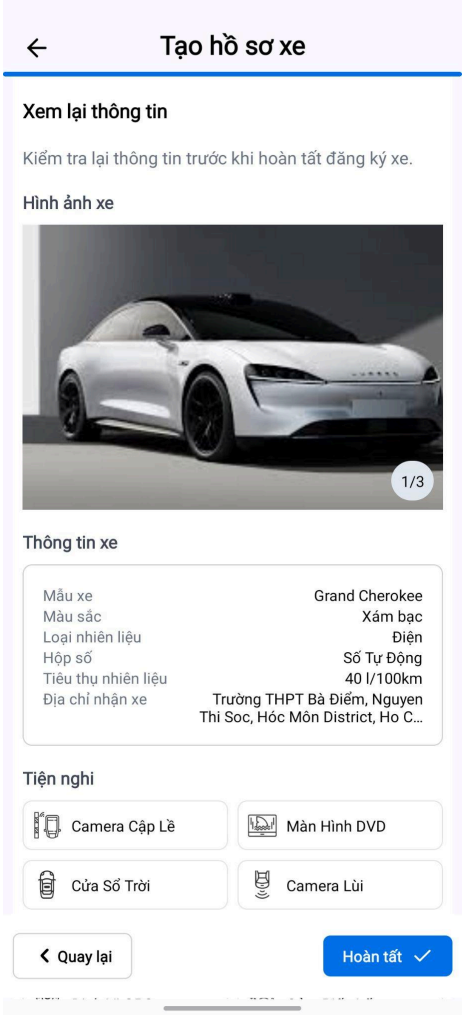
- **Function Trigger:** When the owner clicks the "Save" or "Update Car" button on the "Edit Car Screen" after modifying the car details.
- **Function Description:**
    - **Actors:** Owner
    - **Purpose:** To modify details of an existing car listing.
    - **Interface:** Edit Car Screen
- **Data Processing:**
    - **Receive Data Input:** Receives the updated car details from the Edit Car Screen.
    - **Data Computation:** Validates the input data.
    - **Data Storage:** Updates the car information in the database.
- **Screen Layout:**



- **Function Details:** Update car and details.

3.35.14 Upload Image Car

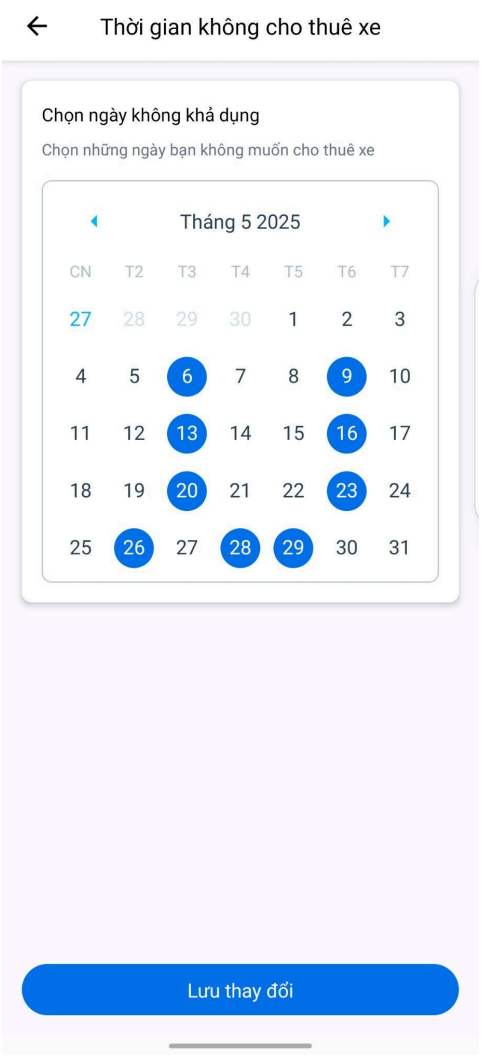
- **Function Trigger:** When the owner selects image files and confirms the upload on the "Edit Car Screen".
- **Function Description:**
    - **Actors:** Owner
    - **Purpose:** To upload photos of the car.
    - **Interface:** Edit Car Screen
- **Data Processing:**
    - **Receive Data Input:** Receives the image files uploaded by the owner.
    - **Data Computation:** May perform image validation (e.g., file size, file type).
    - **Data Storage:** Stores the image files in a storage service and saves the image URLs or paths in the database, associated with the car.
- **Screen Layout:**



- **Function Details:** Upload images of the car.

3.35.15 Create unavailability date

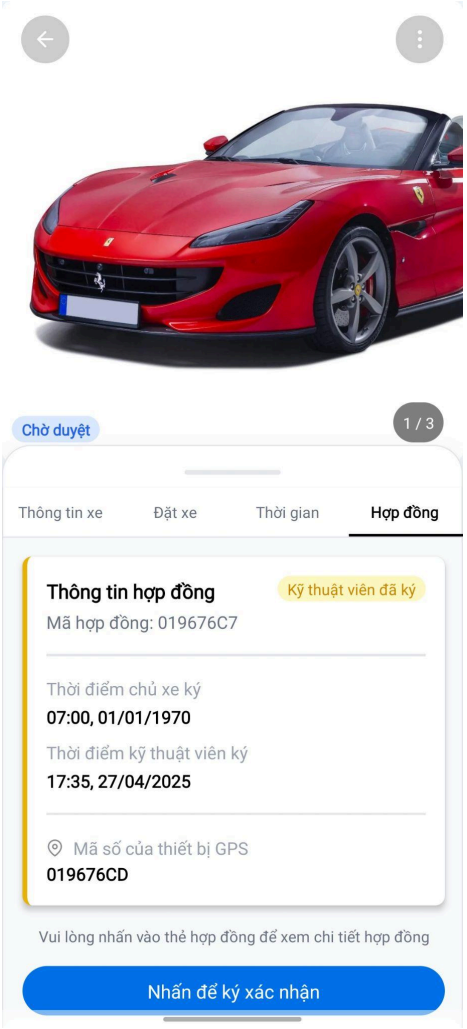
- **Function Trigger:** When the owner selects a date range and clicks the "Save" or "Block Dates" button on the "Unavalibility Schedule Screen".
- **Function Description:**
    - **Actors:** Owner
    - **Purpose:** To block out dates when a car is not available for rent (e.g., for maintenance, personal use).
    - **Interface:** Unavalibility Schedule Screen
- **Data Processing:**
    - **Receive Data Input:** Receives the car ID and the date range from the Unavalibility Schedule Screen.
    - **Data Computation:** No significant computation is needed.
    - **Data Storage:** Stores the unavailability dates in the database, associating them with the car.
- **Screen Layout:**



- **Function Details:** Dates car will be unavailable.

3.35.16 View Contract Car

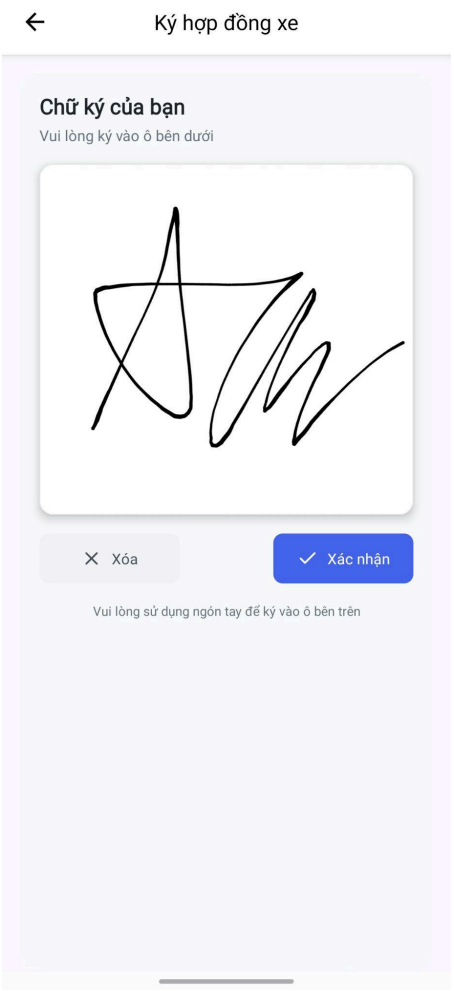
- **Function Trigger:** When the owner clicks the "View Contract" button on the "Contract Car Screen".
- **Function Description:**
    - **Actors:** Owner
    - **Purpose:** To view the car registration contract.
    - **Interface:** Contract Car Screen
- **Data Processing:**
    - **Data Collect:** Retrieves the contract document (or a link to it) from the database or a storage service, associating it with the car.
    - **Data Computation:** No significant computation is needed.
    - **Data Binding:** Displays the car registration contract on the Contract Car Screen.
- **Screen Layout:**



- **Function Details:** Contract that the owner signed.

3.35.17 Contract sign car

- **Function Trigger:** When the owner completes signing their name/initials in the required field and clicks the "Submit" button on the "Signature Screen".
- **Function Description:**
    - **Actors:** Owner
    - **Purpose:** To digitally sign the car registration contract.
    - **Interface:** Signature Screen
- **Data Processing:**
    - **Receive Data Input:** Receives the signature data (e.g., captured signature image, signature data in a specific format).
    - **Data Computation:** May validate the signature data (e.g., ensure a signature is present).
    - **Data Storage:** Stores the signature data and associates it with the car registration contract in the database. May also store the completed contract document (including the signature) in a storage service.
- **Screen Layout:**



- **Function Details:** Owner sign contract.

3.35.18 Tracking location car

- **Function Trigger:** When the owner navigates to the "Map Screen" and selects a specific car to track.
- **Function Description:**
    - **Actors:** Owner
    - **Purpose:** To display a car's current location (if GPS equipped).
    - **Interface:** Map Screen
- **Data Processing:**
    - **Data Collect:** Collects the car's unique ID. Retrieves the car's current location data from the GPS tracking system (or the database, if the location is periodically updated).
    - **Data Computation:** No significant computation is needed.
    - **Data Binding:** Displays the car's location on the Map Screen, typically using a map marker.
- **Screen Layout:**

- **Function Details:** The car must have a GPS to track.

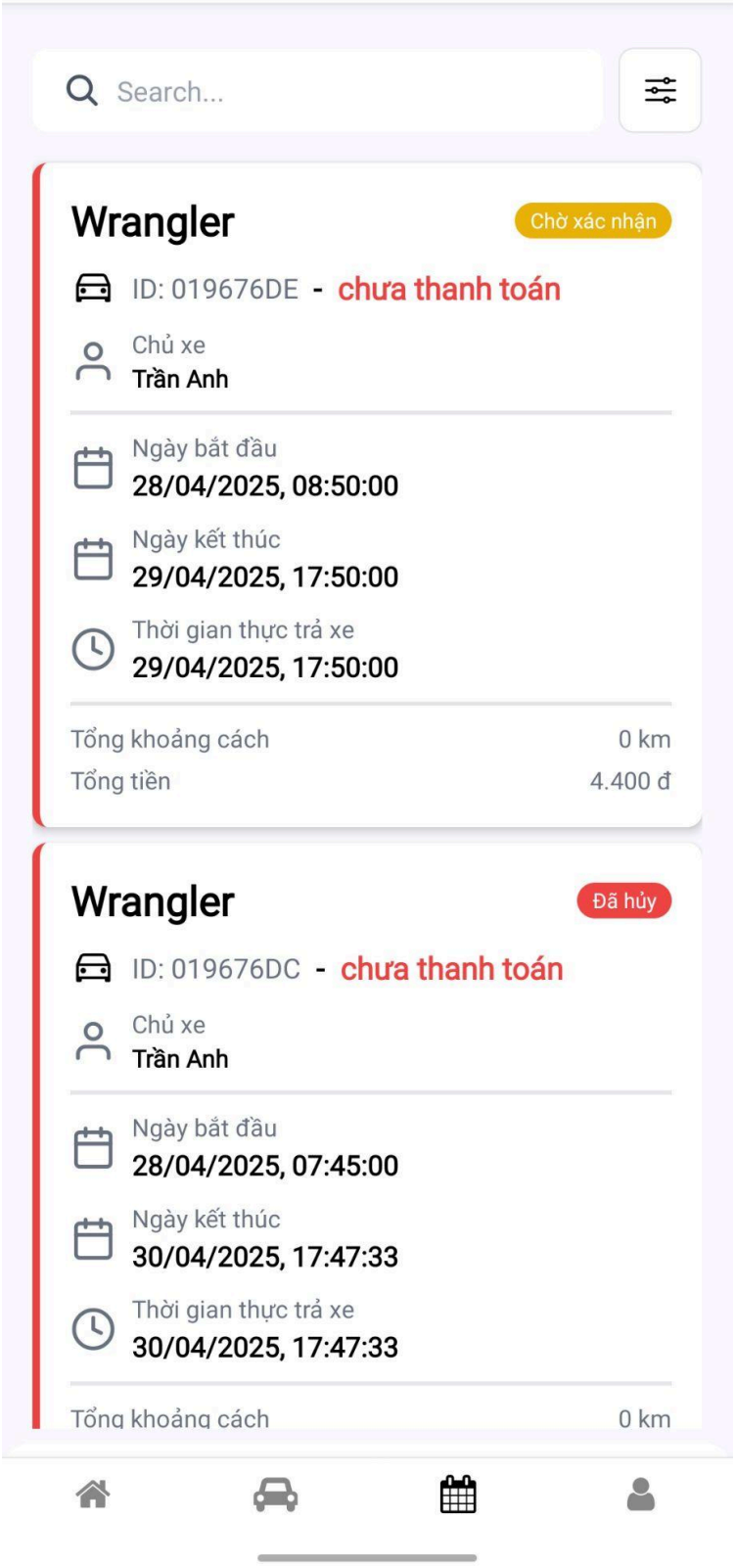
3.36.1 Contract sign booking

- **Function Trigger:** When the user (Driver/Owner) clicks the "Sign Contract" button on the Signature Screen during the booking process.

- **Function Description:**
    - **Actors:** Driver, Owner
    - **Purpose:** To digitally sign the rental contract.
    - **Interface:** Signature Screen
- **Data Processing:**
    - **Receive Data Input:** Receives the user's signature data (e.g., captured signature image, signature data in a specific format).
    - **Data Computation:** May validate the signature data (e.g., ensure a signature is present).
    - **Data Storage:** Stores the signature data and associates it with the booking in the database. May also store the completed contract document (including the signature) in a storage service.
- **Screen Layout:** it to the rental agreement.
- **Function Details:** On clicking "Sign Contract," the system captures the user's signature, saves it, and links it to the rental agreement.

3.36.2 List Booking

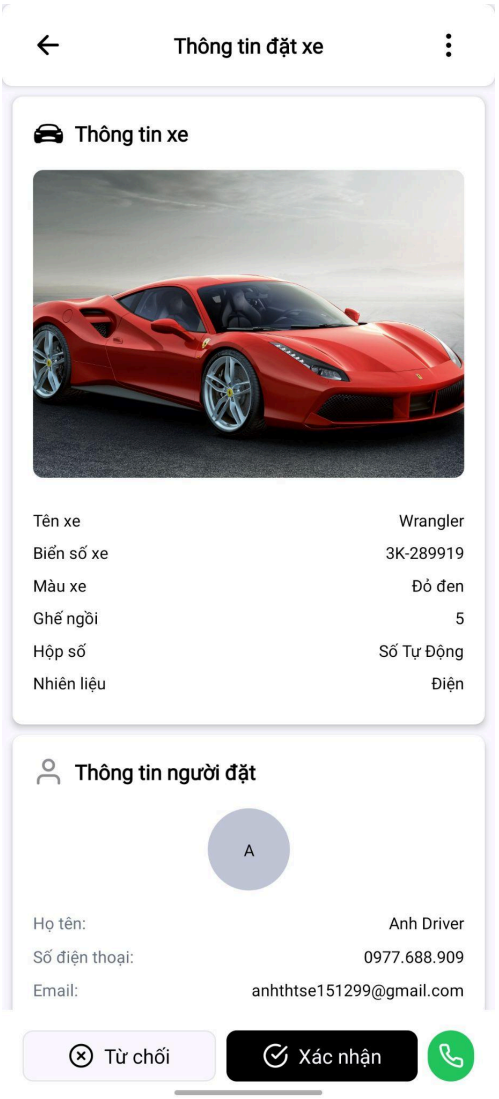
- **Function Trigger:** When the user (Driver/Owner) navigates to the "Booking Screen" or "Home Screen".
- **Function Description:**
    - **Actors:** Driver, Owner
        - **Purpose:** To display a list of bookings (current and past bookings for drivers, all bookings for owner's cars for owners).
- **Interface:** Booking Screen, Home Screen
- **Data Processing:**
    - **Data Collect:**
        - For Drivers: Retrieves a list of bookings associated with the logged-in driver from the database.
        - For Owners: Retrieves a list of all bookings for cars registered by the logged-in owner from the database.
    - **Data Computation:** Filters and sorts the bookings (e.g., by date, status).
    - **Data Binding:** Displays the list of bookings on the Booking Screen or Home Screen, including relevant information (e.g., car details, booking dates, status).
- **Screen Layout:**



- **Function Details:** On navigating to the Booking Screen or Home screen, the user sees a list of their relevant bookings. For owners, the Home Screen shows a summary view while the Booking Screen provides more detailed functionality.

3.36.3 Get Booking by id

- **Function Trigger:** When the user (Driver/Owner) clicks on a specific booking from the "Booking Screen" list or when viewing compensation forms.
- **Function Description:**
    - **Actors:** Driver, Owner
    - **Purpose:** To view detailed information for a specific booking.
    - **Interface:** Booking Detail Screen, Pre and Post Compensation Screen
- **Data Processing:**
    - **Data Collect:** Collects the booking's unique ID. Retrieves the booking details from the database using the ID.
    - **Data Computation:** No significant computation is needed.
    - **Data Binding:** Displays all relevant information about the booking on the Booking Detail Screen (e.g., car details, booking dates, total cost, status, contract information).
- **Screen Layout:**



- **Function Details:** On clicking on a specific booking, the system displays its detailed information.

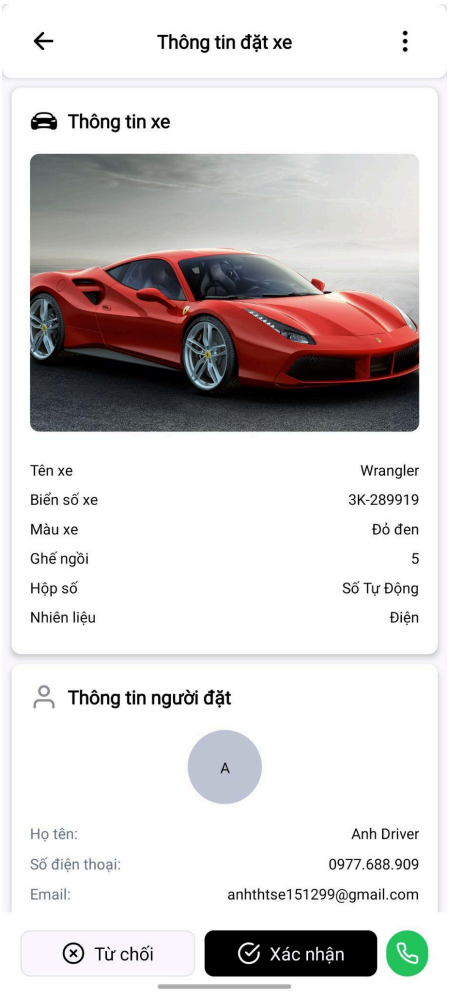
3.36.4 View Contract Booking

- **Function Trigger:** When the user (Driver/Owner) clicks the "View Contract" button on the "Contract Booking Screen" or "Booking Detail Screen."
- **Function Description:**
    - **Actors:** Driver, Owner
    - **Purpose:** To display the rental contract for a specific booking.
    - **Interface:** Contract Booking Screen
- **Data Processing:**
    - **Data Collect:** Collects the booking ID. Retrieves the contract document (or a link to it) from the database or a storage service, associating it with the booking.
    - **Data Computation:** No significant computation is needed.
    - **Data Binding:** Displays the rental contract on the Contract Booking Screen.
- **Screen Layout:**

- **Function Details:** Allows user to see what contract they signed.

3.36.5 Approve Booking

- **Function Trigger:** When the owner clicks the "Approve" button on the "Booking Detail Screen" for a booking with a "Pending" status.
- **Function Description:**
    - **Actors:** Owner
    - **Purpose:** To accept a pending booking request.
    - **Interface:** Booking Detail Screen
- **Data Processing:**
    - **Data Collect:** Collects the booking ID.
    - **Data Computation:** No significant computation is needed.
    - **Data Storage:** Updates the booking's status in the database to "Approved" (or similar). May trigger notifications to the renter.
- **Screen Layout:**

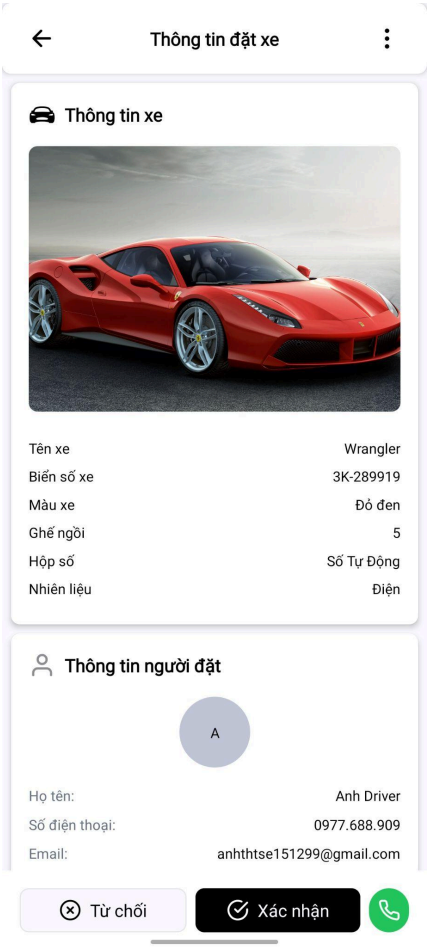


- **Function Details:** The owner accept the booking.

3.36.6 Reject Booking

- **Function Trigger:** When the owner clicks the "Reject" button on the "Booking Detail Screen" for a booking with a "Pending" status.

- **Function Description:**
    - **Actors:** Owner
    - **Purpose:** To decline a pending booking request.
    - **Interface:** Booking Detail Screen
- **Data Processing:**
    - **Data Collect:** Collects the booking ID.
    - **Data Computation:** No significant computation is needed. May require the owner to provide a reason for rejection.
    - **Data Storage:** Updates the booking's status in the database to "Rejected" (or similar). May trigger notifications to the renter.
- **Screen Layout:**

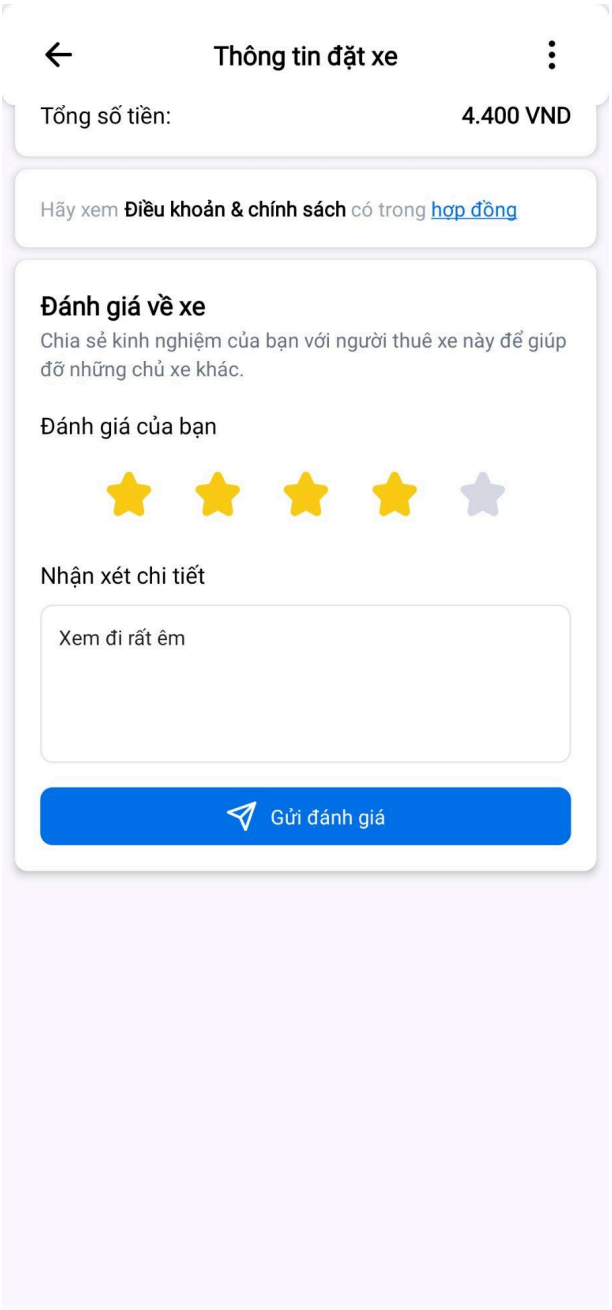


- **Function Details:** Owner declines the booking.

3.36.7 Feedback Booking

- **Function Trigger:** When the owner clicks the "Submit Feedback" button on the "Booking Detail Screen" after a booking has been completed.
- **Function Description:**
    - **Actors:** Owner

    - **Purpose:** To provide feedback or notes about a booking. This might be internal notes for record-keeping.
    - **Interface:** Booking Detail Screen
- **Data Processing:**
    - **Receive Data Input:** Receives the owner's feedback from the Booking Detail Screen.
    - **Data Computation:** No significant computation is needed.
    - **Data Storage:** Stores the feedback in the database, associating it with the booking.
- **Screen Layout:**

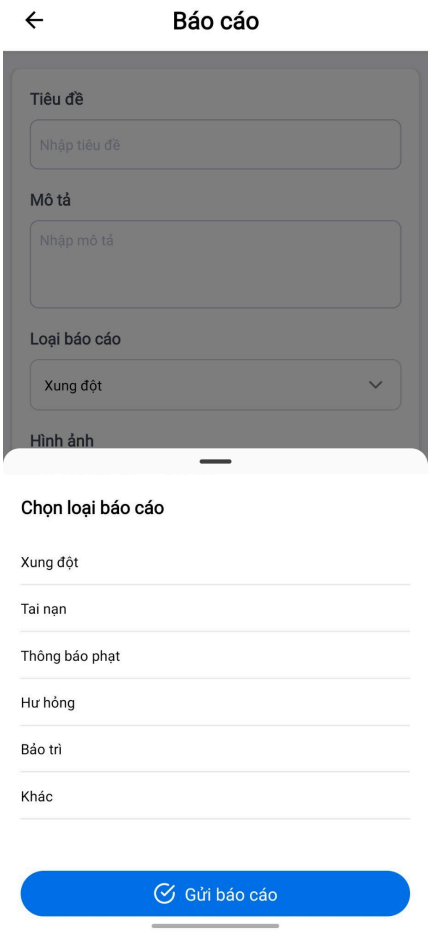


- **Function Details:** Owner adds feedback or notes about booking.

---

a3.36.8 Create Booking Report

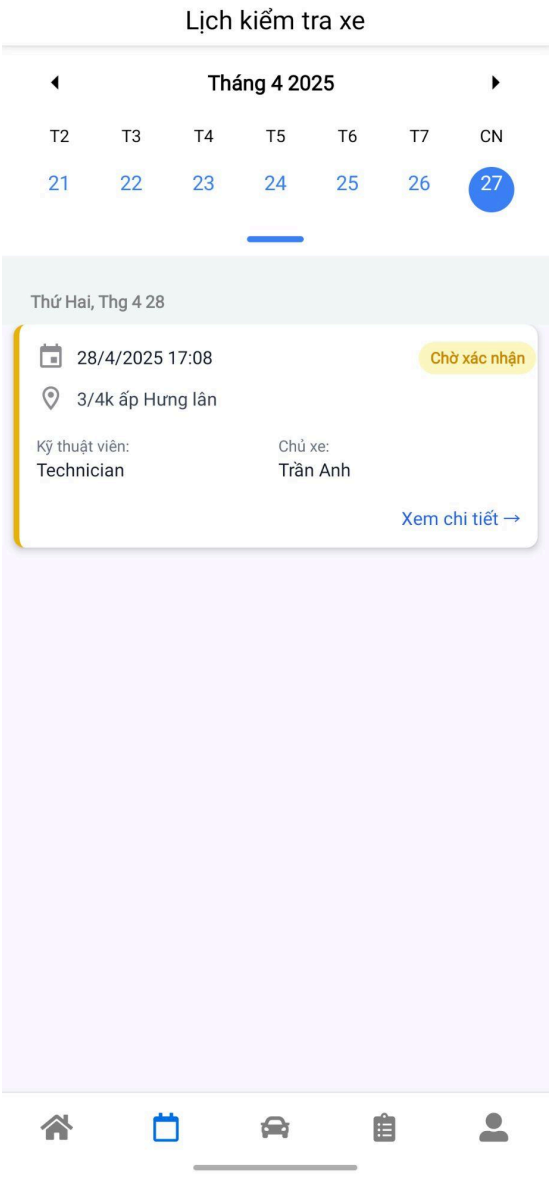
- **Function Trigger:** When the owner clicks the "Submit Report" button on the "Create Booking Report Screen."
- **Function Description:**
    - **Actors:** Owner
    - **Purpose:** To create a report related to a specific booking (e.g., renter behavior, car issues).
    - **Interface:** Create Booking Report Screen
- **Data Processing:**
    - **Receive Data Input:** Receives the report details from the Create Booking Report Screen (e.g., type of issue, description).
    - **Data Computation:** No significant computation is needed at this stage. The data is collected for review and action.
    - **Data Storage:** Stores the booking report information in the database, associating it with the booking.
- **Screen Layout:**



- **Function Details:** Report for specific booking.

3.36.9 Get list schedule

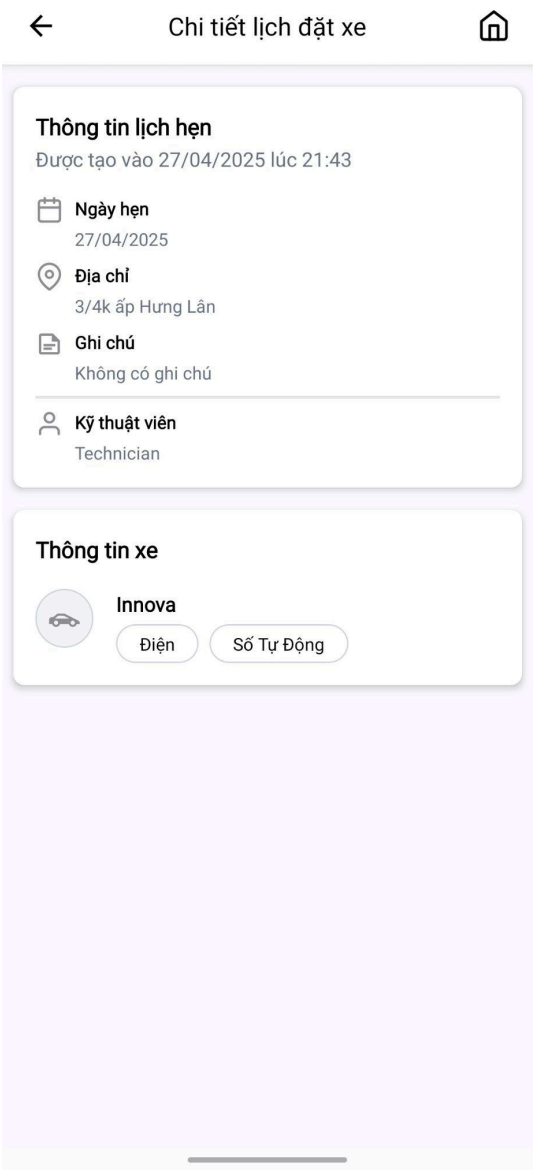
- **Function Trigger:** When the owner navigates to the "View Inspection Schedule Screen."
- **Function Description:**
    - **Actors:** Owner
    - **Purpose:** To display a list of scheduled vehicle inspections for their cars.
    - **Interface:** View Inspection Schedule Screen
- **Data Processing:**
    - **Data Collect:** Retrieves a list of scheduled inspections for all cars registered by the logged-in owner from the database.
    - **Data Computation:** Filters and sorts the schedules.
    - **Data Binding:** Displays the list of inspections.
- **Screen Layout:**



- **Function Details:** Shows the schedule to the owner.

3.36.10 Get schedule by id

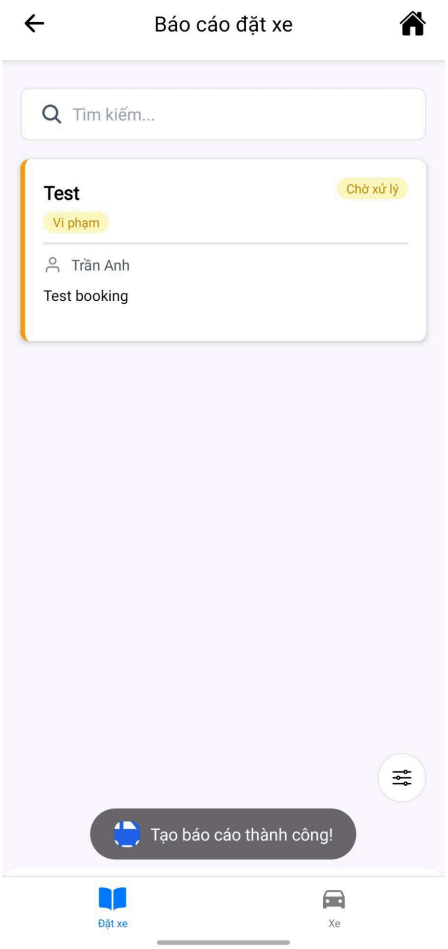
- **Function Trigger:** When the owner clicks on a specific inspection schedule from the "View Inspection Schedule Screen."
- **Function Description:**
    - **Actors:** Owner
    - **Purpose:** To show detailed information for a specific inspection schedule.
    - **Interface:** View Detail Inspection Schedule Screen
- **Data Processing:**
    - **Data Collect:** Collects the inspection schedule's unique ID. Retrieves the schedule details from the database using the ID.
    - **Data Computation:** No significant computation is needed.
    - **Data Binding:** Displays all relevant information about the inspection schedule.
- **Screen Layout:**



- **Function Details:** Details of the specific inspection.

3.36.11 Get list booking reports

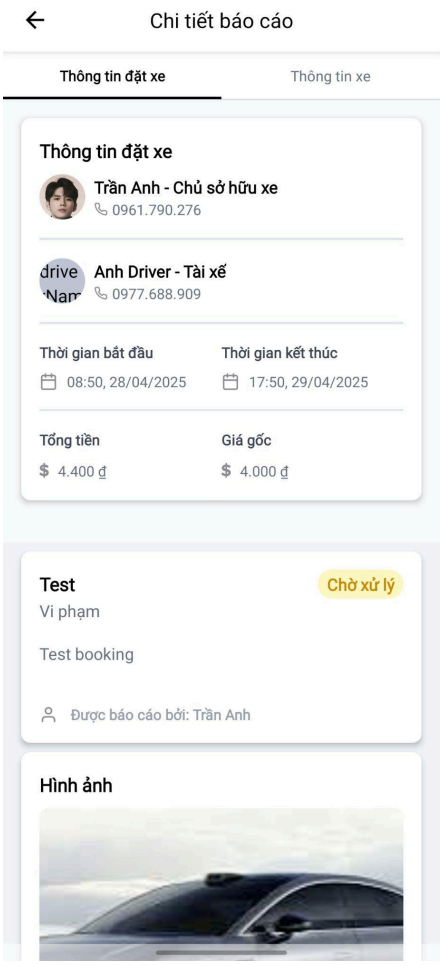
- **Function Trigger:** When the owner navigates to the "List Booking Reports Screen".
- **Function Description:**
    - **Actors:** Owner
    - **Purpose:** To display a list of booking reports created by the owner.
    - **Interface:** List Booking Reports Screen
- **Data Processing:**
    - **Data Collect:** Retrieves a list of booking reports associated with the logged-in owner from the database.
    - **Data Computation:** Filters and sorts the reports.
    - **Data Binding:** Displays the list of booking reports.
- **Screen Layout:**



- **Function Details:** Owner sees the reports.

3.36.12 Get booking report by id

- **Function Trigger:** When the owner clicks on a specific booking report from the "List Booking Reports Screen".
- **Function Description:**
    - **Actors:** Owner
    - **Purpose:** To show detailed information for a specific booking report.
    - **Interface:** Booking Reports Detail Screen
- **Data Processing:**
    - **Data Collect:** Collects the booking report's unique ID. Retrieves the report details from the database using the ID.
    - **Data Computation:** No significant computation is needed.
    - **Data Binding:** Displays all relevant information about the booking report.
- **Screen Layout:**

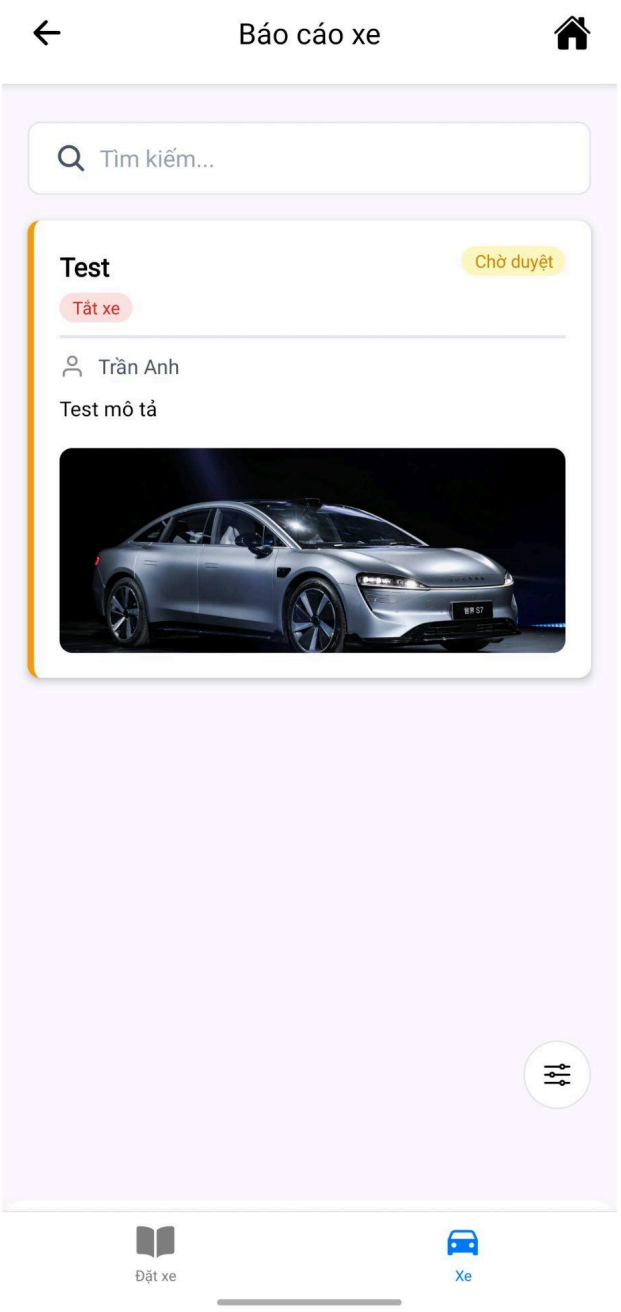


- **Function Details:** Details of the report.

3.36.13 Get list car reports

- **Function Trigger:** When the owner navigates to the "List car Reports Scree".
- **Function Description:**

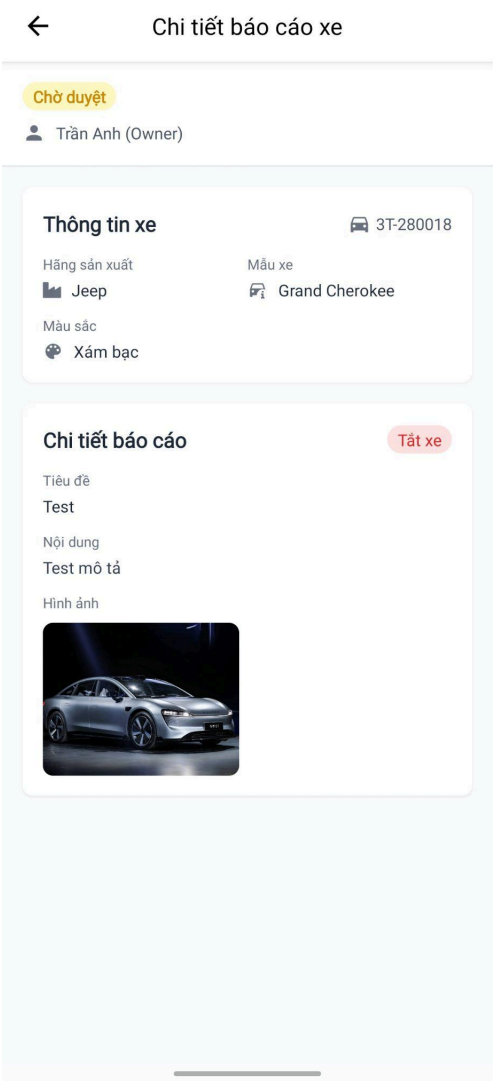
    - **Actors:** Owner
    - **Purpose:** To display a list of car reports created by the owner.
    - **Interface:** List car Reports Scree
- **Data Processing:**
    - **Data Collect:** Retrieves a list of car reports associated with cars owned by the logged-in owner from the database.
    - **Data Computation:** Filters and sorts the reports.
    - **Data Binding:** Displays the list of car reports.
- **Screen Layout:**



- **Function Details:** List of reports for owner's cars.

3.36.14 Get car report by id

- **Function Trigger:** When the owner clicks on a specific car report from the "Car Reports Detail Screen".
- **Function Description:**
    - **Actors:** Owner
    - **Purpose:** To show detailed information for a specific car report.
    - **Interface:** Car Reports Detail Screen
- **Data Processing:**
    - **Data Collect:** Collects the car report's unique ID. Retrieves the report details from the database using the ID.
    - **Data Computation:** No significant computation is needed.
    - **Data Binding:** Displays all relevant information about the car report.
- **Screen Layout:**



- **Function Details:** Detail of the report.

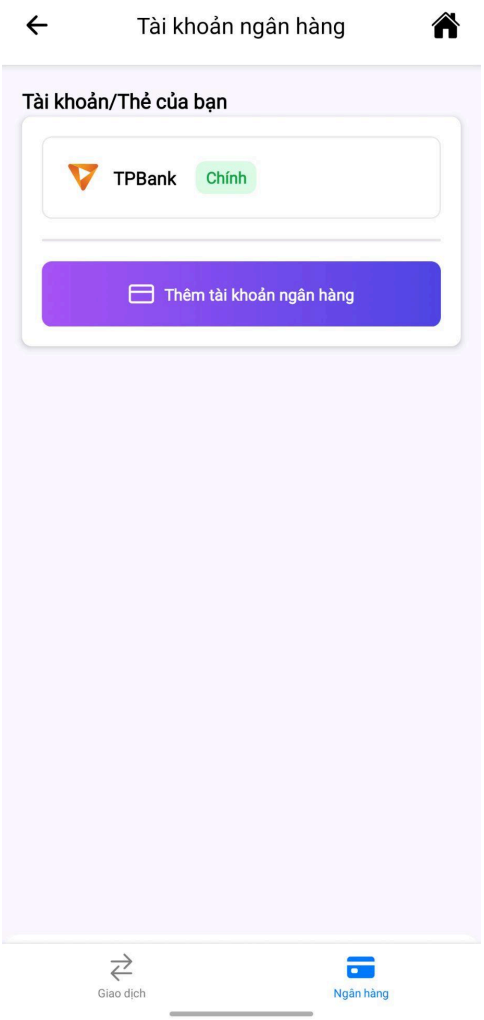
3.37.1 Get list transaction

- **Function Trigger:** When the user (Driver/Owner) navigates to the "Transaction List Screen".
- **Function Description:**
    - **Actors:** Driver, Owner
    - **Purpose:** To display a list of the user's financial transactions.
    - **Interface:** Transaction List Screen
- **Data Processing:**
    - **Data Collect:** Retrieves a list of financial transactions associated with the logged-in user from the database.
    - **Data Computation:** Filters and sorts the transactions (e.g., by date, type, status).
    - **Data Binding:** Displays the list of transactions on the Transaction List Screen.
- **Screen Layout:**

- **Function Details:** Shows list of financial transactions.

3.37.2 Get list bank account

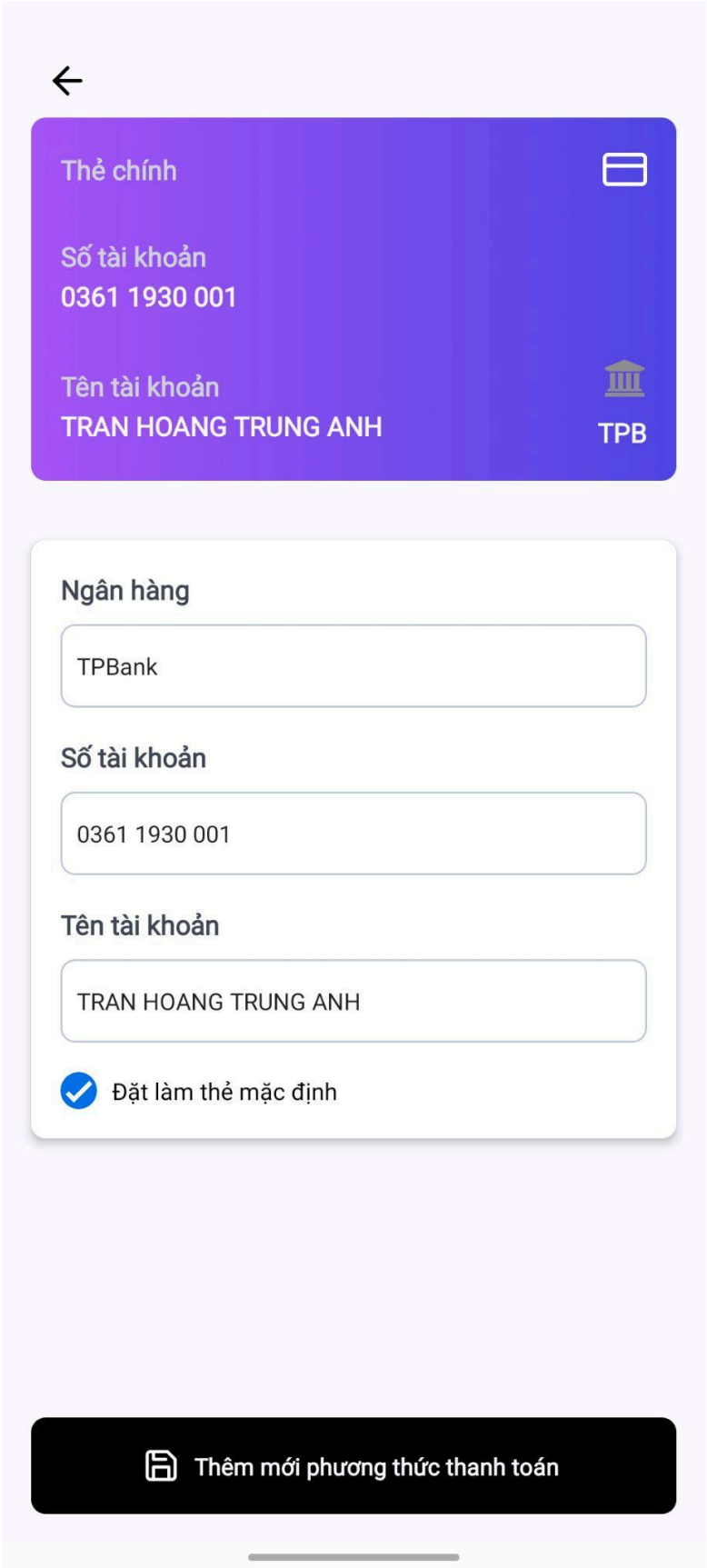
- **Function Trigger:** When the owner navigates to the "List Bank Account Screen".
- **Function Description:**
    - **Actors:** Owner
    - **Purpose:** To display a list of the owner's linked bank accounts for payments.
    - **Interface:** List Bank Account Screen
- **Data Processing:**
    - **Data Collect:** Retrieves a list of bank accounts associated with the logged-in owner from the database.
    - **Data Computation:** No significant computation is needed.
    - **Data Binding:** Displays the list of bank accounts.
- **Screen Layout:**



- **Function Details:** Banks account linked.

3.37.3 Get bank account by id

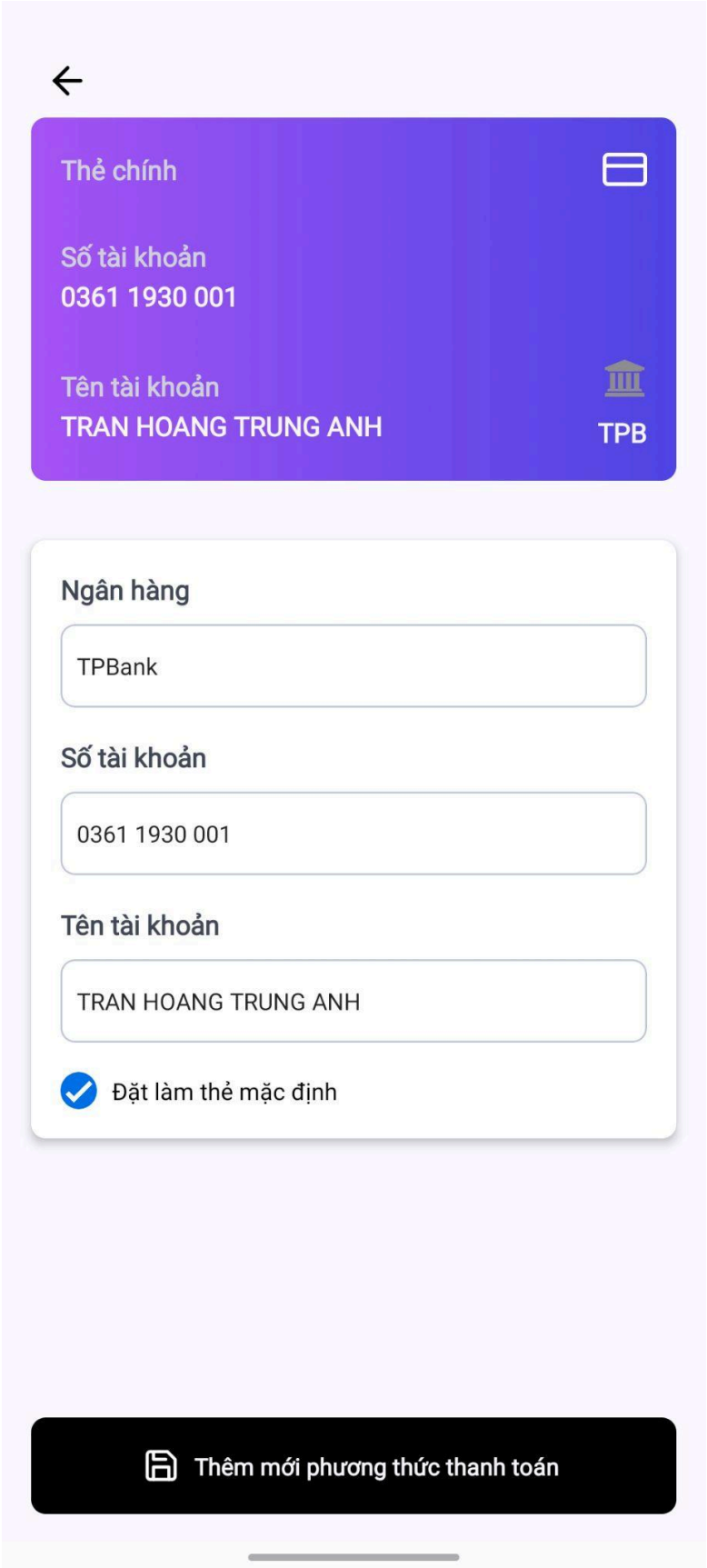
- **Function Trigger:** When the owner clicks on a specific bank account from the "List Bank Account Screen".
- **Function Description:**
    - **Actors:** Owner
    - **Purpose:** To show detailed information for a specific bank account.
    - **Interface:** Edit Bank Account Screen
- **Data Processing:**
    - **Data Collect:** Collects the bank account's unique ID. Retrieves the bank account details from the database using the ID.
    - **Data Computation:** No significant computation is needed.
    - **Data Binding:** Displays all relevant information about the bank account.
- **Screen Layout:**



- **Function Details:** Specific bank detail

3.37.4 Update bank acount

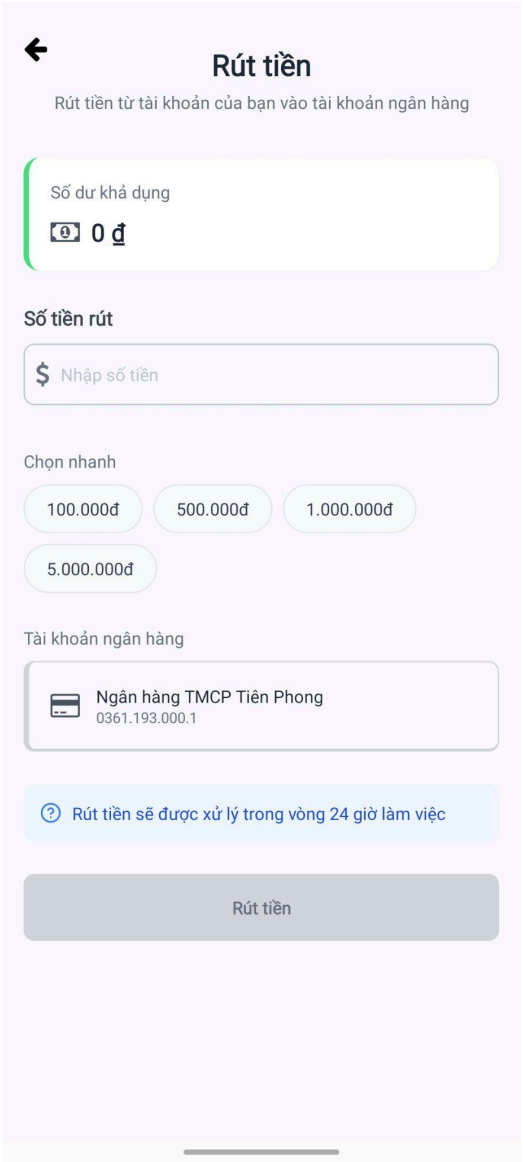
- **Function Trigger:** When the owner clicks the "Save" or "Update" button on the "Edit Bank Account Screen".
- **Function Description:**
    - **Actors:** Owner
    - **Purpose:** To modify details of a linked bank account.
    - **Interface:** Edit Bank Account Screen
- **Data Processing:**
    - **Receive Data Input:** Receives the updated bank account information from the Edit Bank Account Screen.
    - **Data Computation:** Validates the input data (e.g., account number format, routing number format).
    - **Data Storage:** Updates the bank account information in the database.
- **Screen Layout:**



- **Function Details:** Update info on bank detail.

3.37.5 Create Withdraw

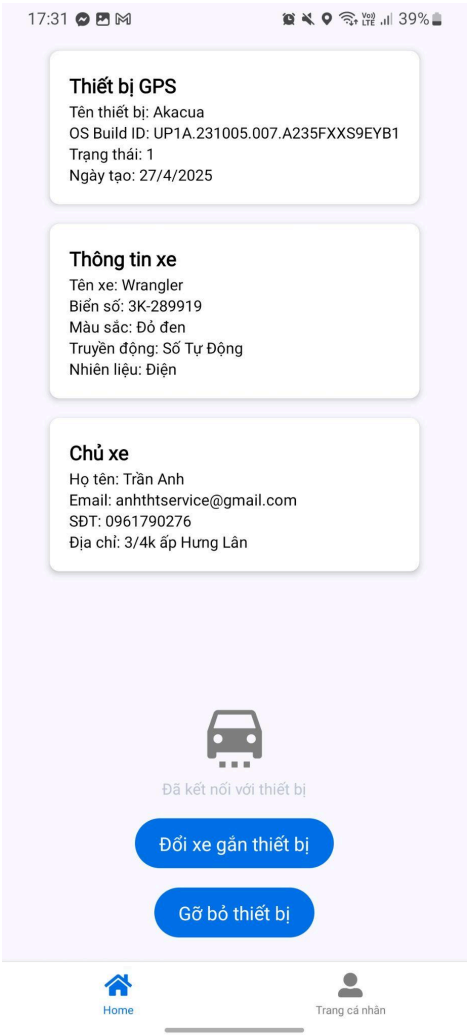
- **Function Trigger:** When the owner clicks the "Submit Withdrawal Request" button on the "Withdraw Request Scree".
- **Function Description:**
    - **Actors:** Owner
    - **Purpose:** To request a withdrawal of the owner's earnings.
    - **Interface:** Withdraw Request Scree
- **Data Processing:**
    - **Receive Data Input:** Receives the withdrawal amount and selected bank account from the Withdraw Request Scree.
    - **Data Computation:** Validates that the withdrawal amount is within the available balance and meets any minimum withdrawal requirements.
    - **Data Storage:** Creates a new withdrawal request record in the database, associating it with the owner and the selected bank account.
- **Screen Layout:**



- **Function Details:** Request payout.

3.38.1 View detail gps device

- **Function Trigger:** When the technician clicks on a specific GPS device listing on the Home Screen.
- **Function Description:**
    - **Actors:** Technician
    - **Purpose:** To display detailed information about a specific GPS tracking device.
    - **Interface:** Home Screen (leading to a "GPS Device Detail Screen" or a modal window)
- **Data Processing:**
    - **Data Collect:** Collects the GPS device's unique ID. Retrieves the device's details from the database using the ID (e.g., serial number, status, battery level, last known location, assigned car).
    - **Data Computation:** No significant computation is needed.
    - **Data Binding:** Displays all relevant information about the GPS device on the Home Screen (or a separate detail screen/modal).
- **Screen Layout:**



- **Function Details:** Details of GPS tracking.

3.38.2 Unassign device

- **Function Trigger:** When the technician clicks the "Unassign Device" button on the Home Screen (likely near the GPS device details or on a dedicated "GPS Device Detail Screen").
- **Function Description:**
    - **Actors:** Technician
    - **Purpose:** To remove the assignment of a GPS device from a car.
    - **Interface:** Home Screen (or a "GPS Device Detail Screen")
- **Data Processing:**
    - **Data Collect:** Collects the GPS device's unique ID.
    - **Data Computation:** No significant computation is needed, although a confirmation prompt might appear.
    - **Data Storage:** Updates the GPS device's record in the database to remove the association with the car.
- **Screen Layout:**

- **Function Details:** Remove device to specific car.

3.38.3 Get List car for staff

- **Function Trigger:** When the technician navigates to the "List Cars Screen".
- **Function Description:**
    - **Actors:** Technician
        - **Purpose:** To retrieve and display a list of cars with 'pending' and 'maintain' status for staff management (presumably for assigning GPS devices).
- **Interface:** List Cars Screen
- **Data Processing:**
    - **Data Collect:** Retrieves a list of cars from the database.
    - **Data Computation:** Filters the list to include only cars with a status of "pending" or "maintain".
    - **Data Binding:** Displays the filtered list of cars on the List Cars Screen, including relevant information (e.g., car model, registration number, status).
- **Screen Layout:**

- **Function Details:** Display cars for manage.

3.38.4 Assign device to car

- **Function Trigger:** When the technician selects a GPS device and a car on the "List Cars Screen" and clicks the "Assign Device" button.
- **Function Description:**
    - **Actors:** Technician
    - **Purpose:** To assign a specific GPS tracking device to a designated car.
    - **Interface:** List Cars Screen
- **Data Processing:**
    - **Data Collect:** Collects the GPS device's unique ID and the car's unique ID.
    - **Data Computation:** No significant computation is needed, but the system should verify that the device is not already assigned to another car.
    - **Data Storage:** Updates the GPS device's record in the database to associate it with the car. Updates the car's record to indicate that a GPS device has been assigned.
- **Screen Layout:**

- **Function Details:** Link device to the car.

---

4. Non-Functional Requirements

4.1 External Interfaces

4.1.1 User Interface

- User Interfaces are provided for different user roles:
    - Owner Mobile Application
    - Driver Mobile Application
    - Technician Tracking Application
    - Web Dashboard (for Admin, Consultant, Technician)
- The authenticated user's UI must be fully shown in the mobile app and web dashboard according to their authorized role.

4.1.2 Communications Interfaces

- The system communicates with external services for various functions.
- The system calls APIs through data transferred by HTTPS for integrating with external services.
- Integrations include:
    - Payment Gateway (e.g., PayOS, VietQR)
    - SMS Service Provider
    - Email Service Provider
    - Mapping Service (e.g., Mapbox, Geoapify)
    - Optional Identity Verification Service
    - Cloud Storage for images (e.g. Cloudinary)
    - Receiving data from GPS tracking devices.

4.2 Quality Attributes

4.2.1 Usability

- The mobile applications (Owner and Driver) should be friendly and easy for users to use without extensive training.
- The Technician Tracking Application should be intuitive for its specific tasks, requiring minimal training beyond initial process understanding.
- The Web Dashboard for Admin, Consultants, and Technicians should require no more than 2 days of training for general use by competent staff.

4.2.2 Reliability

Requirements ensuring the system's dependable operation.

- The mobile applications automatically try to reload content or reconnect when the internet connection is restored after interruption.
- Key system services (e.g., booking, payment processing, tracking data reception) must maintain high availability.

4.2.3 Performance

Requirements for protecting system data and access.

- Users are divided into several roles: Admin, Consultant, Technician, Driver, and Owner.
- All input data across all interfaces is validated before being saved to the database.
- Each role only has access to a specific set of features and data according to the defined authorization matrix.

- The system always checks authentication (verifying user identity) and authorization (verifying user permissions for the requested action) before carrying out any feature or providing access to data.

5. Requirement Appendix

5.1 Business Rules

ID	Rule Definition
BR-01	Username must be 3-50 characters and not empty.
BR-02	Email must be valid, unique, and not empty.
BR-03	Passwords must be at least 6 characters and not empty.
BR-04	Address and phone number must not be empty.
BR-05	Date of birth must be earlier than the current date.
BR-06	Only the admin can create employee accounts.
BR-07	Employee roles are limited to "consultant" or "technician".
BR-08	Car models must be selected; license plates must be 12 characters.
BR-09	Fuel consumption and daily rental price must be greater than 0 and not empty.
BR-10	Car description must not exceed 1000 characters.
BR-11	Booking requires a valid driver's license and max 1 active bookings per driver.
BR-12	Booking duration must be 1–30 days and start time must be greater than current time at least 1.5 hours.
BR-13	Booking must not overlap with existing bookings.
BR-14	Base Price for booking = daily price * days Platform Fee = 0.1 * base price Total Price = base price + platform fee
BR-15	Booking time must be between 6:00 and 22:00.
BR-16	After the trip ends, final fuel level and cleaning photos are required.
BR-17	Inspection photos allow max 1 image per type and accept only these image extensions ".jpg", ".jpeg", ".png".
BR-18	Expiry date of inspection and license must be today or later.
BR-19	Car manufacturer and model names must not exceed 100 characters and cannot be empty.
BR-20	Bank account number must be 5–20 characters; name 3–100 characters.
BR-21	Amenity name ≤ 50 characters; description ≤ 500 characters; icon must be SVG.
BR-22	Feedback must have content (≤ 500 characters) and a score from 1–5.
BR-23	License number must be 12 digits and license must be provided both front and back images.
BR-24	Latitude must be -90 to 90, Longitude -180 to 180.
BR-25	Report description must be ≤ 1000 characters; images ≤ 10 MB and only allow image extensions ".jpg", ".jpeg", ".png"
BR-26	Image Types: exterior car image, fuel gauge image, parking location, car key, trunk space are required to provide in inspection ,pre-booking and post-booking

BR-27	Technician signs and confirms the inspection contract before the owner can sign.
BR-28	Phone number, driving license and bank account number must be encrypted in the database to ensure security.
BR-29	Technicians must mark a schedule to be in-progress no later than 15 minutes after the scheduled time.
BR-30	Technicians must approve or reject schedules within one hour from scheduled time.
BR-31	Manufacturer logo must be SVG.
BR-32	Consultant can only delete schedules with scheduled time greater than now.
BR-33	A technician cannot be assigned a new inspection schedule if they already have a non-handled schedule within one hour (before or after) of the requested time.
BR-34	Sending email to user for every booking step
BR-35	User can edit start time and end time for pending booking
BR-36	User can only edit start time for approve/ready for pickup booking
BR-37	Users can only edit end time for ongoing booking. If the new end time is larger than the old end time, user have to make payment for the rest. If users don't make payment in 15 minutes, revert back to the old end time.
BR-38	User return car later than end time for 6 hours, they will be cost 120% for each day Booking Excess day Fee = excess days * (daily price * 0.12)
BR-39	User can only have 1 ongoing booking at a time

5.2 Common Requirements

- User Login: User is authenticated with the system.

- User Authorization: Authorize users into systems based on their roles.

5.3 Application Messages List

Message ID	Message Text	Message Type	Context/Trigger
MSG_AUTH_01	OTP sent to your phone number.	Information	User Registration (Send OTP)
ERR_AUTH_01	Invalid phone number format.	Validation	User Registration (Send OTP validation)
ERR_AUTH_03	OTP verification failed.	Error	User Registration (Verify OTP)
MSG_AUTH_02	Account created successfully.	Success	User Registration (Create Account)
ERR_AUTH_04	Email address already registered.	Error	User Registration (Create Account - duplicate email)
ERR_AUTH_07	Invalid login credentials.	Error	Login Screen (Login failure)
MSG_LOGIN_01	Login successful.	Success	Login Screen (Successful login)
MSG_LIC_01	License information saved.	Success	Edit License Screen (Create License)

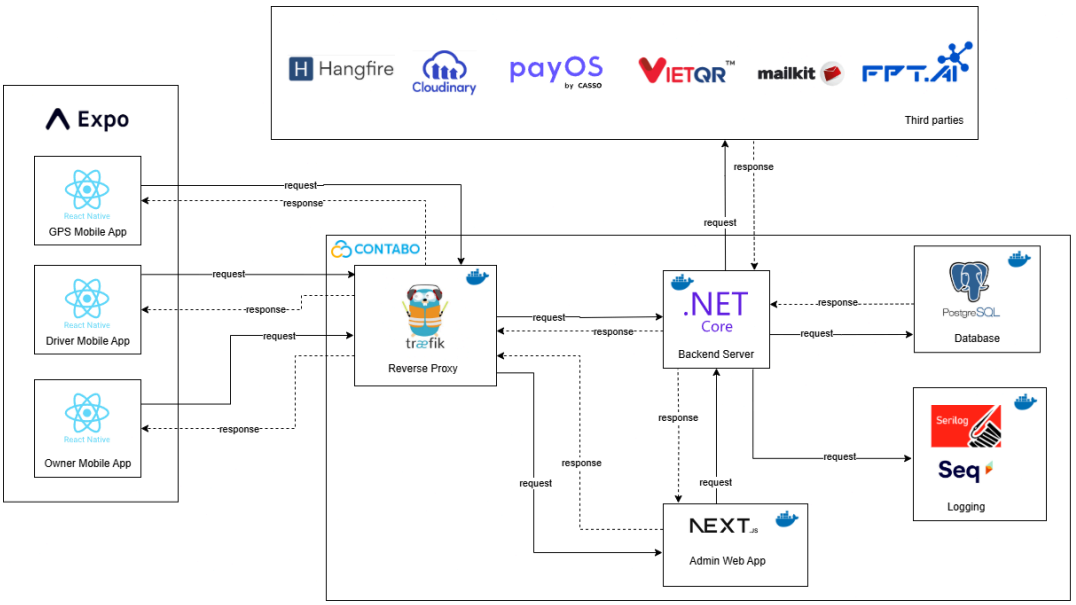
MSG_LIC_02	License image uploaded.	Success	Edit License Screen (Upload Image License)
ERR_LIC_03	Invalid license number format.	Validation	Edit License Screen (Create License validation)
INFO_CAR_01	Loading cars...	Information	List Cars Screen (Fetching cars)
INFO_CAR_02	No cars available for rent.	Information	Driver Cars Screen (List is empty)
ERR_CAR_01	Failed to load cars. Please try again.	Error	List Cars Screen (Fetch cars failure)
ERR_CAR_02	Failed to load car details.	Error	Car Detail Screen (Fetch car by id failure)
MSG_CAR_01	Car deleted successfully.	Success	Owner Cars Screen (Delete Car)
MSG_CAR_02	Car disabled successfully.	Success	Owner Car Detail Screen (Disable Car)
MSG_CAR_04	Car added successfully.	Success	Owner Edit Car Screen (Create Car)
MSG_CAR_05	Car updated successfully.	Success	Owner Edit Car Screen (Update Car)
ERR_CAR_07	Failed to add car. Please check input details.	Error	Owner Edit Car Screen (Create Car failure/validation)
ERR_CAR_08	Failed to update car. Please try again.	Error	Owner Edit Car Screen (Update Car failure)
ERR_CAR_10	Required car details are missing.	Validation	Owner Edit Car Screen (Create/Update Car validation)
MSG_BOOK_01	Booking request submitted.	Success	Driver Create Booking Screen (Create Booking)
ERR_BOOK_01	Failed to create booking. Please try again.	Error	Driver Create Booking Screen (Create Booking failure)
ERR_BOOK_02	Car is unavailable for the selected dates.	Error	Driver Create Booking Screen (Create Booking - availability conflict)
MSG_BOOK_06	Booking approved successfully.	Success	Owner Booking Detail Screen (Approve Booking)
MSG_BOOK_07	Booking rejected successfully.	Success	Owner Booking Detail Screen (Reject Booking)
MSG_BOOK_03	Booking cancelled successfully.	Success	Driver Booking Detail Screen (Cancel Booking)
WARN_BOOK_01	Are you sure you want to cancel this booking?	Warning	Driver Booking Detail Screen (Confirm Cancel Booking)
MSG_BOOK_04	Trip started.	Success	Driver Booking Detail Screen (Start-trip Booking)
MSG_BOOK_05	Trip completed.	Success	Driver Booking Detail Screen (Complete Booking)

ERR_BOOK_12	Cannot complete trip before the scheduled end time.	Error	Driver Booking Detail Screen (Complete Booking - too early)
MSG_PAY_01	Payment successful.	Success	Driver Booking Detail Screen (Payment Booking)
ERR_PAY_01	Failed to process payment.	Error	Driver Booking Detail Screen (Payment Booking failure)
MSG_FEED_01	Feedback submitted. Thank you!	Success	Driver/Owner Booking Detail Screen (Feedback Booking)
MSG_REP_01	Report submitted successfully.	Success	Create Booking/Car Report Screen (Submit Report)
ERR_REP_01	Failed to submit report.	Error	Create Booking/Car Report Screen failure
ERR_REP_02	Report description is required.	Validation	Create Booking/Car Report Screen validation
MSG_PROF_01	Profile updated successfully.	Success	Edit Profile Screen (Update user profile)
ERR_PROF_02	Failed to update profile. Please check your inputs.	Error	Edit Profile Screen (Update user profile failure/validation)
MSG_PASS_01	Password updated successfully.	Success	Change password Screen (Update password account)
ERR_PASS_01	Failed to update password.	Error	Change password Screen (Update password account failure)
ERR_PASS_02	Current password is incorrect.	Validation	Change password Screen (Update password account validation)
ERR_PASS_03	New passwords do not match.	Validation	Change password Screen (Update password account validation)
MSG_CONT_01	Contract signed successfully.	Success	Signature Screen (Contract sign)
ERR_CONT_01	Failed to sign contract.	Error	Signature Screen failure
ERR_CONT_02	Signature is required.	Validation	Signature Screen validation
MSG_SCHED_03	Inspection schedule approved.	Success	Dashboard Todo Page / Approve Inspection Page (Approve Inspection Schedule)
MSG_SCHED_04	Inspection schedule rejected.	Success	Dashboard Todo Page (Reject Inspection Schedule)
MSG_GPS_01	Device unassigned successfully.	Success	Technician Home Screen (Unassign device)
MSG_TECH_01	Device assigned to car successfully.	Success	Technician List Cars Screen (Assign device to car)
ERR_TECH_02	Failed to assign device to car.	Error	Technician List Cars Screen failure

IV. Software Design Description

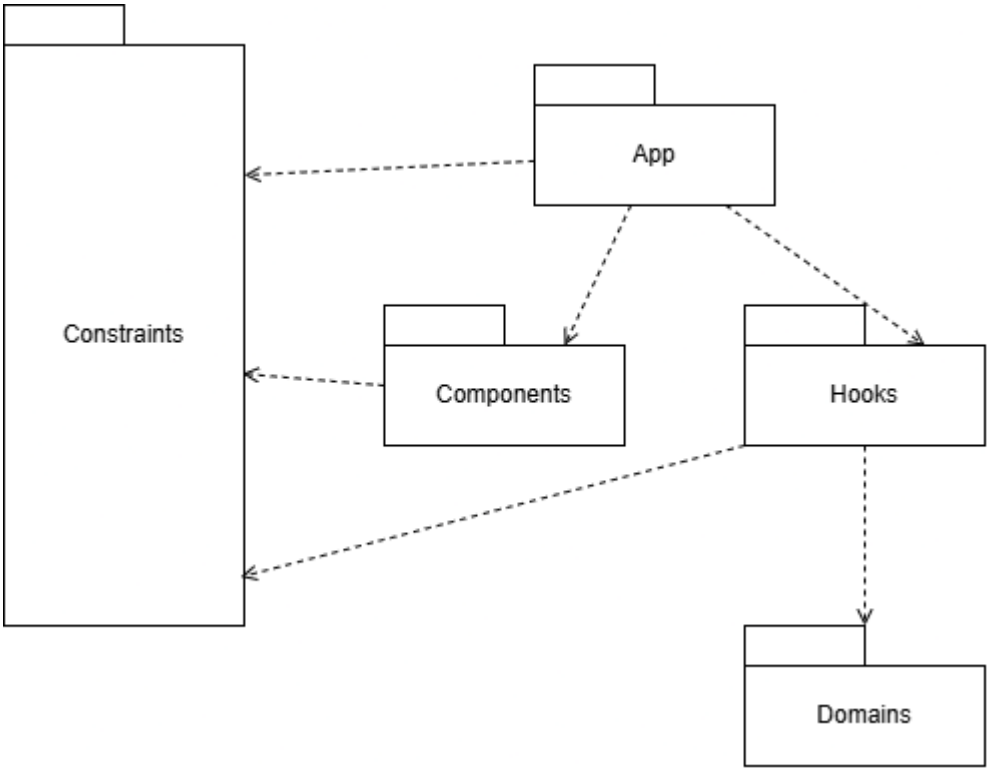
1. System Design

1.1 System Architecture

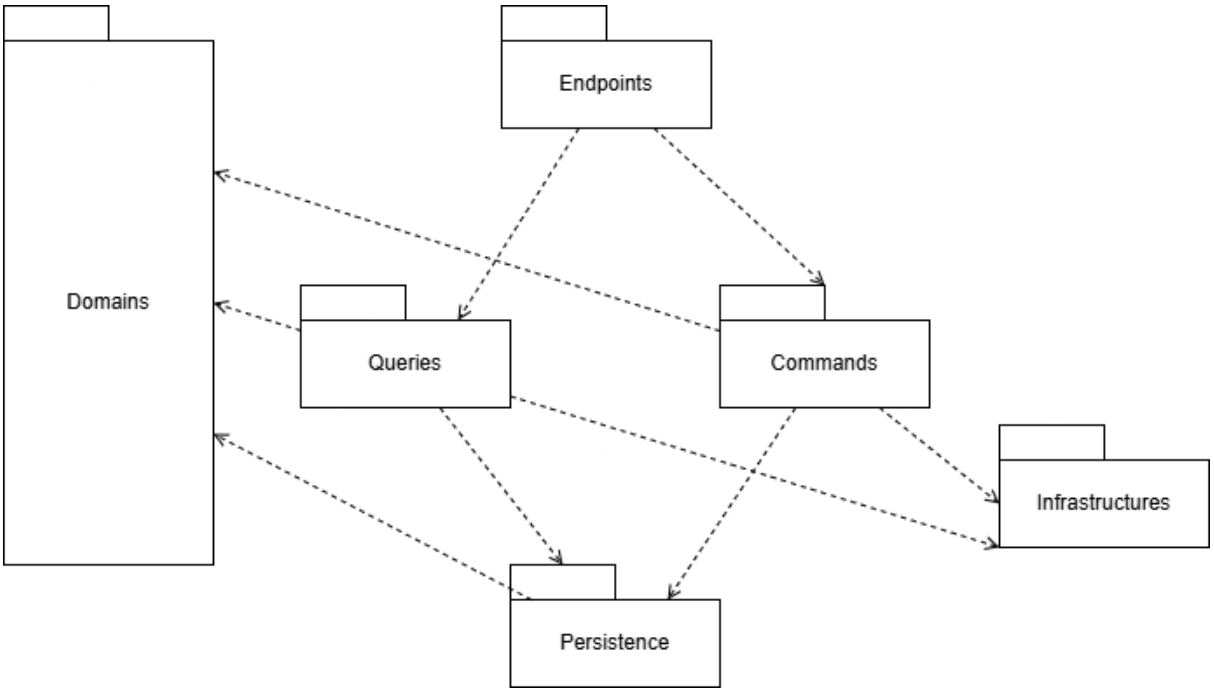


1.2 Package Diagram

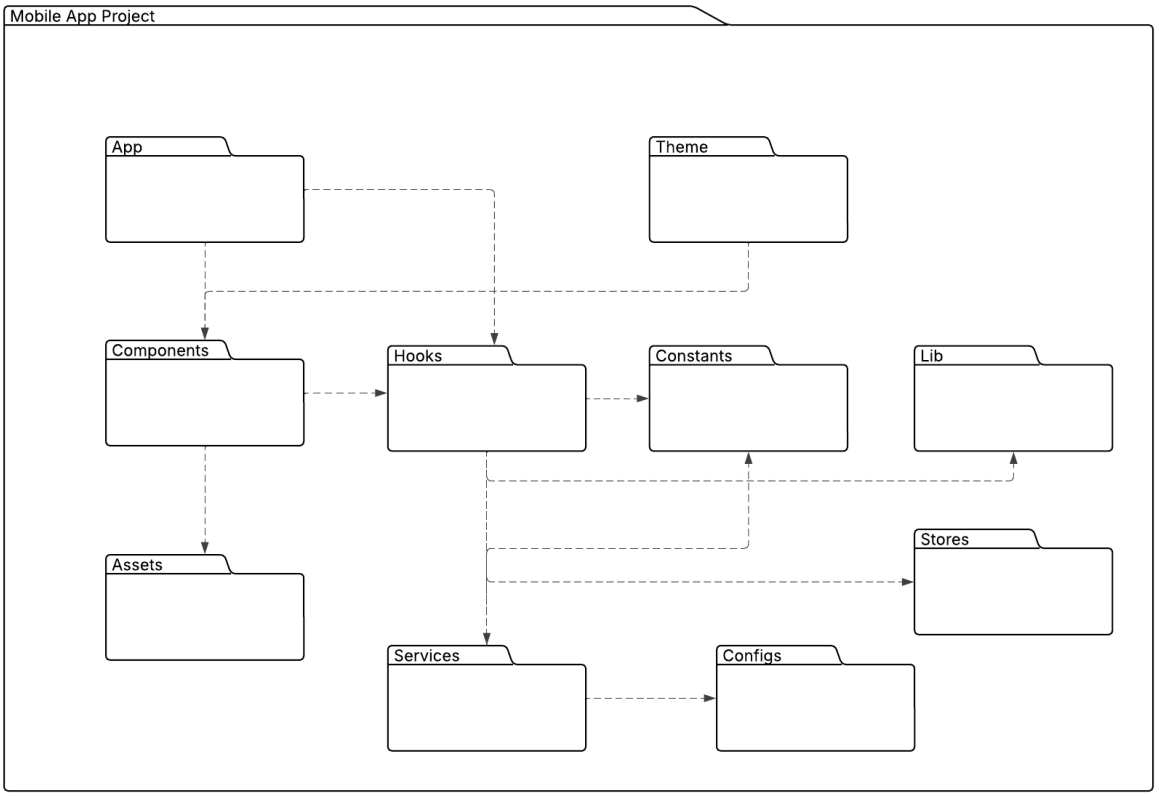
1.2.1 Admin Web App



1.2.2 Backend Server



1.2.3 Mobile App



Package Descriptions

No	Package Name	Description
1	App	Handles application routing and entry points, following framework conventions.
2	Assets	Stores static assets like images and fonts.
3	Components	Contains reusable UI building blocks, including screens, forms, cards, layouts, and UI plugins, and custom layouts.
4	Configs	Sets up default configurations for services like Axios and Calendar.
5	Constants / Models/ Domains	Defines fixed values like schemas for form validation, data models, enums for predefined choices, and base objects within the system (Enums, Constraints, Base Entity).
6	Hooks	Provides custom React hooks for querying data and performing mutations \(updates, creates, deletes\) and logic for fetching/mutating data to backend.
7	Lib	Includes utility functions and helper modules used throughout the application.
8	Services/Endpoints	Manages API endpoint calls to interact with backend systems and data sources; Declares API endpoints.
9	Stores	Implements state management using Zustand for managing application state.
10	Theme	Defines the application's visual styling and appearance, including colors, typography, and spacing.
11	Queries/Commands	Process fetching and mutating data use cases.
12	Infrastructures	Contains 3rd parties' implementations.
13	Persistence	Contains database configurations, ORM Configurations.

2. Database Design

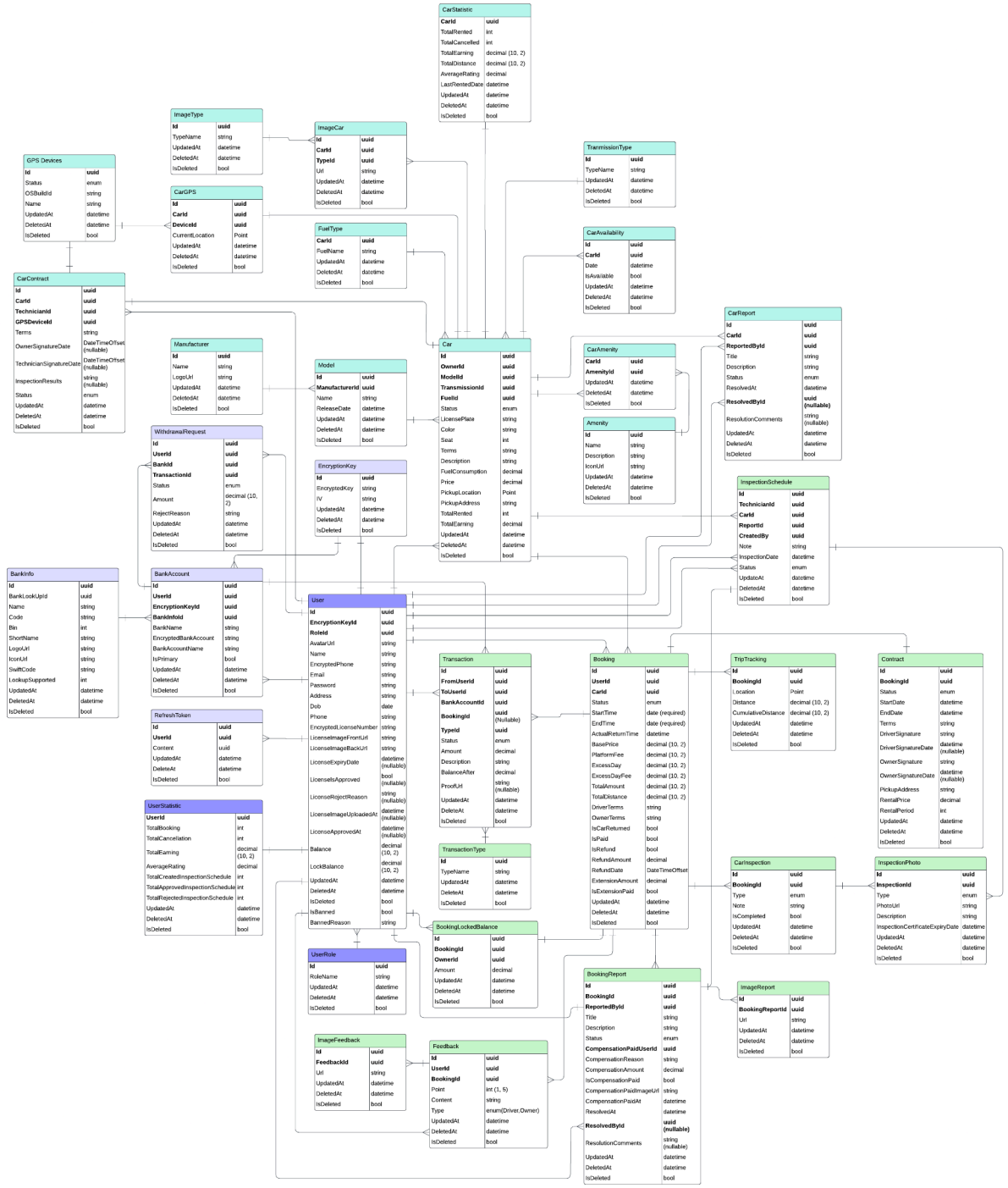


Table Descriptions

#	Entity	Description
1	Amenities	Store amenities of a car. - Primary Key: Id - Foreign Key:
2	BankAccount	Store bank accounts of a user, which is used for send request of withdrawal.

		- Primary Key: Id - Foreign Key: EncryptionKeyId, BankInfoId,UserId
3	BankInfo	Store bank info of all banks, which is used for withdrawal requests. - Primary Key: Id - Foreign Key:
4	Booking	Store all bookings that are created by drivers. - Primary Key: Id - Foreign Key: UserId, CarId
5	BookingLockedBalance	Store current payment amount belong to a booking, to prevent owner withdraw current balance - Primary Key: Id - Foreign Key: BookingId, OwnerId
6	BookingReport	Store reports of a booking, including fee, accident, conflict,... - Primary Key: Id, - Foreign Key: BookingId, ReportedById, ResolvedById
7	Car	Store cars of platform. - Primary Key: Id, - Foreign Key: EncryptionKeyId, RoleId
8	CarAmenity	Store amenities for each car - Primary Key: Id - Foreign Key: Carid, AmenityId
9	CarAvailability	Store available time of a car - Primary Key: Id - Foreign Key: CarId
10	CarContract	Store a contract of a car, which is created by system when the car is created in platform, which is just tagged as Pending. - Primary Key: Id, - Foreign Key: CarId, TechnicianId, GPSDeviceId
11	CarGPS	Store GPS Device's location, which is attached in the car already. - Primary Key: Id, - Foreign Key: Car Id, DeviceId
12	CarInspection	Store car reviews before and after each booking, which ensures cars' overall condition to save proofs for conflicts. - Primary Key: Id - Foreign Key: BookingId
13	CarReport	Store all reports for owner's cars. Owner create report when they need assistance. - Primary Key: Id - Foreign Key: CarId, ReportedById, ResolvedById
14	CarStatistic	Store cars' statistics for overall review. - Primary Key: Id - Foreign Key: CarId
15	Contract	Store bookings' contracts between drivers and owners - Primary Key: Id - Foreign Key: BookingId
16	EncryptionKey	Store encryption information of entities for encrypting datas. - Primary Key: Id

17	Feedback	Store feedbacks of booking when finish a booking, which is describe personal thoughts about the opposite - Primary Key: Id - Foreign Key: UserId, BookingId
18	FuelType	Store types of fuel that car used. - Primary Key: Id
19	GPSDevice	Store information on all devices that are used for collecting GPS Coordination and send that information back to platform's server. - Primary Key: Id
20	ImageCar	Store images of a car, including the car itself and its paperworks - Primary Key: Id - Foreign Key: CarId, TypeId
21	ImageFeedback	Store image of a feedbacks. - Primary Key: Id - Foreign Key: FeedbackId,
22	ImageReport	Store images of a user report. - Primary Key: Id - Foreign Key: ReportId
23	ImageType	Store types of an image, which is used for specifying the purpose of an image usage. - Primary Key: Id - Foreign Key: TypeId
24	InspectionPhoto	Store images of a car, which is taken by the drivers itself for evidence. - Primary Key: Id - Foreign Key: InspectionId
25	InspectionSchedule	Store dates of inspections, which is used for technicians and consultants to know dates and time for inspecting purposes. - Primary Key: Id - Foreign Key: TechnicianId, CarId, ReportId
26	Manufacturer	Store manufacturers of a car, to know which brand produces the car. - Primary Key: Id
27	Model	Store models of manufacturers, to know which model the car is . - Primary Key: Id - Foreign Key: ManufacturerId
28	RefreshToken	Store refresh token of the users, which is used for revalidate the JWT Token. - Primary Key: Id - Foreign Key: UserId
29	Transaction	Store transactions of system, which is used for control the money flow of the system. - Primary Key: Id - Foreign Key: FromUserId, ToUserId, BankAccountId, TypeId
30	TransactionType	Store types of transaction, to distinguish transactions. - Primary Key: Id

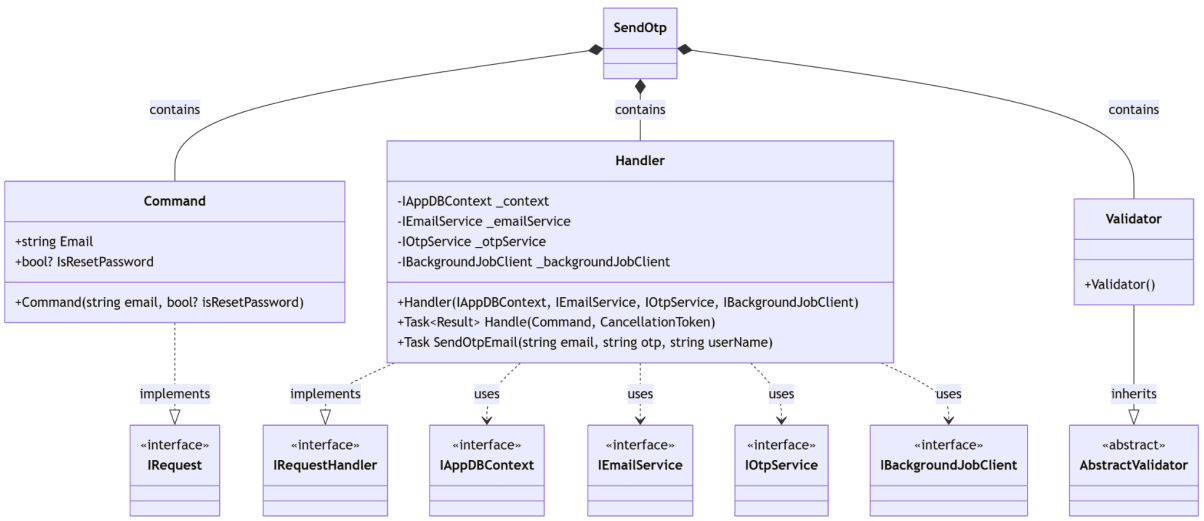
31	TransmissionType	Store type informations of transmission type - Primary Key: Id
32	TripTracking	Store all the locations that are received from the GPS Devices for logging the history of car locations. - Primary Key: Id - Foreign Key: BookingId
33	User	Store users of the platform. - Primary Key: Id - Foreign Key: EncryptionKeyId, RoleId
34	UserRole	Store role informations of users. - Primary Key: Id
35	UserStatistic	Store statistics of activities of users in the platform. - Primary Key: Id - Foreign Key: UserId
36	WithdrawalRequest	Store withdrawal requests of owners who want to retrieve profits from the platform. - Primary Key: Id - Foreign Key: UserId, StatusId, BankId, TransactionId

3. Detailed Design

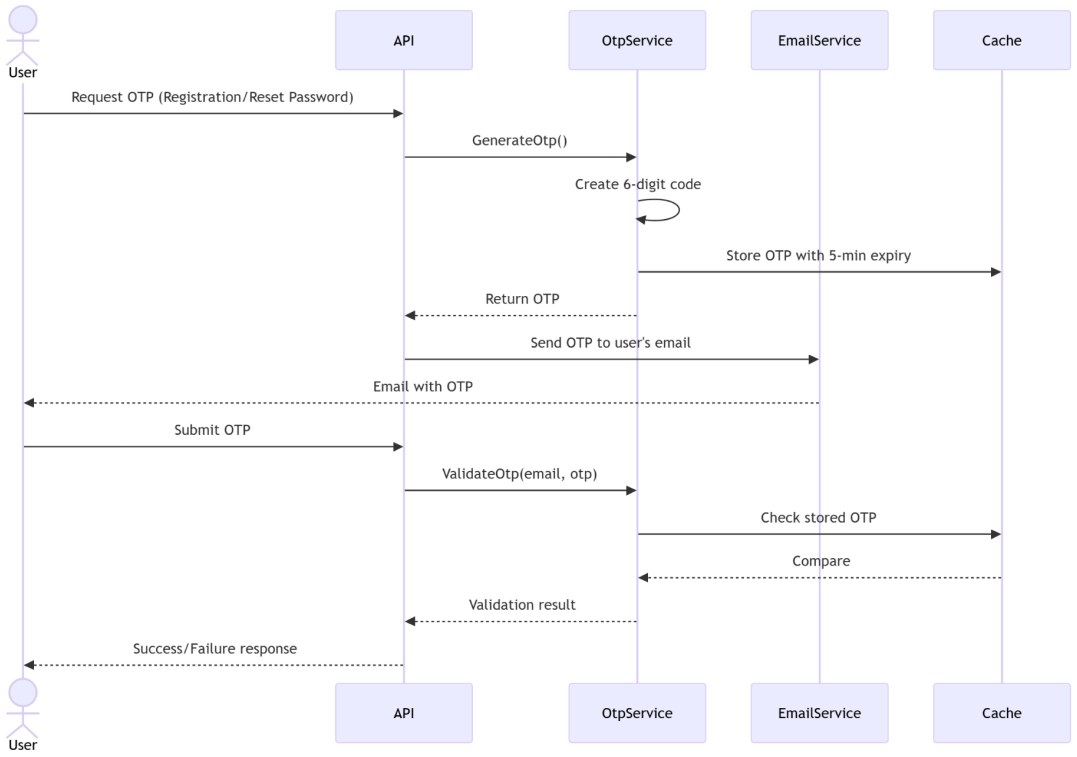
3.1 Authentication and Authorization

3.1.1 One-Time Passwords (OTP)

3.1.1.1 Class Diagram

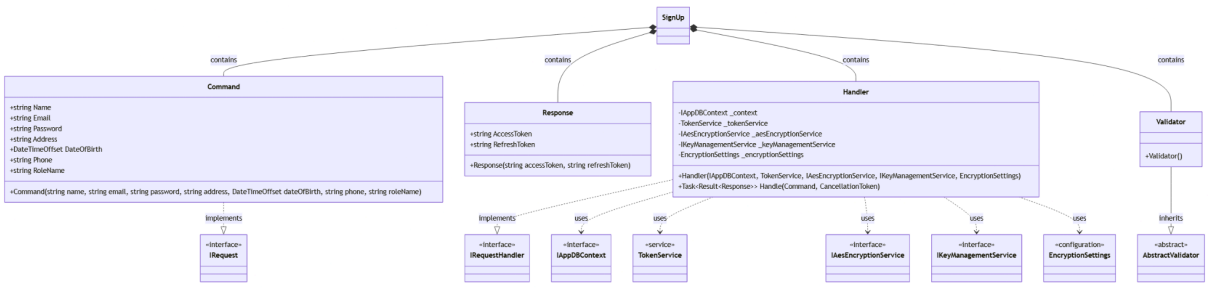


3.1.1.2 Sequence Diagram

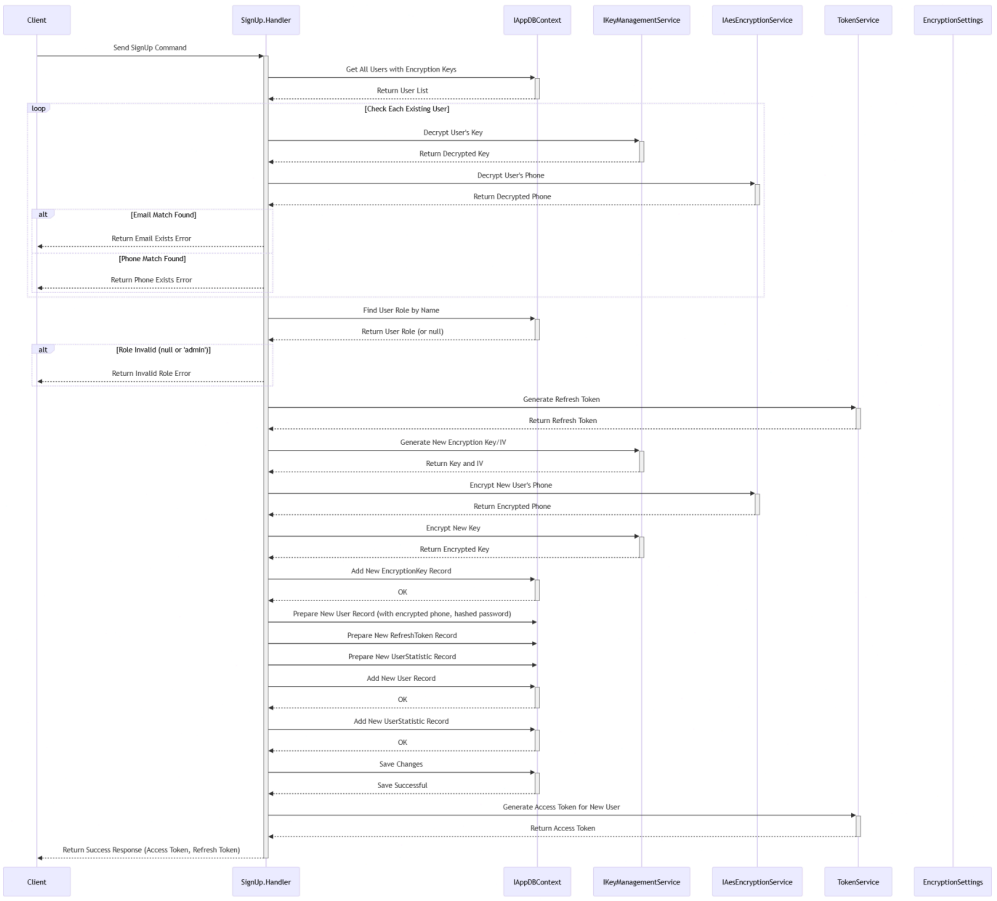


3.1.2 Sign up

3.1.2.1 Class Diagram

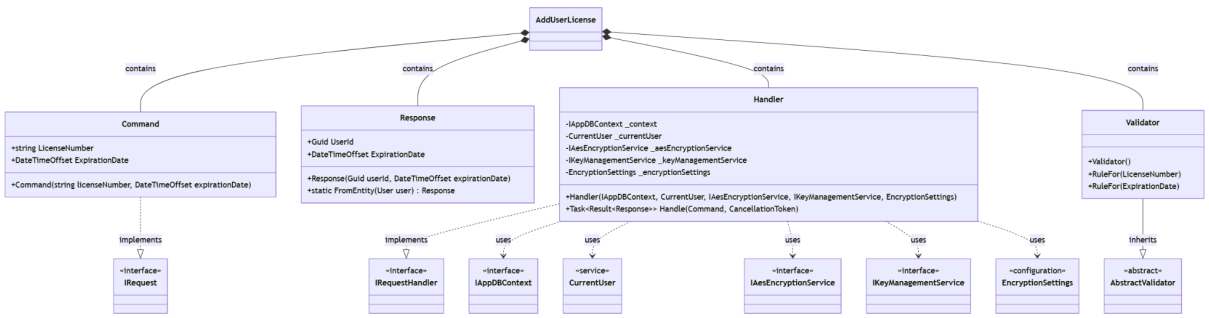


3.1.2.2 Sequence Diagram

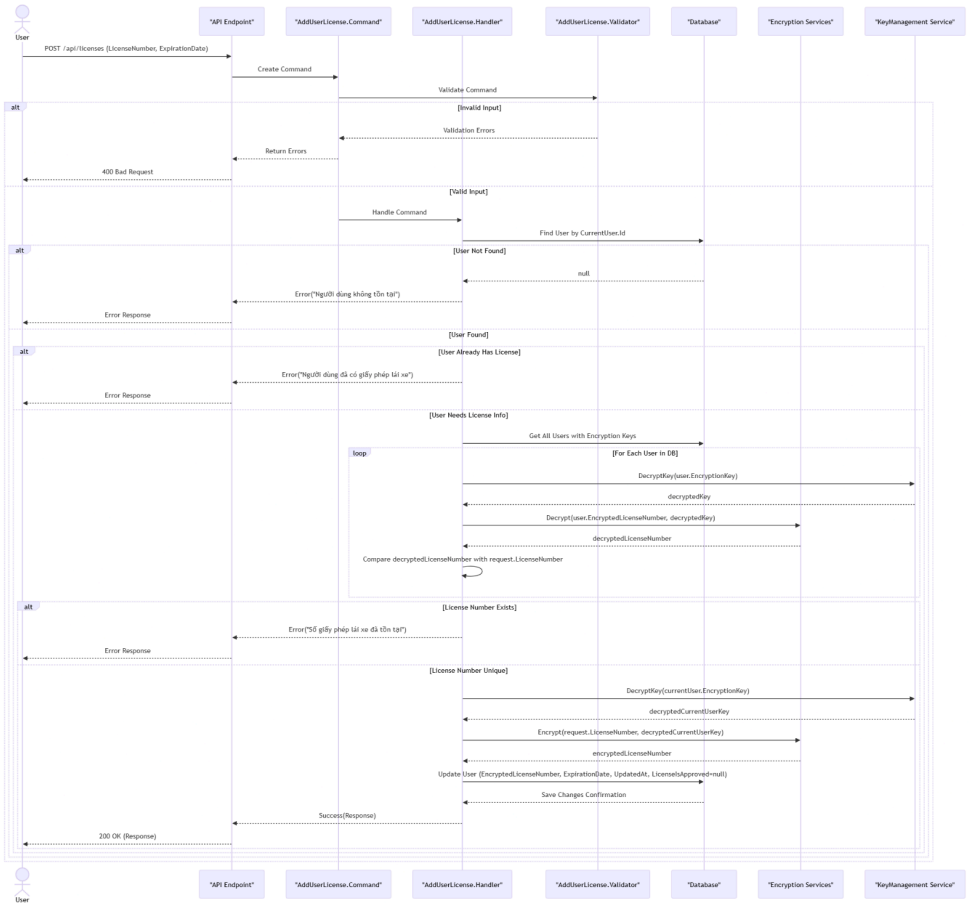


3.1.3 Add License

3.1.3.1 Class Diagram

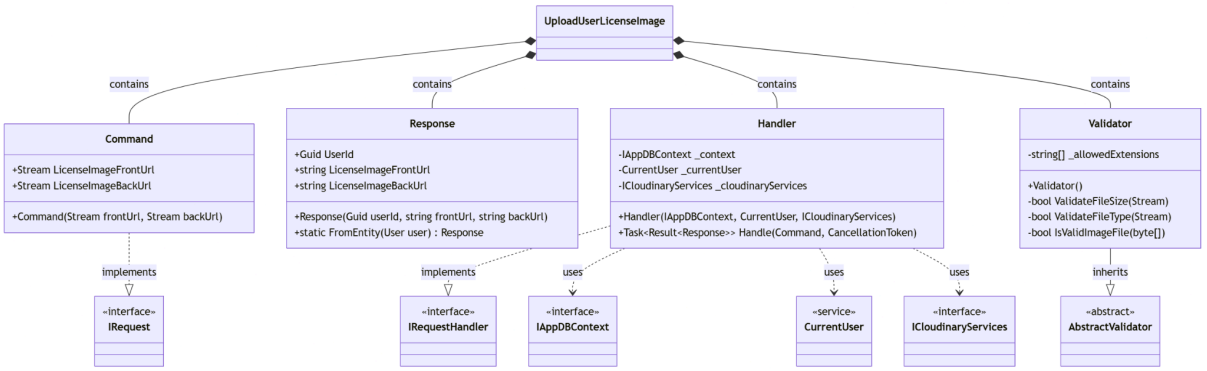


3.1.3.2 Sequence Diagram

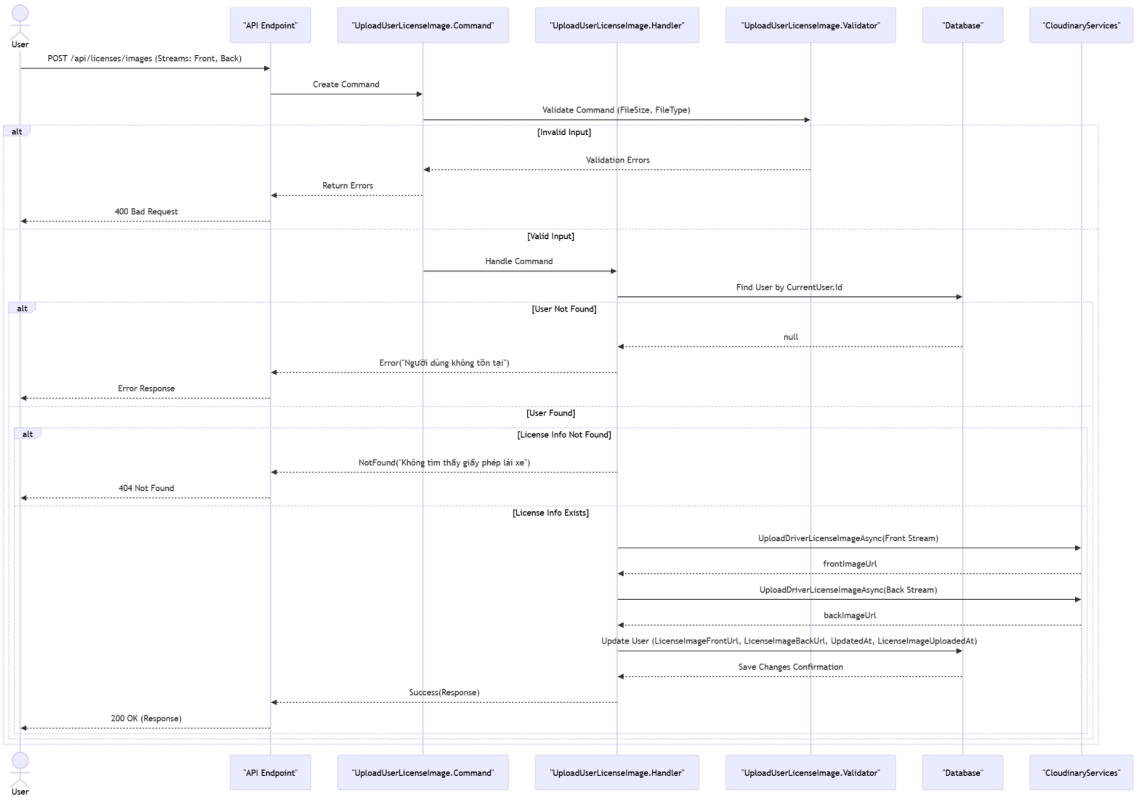


3.1.4 Upload License Image

3.1.4.1 Class Diagram

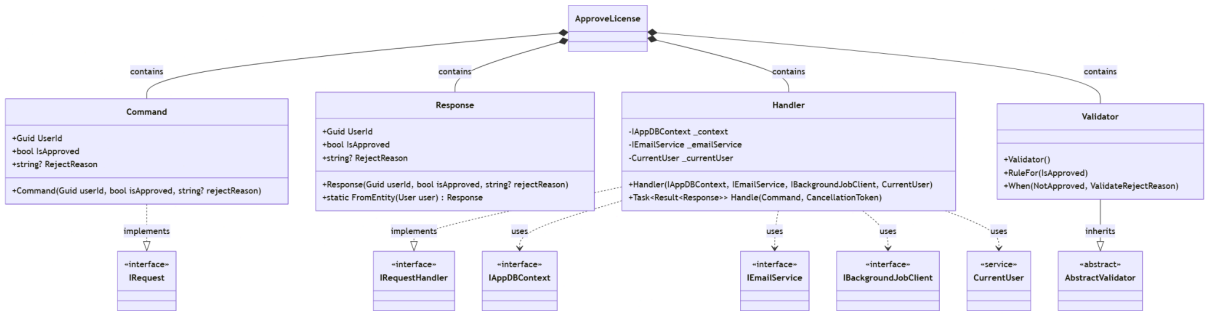


3.1.4.2 Sequence Diagram

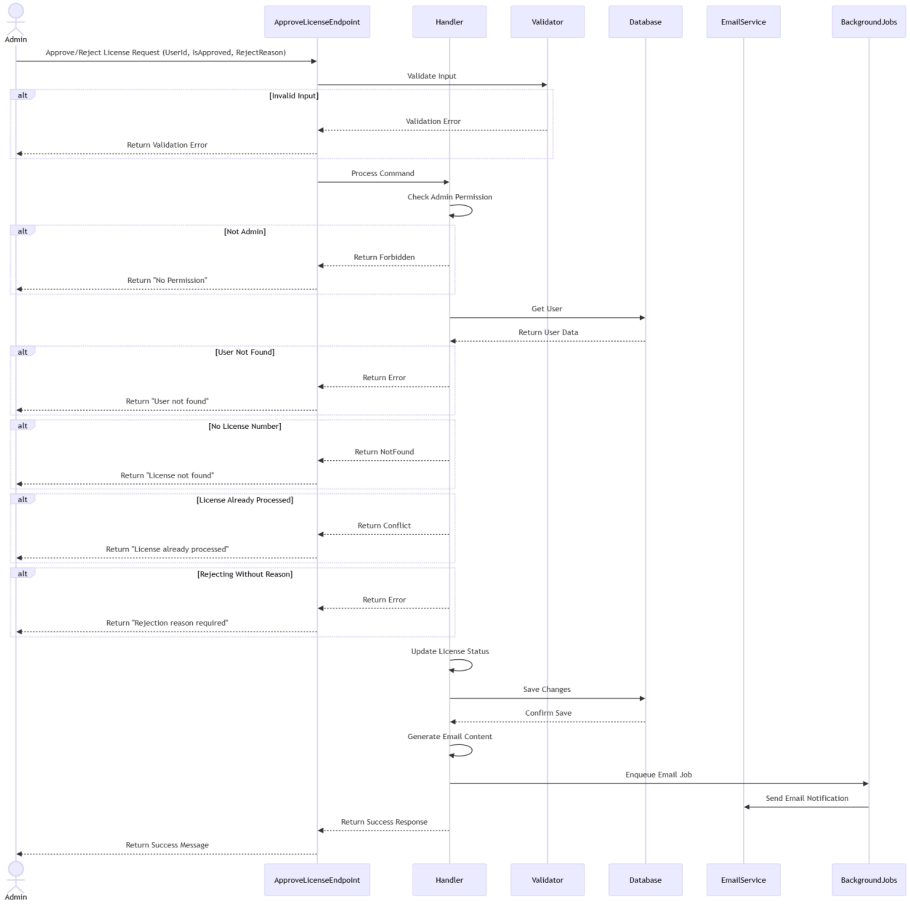


3.1.5 Admin Approve License

3.1.5.1 Class Diagram



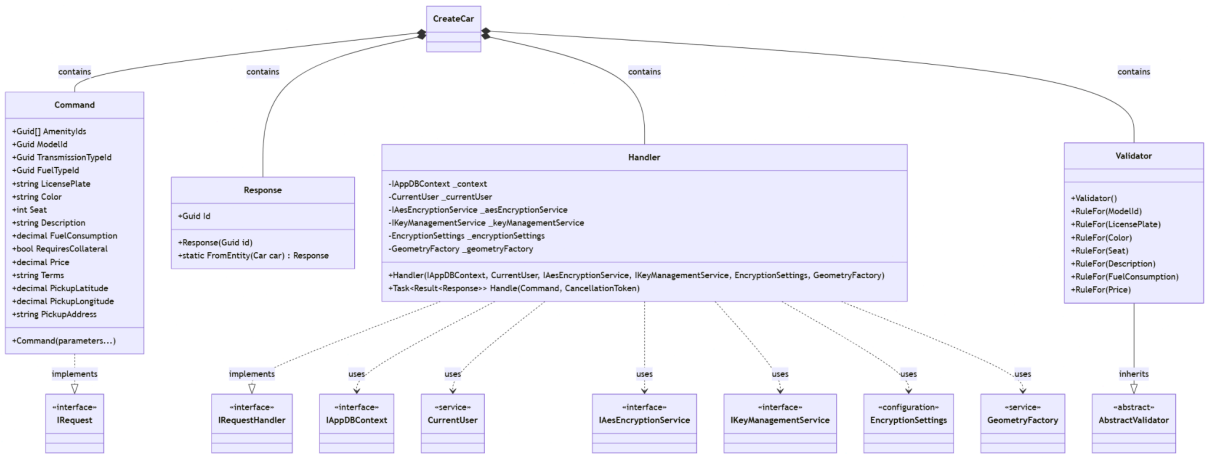
3.1.5.2 Sequence diagram



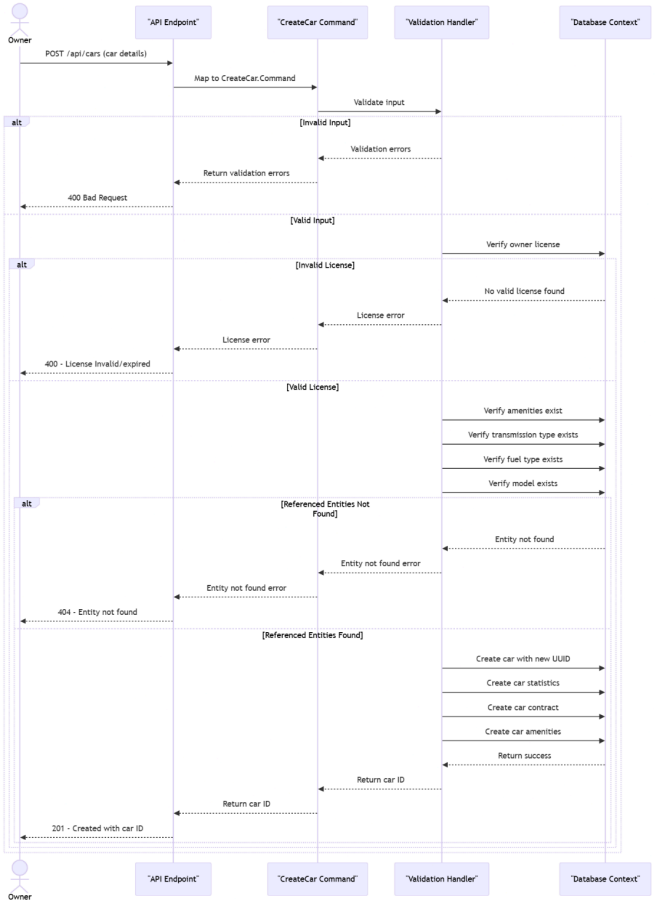
3.3 Car Management

3.3.1 Car Creation

3.3.1.1 Class Diagram

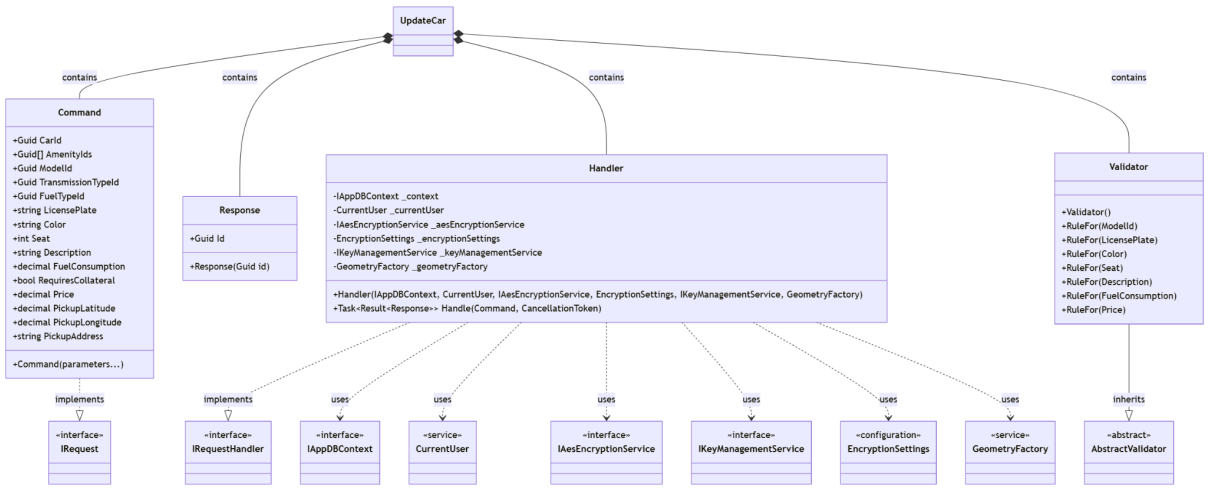


3.3.1.2 Sequence diagram

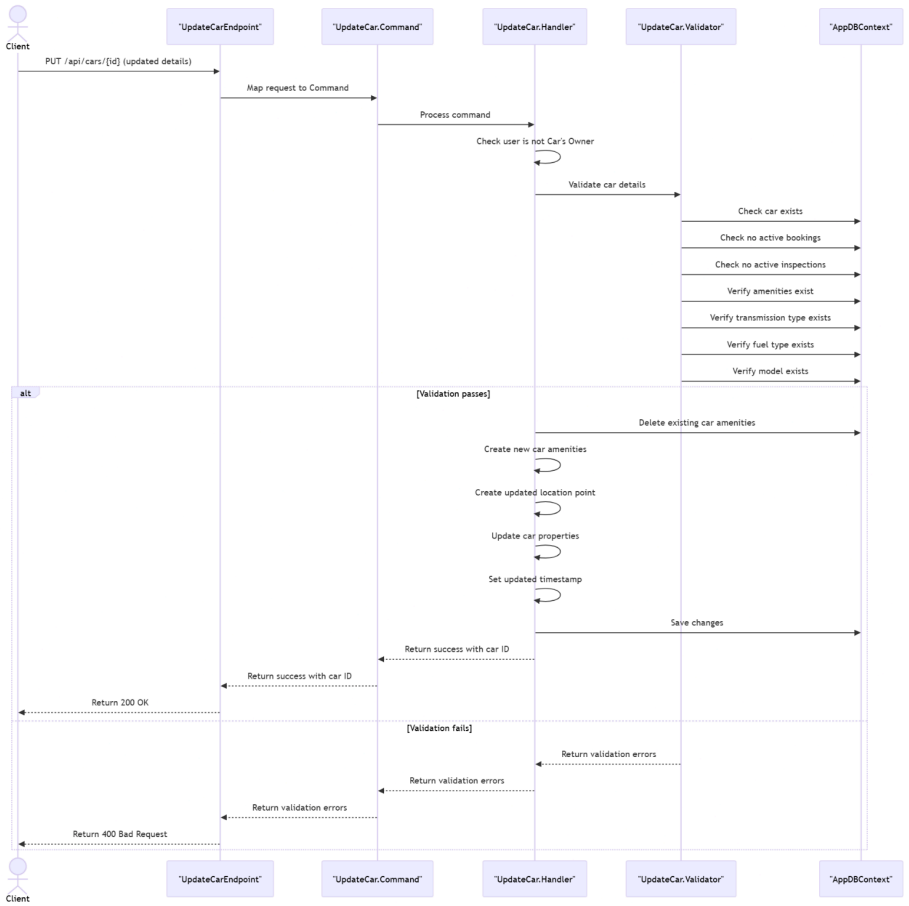


3.3.2 Car Update

3.3.2.1 Class Diagram

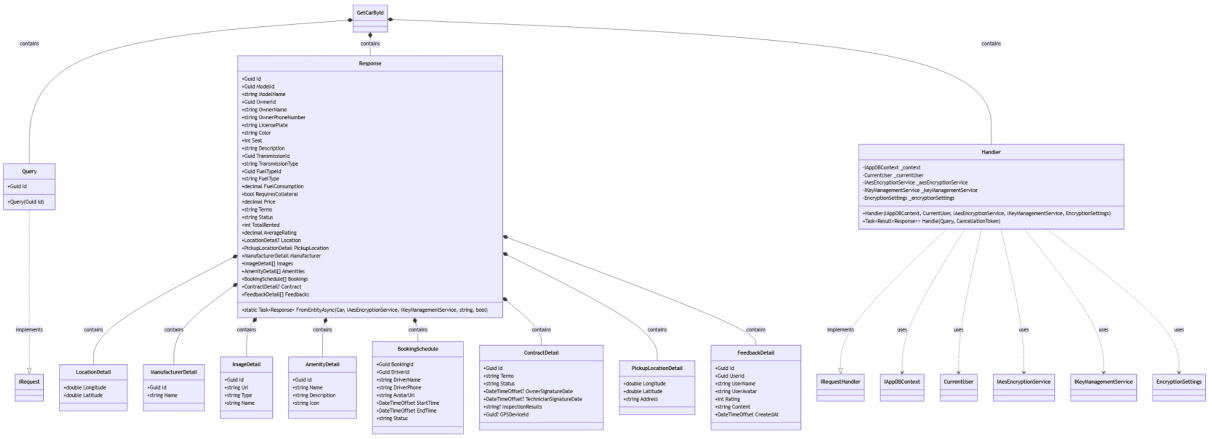


3.3.2.1 Sequence Diagram

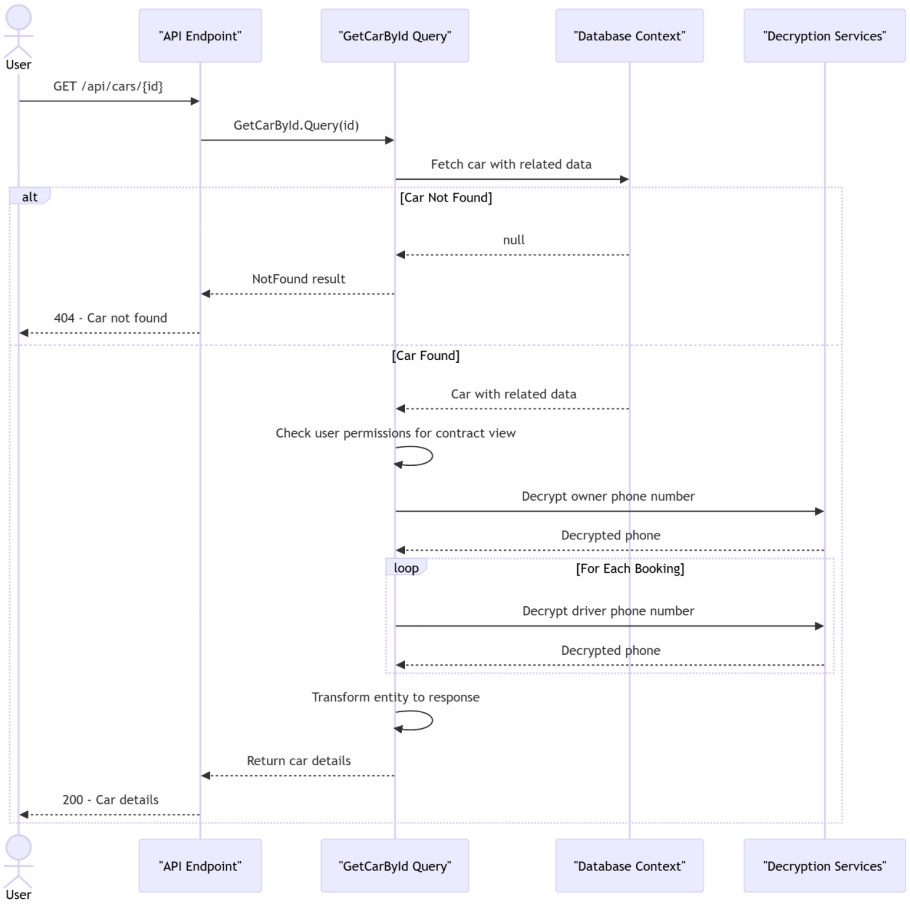


3.3.3 Get Car Detail

3.3.3.1 Class Diagram

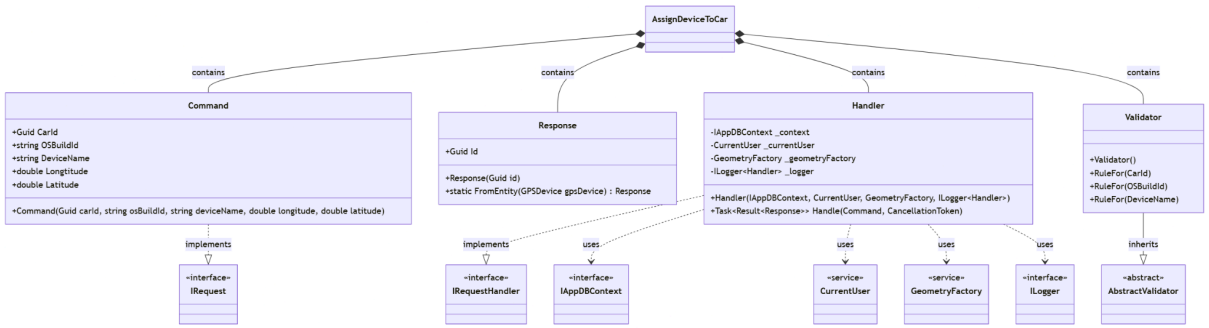


3.3.3.2 Sequence Diagram

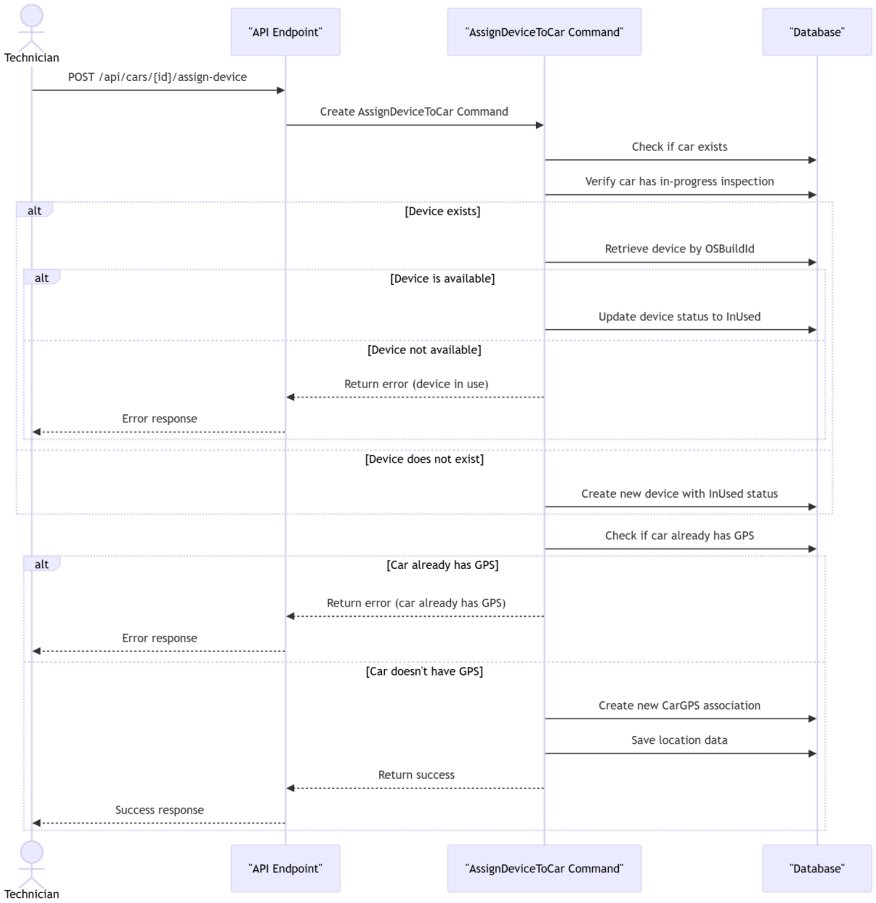


3.3.4 Assigning a GPS Device to a Car

3.3.4.1 Class Diagram

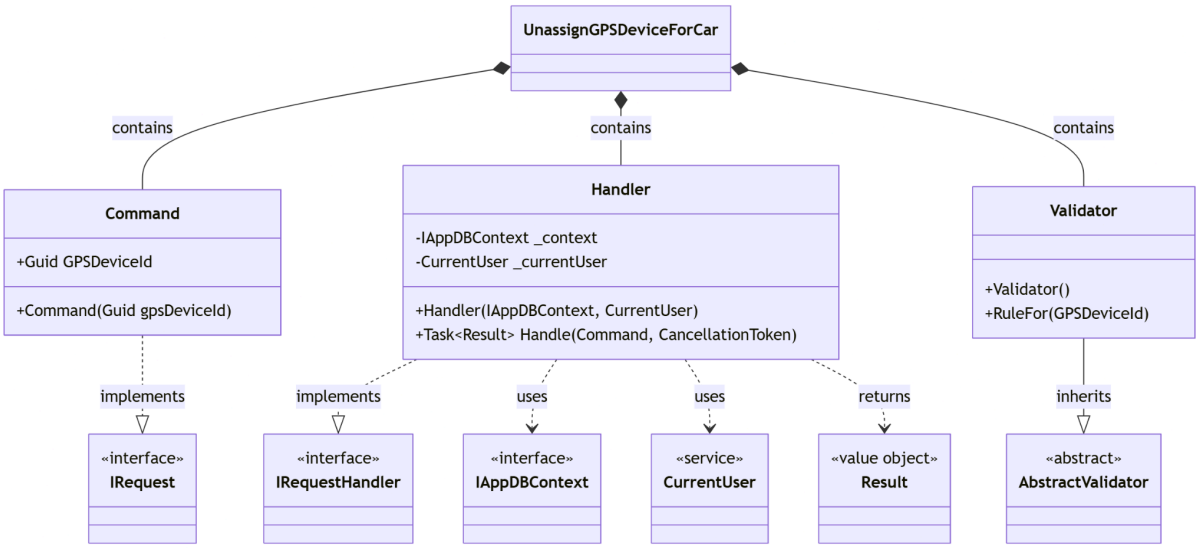


3.3.4.2 Sequence Diagram

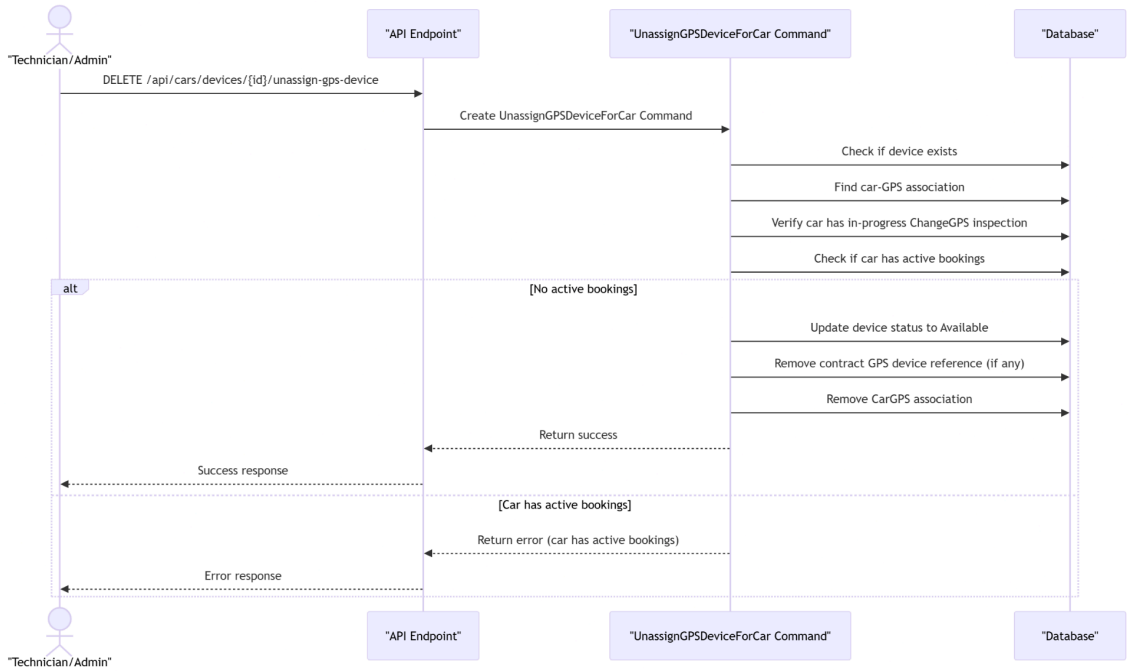

Technician
"API Endpoint"
"AssignDeviceToCar Command"
"Database"
POST /api/cars/{id}/assign-device
Create AssignDeviceToCar Command
Check if car exists
Verify car has in-progress inspection
alt
[Device exists]
Retrieve device by OSBuildId
alt
[Device is available]
Update device status to InUsed
[Device not available]
Return error (device in use)
Error response
[Device does not exist]
Create new device with InUsed status
Check if car already has GPS
alt
[Car already has GPS]
Return error (car already has GPS)
Error response
[Car doesn't have GPS]
Create new CarGPS association
Save location data
Return success
Success response
Technician
"API Endpoint"
"AssignDeviceToCar Command"
"Database"

3.3.5 Unassigning a GPS Device from a Car

3.3.5.1 Class Diagram



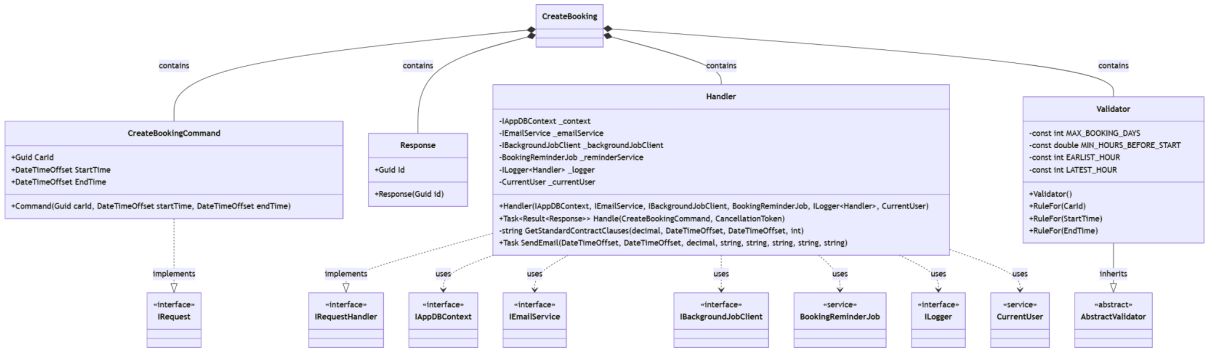
3.3.5.2 Sequence Diagram



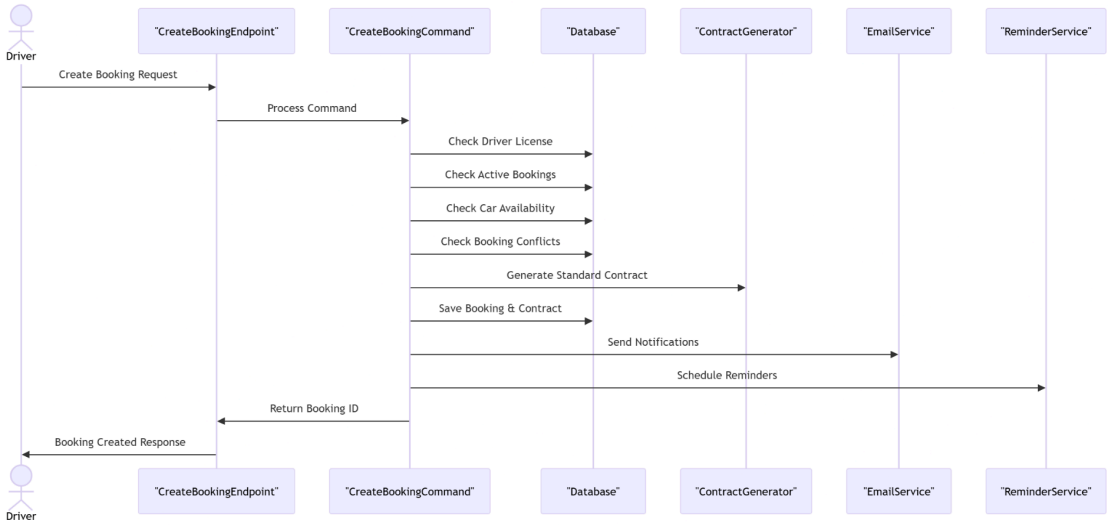
3.4 Booking Management

3.4.1 Booking Creation

3.4.1.1 Class Diagram

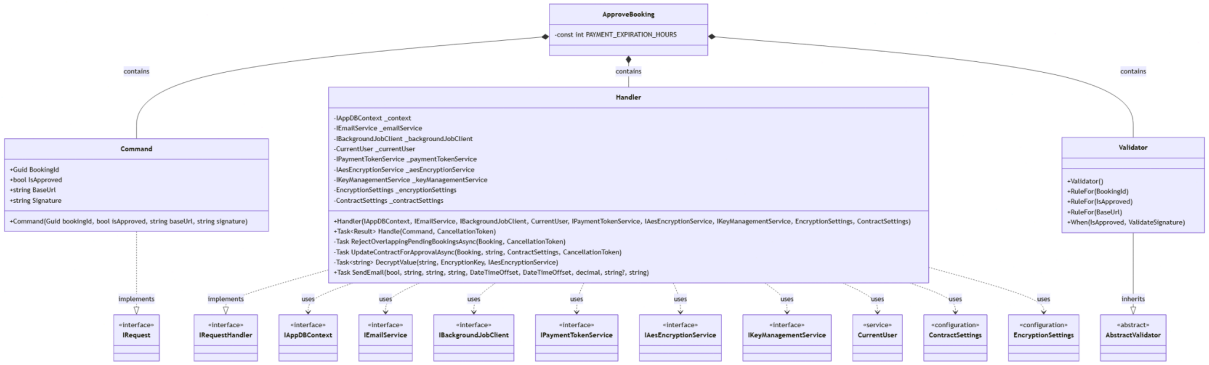


3.4.1.2 Sequence diagram

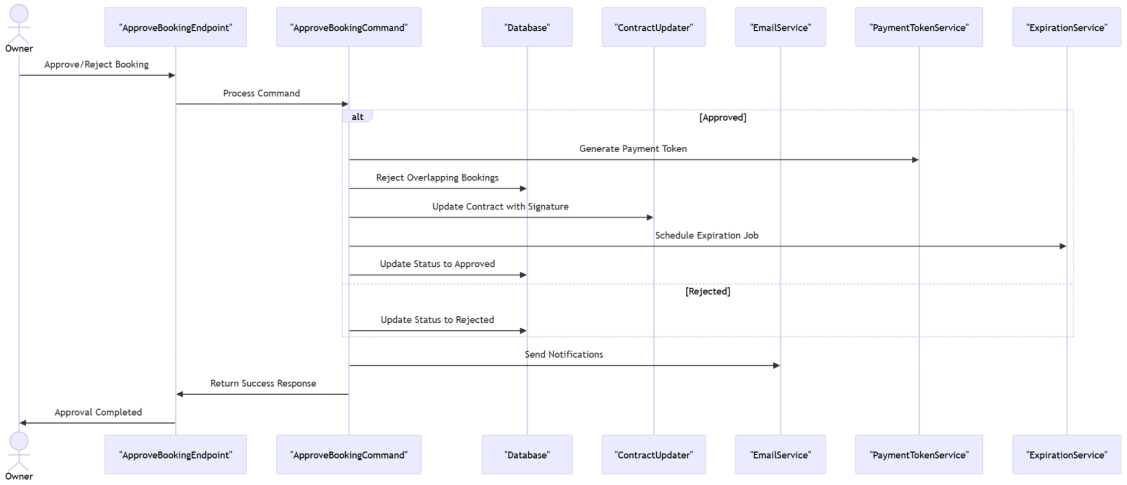


3.4.2 Booking Approval

3.4.2.1 Class Diagram

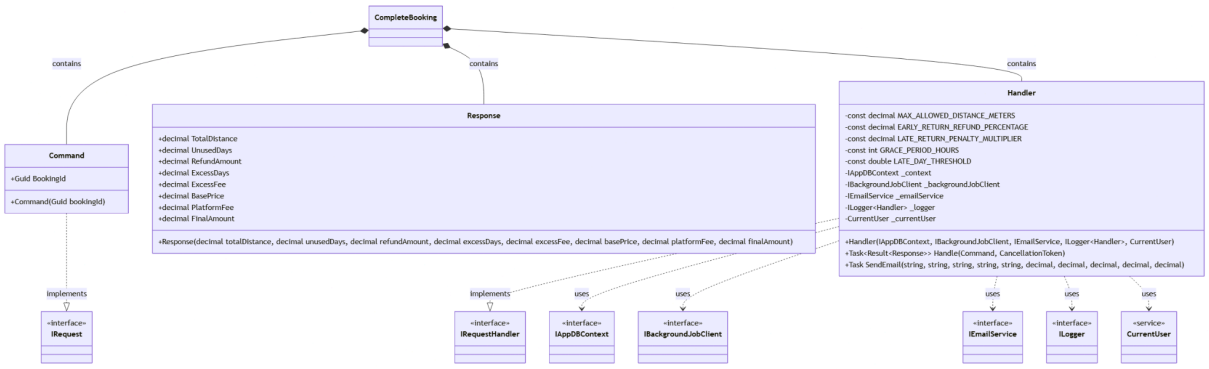


3.4.2.2 Sequence diagram

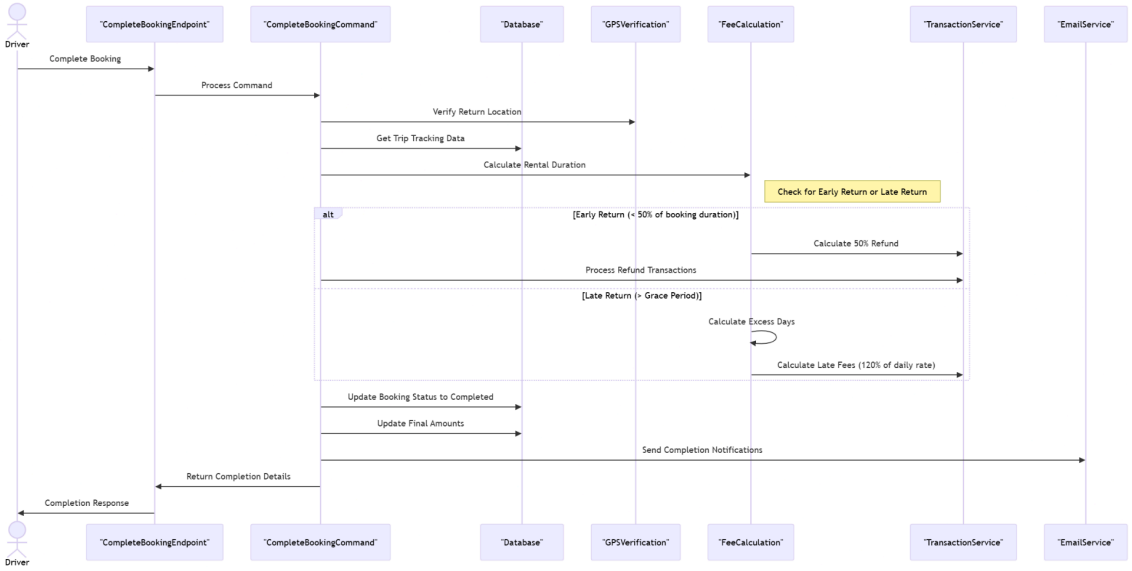


3.4.3 Booking Completion

3.4.3.1 Class Diagram

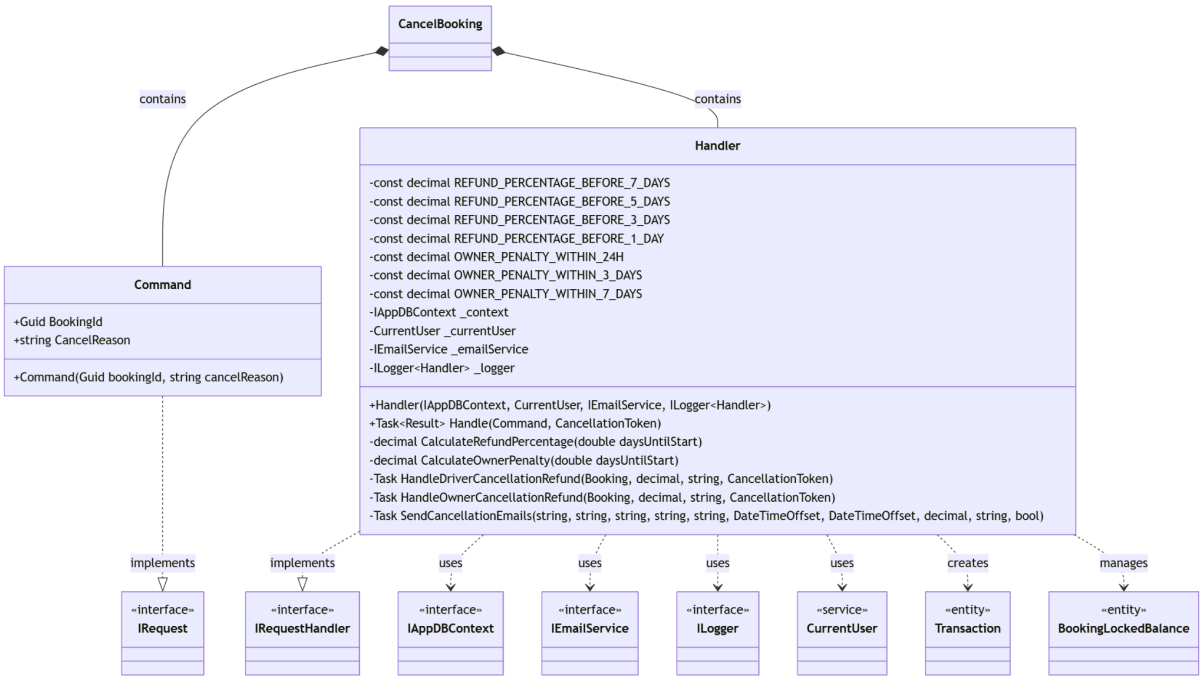


3.4.3.2 Sequence diagram

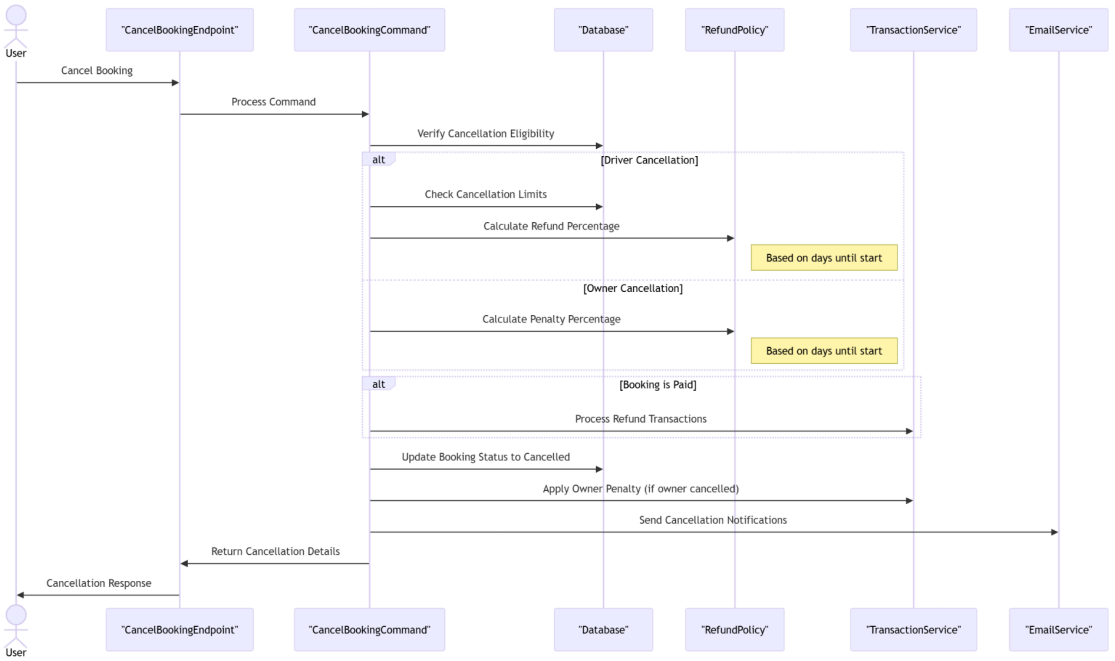


3.4.4 Booking Cancellation

3.4.4.1 Class Diagram



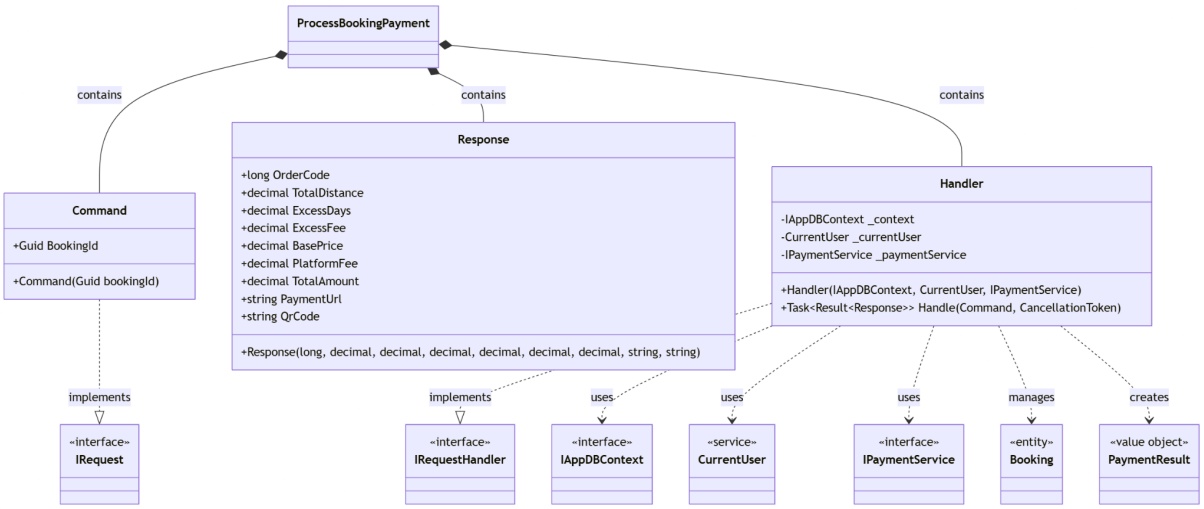
3.4.4.2 Sequence diagram



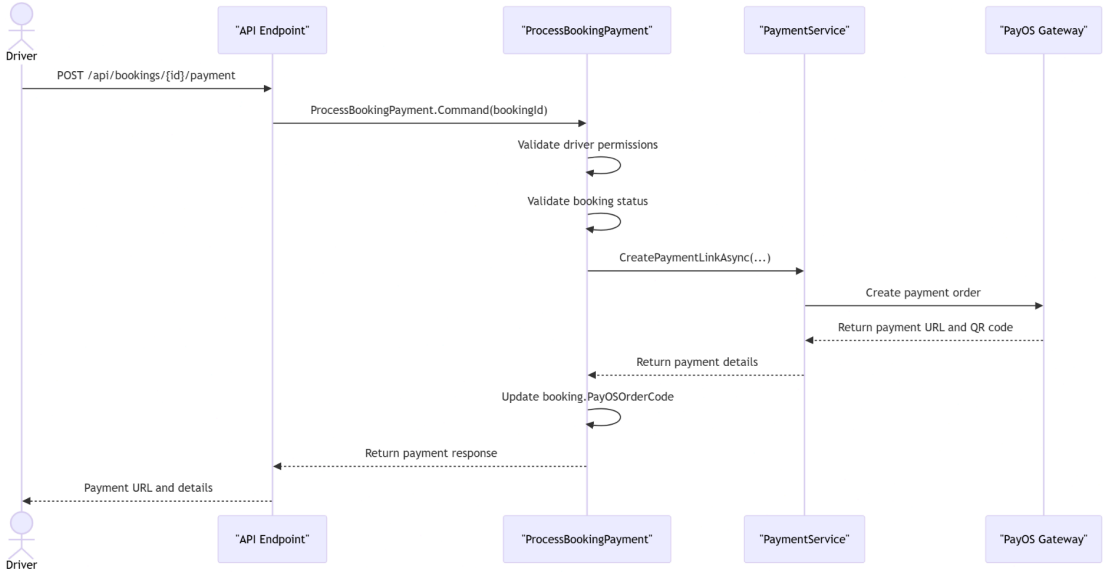
3.4.5 Payment Initiation

3.4.5.1 Direct Payment Process

3.4.5.1.1 Class Diagram

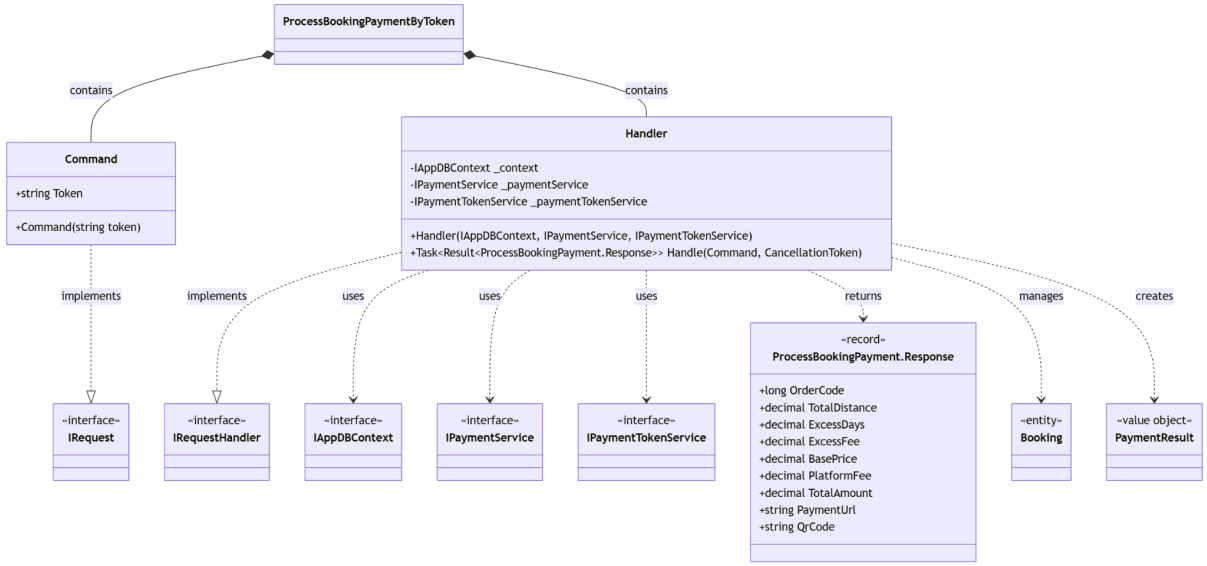


3.4.5.1.2 Sequence diagram

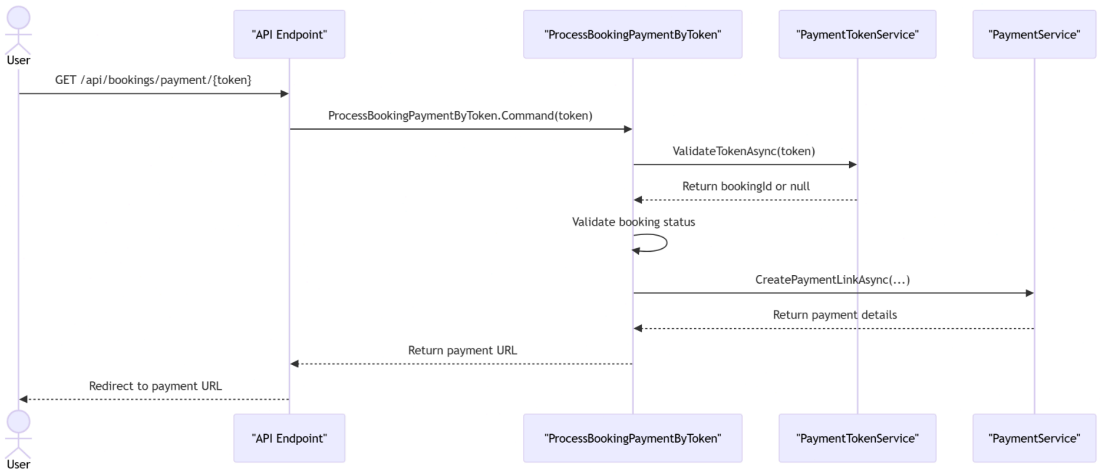


3.4.5.2 Token-based Payment

3.4.5.2.1 Class Diagram

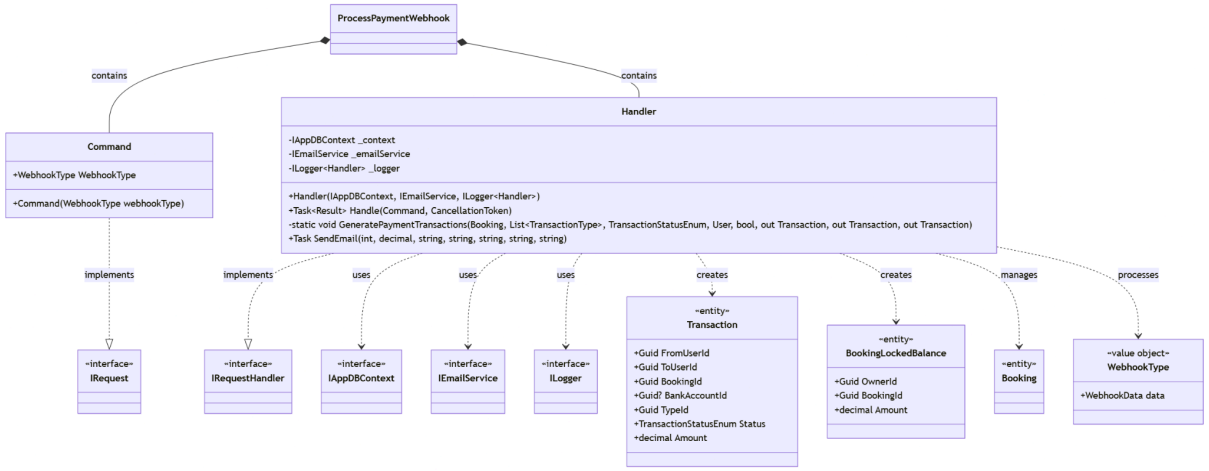


3.4.5.2.2 Sequence diagram

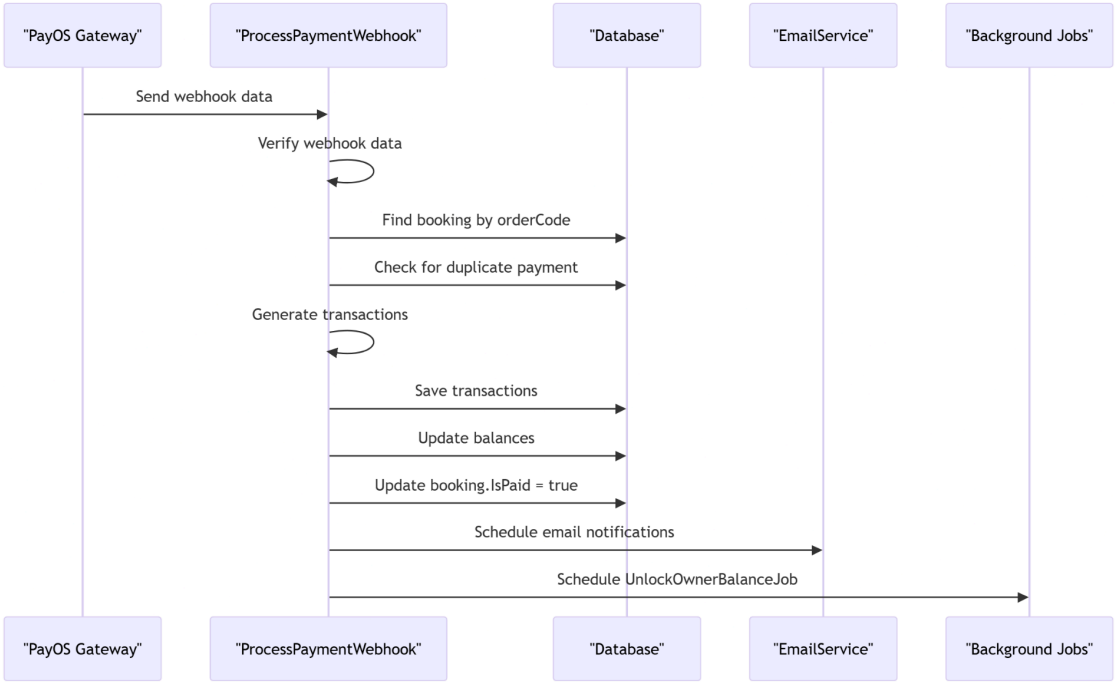


3.4.5.3 Payment Webhook Processing

3.4.5.3.1 Class Diagram

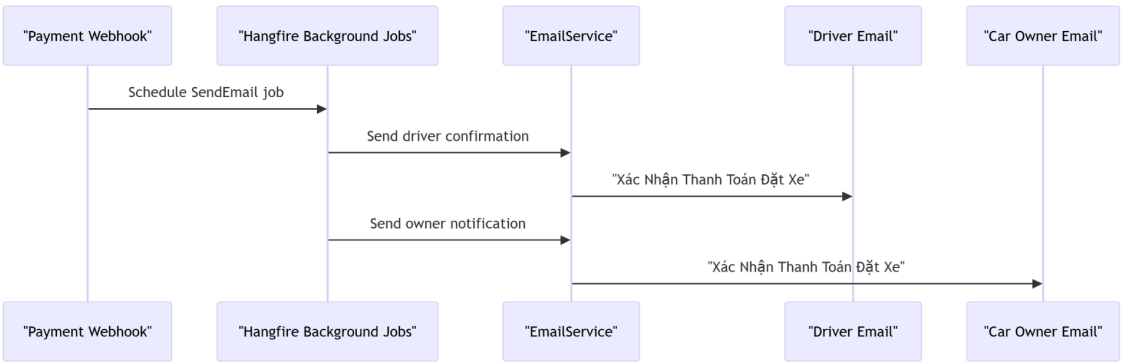


3.4.5.3.2 Sequence diagram



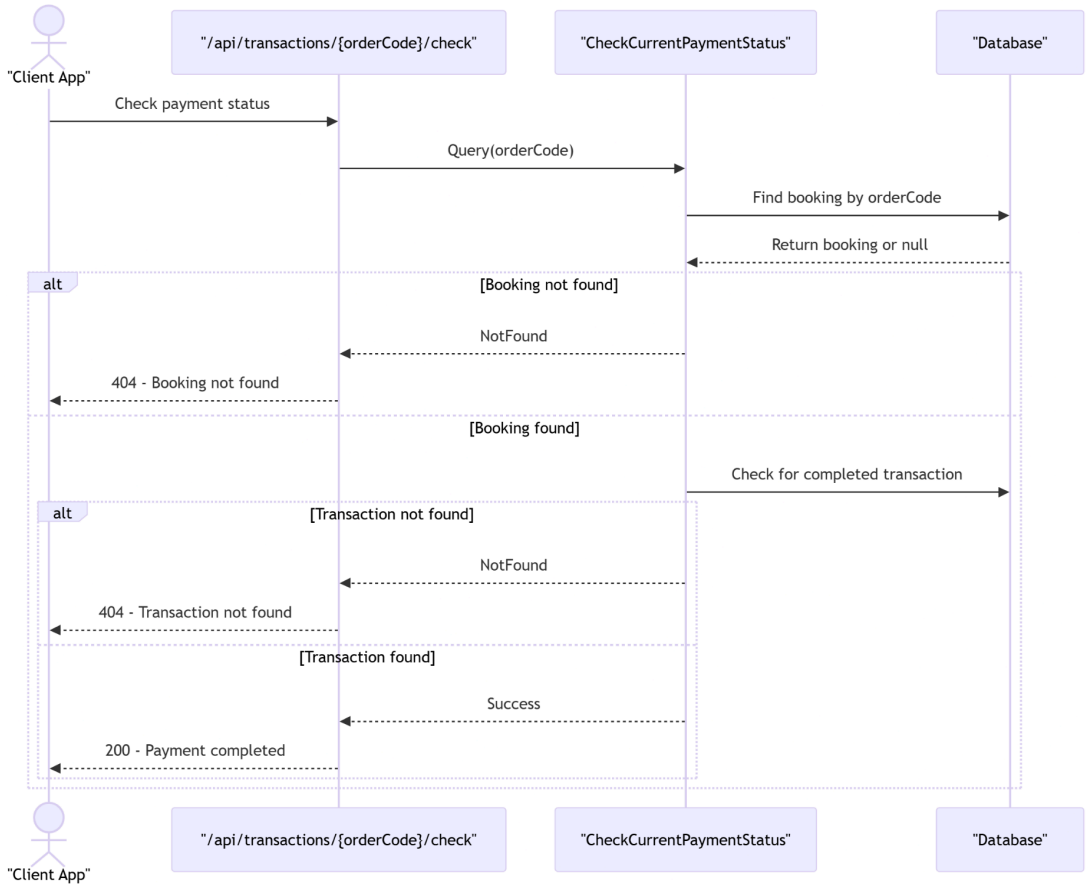
3.4.5.4 Email Notifications

3.4.5.4.1 Sequence diagram



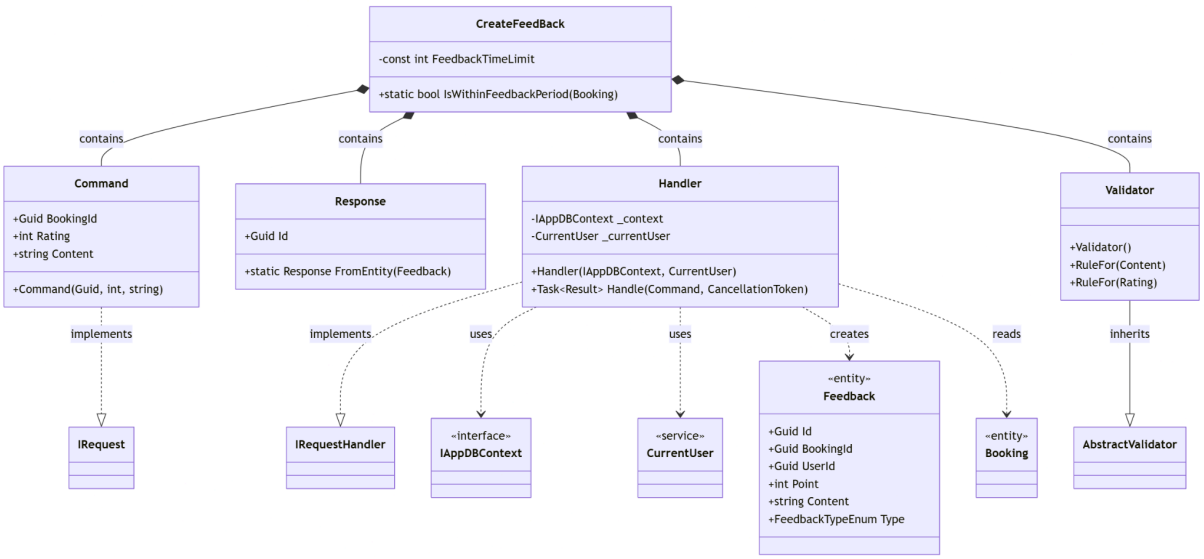
3.4.5.5 Payment Status Checking

3.4.5.5.1 Sequence diagram

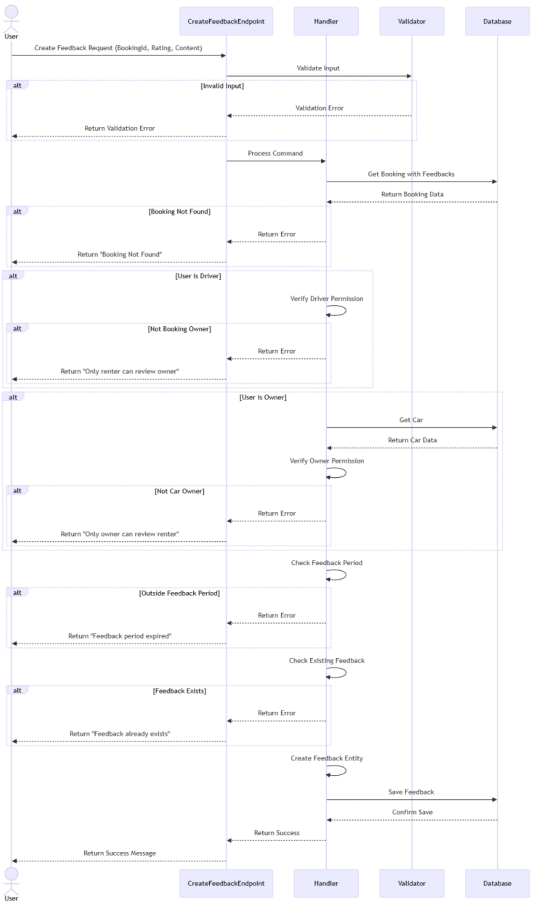


3.4.6 Create Feedback

3.4.6.1 Class Diagram



3.4.6.2 Sequence diagram



V. Software Testing Documentation

1. Scope of Testing

The scope of this testing document is to define the tests for the main flow. The main flow includes:

1. User register new account and upload driving license
2. Owners add new car information and relevant licenses and images.
3. Consultant create car registration verification schedule and assign it for a technician
4. Technician in-progress and approve schedule to allow car to be available for driver to book or reject it.
5. Drivers create booking for specific car
6. Owner approve, reject or cancel booking

2. Test Strategy

2.1 Testing Types

1. Functional Testing

- Objective: Validate core functionality of main flows.
- Technique: Black-box testing.
- Completion Criteria: All high-priority test cases pass, and defects are resolved.

2. Usability Testing

- Objective: Evaluate user experience and intuitiveness of the UI/UX.
- Technique: User observation and feedback.
- Completion Criteria: Users can complete the main flows without significant confusion.

2.2 Test Levels

Type of Tests	Test Level			
	Unit	Integration	System	Acceptance
Functional Testing	X			
Usability Testing				X

2.3 Supporting Tools

Purpose	Tool	Vendor/In-house	Version
API Testing	Swagger	In-house	V1
API Testing	Postman	In-house	V11.41.4

3. Test Plan

3.1 Human Resources

Worker/Doer	Role	Specific Responsibilities/Comments
Mã Ngọc Long Bảo	Leader	Planning, execute, verify test result
Phạm Đức Thịnh	Member	Planning, execute, verify test result
Nguyễn Huỳnh Anh Tài	Member	Planning, execute, verify test result
Trần Hoàng Trung Anh	Member	Planning, execute, verify test result

3.2 Test Environment

Purpose	Tool	Provider	Version
System Testing	Browser, Mobile	Chrome, Edge	V1
Unit Testing	VSCode, TestContainers	Microsoft	V1

3.3 Test Milestones

Milestone Task	Start Date	End Date
Testing Document	20/03/2025	25/04/2024

4. Test Cases

- Other test reports: test-report.html, PCDACO-performance-report-20.pdf
- Detail Unit test: Report5_Unit Test.xlsx
- Detail User Acceptance test: Report5_Test Report.xlsx

5. Test Reports

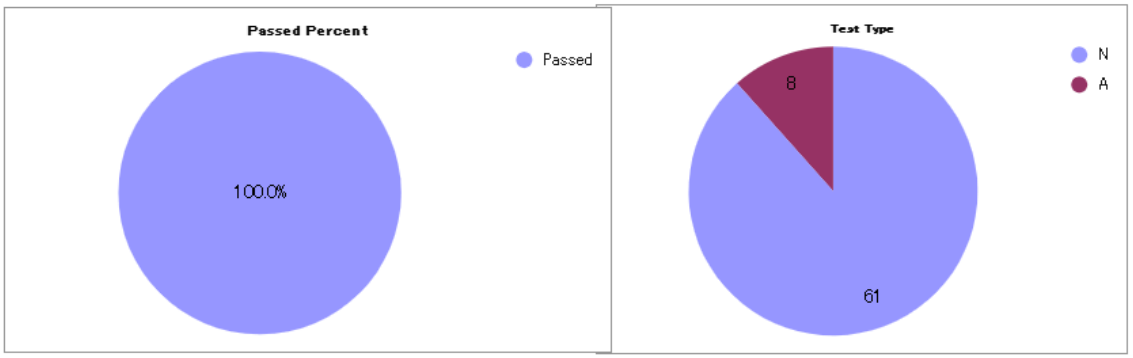
5.1. Unit test

UNIT TEST REPORT

Project Name	*FreeDriver - Platform connecting drivers and*	**Creator**	BaoMNL
Project Code	*SP25SE136*	**Reviewer/Approver**	MinhTTH
Document Code	*SP25SE136_Test Report_v1.0*	**Issue Date**	*03/20/2025*
Notes			

No	Function code	Passed	Failed	Untested	N	A	B	Total Test Cases
1	UT01	3	0	0	2	1	0	3
2	UT02	5	0	0	4	1	0	5
3	UT03	4	0	0	3	1	0	4
4	UT04	8	0	0	7	1	0	8
5	UT05	6	0	0	4	0	0	6
6	UT06	8	0	0	5	3	0	8
7	UT07	3	0	0	0	0	0	3
8	UT08	9	0	0	0	0	0	9
9	UT09	5	0	0	0	0	0	5
10	UT10	7	0	0	0	0	0	7
11	UT11	3	0	0	0	0	0	3
12	UT12	3	0	0	0	0	0	3
13	UT13	7	0	0	0	0	0	7
14	UT14	8	0	0	0	0	0	8
15	UT15	5	0	0	0	0	0	5
16	UT16	5	0	0	4	1	0	5
17	UT17	5	0	0	0	0	0	5
18	UT18	6	0	0	0	0	0	6
19	UT19	5	0	0	0	0	0	5
20	UT20	9	0	0	9	0	0	9
21	UT21	9	0	0	9	0	0	9
22	UT22	3	0	0	3	0	0	3
23	UT23	7	0	0	7	0	0	7
24	UT24	4	0	0	4	0	0	4
	Sub total	137	0	0	61	8	0	137

Test coverage	100.00 %
Test successful coverage	100.00 %
Normal case	44.53 %
Abnormal case	5.84 %
Boundary case	0.00 %



5.2 User acceptance test

TEST STATISTICS

Project Name	*FreeDriver - Platform connecting drivers a*	Creator	BaoMNL, ThinhPD, TaiNHA, AnhTHT
Project Code	*SP25SE136*	Reviewer/Approver	MinhTTH
Document Code	*SP25SE136_Test Report_v1.0*	Issue Date	*03/20/2025*
Notes			

No	Module code	Passed	Failed	Pending	N/A	Number of test cases
1	Authentication	14	0	0	0	18
2	Driver Use Case	9	0	0	0	11
3	Owner Use Case	29	0	0	0	35
4	Admin Use Case	106	0	0	0	125
4	Consultant Use Case	117	0	0	0	128
4	Technician Use Case	40	0	0	0	45
	Sub total	315	0	0	0	362

Test coverage	87.02 %
Test successful coverage	87.02 %

VI. Release Package & User Guides

1. Deliverable Package

No.	Deliverable Item	Description
1	Project Schedule/Tracking	Trello, Google Sheet
2	Project Backlog	Trello
3	Source Codes	GitHub
4	Docker Image	pcdaco-be pcdaco-fe
4	Database Script(s)	PCDACODB_Data.sql PCDACODB_Schema.sql
5	Final Report Document	Report7_Final Project Report.docx
6	Test Cases Document	Report5_Test Documentation.docx Report5_Test Report.xlsx Report5_Unit Test.xlsx
7	Slide	SEP490 - Final

2. Installation Guides

2.1 System Requirements

PC	Minimum	Recommended
Internet Connection	LAN, internet access capability	LAN, Wi-fi (16 Mbps)
CPU	Intel Core i5 1.5Ghz	Intel Core i7 2.6Ghz
Memory	4GB RAM	8GB RAM
Storage	128GB	256GB
Web Browser	Chrome (v69)	Chrome latest stable version

2.1.2 Mobile Application

Mobile	Minimum	Recommended
Internet Connection	Wi-fi	LAN, Wi-fi (16 Mbps)
CPU	Kirin 659	Snapdragon 712
Memory	4GB RAM	8GB RAM
Storage	16GB	32GB

2.2 Installation Instruction

2.2.1 VSCode

2.2.1.1 System Requirements

- **Operating Systems:** Windows 7 or later, macOS 10.11 or later, and popular Linux distributions.
- **Hardware:** A modern computer with adequate RAM and processing power.
- **Dependencies:** None; VSCode is a self-contained application.

2.2.1.2 Installation Steps

1. Visit the official Visual Studio Code download page.
2. Select the installer package that matches your operating system.
3. Download and run the installer.
4. Follow the on-screen instructions to complete the installation.
5. Launch VSCode and consider installing extensions to tailor the editor to your needs.

2.2.2 DBeaver

2.2.2.1 System Requirements

- **Operating Systems:** Windows, macOS, and Linux.
- **Hardware:** Standard computing resources are sufficient.
- **Dependencies:** DBeaver is typically bundled with its own Java Runtime Environment (JRE).

2.2.2.2 Installation Steps

1. Go to the official DBeaver download page.
2. Choose the version that suits your operating system—usually the Community Edition.
3. Download the installer or archive file.
4. Run the installer and complete the setup by following the wizard's instructions.
5. Launch DBeaver and set up your database connections as required.

2.2.3 Postman

2.2.3.1 System Requirements

- **Operating Systems:** Windows, macOS, and Linux.

- **Hardware:** A typical computer setup; Postman runs smoothly on most systems.
- **Dependencies:** None; Postman is distributed as a standalone package.

2.2.3.2 Installation Steps

1. Navigate to the Postman download page.
2. Select the installer corresponding to your operating system.
3. Download and execute the installer.
4. Follow the installation prompts to complete the setup.
5. Open Postman, sign in (or create an account), and start using the tool for API development and testing.

2.2.4 Docker Desktop

2.2.4.1 System Requirements

- **Operating Systems:**
  - **Windows:** Windows 10 Pro, Enterprise, or Education (with Hyper-V support) or Windows 10 Home with WSL 2.
  - **macOS:** macOS 10.15 (Catalina) or later.
- **Hardware:** A computer with virtualization support enabled in the BIOS.
- **Dependencies:**
  - On Windows, ensure virtualization and (if necessary) WSL 2 are enabled.
  - On macOS, ensure you meet the system requirements for Docker Desktop.

2.2.4.2 Installation Steps

1. Visit the Docker Desktop download page.
2. Choose the appropriate installer for your operating system.
3. Download the installer and run it.
4. Follow the installation instructions and accept any required license agreements.
5. Launch Docker Desktop and complete any initial configuration (such as enabling WSL 2 on Windows if needed).

2.2.5 Figma

2.2.5.1 System Requirements

- **Operating Systems:** Since Figma is primarily web-based, any OS with a modern web browser (Chrome, Firefox, Safari, etc.) works. For the desktop app, see the specific requirements on the Figma website.
- **Hardware:** A device capable of running a modern browser; for the desktop version, refer to Figma's system recommendations.
- **Dependencies:** An active internet connection is required for the web version.

2.2.5.2 Installation Steps

1. **For the Web Version:**
   - Navigate to the Figma website and sign up or log in to your account.
2. **For the Desktop Application:**
   - Visit the Figma downloads page.
   - Download the installer for your operating system.
   - Run the installer and follow the on-screen instructions.
   - Launch Figma and sign in to start designing.

2.2.6 NeoVim

2.2.6.1 System Requirements

- **Operating Systems:** Windows, macOS, and Linux.
- **Hardware:** A standard computer setup is sufficient.
- **Dependencies:** Some plugins may require additional tools (e.g., Node.js) and further configuration.

2.2.6.2 Installation Steps

1. **On Windows:**
   - Download the installer from the NeoVim releases page or use a package manager such as Scoop or Chocolatey.
2. **On macOS:**
   - Install via Homebrew by running: $$ brew install neovim $$
   - Alternatively, download a prebuilt package from the official website.
3. **On Linux:**
   - Use your distribution's package manager, for example: $$ sudo apt-get install neovim \quad \text{or} \quad sudo dnf install neovim $$
   - For the latest version, consider using a snap package or compiling from source.
4. After installation, configure NeoVim by creating an `init.vim` file in your configuration directory (typically `~/.config/nvim`). Customize your settings and install desired plugins to enhance your workflow.

2.2.7 Setup Backend

- Prerequisites:
  - Git: Ensure Git is installed on your system. (Download Git)
  - .NET 8 SDK: The project targets .NET 8. Download and install the SDK from the official Microsoft website. (Download .NET 8)
  - .Net 8 Runtime: (.NET 8 Runtime)
  - Docker Desktop: Docker is required to run the PostgreSQL/PostGIS database container. (Download Docker Desktop) Make sure Docker Desktop is running before proceeding.
  - Taskfile: (taskfile)
  - (Optional) Code Editor/IDE: Visual Studio 2022 or Visual Studio Code with C# extensions are recommended.
- Setup Steps:
  1. **Clone the Repository:** Open your terminal or command prompt and clone the project repository:

```
git clone https://github.com/PCDACO/PCDACO-BE.git
```

  2. **Navigate to Project Directory**

```
cd PCDACO-BE
```

3. **Configure Application Settings:**

- Create a .env file in the src directory of your project.

```
CONNECTION_STRING="User ID=postgres;Password=postgres;Host=localhost;Port=5432;Database=PCDACODB"

ISSUER="https://pvekpowymp.io"
AUDIENCE="https://ldwptnvwhc.app"
SECRET_KEY="tAlTTVevURpLmmhnCwml9wVhmGrAHX69joB5DbsTQYiVlQIkJwXKwvzFTLnhkHGQ"
TOKEN_EXPIRATION_IN_MINUTES=60

CLOUDINARY_URL=cloudinary://893831472878937:xmLYe7EIekjJjNfMsUic5G4iyLw@ds2bfbfyd

# Encryption
MASTER_KEY="dnjGHqR9O/2hKCQUgImXcEjZ9YPaAVcfz4l5VcTBLcY="

# PayOS
PAYOS_CLIENT_ID=4c6c2974-206a-4e6e-a3c3-f27a64cfa372
PAYOS_API_KEY=8c0bc12d-2490-4820-af34-5cb325695df6
PAYOS_CHECKSUM_KEY=a7d31fe00b2dfb6791e6c5129dd0af71879b7c9d53d9c9439a0f01d159cf304c

RETURN_URL=https://koi.persiehomeserver.com
CANCEL_URL=https://koi.persiehomeserver.com

# MailKit
MAIL_SETTINGS_MAIL=mglongbao@gmail.com
MAIL_SETTINGS_DISPLAY_NAME=Free Driver
MAIL_SETTINGS_PASSWORD=uyfy ycce qjwt szyd
MAIL_SETTINGS_HOST=smtp.gmail.com
MAIL_SETTINGS_PORT=587

SEQ_URL=http://localhost:5341
SEQ_API_KEY=U2NmJSTTtKWUP7s004iv
OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:5341/ingest/otlp/v1/traces
OTEL_EXPORTER_OTLP_PROTOCOL=http/protobuf

# Contract
COLLATERAL_PRICE=20000000
```

4. **Start the Database Container:**

   The project uses a PostGIS database managed via Docker Compose *(docker-compose.yml)*.

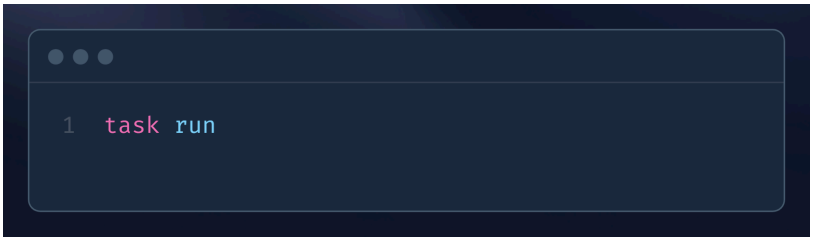
- **Start Docker Desktop**: Ensure Docker Desktop is running.
- **Run Docker Compose:** Open a terminal in the project's root directory (where docker-compose.yml is located) and run:

```
docker-compose up -d
```

This command will:

- Automatically download the *postgis/postgis:13-3.1* image if it's not already available locally
- Create and start a container named *postgis_container,* running it in the background.
- Set up a database named *PCDACODB* with the default username *postgres* and password *postgres*.
- Expose the database on *localhost:5432*, as specified in the *docker-compose.yml*.
- You'll then be able to connect to the database using any Postgres client or application.

5. **Run project:** Open your terminal or command prompt and run this command

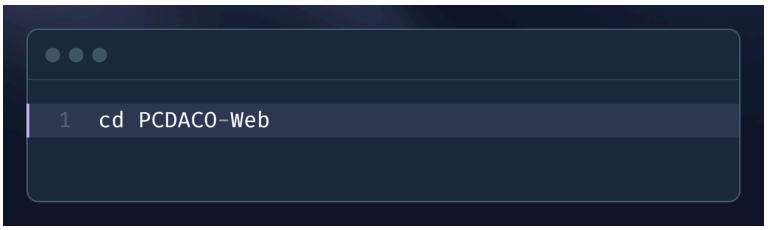


2.2.8 Setup Web App

- Prerequisites:
  - Node: Ensure Node is installed on your system. Recommend v22.15.0 (Download Node)
  - Code Editor/IDE: Visual Studio Code with React extensions are recommended.
- Setup steps

1. **Clone the Repository:** Open your terminal or command prompt and clone the project repository:

2. **Navigate to Project Directory**



3. **Configure Application Settings:**
   - Create a .env file in the src directory of your project.

```
NEXT_PRIVATE_API_URL=http://144.91.64.170:8080

NEXT_PUBLIC_MAPBOX_ACCESS_TOKEN=pk.eyJ1IjoiYW5odGh0MTM4IiwiYSI6ImNtOGExOHI2bDEwb2cybHF1M2l4aWxr
tDF2HG0qSTk3Q
NEXT_PUBLIC_MAPBOX_URL=https://api.mapbox.com

```

4. **Install library and start project**

```
bun i
>> bun run dev
```

2.2.9 Setup Mobile App

2.2.9.1 Owner App

- Prerequisites:
  - Node: Ensure Node is installed on your system. Recommend v22.15.0 (Download Node)
  - Code Editor/IDE: Visual Studio Code with React Native extensions are recommended.
- Setup steps
  1. **Clone the Repository:** Open your terminal or command prompt and clone the project repository:

```
git clone https://github.com/PCDACO/PCDACO-moblie.git
```

  2. **Navigate to Project Directory**

```
cd PCDACO-mobile
```

  3. **Configure Application Settings:**
     - Create a .env file in the src directory of your project.

```
#api url
EXPO_PUBLIC_API_URL=https://pcdaco-api.persiehomeserver.com

#mapbox
EXPO_PUBLIC_MAPBOX_ACCESS_TOKEN=pk.eyJ1IjoiYW5odGh0MTM4IiwiYSI6ImNtOGExOHI2bDEwb2cybHF1M2l4aWx
tDF2HG0qSTk3Q
EXPO_PUBLIC_MAPBOX_URL=https://api.mapbox.com

#Geoapify
EXPO_PUBLIC_GEOAPIFY_URL=https://api.geoapify.com/v1
EXPO_PUBLIC_GEOAPIFY_API_KEY=102afaeaecf440baaaadfb78f8036389

#ai
EXPO_PUBLIC_AI_API_KEY=7uMz1EqukYe72VIbLeMeaGV3EeqhuTsq
EXPO_PUBLIC_AI_API_URL=https://api.fpt.ai/vision
```

4. **Install library and start project**

```
bun i
>> bun start
```

2.2.9.2 Driver App

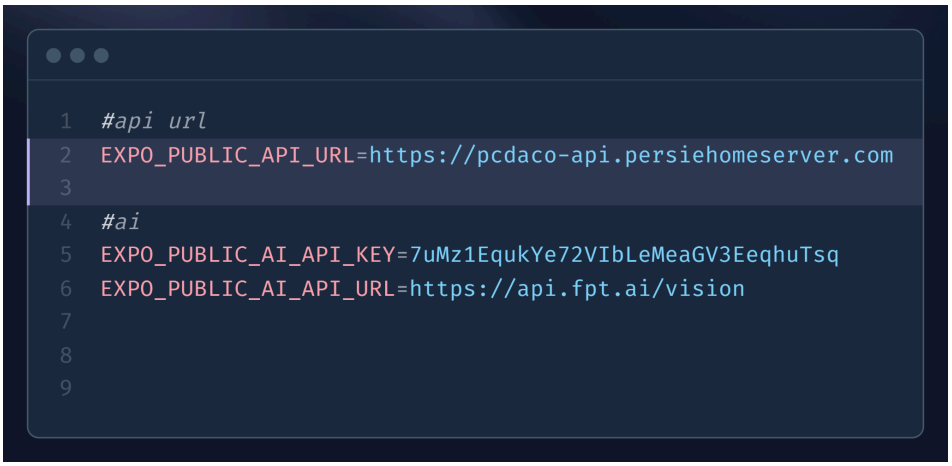
- Prerequisites:
  - Node: Ensure Node is installed on your system. Recommend v22.15.0 (Download Node)
  - Code Editor/IDE: Visual Studio Code with React Native extensions are recommended.
- Setup steps
  1. **Clone the Repository:** Open your terminal or command prompt and clone the project repository:

```
git clone https://github.com/PCDACO/PCDACO-Driver.git
```

2. **Navigate to Project Directory**

```
cd PCDACO-Driver
```

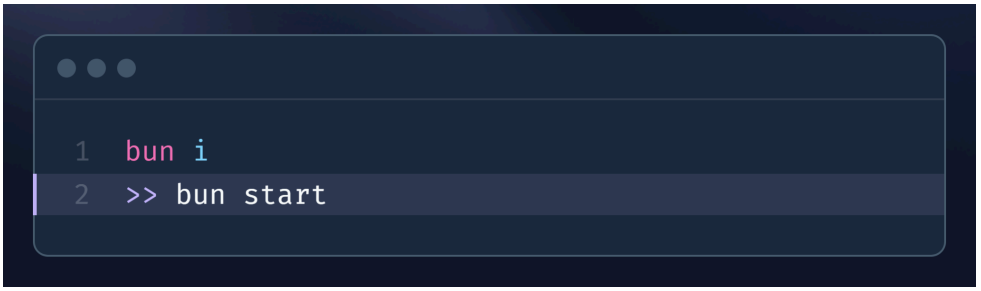
3. **Configure Application Settings:**
    - Create a .env file in the src directory of your project.



```
#api url
EXPO_PUBLIC_API_URL=https://pcdaco-api.persiehomeserver.com

#ai
EXPO_PUBLIC_AI_API_KEY=7uMz1EqukYe72VIbLeMeaGV3EeqhuTsq
EXPO_PUBLIC_AI_API_URL=https://api.fpt.ai/vision
```

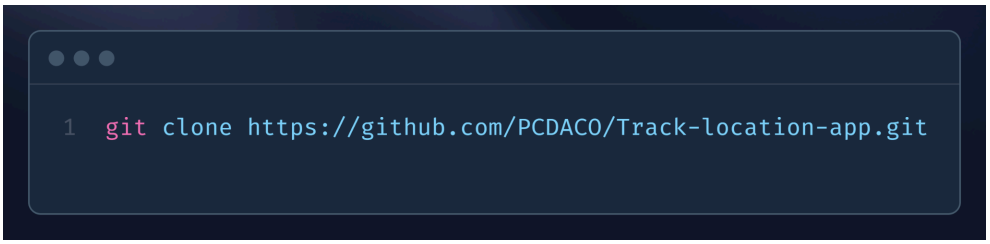
4. **Install library and start project**



```
bun i
>> bun start
```

2.2.9.3 Tracking Location App

- Prerequisites:
    - Node: Ensure Node is installed on your system. Recommend v22.15.0 (Download Node)
    - Code Editor/IDE: Visual Studio Code with React Native extensions are recommended.
- Setup steps
    1. **Clone the Repository:** Open your terminal or command prompt and clone the project repository:



```
git clone https://github.com/PCDACO/Track-location-app.git
```

2. **Navigate to Project Directory**

```
cd Track-location-app
```

3. **Configure Application Settings:**
    - Create a .env file in the src directory of your project.

```
#api url
EXPO_PUBLIC_API_URL=https://pcdaco-api.persiehomeserver.com
```

4. **Install library and start project**

```
bun i
>> bun start
```

3. User Manual

3.1 Overview

3.1.1 Owner User Manual

- Sign in and Register an account
- Add user license
- Update user profile
- Add a new car
- View and sign car contract with technician when technician verifies car
- View personal cars
- View incoming requests from drivers
- Approve/Reject booking
- Upload images to check car before booking
- Upload images to check car after driver completes booking
- Set car unavailability for specific times
- Turn car to be online for driver can book
- Turn car to be offline if they do not want to allow drivers to book
- Track car location
- Send withdrawal request
- View earnings from car rentals, withdraw payments, and view transaction history.
- Leave ratings and feedbacks on driver

3.1.2 Driver User Manual

- Sign in and Register an account
- Add user license
- Search for available car
- Create booking requests for cars, including the rental period and specific requirements
- Start booking trips
- Complete booking
- View history of previous car bookings, including rental details and feedback.
- Track ongoing rides.
- Pay for car rental
- Leave ratings and feedbacks on owner

3.1.3 Admin User Manual

- Sign in
- Manage users (car owners, drivers, technician, consultant)
- Monitors owner withdrawal requests
- Monitors financial transactions between driver and owner
- Monitors users including owners, drivers, technician, consultant
- Ban or unban user
- Create Staff
- View bookings and cars
- View statistics
- Approve User License
- Manage gps devices, amenities, manufacturers, models, transmission types, fuel types,

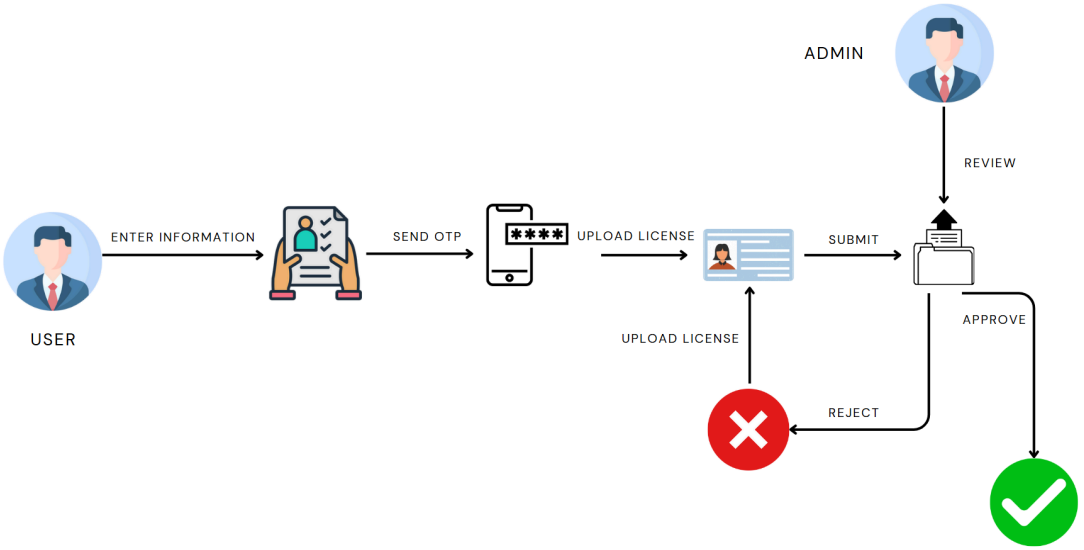
3.1.4 Consultant User Manual

- Sign in and update profile
- View personal statistics
- View recent activity
- View cars and car detail
- View booking reports and booking report detail
- View car report and car report detail
- Create or Update or Delete schedule for car verification, incident, change gps devices for car.
- Assign compensation to user
- Approve booking report
- Approve car report

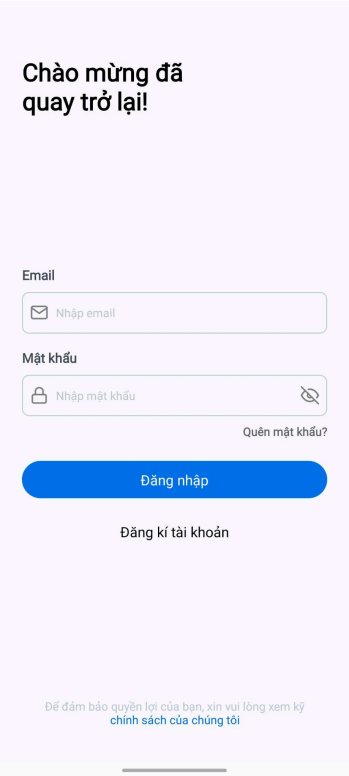
3.1.5 Technician User Manual

- Sign in and update profile
- View personal statistics
- View recent activity
- View personal inspection schedule according to month, week, day
- In-progress schedule
- View and sign car contract with owner when handling car verification schedule
- Upload image for inspection
- Approve or Reject a schedule

3.2 USER REGISTER

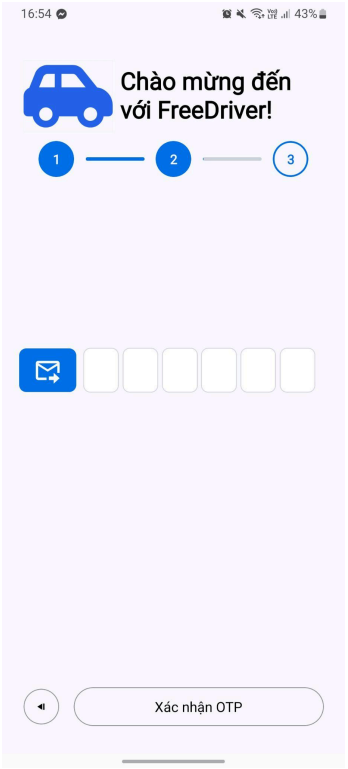


- Open Free Driver app, click “Đăng ký tài khoản”

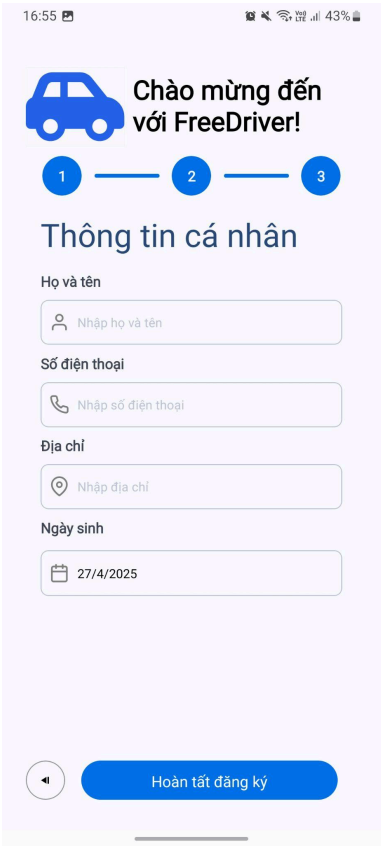


- Enter Email and Password
- Click “Tôi đã đọc và đồng ý…”

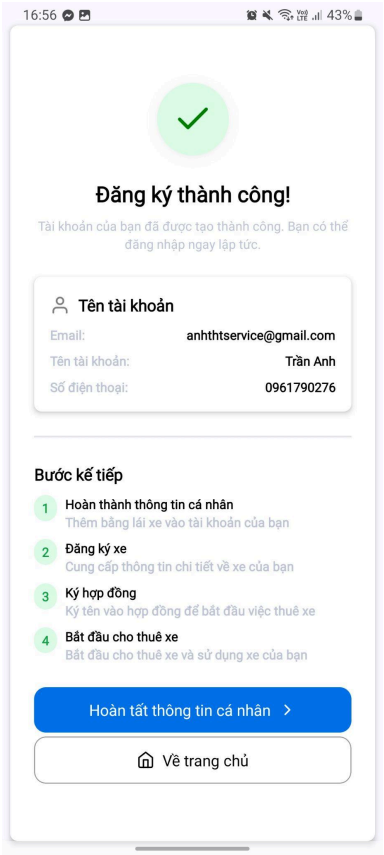
- Open email and get OTP



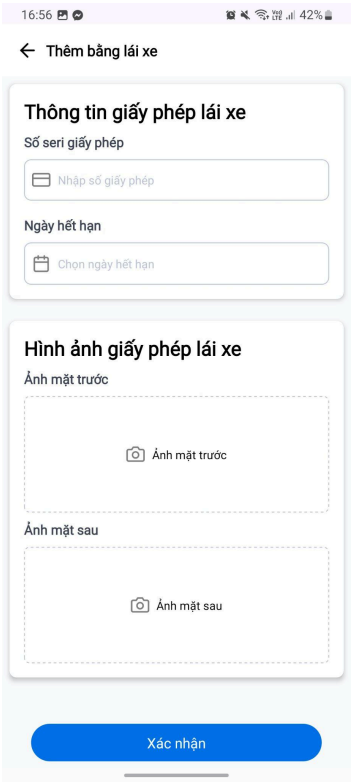
- Enter Full Name, phone number, address, DoB



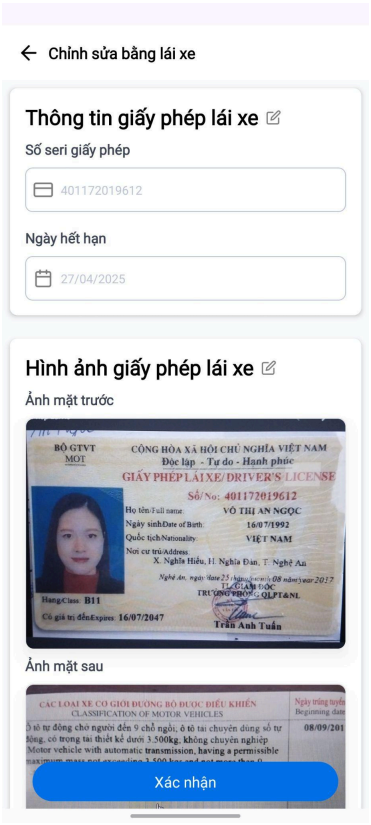
- Success Register, Click “Hoàn tất thông tin cá nhân”



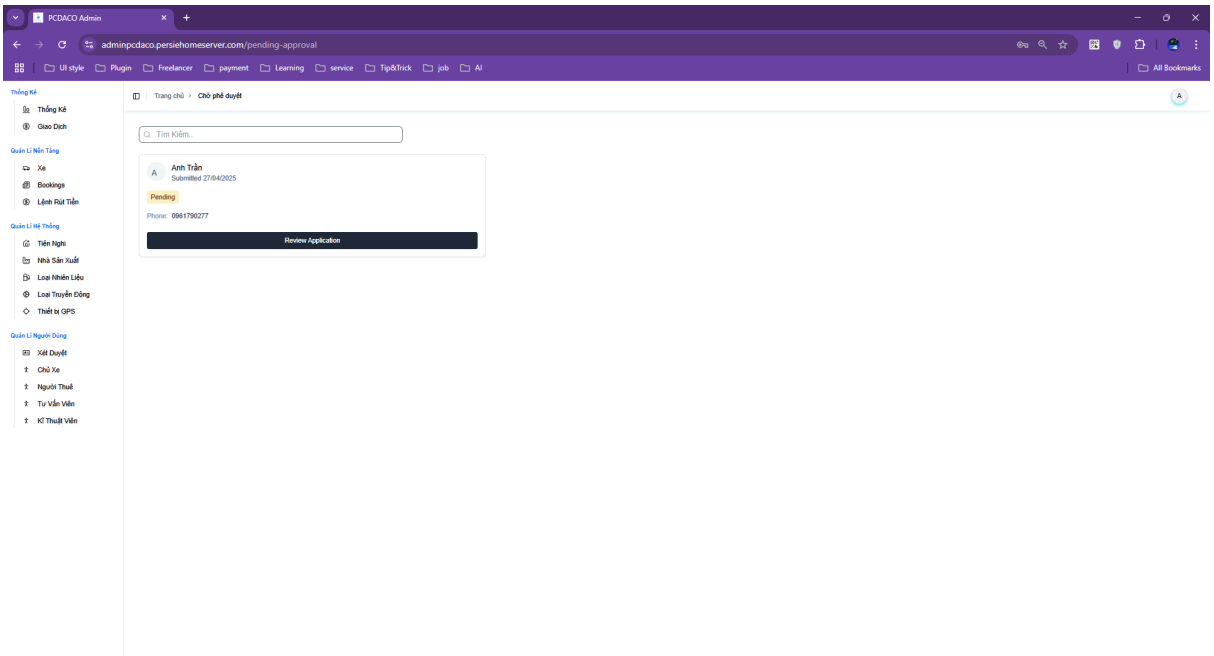
- Upload driving license



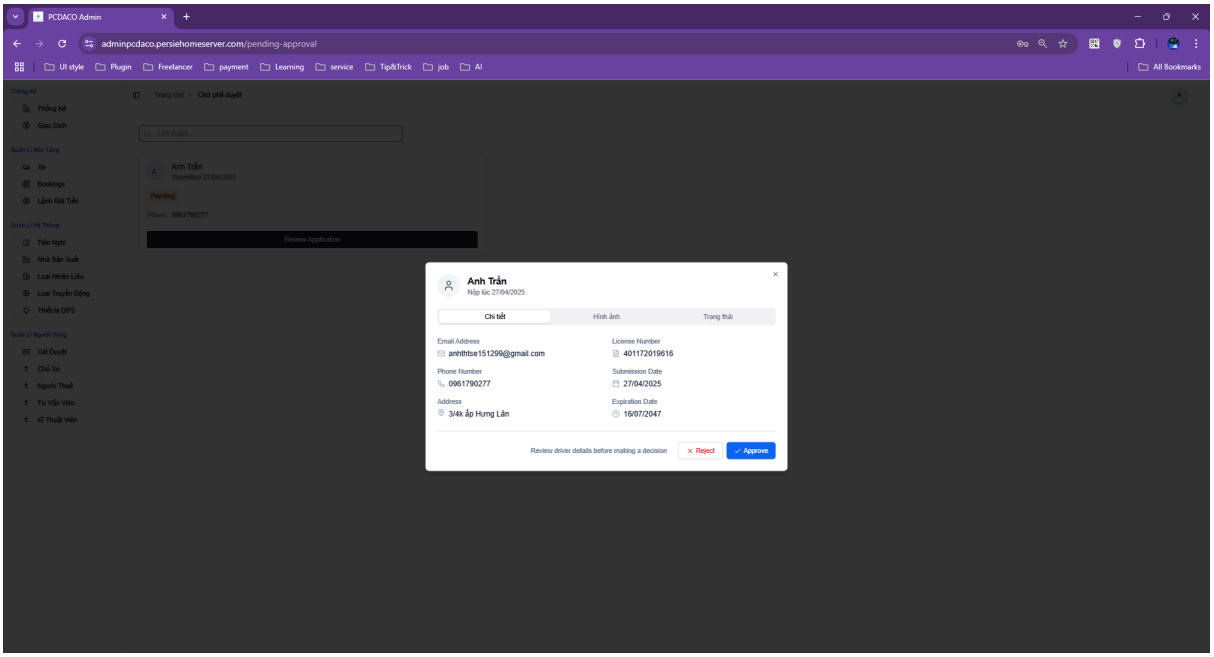
- Check all data is correct, click “Xác nhận”



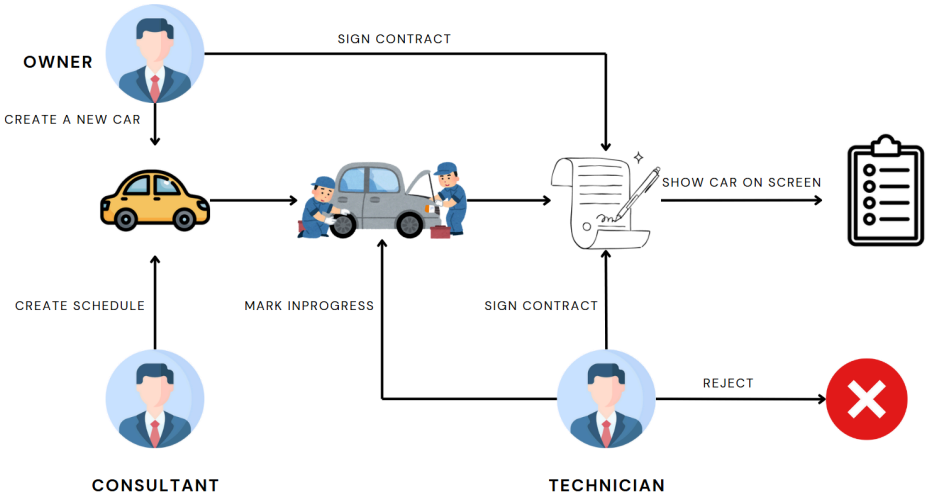
- Admin sign into system and select menu “Xét duyệt”
- Click “Review Application”



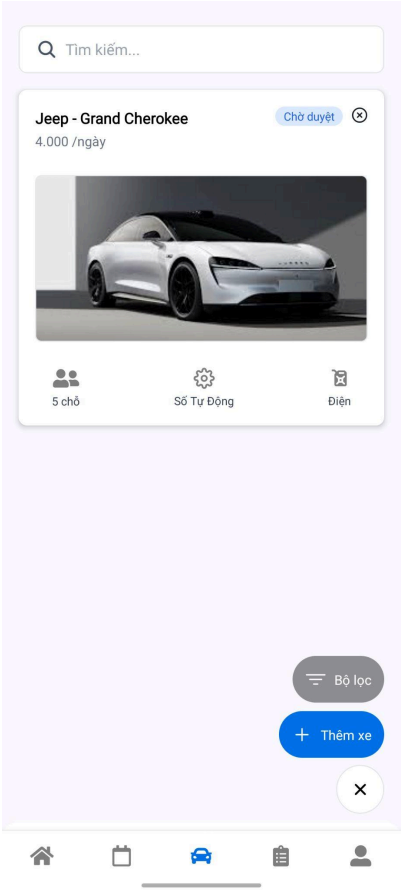
- Review detail of driving license, then click “Approve”



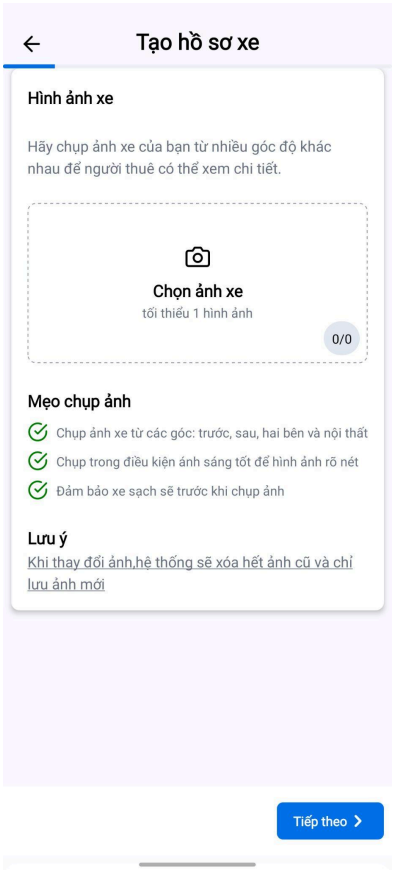
3.3 OWNER CREATE CAR

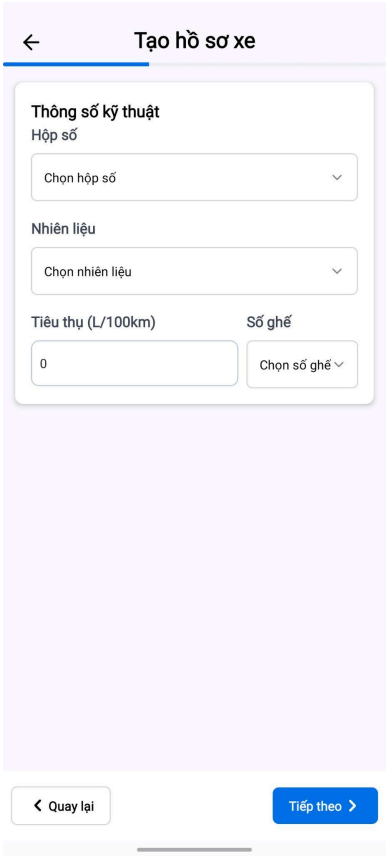


- Click button plus icon to move create car

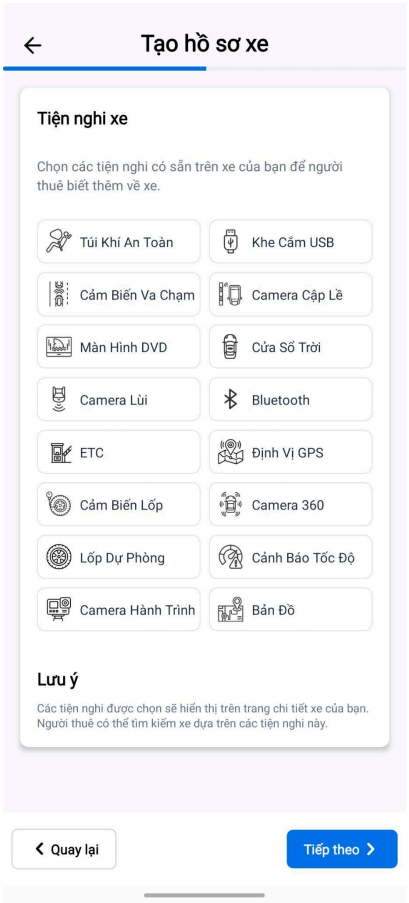


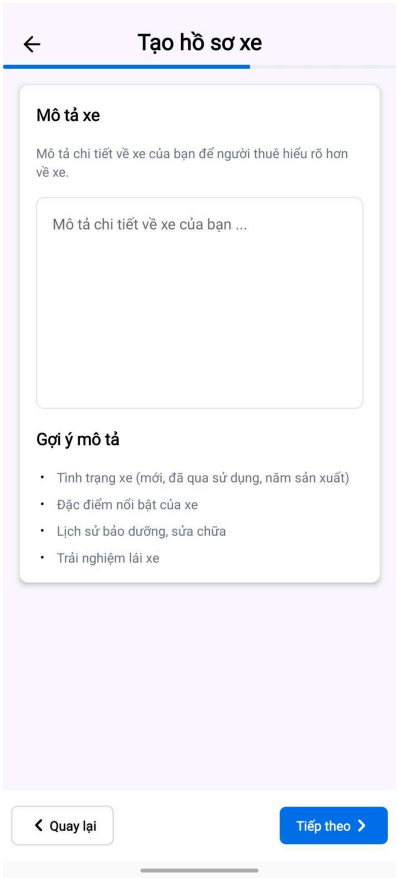
- Owner fills in all field required

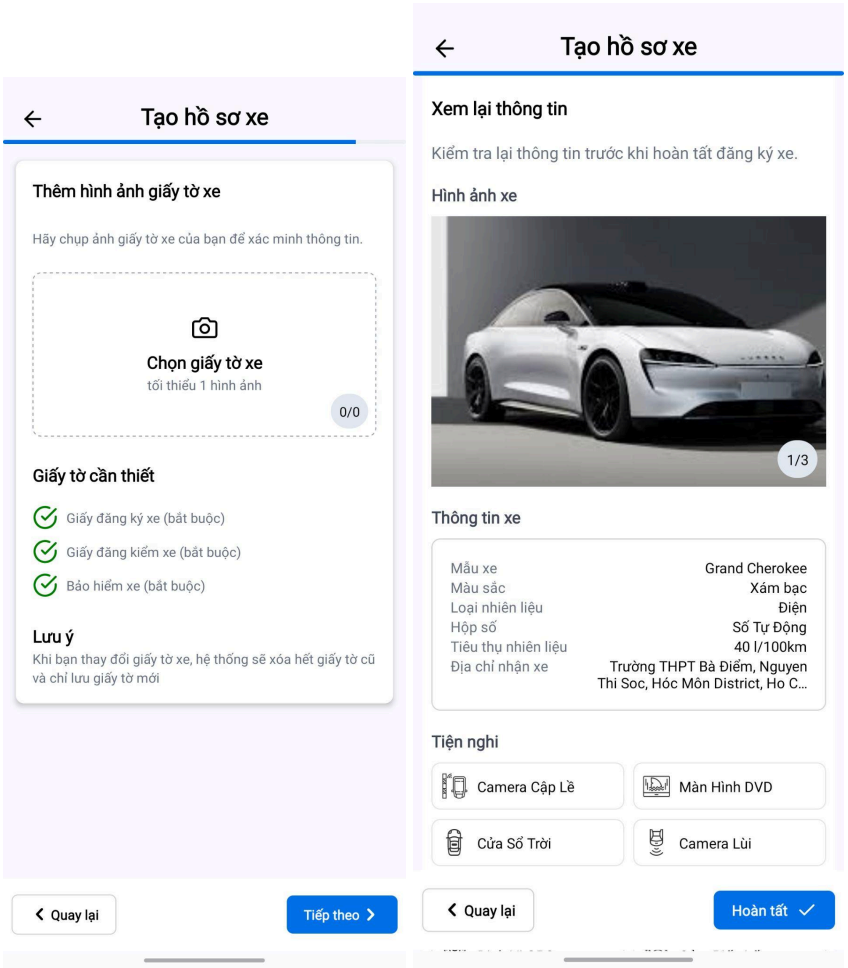

Tạo hồ sơ xe
Hình ảnh xe
Hãy chụp ảnh xe của bạn từ nhiều góc độ khác nhau để người thuê có thể xem chi tiết.
Chọn ảnh xe
tối thiểu 1 hình ảnh
0/0
Mẹo chụp ảnh
Chụp ảnh xe từ các góc: trước, sau, hai bên và nội thất
Chụp trong điều kiện ánh sáng tốt để hình ảnh rõ nét
Đảm bảo xe sạch sẽ trước khi chụp ảnh
Lưu ý
Khi thay đổi ảnh,hệ thống sẽ xóa hết ảnh cũ và chỉ lưu ảnh mới
Tiếp theo

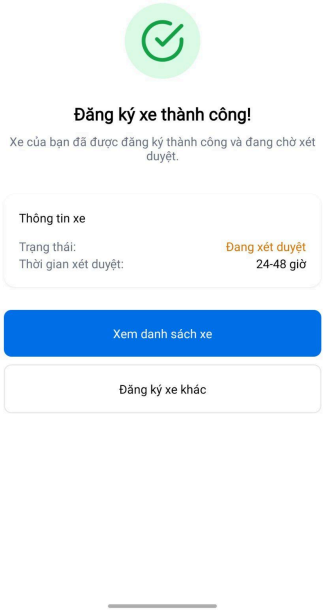

Tạo hồ sơ xe
Thông số kỹ thuật
Hộp số
Chọn hộp số
Nhiên liệu
Chọn nhiên liệu
Tiêu thụ (L/100km)
Số ghế
0
Chọn số ghế
Quay lại
Tiếp theo

Tạo hồ sơ xe
Thông tin xe
Mẫu xe
Nhấn chọn mẫu xe
Vị trí để xe
Nhập vị trí để xe
Biển số xe
Nhập biển số xe
Màu sắc
Nhập màu sắc
Quay lại
Tiếp theo

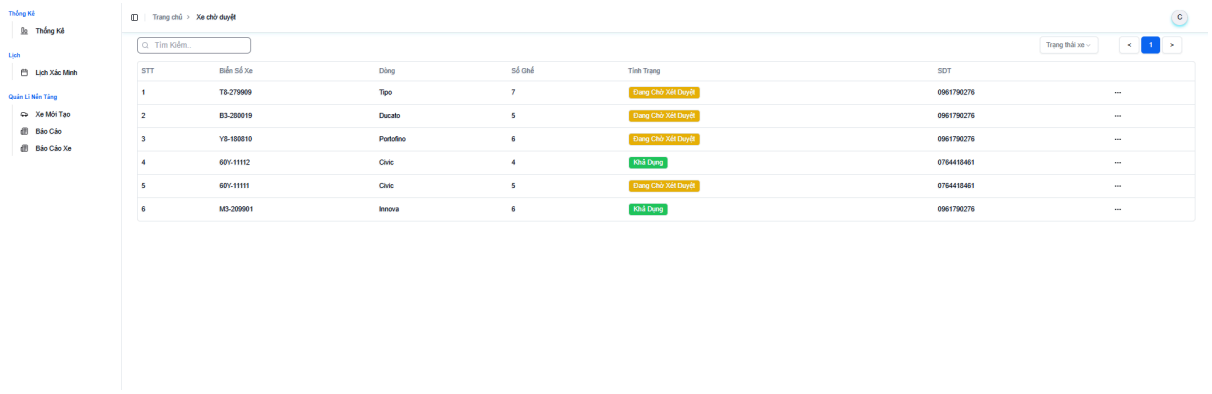

Tạo hồ sơ xe
Tiện nghi xe
Chọn các tiện nghi có sẵn trên xe của bạn để người thuê biết thêm về xe.
Túi Khí An Toàn
Khe Cắm USB
Cảm Biến Va Chạm
Camera Cập Lề
Màn Hình DVD
Cửa Sổ Trời
Camera Lùi
Bluetooth
ETC
Định Vị GPS
Cảm Biến Lốp
Camera 360
Lốp Dự Phòng
Cảnh Báo Tốc Độ
Camera Hành Trình
Bản Đồ
Lưu ý
Các tiện nghi được chọn sẽ hiển thị trên trang chi tiết xe của bạn. Người thuê có thể tìm kiếm xe dựa trên các tiện nghi này.
Quay lại
Tiếp theo

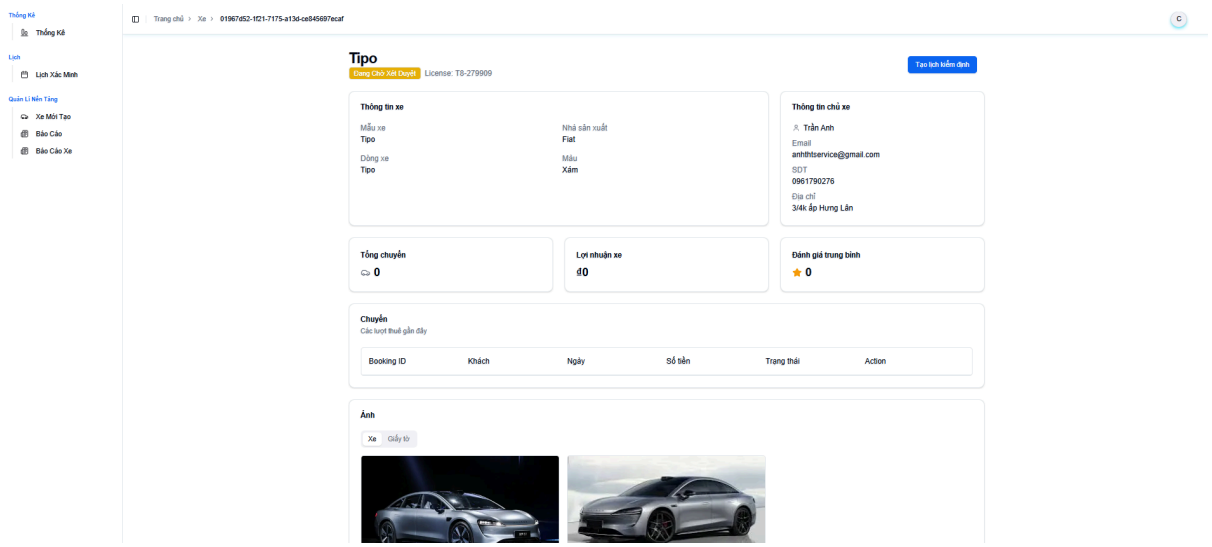

Tạo hồ sơ xe
Mô tả xe
Mô tả chi tiết về xe của bạn để người thuê hiểu rõ hơn về xe.
Mô tả chi tiết về xe của bạn ...
Gợi ý mô tả
Tình trạng xe (mới, đã qua sử dụng, năm sản xuất)
Đặc điểm nổi bật của xe
Lịch sử bảo dưỡng, sửa chữa
Trải nghiệm lái xe
Quay lại
Tiếp theo


Tạo hồ sơ xe
Thêm hình ảnh giấy tờ xe
Hãy chụp ảnh giấy tờ xe của bạn để xác minh thông tin.
Chọn giấy tờ xe
tối thiểu 1 hình ảnh
0/0
Giấy tờ cần thiết
Giấy đăng ký xe (bắt buộc)
Giấy đăng kiểm xe (bắt buộc)
Bảo hiểm xe (bắt buộc)
Lưu ý
Khi bạn thay đổi giấy tờ xe, hệ thống sẽ xóa hết giấy tờ cũ và chỉ lưu giấy tờ mới
Quay lại
Tiếp theo
Tạo hồ sơ xe
Xem lại thông tin
Kiểm tra lại thông tin trước khi hoàn tất đăng ký xe.
Hình ảnh xe
1/3
Thông tin xe
Mẫu xe Grand Cherokee
Màu sắc Xám bạc
Loại nhiên liệu Điện
Hộp số Số Tự Động
Tiêu thụ nhiên liệu 40 l/100km
Địa chỉ nhận xe Trường THPT Bà Điểm, Nguyen Thi Soc, Hóc Môn District, Ho C...
Tiện nghi
Camera Cập Lề
Màn Hình DVD
Cửa Sổ Trời
Camera Lùi
Quay lại
Hoàn tất

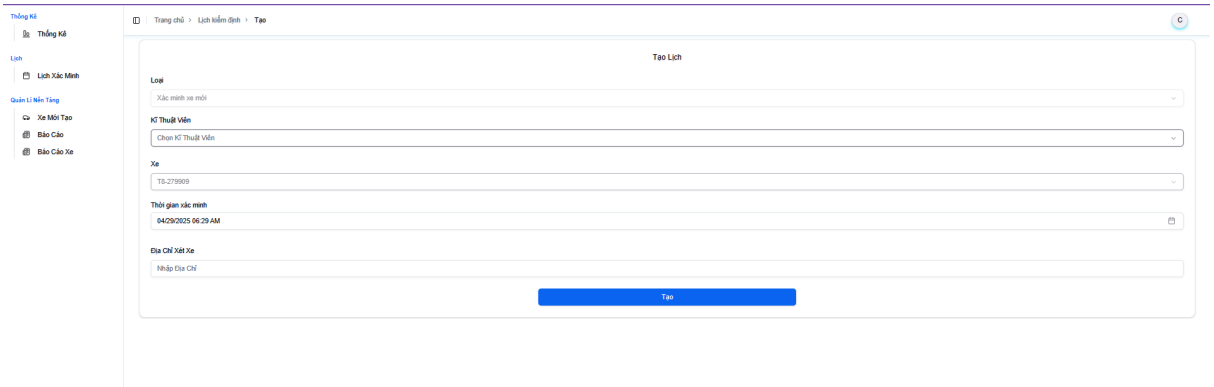

Đăng ký xe thành công!
Xe của bạn đã được đăng ký thành công và đang chờ xét duyệt.
Thông tin xe
Trạng thái: Đang xét duyệt
Thời gian xét duyệt: 24-48 giờ
Xem danh sách xe
Đăng ký xe khác

- Consultant signs in system and click menu "Xe mới tạo"

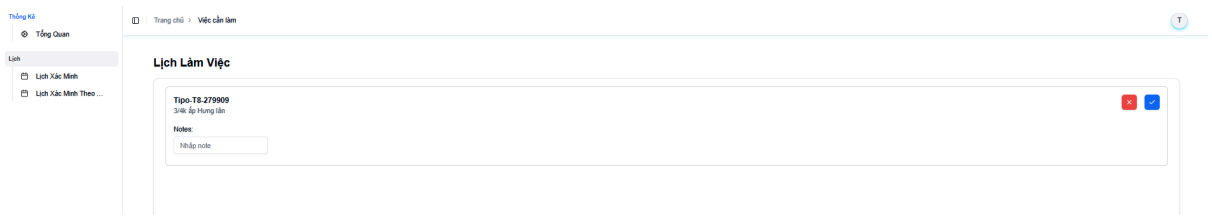




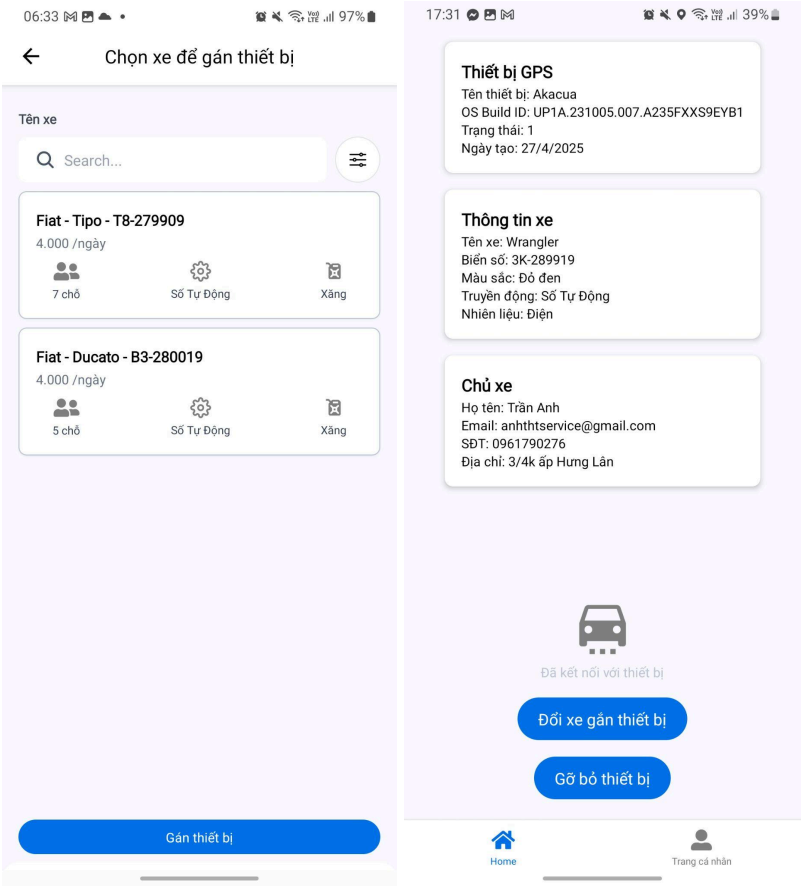
- Consultant clicks button "Xem chi tiết" and continue click button "Tạo lịch kiểm định"
- Consultant fills all required fields.



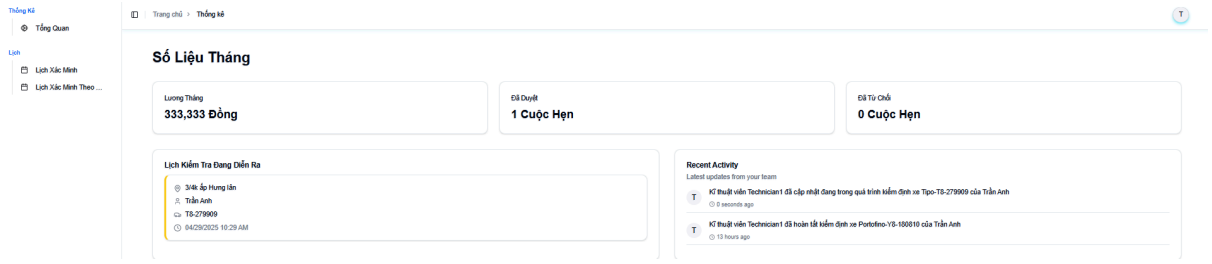
- Technician signs in the system and click menu "Lịch Xác Minh"
- Technician clicks button check to in progress inspection schedules.



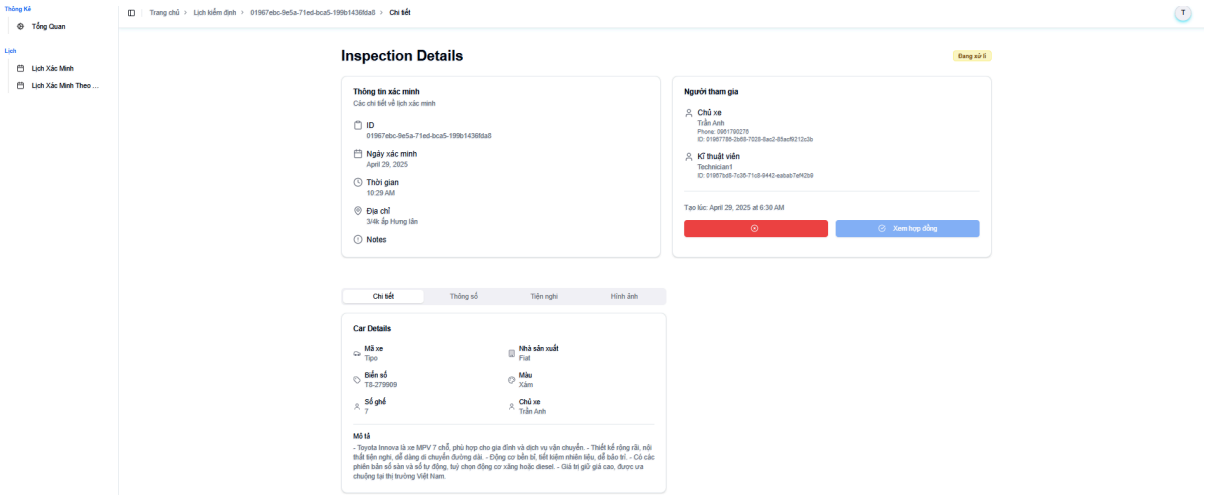
- Technician signs in app tracking and choose a schedules vehicle



- Technician moves back to the statistics screen and clicks the card to see detail.

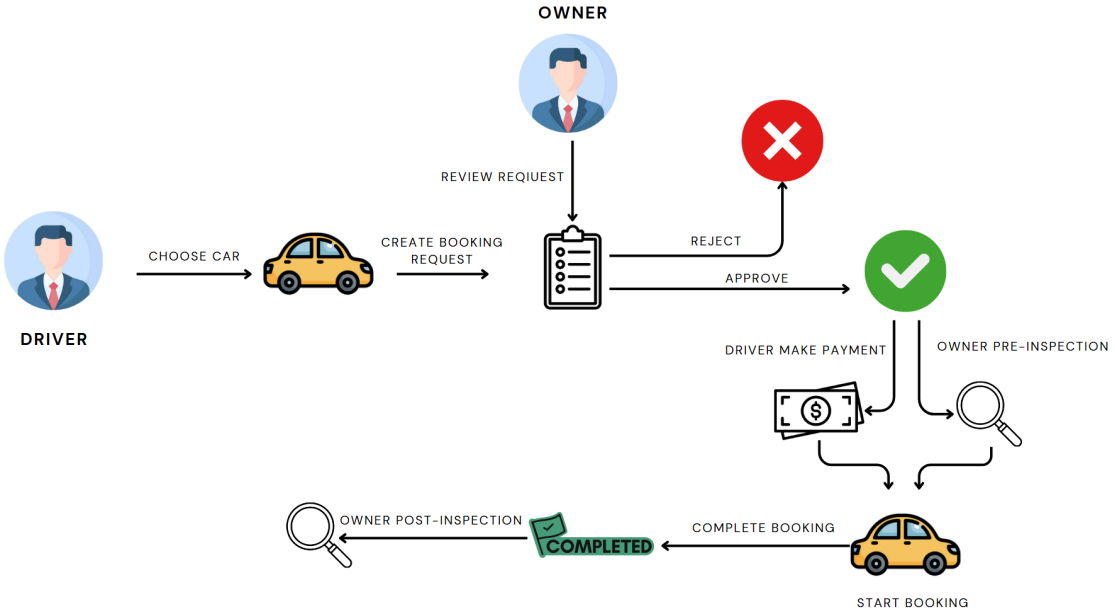


- Technician clicks button "Xem hợp đồng"
- Technician and Owner will sign and click button "Xác nhận"
- Technician fills all require field and click button "Xác nhận"

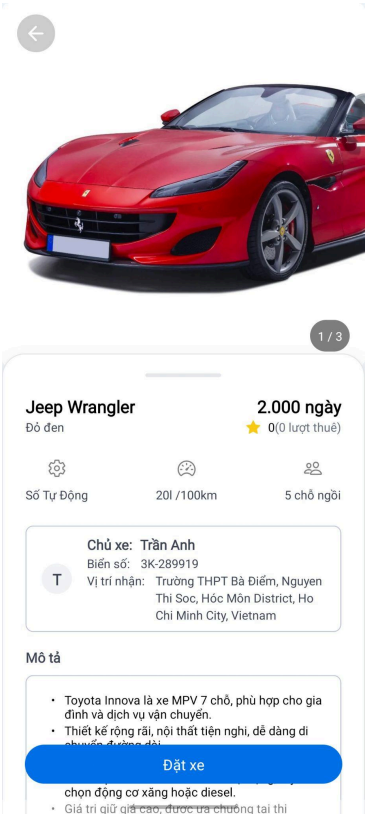


-

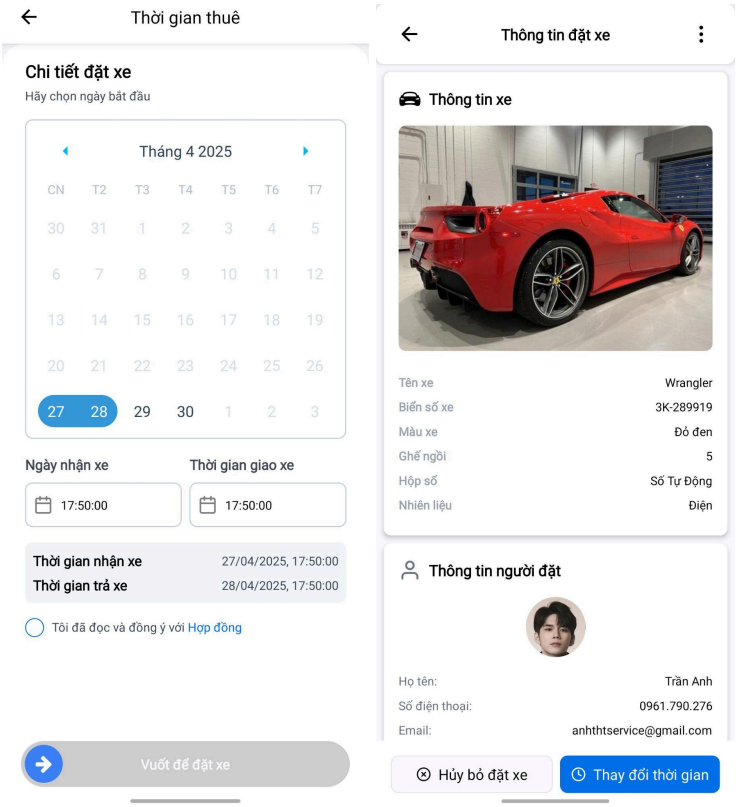
3.4 DRIVER CREATE BOOKING



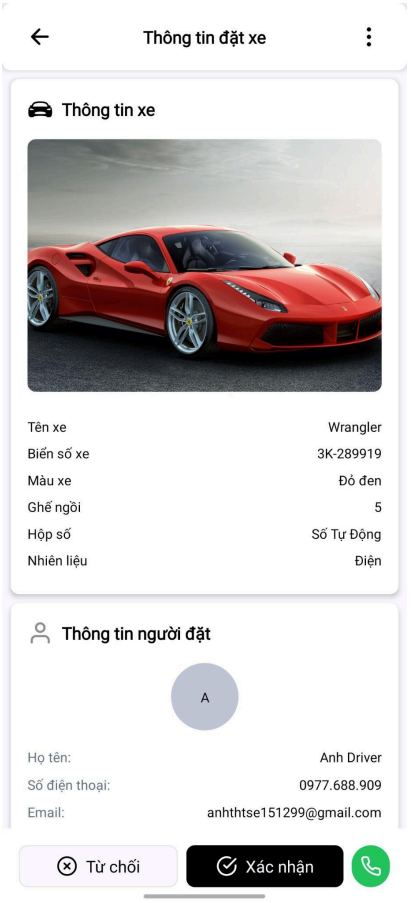
- Driver clicks button "Đặt xe"



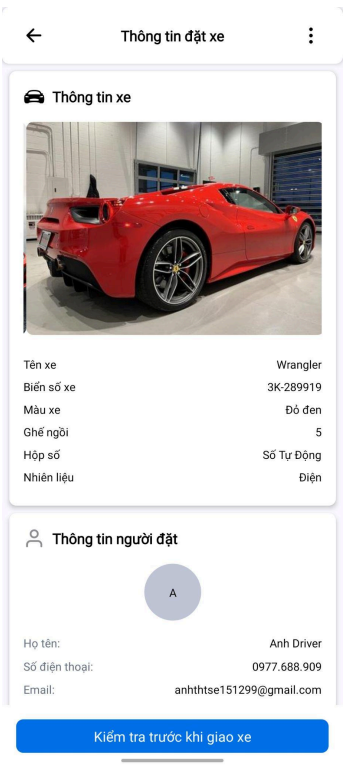
- Driver fills all required fields and drag "Xác nhận"



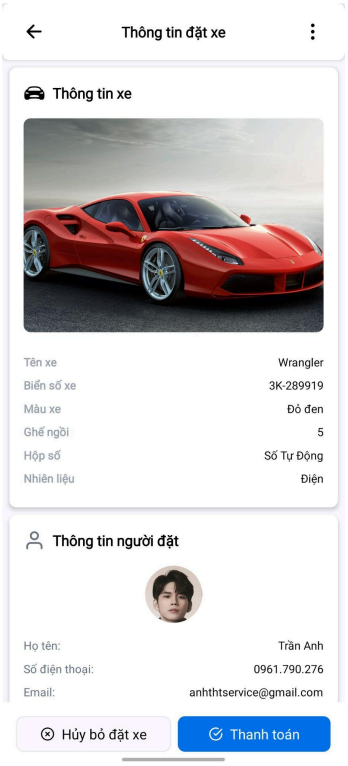
- Owner clicks button "Chấp nhận đặt xe" and signature



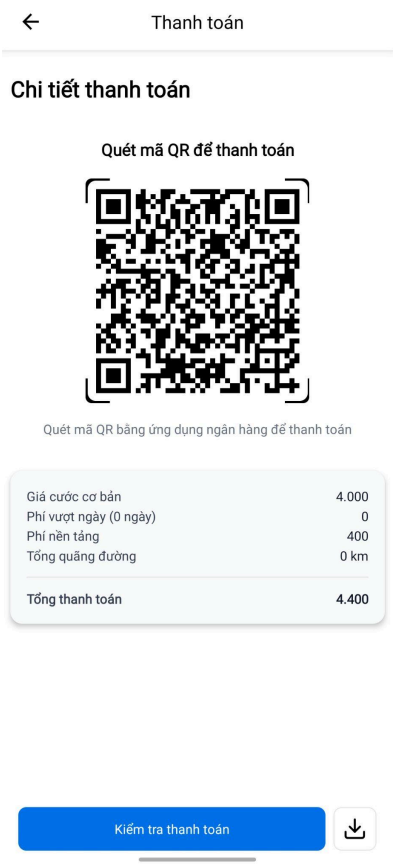
- Owner clicks button "Kiểm tra trước khi giao xe" and fills all require



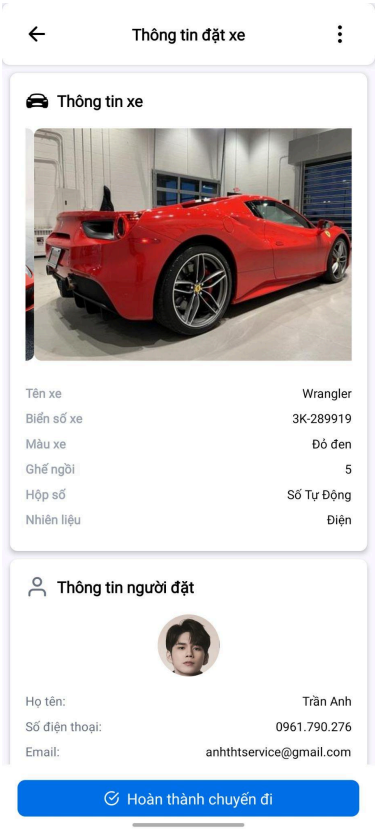
- Driver click button "Thanh toán" moving to payment screen



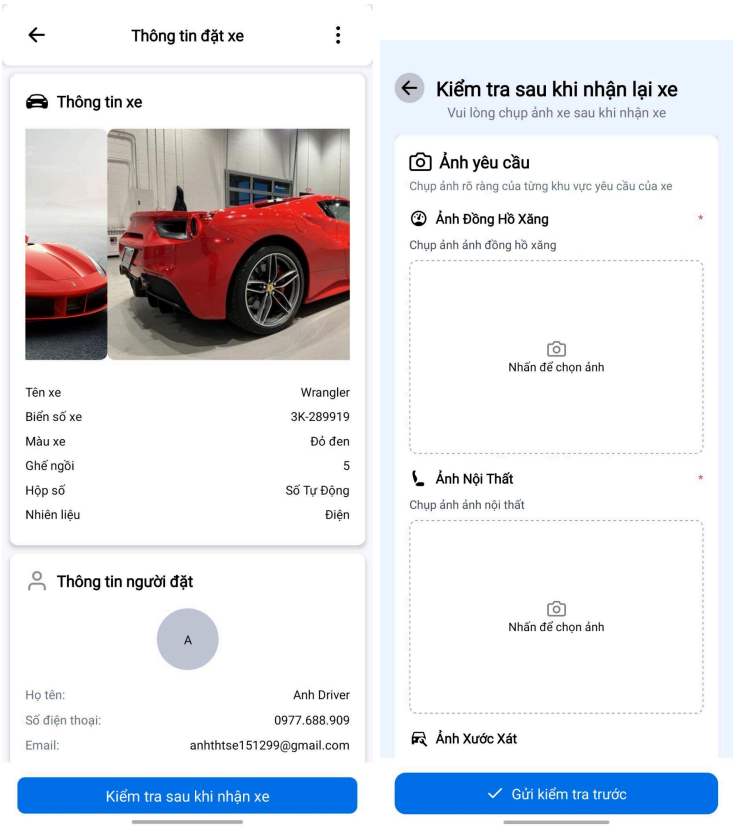
- Driver checkout by button "Xác nhận"



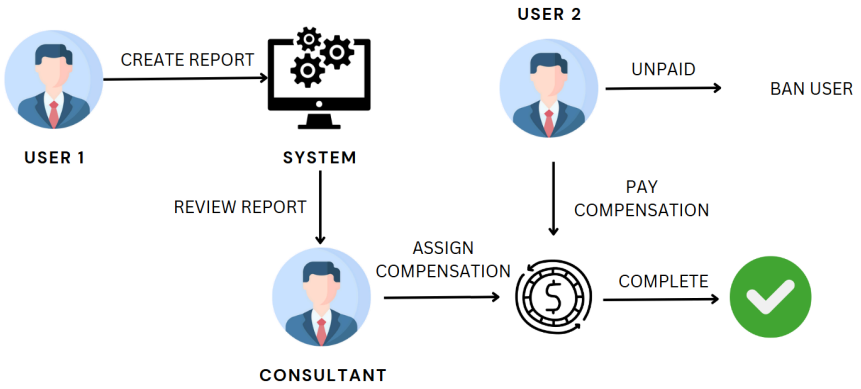
- Driver clicks button "Bắt đầu chuyến đi"



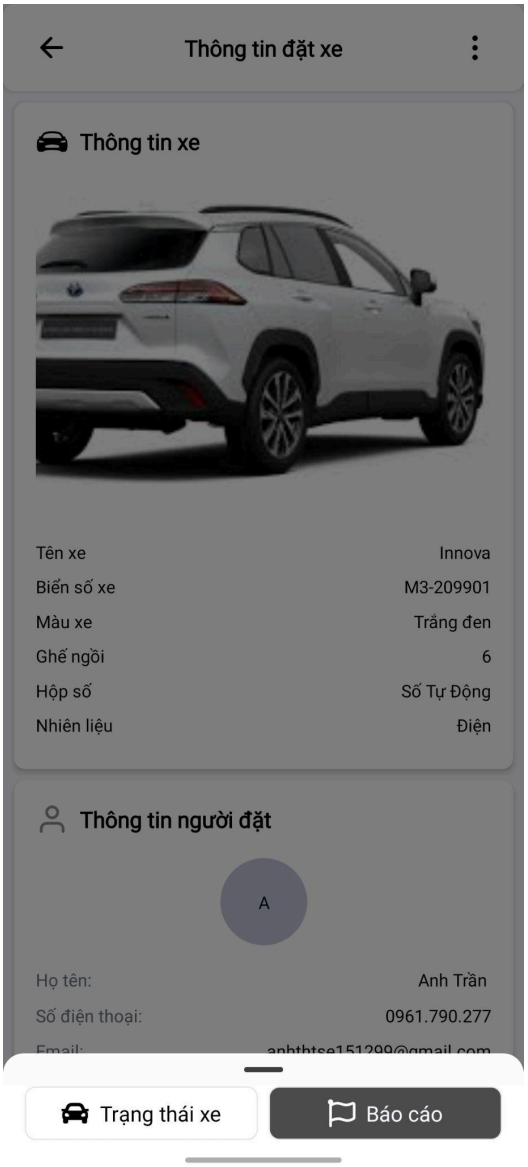
- Driver clicks button "Hoàn thành chuyển đi"
- Owner click button "Kiểm tra sau khi nhận xe"



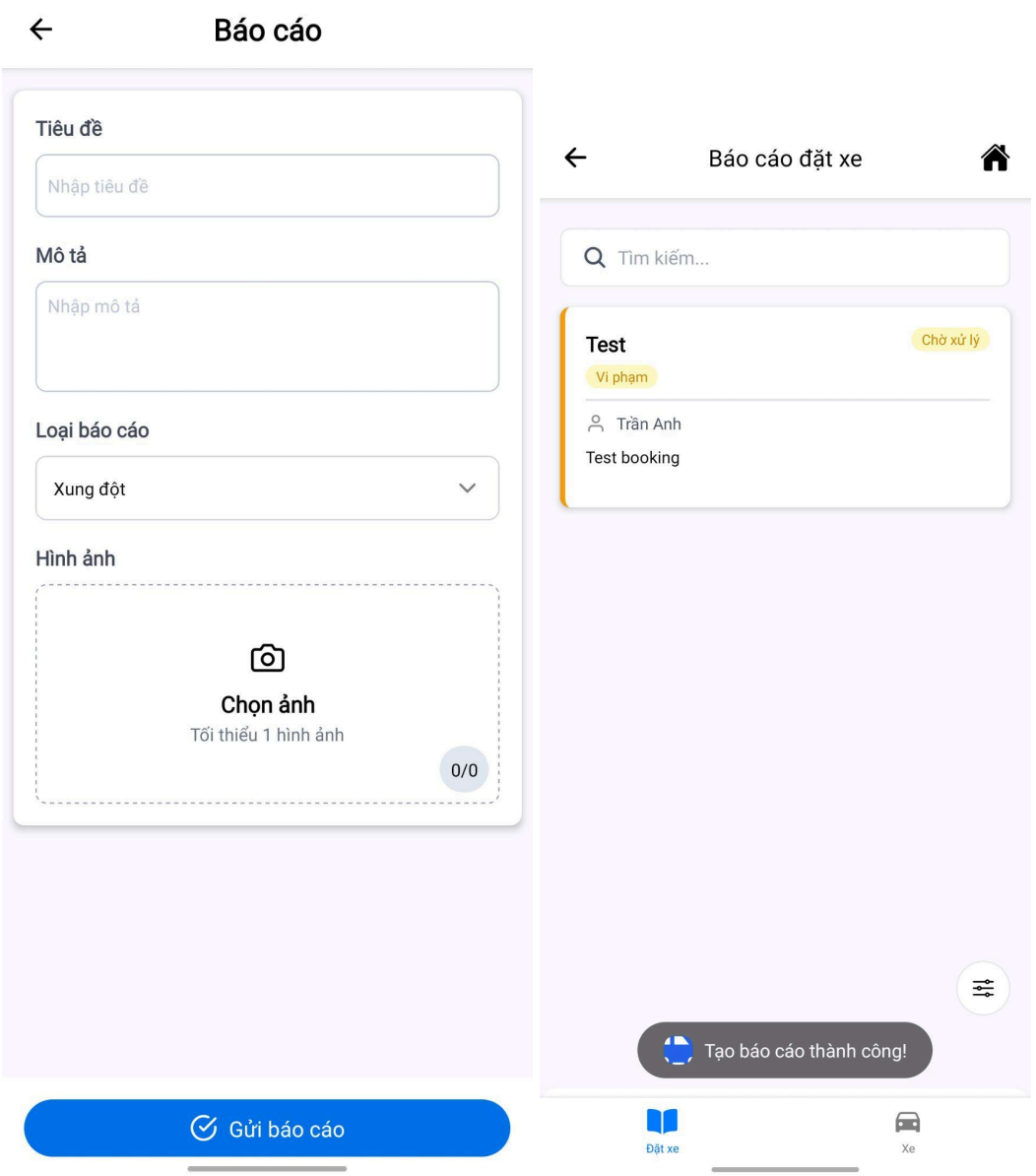
3.5 USER CREATE REPORT



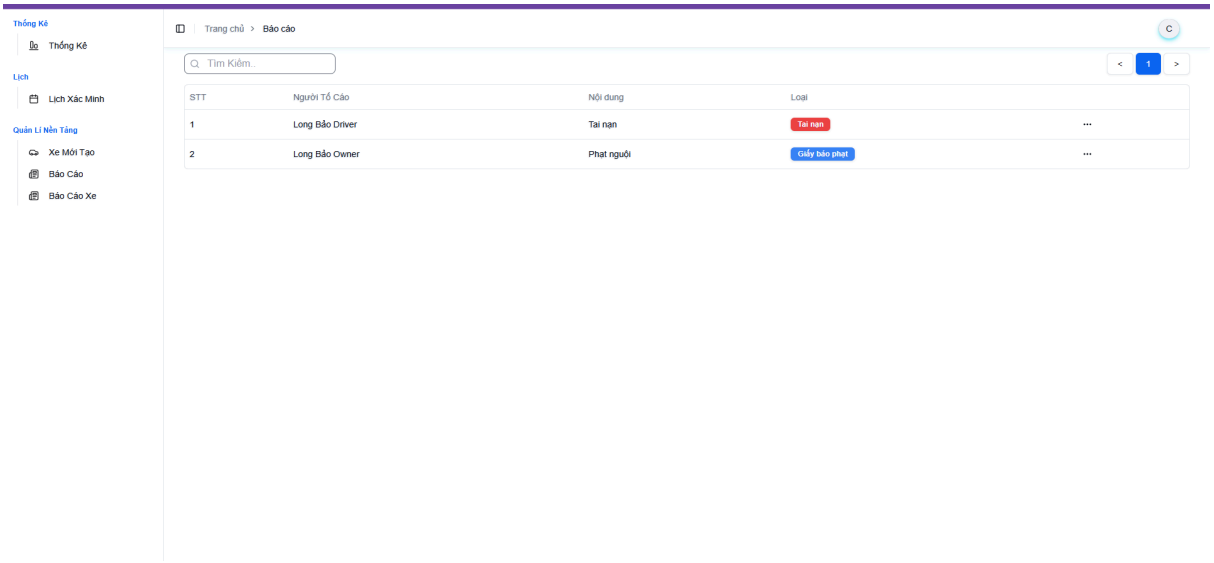
- Owner/ Driver will clicks menu action and click to button "Báo cáo"



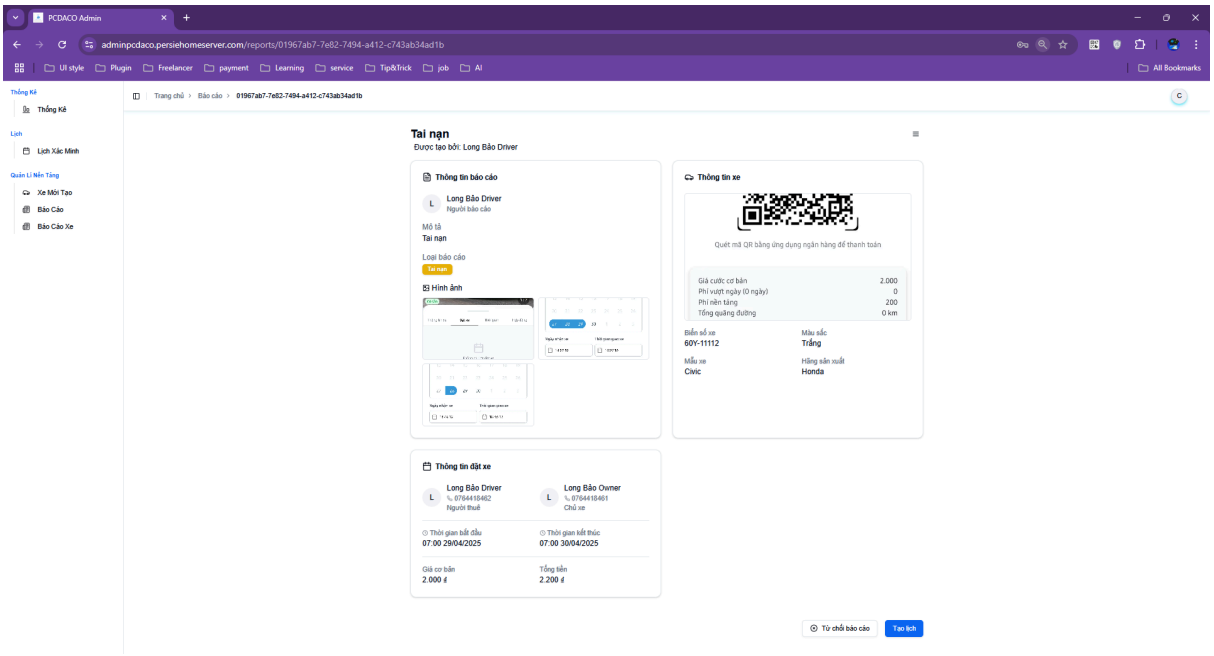
- Owner/ Driver will fills all require field and click "Gửi báo cáo"



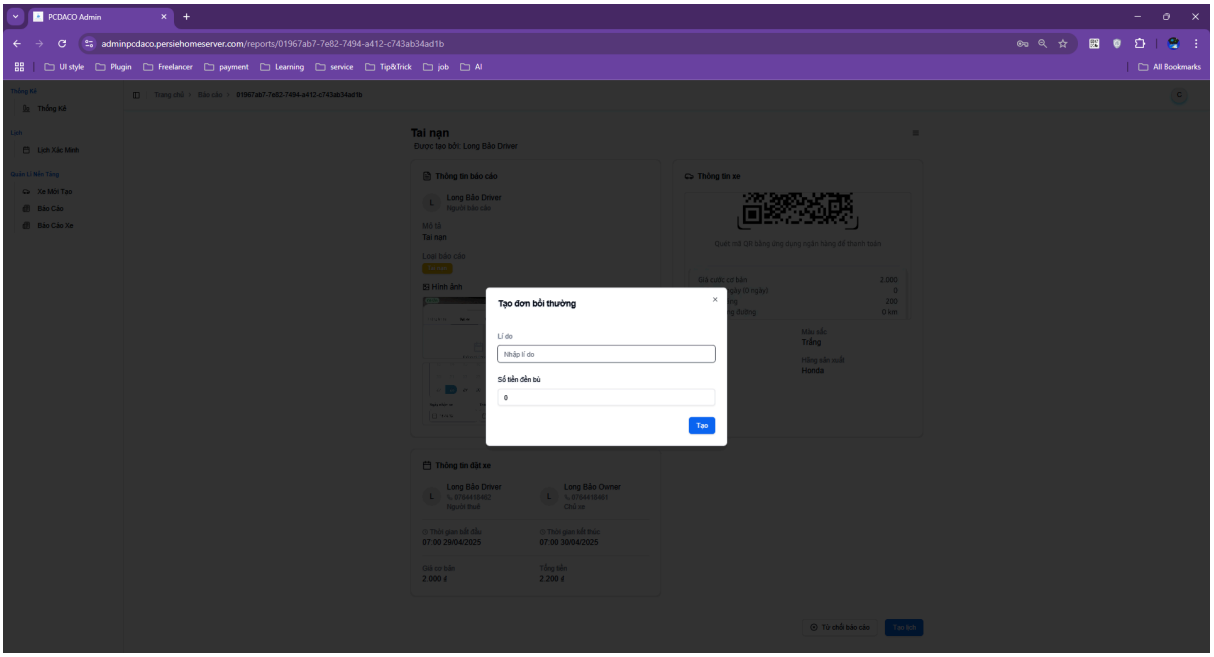
- Consultant will signs into system and select menu "Báo cáo"



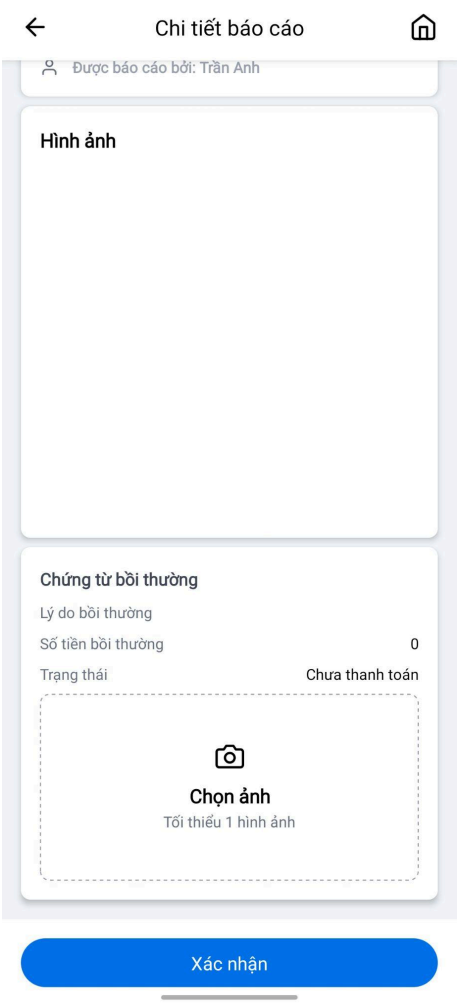
- Consultant will clicks "Chi tiết" to view detail report



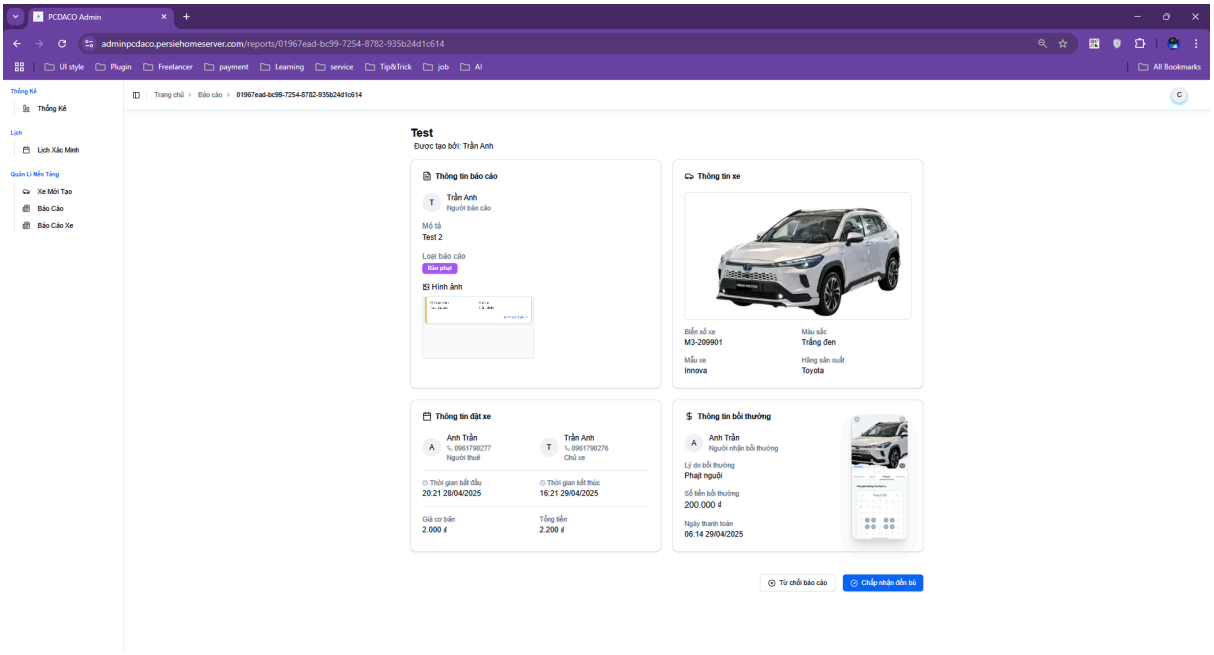
- Consultant clicks "Tạo đơn bồi thường" and fills all required fields before click "Tạo"



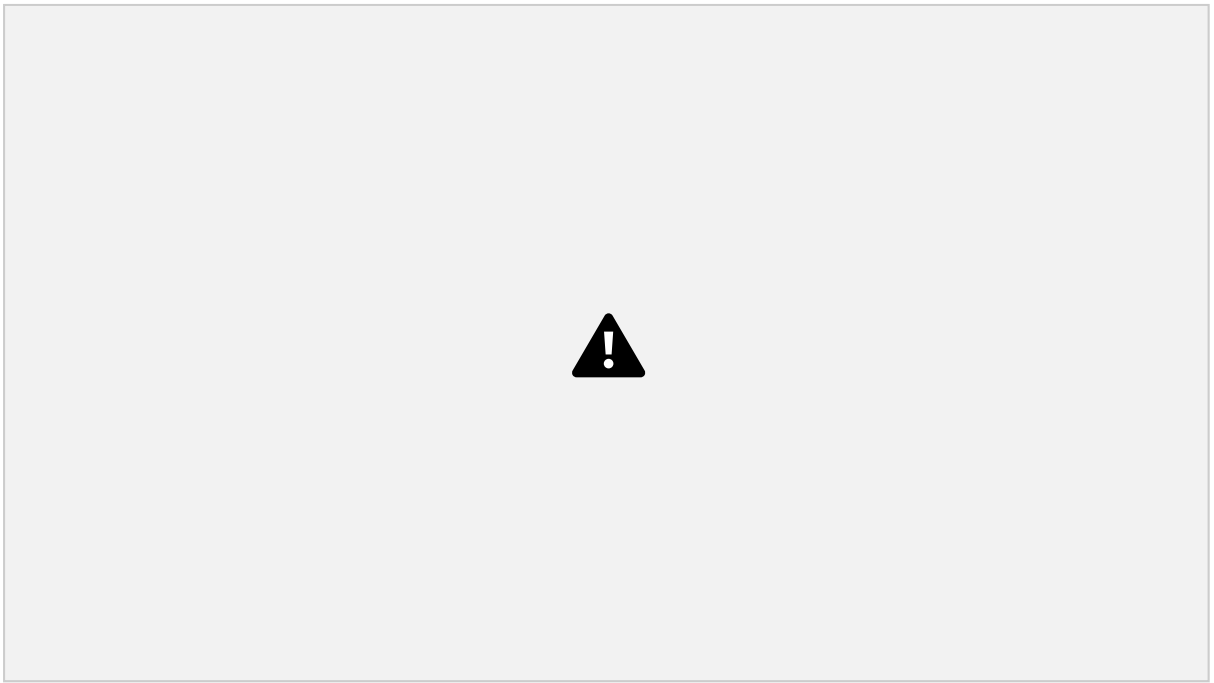
- Driver will upload image compensation proof and click "Gửi bằng chứng"



- Consultant clicks “Chấp nhận đền bù”



3.6 USER CREATE WITHDRAWAL



- User clicks on bank account item will show bottom sheet then click "Rút tiền" to move on the withdraw screen

- User will fills all the required fields then click "Rút tiền"

Rút tiền

Rút tiền từ tài khoản của bạn vào tài khoản ngân hàng

Số dư khả dụng

0 ₫

Số tiền rút

$ Nhập số tiền

Chọn nhanh

100.000đ 500.000đ 1.000.000đ 5.000.000đ

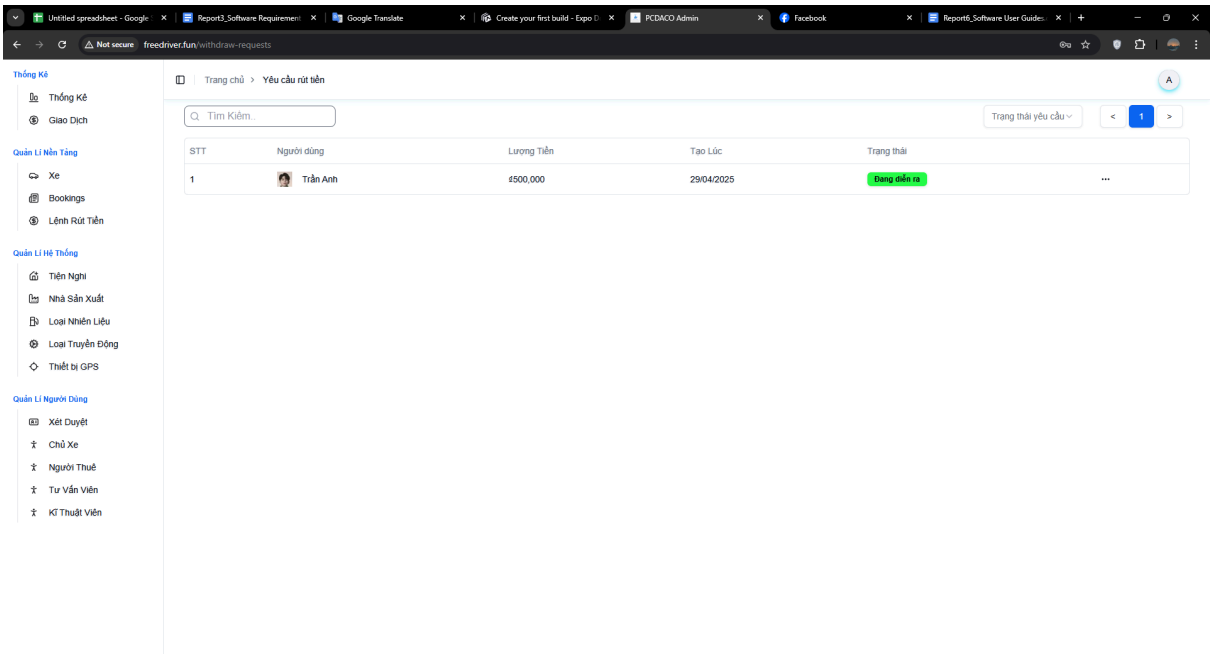
Tài khoản ngân hàng

Ngân hàng TMCP Tiên Phong
0361.193.000.1

Rút tiền sẽ được xử lý trong vòng 24 giờ làm việc

Rút tiền

- Admin signs into system then click menu "Lệnh rút tiền"



- Admin click "chuyển tiền" to move on payment screen